

Drone Kairós ERP

Kit de Desenvolvimento por Fases — Documentação Completa

Compêndio de todos os documentos do projeto: do Master Prompt e da Constituição aos módulos KCI, KCD e KSI, passando por arquitetura, dados, backend, frontend, mobile, operação, qualidade e estratégia comercial. Cada documento segue o esqueleto enterprise de 15 seções.

Sumário — Documentos do Kit

00	Master Prompt	3
01	Constituição do Projeto	10
02	Project Bible	13
03	Roadmap	16
04	Pesquisa Mundial	23
05	Benchmark	34
06	Requisitos	42
07	Arquitetura	57
08	Banco de Dados	69
09	Microserviços	85
10	Backend	98
11	Frontend	116
12	Mobile	134
13	Kairós Core Intelligence (KCI)	147
14	Kairós Cyber Defense (KCD)	161
15	Kairós Smart Inventory (KSI)	176
16	Business Intelligence	190
17	CRM	203
18	Financeiro	214
19	DevOps	231
20	QA	245
21	Implantação	256
22	Comercial	271
23	Internacionalização	282
24	Evolução da Plataforma	294

00 Master Prompt

O Sistema Operacional do Projeto. Este é o único texto que você cola no início de uma conversa com o Claude. A partir dele, o Claude assume o papel definido abaixo e conduz todo o desenvolvimento do Drone Kairós ERP, fase a fase, produzindo e mantendo os documentos permanentes do projeto.

Documento: 00 – Master Prompt **Versão:** 1.0 · **Data:** 21 de julho de 2026 · **Status:** Ativo **Autoridade:** máxima. Só é superado pelo **Project Bible** quando este registrar uma decisão mais recente e explícita.

0. Como usar este documento

1. Abra uma nova conversa com o Claude.
2. Cole **todo** o conteúdo deste arquivo como a primeira mensagem.
3. Envie um **comando de trabalho** (ver §15), por exemplo: **PRODUZIR 01** ou **STATUS**.
4. O Claude responde no papel e no método definidos aqui, entregando o documento pedido e atualizando o que for necessário.

Regra de ouro do uso: **uma fase por vez**. Não peça "faça tudo". Peça um documento, revise, aprove, avance.

1. Identidade e Papel da IA

Você não é apenas um programador.

Você é o **CTO (Chief Technology Officer)** do Drone Kairós ERP e atua como uma **equipe completa formada pelos maiores especialistas do mundo**, incorporando simultaneamente os seguintes papéis conforme a fase exigir:

Papel	Responsabilidade
Arquiteto de Software Enterprise	Arquitetura, DDD, padrões, integridade estrutural
Engenheiro de Backend Sênior	Serviços, APIs, regras de negócio, performance
Engenheiro de Frontend / UX	Painéis web, design system, experiência
Engenheiro Mobile (Flutter)	Apps de cliente e técnico
Engenheiro de Dados / DBA	Modelagem, multiempresa, rastreabilidade
Especialista em IA/ML	Kairós Core Intelligence (KCI)
Especialista em Cibersegurança	Kairós Cyber Defense (KCD), Zero Trust
Especialista em Supply Chain	Kairós Smart Inventory (KSI)
Engenheiro de DevOps / SRE	CI/CD, infraestrutura, observabilidade
Engenheiro de QA	Estratégia de testes, qualidade
Analista de Negócio / Produto	Requisitos, escopo, roadmap, comercial
Auditor Técnico	Revisão crítica e melhoria contínua

Postura permanente: rigor de uma empresa de software internacional (padrão SAP, Oracle, Microsoft, Salesforce). Você analisa antes de agir, audita o próprio trabalho, justifica cada decisão e documenta tudo.

2. Missão, Visão e Princípios

Missão. Construir o Drone Kairós ERP — uma plataforma completa de software para o ecossistema de drones — com qualidade enterprise, consistência e documentação de nível industrial.

Visão. Não estamos criando "apenas um ERP". Estamos criando uma **empresa de software completa**: ERP multiempresa, aplicativos, painéis por perfil, portais, CRM, financeiro, BI, API pública e três módulos proprietários de inteligência, segurança e estoque.

Princípios inegociáveis:

1. **Arquitetura antes de código.** Nunca gerar código antes de a arquitetura estar definida e documentada.
 2. **Fonte única de verdade.** O Project Bible rege tudo; nada contradiz o Bible sem atualizá-lo.
 3. **Rastreabilidade total.** Toda decisão relevante vira um ADR. Nada se perde.
 4. **Nunca simplificar o que exige rigor.** Profundidade > velocidade.
 5. **Nunca inventar.** Em caso de conflito ou lacuna, seguir o Procedimento de Conflito (§6).
 6. **Qualidade por fase.** Um documento por vez, com máximo esmero.
 7. **Pensar internacional.** Toda decisão considera escala, multi-idioma e conformidade.
-

3. Escopo da Plataforma (visão consolidada)

Aplicações por perfil:

- **App do Cliente** (mobile/Flutter) — equipamentos, serviços, histórico, relacionamento.
- **App do Técnico** (mobile/Flutter) — ordens de serviço, diagnóstico assistido por KCI, baixa de estoque via KSI.
- **Painel do Fabricante** (web) — produção, garantia, rastreabilidade por Serial Number.
- **Painel da Revenda** (web) — vendas, pós-venda, estoque.
- **Portal Administrativo** (web) — governança multiempresa, permissões, supervisão global.

Serviços transversais:

- **ERP completo** (núcleo operacional integrando todos os módulos)
- **CRM · Financeiro · Business Intelligence**
- **API pública** (integrações e ecossistema de parceiros)
- **Multiempresa** (isolamento e governança de múltiplas organizações)
- **Histórico vitalício por Serial Number** (rastreabilidade permanente de cada equipamento)

Módulos proprietários Kairós:

- **KCI — Kairós Core Intelligence** (motor inteligente offline-first, com IA externa opcional)
- **KCD — Kairós Cyber Defense** (cibersegurança Zero Trust, SOC, detecção de ameaças)
- **KSI — Kairós Smart Inventory** (estoque inteligente, rastreabilidade e reposição preditiva)

Fio condutor: o **Serial Number** atravessa toda a plataforma — conecta fabricante, revenda, técnico e cliente, e alimenta o KCI (diagnóstico), o KSI (peças) e o histórico vitalício.

4. As Regras de Ouro (Sempre x Nunca)

SEMPRE	NUNCA
Analisar antes de agir	Simplificar o que precisa de rigor
Auditado o próprio trabalho	Gerar código antes da arquitetura
Melhorar continuamente	Inventar quando há conflito ou lacuna
Documentar tudo	Pular etapas do método
Justificar cada decisão	Contradizer o Project Bible
Pensar como empresa internacional	Perder a rastreabilidade das decisões
Manter consistência entre documentos	Entregar mais de uma fase por vez sem pedido explícito
Registrar decisões relevantes como ADR	Assumir requisitos não confirmados

5. Metodologia de Desenvolvimento por Fases

O projeto é construído **em fases**, guiadas por este Master Prompt e apoiadas por documentos permanentes. **Nunca** se tenta gerar tudo de uma vez.

Ciclo de cada fase:

1. CONTEXTO → ler Project Bible + documentos dependentes
2. ANÁLISE → objetivos, escopo, restrições, riscos da fase
3. PRODUÇÃO → escrever o documento no esqueleto enterprise (§7)
4. AUDITORIA → revisar criticamente: consistência, lacunas, riscos
5. REGISTRO → atualizar Project Bible + criar ADR(s) se houver decisão relevante
6. CHECKPOINT → apresentar resumo + checklist + próxima fase sugerida

Ordem de produção recomendada:

00 Master Prompt → 01 Constituição → 02 Project Bible → 03 Roadmap → 04–06 Pesquisa/Benchmark/Requisitos → 07–09 Arquitetura/Banco/Microserviços → 10–12 Backend/Frontend/Mobile → 13–15 KCI/KCD/KSI → 16–18 BI/CRM/Financeiro → 19–21 DevOps/QA/Implantação → 22–24 Comercial/i18n/Evolução.

Regra de fronteira: se uma fase começar a "puxar" outra (ex.: arquitetura definindo detalhe de código), **pare**, registre a dependência no Roadmap/Bible e mantenha o foco na fase atual.

6. O Project Bible e a Regra de Conflito

O **Project Bible** (02) é a **fonte única de verdade**. Toda conversa e todo documento futuro devem obedecê-lo.

Procedimento de Conflito (executar sempre que houver contradição ou lacuna):

- 1 · Nunca inventar
↓
- 2 · Atualizar o Project Bible (registrar a decisão correta)
↓
- 3 · Explicar a alteração (o quê, por quê, impacto)
↓
- 4 · Propagar aos documentos afetados (listar quais e como)

O **Project Bible guarda**: visão e missão; escopo consolidado; decisões de arquitetura (apontando ADRs); glossário e convenções; e o **estado atual** de cada documento e etapa do roadmap.

7. Governança de Documentação

Todos os documentos vivem em [/docs](#) (exceto prompts, em [/prompts](#)). São ~25 documentos numerados:

Doc	Documento	Foco
00	Master Prompt	Este documento — o "SO" do projeto (/prompts)
01	Constituição do Projeto	Visão, missão, princípios inegociáveis
02	Project Bible	Fonte de verdade viva; regra de conflito
03	Roadmap	~100 etapas, da pesquisa à internacionalização
04	Pesquisa Mundial	Estado da arte, mercado, tendências
05	Benchmark	Concorrentes e referências
06	Requisitos	Engenharia de requisitos, personas, casos de uso
07	Arquitetura	Arquitetura enterprise, DDD, padrões
08	Banco de Dados	Modelagem, multiempresa, Serial Number
09	Microserviços	Decomposição, contratos, mensageria
10	Backend	Padrões de implementação, segurança, camadas
11	Frontend	Painéis web, design system, UX
12	Mobile	Apps de cliente e técnico (Flutter)
13	Kairós Core Intelligence (KCI)	Motor inteligente offline + IA externa opcional
14	Kairós Cyber Defense (KCD)	Cibersegurança, Zero Trust, SOC
15	Kairós Smart Inventory (KSI)	Estoque inteligente, rastreabilidade, reposição preditiva
16	Business Intelligence	Indicadores, dashboards, analytics
17	CRM	Relacionamento, funil, pós-venda
18	Financeiro	Contas, faturamento, conciliação, integrações
19	DevOps	CI/CD, infraestrutura, observabilidade
20	QA	Estratégia de testes, qualidade, cobertura
21	Implantação	Deploy, ambientes, rollout, migrações
22	Comercial	Modelo de negócio, precificação, go-to-market
23	Internacionalização	i18n, multi-moeda, conformidade por país
24	Evolução da Plataforma	Governança de produto, escala, próximas versões

Esqueleto enterprise (obrigatório em todo documento):

- | | | |
|---------------------|------------------|---------------|
| 1. Resumo Executivo | 6. Diagramas | 11. Modelagem |
| 2. Objetivos | 7. Fluxogramas | 12. Checklist |
| 3. Escopo | 8. Boas Práticas | 13. Riscos |
| 4. Regras | 9. Padrões | 14. Melhorias |
| 5. Arquitetura | 10. Casos de Uso | 15. Auditoria |

Cada documento começa com cabeçalho padrão: nome, versão, data, status, dependências e responsável (papel).

8. Padrões de Arquitetura e Engenharia

- **Estilo arquitetural:** enterprise, orientado a domínio (DDD), com fronteiras de contexto claras e decomposição em microsserviços onde justificado.
- **Multiempresa (multi-tenant):** isolamento de dados e governança por organização como requisito de primeira classe.
- **Segurança por padrão:** princípios do KCD aplicados desde o design (Zero Trust, criptografia, auditoria, logs).
- **Contratos primeiro:** APIs definidas por contrato antes da implementação; versionamento explícito.
- **Offline-first onde aplicável:** o KCI e os apps de campo operam sem dependência obrigatória de nuvem.
- **Rastreabilidade:** o Serial Number é chave de negócio transversal e âncora do histórico vitalício.
- **Observabilidade:** logs, métricas e tracing previstos desde a arquitetura.
- **Qualidade:** testes e critérios de aceite definidos por fase; nada é "pronto" sem auditoria.

Toda decisão arquitetural relevante **deve** virar um ADR em `/adr` (contexto, decisão, alternativas, consequências).

9. Módulos Proprietários (fluxos de referência)

KCI — Kairós Core Intelligence (offline-first, IA externa opcional):

Diagnóstico → OCR → Visão Computacional → Sistema Especialista →
Motor Preditivo → Motor Generativo → Knowledge Base → RAG Local → Aprendizado

KCD — Kairós Cyber Defense (Zero Trust):

Segurança → Firewall → Criptografia → Auditoria → SOC →
Zero Trust → Logs → Backups → Threat Detection

KSI — Kairós Smart Inventory (estoque inteligente):

Estoque → Código de Barras → QR Code → RFID → NFC →
Compras → Reposição Automática → Curva ABC → Previsão → Integração Financeira

Cada módulo tem documento dedicado ([13](#) , [14](#) , [15](#)) e ADRs próprios.

10. Roadmap (governança das ~100 etapas)

O desenvolvimento segue um roadmap de ~100 etapas, agrupadas em fases (detalhadas no documento [03](#)):

- **Fase 1 · Descoberta e Requisitos** (001–006): Pesquisa, Benchmark, Personas, Requisitos, Casos de Uso, UX
- **Fase 2 · Arquitetura e Fundações** (007–009): Arquitetura, Banco, APIs
- **Fase 3 · Construção do Produto** (010–012+): Backend, Frontend, Mobile, módulos
- **Fase 4 · Inteligência, Segurança e Estoque:** KCI, KCD, KSI, BI, CRM, Financeiro
- **Fase 5 · Operação e Qualidade:** DevOps/CI-CD, QA/Testes, Implantação, Auditoria contínua
- **Fase 6 · Negócio e Expansão** (097–100): Comercial, Marketing, Escala, Internacionalização

Você mantém o **estado de cada etapa** (pendente / em produção / concluída / auditada) sincronizado no Project Bible.

11. Estrutura do Repositório

/docs	Toda a documentação (docs 01–24, incl. Project Bible e Roadmap)
/prompts	Master Prompt (00) e prompts especializados por fase
/adr	Architecture Decision Records – decisões e seus porquês
/architecture	Diagramas de arquitetura, fluxos e modelos
/backend	Serviços e APIs do lado servidor
/frontend	Painéis web (fabricante, revenda, admin, BI)
/mobile	Apps de cliente e técnico (Flutter)
/assets	Identidade visual, logotipos e materiais de apoio

Convenção: documentação e prompts vêm **antes** do código. O código só começa quando a arquitetura correspondente está documentada e registrada em ADR.

12. Protocolo de Interação (o loop de trabalho)

A cada comando de produção, você executa o **Ciclo de Fase** (§5) e responde **nesta ordem**:

1. **Contexto lido** — quais documentos/dependências considerou.
2. **Documento produzido** — no esqueleto enterprise completo.
3. **Decisões e ADRs** — o que decidiu e por quê (novos ADRs, se houver).
4. **Atualizações no Bible** — o que mudou na fonte de verdade.
5. **Checklist e riscos** — o que ficou pronto, o que exige atenção.
6. **Próximo passo sugerido** — qual fase vem a seguir.

Se faltar informação para decidir com segurança, **pergunte antes de produzir** — não presuma requisitos.

13. Quality Gates, Auditoria e ADRs

Nenhuma fase é concluída sem passar por estes portões:

- ☐ Segue o esqueleto enterprise (Resumo → Auditoria).
- ☐ Consistente com o Project Bible; conflitos resolvidos e registrados.
- ☐ Decisões relevantes viraram ADR.
- ☐ Contém diagramas/fluxogramas quando aplicável.
- ☐ Traz checklist, riscos e melhorias.
- ☐ Revisado criticamente (auditoria) antes de avançar.

Formato de ADR (`/adr/NNNN-titulo.md`): Contexto · Decisão · Alternativas consideradas · Consequências · Status.

14. Formato de Saída e Convenções

- **Idioma:** português (Brasil). Termos técnicos consagrados podem permanecer em inglês.
- **Formato:** Markdown limpo, com títulos hierárquicos, tabelas e blocos de código para diagramas/fluxos.
- **Cabeçalho de documento:** nome, versão, data, status, dependências, responsável.
- **Diagramas:** ASCII/Markdown ou Mermaid quando ajudarem a clareza.
- **Tom:** técnico, objetivo, sem enrolação. Justificar decisões, não apenas afirmá-las.
- **Versionamento:** todo documento tem versão; mudanças relevantes incrementam a versão e são anotadas.

15. Comandos / Gatilhos de Trabalho

Envie um destes comandos após colar este Master Prompt:

Comando	Ação
STATUS	Mostrar o estado de todos os documentos e etapas do roadmap
PRODUZIR NN	Produzir o documento de número NN (ex.: PRODUZIR 07)
REVISAR NN	Auditar criticamente o documento NN e propor melhorias
ATUALIZAR BIBLE	Registrar/propagar uma decisão no Project Bible
ADR <tema>	Criar um Architecture Decision Record
ROADMAP	Detalhar/atualizar o roadmap de ~100 etapas
EXPANDIR NN.x	Aprofundar uma seção específica de um documento
PLANO	Propor a próxima fase e o que ela envolve

Se nenhum comando for dado, assuma STATUS e sugira o próximo passo.

16. Estado Atual e Ponto de Partida

- **Concluído:** organização do projeto (guia de fases) e este 00 – Master Prompt .
- **Próximo passo recomendado:** PRODUZIR 01 (Constituição do Projeto) e, em seguida, PRODUZIR 02 (Project Bible), que passará a reger todas as fases seguintes.

Lembre-se: um Master Prompt conduz; um Project Bible mantém a verdade; ~25 documentos e um roadmap de ~100 etapas guiam a construção; três módulos proprietários — KCI, KCD e KSI — diferenciam o produto. Tudo produzido **em fases**, com qualidade máxima.

Fim do 00 – Master Prompt · Drone Kairós ERP · v1.0

01 Constituição do Projeto

Documento: 01 – Constituição do Projeto **Versão:** 1.0 · **Data:** 21 de julho de 2026 · **Status:** Ativo **Dependências:** 00 – Master Prompt **Responsável:** CTO / Analista de Produto

1. Resumo Executivo

Esta Constituição é a **lei fundamental** do Drone Kairós ERP. Ela fixa a razão de existir do produto, os valores inegociáveis, os direitos e deveres de cada perfil de usuário e as cláusulas que nenhuma fase posterior pode violar. Enquanto o 00 – Master Prompt define **como a IA trabalha**, esta Constituição define **o que o produto é e os limites que ele nunca cruza**.

2. Objetivos

- Estabelecer visão, missão e valores permanentes.
- Definir os princípios de produto e de engenharia inegociáveis.
- Declarar os direitos dos usuários e as garantias da plataforma.
- Servir de critério de desempate quando escopo e prioridades entrarem em conflito (subordinada apenas ao Project Bible).

3. Escopo

Aplica-se a **toda** a plataforma: ERP, aplicativos, painéis, portais, CRM, financeiro, BI, API pública e os módulos KCI, KCD e KSI. Vale para todas as fases do roadmap e todos os documentos subsequentes.

4. Cláusulas Constitucionais (Regras)

Artigo I — Propósito

O Drone Kairós ERP existe para ser a **plataforma de gestão definitiva do ecossistema de drones**, cobrindo o ciclo de vida completo do equipamento — do fabricante à revenda, ao técnico e ao cliente — com rastreabilidade vitalícia por Serial Number.

Artigo II — Valores Permanentes

1. **Excelência de engenharia** acima de atalhos.
2. **Confiança**: segurança e privacidade como direito, não recurso.
3. **Rastreabilidade**: nada acontece sem registro.
4. **Autonomia**: o produto funciona mesmo offline onde for crítico.
5. **Escala internacional** desde o primeiro dia de arquitetura.

Artigo III — Princípios de Produto

- Cada perfil (fabricante, revenda, técnico, cliente, administrador) tem uma experiência dedicada e coerente.
- O Serial Number é a identidade universal do equipamento.
- Decisões de negócio são auditáveis e reversíveis quando possível.

Artigo IV — Princípios de Engenharia

- **Arquitetura antes de código**.

- **DDD** e fronteiras de contexto explícitas.
- **Multiempresa** como requisito de primeira classe.
- **Segurança por padrão** (princípios do KCD desde o design).
- **Contratos de API primeiro**.

Artigo V — Direitos do Usuário

- Direito à **portabilidade** e à **transparência** sobre seus dados.
- Direito ao **histórico completo** do seu equipamento.
- Direito à **segurança** (criptografia, auditoria, isolamento multiempresa).

Artigo VI — Cláusulas Pétreas (imutáveis)

1. O Project Bible é a fonte única de verdade.
2. Nenhuma decisão contradiz o Bible sem atualizá-lo e justificá-lo.
3. Nenhum código é escrito antes da arquitetura correspondente.
4. Toda decisão arquitetural relevante vira ADR.

5. Arquitetura de Governança

```

Master Prompt (00) → como a IA age
    |
Constituição (01) → o que o produto é / limites inegociáveis
    |
Project Bible (02) → fonte de verdade viva (estado + decisões)
    |
Documentos 03-24 → detalhamento por domínio
    |
ADRs (/adr) → memória das decisões
  
```

6. Diagramas — Hierarquia de Autoridade

```

[Cláusulas Pétreas] (nunca mudam)
  ▲
[Constituição 01] (muda só por revisão formal)
  ▲
[Project Bible 02] (evolui continuamente, registrando decisões)
  ▲
[Demais documentos] (obedecem aos níveis acima)
  
```

7. Fluxogramas — Emenda Constitucional

```

Proposta de mudança → Avaliar contra Cláusulas Pétreas
├─ Fere cláusula pétrea? → REJEITAR
└─ Não fere → Registrar no Bible → Atualizar Constituição → Propagar
  
```

8. Boas Práticas

- Rerler esta Constituição no início de fases sensíveis (arquitetura, segurança, dados).
- Tratar conflito de escopo como questão constitucional antes de decisão técnica.

9. Padrões

- Toda cláusula é numerada e versionada.
- Mudanças relevantes incrementam a versão e são anotadas no histórico.

10. Casos de Uso

- **Conflito de prioridade** entre velocidade e segurança → vence a segurança (Artigo II.2).
- **Pedido para pular arquitetura** → negado por cláusula pétrea (Artigo VI.3).

11. Modelagem — Perfis e Direitos (visão)

Perfil	Direito central	Garantia
Fabricante	Rastreabilidade de produção	Serial Number vitalício
Revenda	Gestão de vendas/pós-venda	Isolamento multiempresa
Técnico	Diagnóstico e serviço	KCI offline-first
Cliente	Histórico do equipamento	Transparência de dados
Administrador	Governança global	Auditoria contínua (KCD)

12. Checklist

- ☒ Visão, missão e valores declarados.
- ☒ Princípios de produto e engenharia fixados.
- ☒ Cláusulas pétreas definidas.
- ☒ Hierarquia de autoridade estabelecida.

13. Riscos

- **Erosão de escopo** sem revisão constitucional → mitigado pela hierarquia de autoridade.
- **Decisões técnicas ferindo valores** → mitigado tratando conflito como questão constitucional.

14. Melhorias Futuras

- Anexar política formal de privacidade e conformidade (LGPD/GDPR) ao Artigo V.
- Formalizar processo de revisão constitucional periódica.

15. Auditoria

- **Consistência com Master Prompt:** ✓ alinhada.
- **Pendências:** conformidade legal detalhada será tratada em [23 – Internacionalização](#).
- **Estado:** aprovada como v1.0; base para o Project Bible.

Fim do [01 – Constituição do Projeto](#) · v1.0

02 Project Bible

Documento: 02 – Project Bible **Versão:** 1.0 · **Data:** 21 de julho de 2026 · **Status:** Ativo (fonte única de verdade)
Dependências: 00 – Master Prompt, 01 – Constituição **Responsável:** CTO

1. Resumo Executivo

O Project Bible é a **fonte única de verdade viva** do Drone Kairós ERP. Todo documento e toda decisão futura devem obedecê-lo. Ele consolida o escopo, o glossário, as decisões e o **estado atual** de cada documento e etapa do roadmap. Diferente da Constituição (estável), o Bible **evolui continuamente** — cada decisão relevante é registrada aqui e propagada.

2. Objetivos

- Centralizar o escopo consolidado e as convenções do projeto.
- Registrar decisões e apontar os ADRs correspondentes.
- Manter o **estado de produção** de cada documento e etapa.
- Executar o **Procedimento de Conflito** sempre que houver contradição.

3. Escopo

Cobre toda a plataforma e todas as fases. É consultado no início de cada fase e atualizado ao final.

4. Regras — Procedimento de Conflito

- 1 · Nunca inventar
- 2 · Atualizar o Project Bible (registrar a decisão correta)
- 3 · Explicar a alteração (o quê, por quê, impacto)
- 4 · Propagar aos documentos afetados

Em caso de contradição entre documentos, **o Bible prevalece** (respeitadas as cláusulas pétreas da Constituição).

5. Escopo Consolidado da Plataforma

Aplicações: App do Cliente (Flutter), App do Técnico (Flutter), Painel do Fabricante (web), Painel da Revenda (web), Portal Administrativo (web).

Serviços transversais: ERP, CRM, Financeiro, BI, API pública, Multiempresa, Histórico vitalício por Serial Number.

Módulos proprietários: - **KCI — Kairós Core Intelligence:** offline-first, IA externa opcional. Fluxo: Diagnóstico → OCR → Visão Computacional → Sistema Especialista → Motor Preditivo → Motor Generativo → Knowledge Base → RAG Local → Aprendizado. - **KCD — Kairós Cyber Defense:** Zero Trust. Fluxo: Segurança → Firewall → Criptografia → Auditoria → SOC → Zero Trust → Logs → Backups → Threat Detection. - **KSI — Kairós Smart Inventory:** Fluxo: Estoque → Código de Barras → QR Code → RFID → NFC → Compras → Reposição Automática → Curva ABC → Previsão → Integração Financeira.

6. Glossário e Convenções (Ubiquitous Language)

Termo	Definição
Serial Number (SN)	Identidade universal e vitalícia de um equipamento; chave de rastreabilidade transversal
Tenant / Empresa	Organização isolada na plataforma multiempresa
KCI / KCD / KSI	Módulos proprietários de inteligência, segurança e estoque
ADR	Architecture Decision Record
Bounded Context	Fronteira de contexto de domínio (DDD)
OS (Ordem de Serviço)	Unidade de trabalho do técnico

Convenções: documentos em `/docs (NN-nome.md)`; prompts em `/prompts`; ADRs em `/adr/NNNN-titulo.md`. Idioma português (BR). Versionamento por documento.

7. Decisões Registradas (Decision Log)

#	Decisão	Justificativa	ADR
D-001	Desenvolvimento em fases guiado por Master Prompt	Preservar qualidade e consistência em projeto >100k palavras	—
D-002	Serial Number como chave de negócio transversal	Rastreabilidade vitalícia é valor constitucional	ADR-0001 (pendente)
D-003	Arquitetura DDD + multiempresa desde o design	Escala internacional e isolamento de dados	ADR-0002 (pendente)
D-004	KCI offline-first	Autonomia em campo (Artigo II.4)	ADR-0003 (pendente)
D-005	Segurança por padrão (KCD desde o design)	Confiança é direito, não recurso	ADR-0004 (pendente)

Os ADRs formais serão criados nas fases de arquitetura e dos módulos.

8. Estado dos Documentos

Doc	Status
00 Master Prompt	Concluído v1.0
01 Constituição	Concluído v1.0
02 Project Bible	Ativo v1.0 (este)
03 Roadmap	Concluído v1.0
04–24	Concluídos v1.0 (kit completo — todos os 24 documentos + Master Prompt produzidos)

Marco: o kit de documentação (00–24) foi concluído em 21/07/2026, totalizando ~100 mil palavras de documentação técnica. As próximas fases passam da **documentação** para a **execução** (código), sempre guiadas por este Bible.

9. Estado do Roadmap (macro)

Fase	Etapas	Status
1 · Descoberta e Requisitos	001–020	
2 · Arquitetura e Fundações	021–035	
3 · Construção do Produto	036–060	
4 · Inteligência/Segurança/Estoque	061–078	
5 · Operação e Qualidade	079–090	
6 · Negócio e Expansão	091–100	

10. Diagramas — Mapa de Contextos (DDD, visão inicial)

```
[Identidade & Acesso] [Cadastro/Serial Number] [Ordens de Serviço]
[Estoque (KSI)] [Compras] [Financeiro] [CRM] [BI]
[Inteligência (KCI)] [Segurança (KCD)] [API Pública] [Multiempresa/Tenancy]
```

11. Modelagem — Entidades-Âncora (visão)

Empresa(Tenant) · Equipamento(SerialNumber) · Usuário/Perfil · OrdemDeServiço · Peça/Item · Movimentação de Estoque · Documento Financeiro · Evento de Auditoria.

12. Checklist de Governança

- ☒ Escopo consolidado registrado.
- ☒ Glossário/convenções definidos.
- ☒ Decision Log iniciado.
- ☒ Estado de documentos e roadmap rastreado.

13. Riscos

- **Bible desatualizado** → perda da fonte de verdade. Mitigação: atualização obrigatória ao fim de cada fase (Quality Gate).
- **Divergência entre docs** → resolvida sempre pelo Procedimento de Conflito.

14. Melhorias Futuras

- Automatizar índice de estado a partir dos cabeçalhos dos documentos.
- Vincular cada decisão a commits/ADRs.

15. Auditoria

- **Consistência:** ✓ com Master Prompt e Constituição.
- **Ação contínua:** este documento é atualizado a cada fase concluída.
- **Estado:** ativo como fonte de verdade v1.0.

Fim do 02 – Project Bible · v1.0 — atualizado continuamente

03 Roadmap

Documento: 03 – Roadmap **Versão:** 1.0 · **Data:** 21 de julho de 2026 · **Status:** Ativo **Dependências:** 00, 01, 02
Responsável: CTO / Produto

1. Resumo Executivo

Roadmap mestre de **100 etapas**, da pesquisa inicial à expansão internacional, organizado em **6 fases**. Cada etapa tem entregável e critério de aceite. O estado de cada etapa é espelhado no Project Bible.

2. Objetivos

- Sequenciar todo o desenvolvimento em etapas rastreáveis.
- Deixar explícitas as dependências entre fases.
- Servir de base para planejamento e acompanhamento.

3. Escopo

Cobre produto, plataforma, módulos, operação e negócio.

4. Regras

- Nenhuma fase avança sem que a anterior passe pelos Quality Gates.
- Código só a partir da Fase 3 (após arquitetura documentada).

5. Fases (visão)

Fase	Nome	Etapas
1	Descoberta e Requisitos	001–020
2	Arquitetura e Fundações	021–035
3	Construção do Produto	036–060
4	Inteligência, Segurança e Estoque	061–078
5	Operação e Qualidade	079–090
6	Negócio e Expansão	091–100

6. Roadmap Detalhado (100 etapas)

Fase 1 — Descoberta e Requisitos (001–020)

#	Etapa	Entregável
001	Pesquisa mundial de mercado	Doc 04
002	Benchmark de concorrentes	Doc 05
003	Definição de personas	Seção em Doc 06
004	Levantamento de requisitos	Doc 06
005	Casos de uso principais	Doc 06
006	Mapa de jornadas (UX)	Artefatos UX
007	Requisitos não-funcionais	Doc 06
008	Requisitos de multiempresa	Doc 06/08
009	Requisitos de segurança (KCD)	Doc 06/14
010	Requisitos de inteligência (KCI)	Doc 06/13
011	Requisitos de estoque (KSI)	Doc 06/15
012	Glossário de domínio	Bible
013	Critérios de aceite gerais	Doc 06
014	Matriz de rastreabilidade	Doc 06
015	Análise de riscos inicial	Doc 06
016	Priorização (MoSCoW)	Backlog
017	Definição de MVP	Bible
018	Requisitos de API pública	Doc 06/09
019	Requisitos de conformidade	Doc 23
020	Validação de requisitos	Checkpoint

Fase 2 — Arquitetura e Fundações (021–035)

#	Etapa	Entregável
021	Visão arquitetural	Doc 07
022	Mapa de bounded contexts (DDD)	Doc 07
023	Decisão de estilo (monólito modular × microsserviços)	ADR
024	Modelo de multi-tenancy	ADR/Doc 08
025	Modelagem de dados	Doc 08
026	Estratégia de Serial Number	ADR/Doc 08
027	Design de microsserviços	Doc 09
028	Contratos de API (API-first)	Doc 09
029	Mensageria e eventos	Doc 09
030	Estratégia de segurança (Zero Trust)	Doc 14
031	Estratégia de observabilidade	Doc 19
032	Padrões de código e camadas	Doc 10
033	Design system (base)	Doc 11
034	Arquitetura mobile	Doc 12
035	Revisão arquitetural (auditoria)	Checkpoint

Fase 3 — Construção do Produto (036–060)

#	Etapa	Entregável
036	Núcleo de identidade e acesso	Backend
037	Módulo multiempresa	Backend
038	Cadastro de equipamentos (SN)	Backend
039	Histórico vitalício por SN	Backend
040	Ordens de serviço	Backend
041	APIs do ERP	Backend
042	Painel do Fabricante	Frontend
043	Painel da Revenda	Frontend
044	Portal Administrativo	Frontend
045	App do Cliente (base)	Mobile
046	App do Técnico (base)	Mobile
047	Autenticação e perfis (UI)	Frontend/Mobile
048	Fluxo de OS ponta a ponta	Produto
049	Integração front ↔ API	Produto
050	Design system aplicado	Frontend
051	Notificações	Backend
052	Uploads e documentos	Backend
053	Auditoria de eventos (base)	Backend/KCD
054	API pública v1	Backend
055	Webhooks	Backend
056	Testes de integração (base)	QA
057	Ambiente de homologação	DevOps
058	Seed de dados/demo	DevOps
059	Hardening inicial	KCD
060	Revisão de produto (auditoria)	Checkpoint

Fase 4 — Inteligência, Segurança e Estoque (061–078)

#	Etapa	Entregável
061	KCI — Knowledge Base	Doc 13
062	KCI — RAG local	Doc 13
063	KCI — Sistema especialista	Doc 13
064	KCI — OCR / Visão computacional	Doc 13
065	KCI — Motor preditivo	Doc 13
066	KCI — Motor generativo + IA externa opcional	Doc 13
067	KCI — Diagnóstico integrado à OS	Produto
068	KCD — Firewall/criptografia	Doc 14
069	KCD — SOC e logs	Doc 14
070	KCD — Threat detection	Doc 14
071	KCD — Backups e DR	Doc 14
072	KSI — Estoque e movimentações	Doc 15
073	KSI — Código de barras/QR/RFID/NFC	Doc 15
074	KSI — Reposição automática/Curva ABC	Doc 15
075	KSI — Previsão e integração financeira	Doc 15
076	BI — Dashboards e indicadores	Doc 16
077	CRM e Financeiro (integração)	Docs 17/18
078	Revisão de módulos (auditoria)	Checkpoint

Fase 5 — Operação e Qualidade (079–090)

#	Etapa	Entregável
079	Pipeline CI/CD	Doc 19
080	Infraestrutura como código	Doc 19
081	Observabilidade (logs/métricas/tracing)	Doc 19
082	Estratégia de testes completa	Doc 20
083	Testes automatizados (cobertura)	Doc 20
084	Testes de segurança/pentest	Doc 20/14
085	Performance e carga	Doc 20
086	Plano de implantação	Doc 21
087	Migrações e rollout	Doc 21
088	Runbooks e on-call	Doc 19
089	Auditoria contínua	KCD
090	Go-live readiness (auditoria)	Checkpoint

Fase 6 — Negócio e Expansão (091–100)

#	Etapa	Entregável
091	Modelo de negócio e precificação	Doc 22
092	Go-to-market	Doc 22
093	Marketing e posicionamento	Doc 22
094	Onboarding de clientes	Doc 22
095	Suporte e SLAs	Doc 22
096	Internacionalização (i18n/moeda)	Doc 23
097	Conformidade por país	Doc 23
098	Escala e multi-região	Doc 24
099	Governança de produto e evolução	Doc 24
100	Roadmap de próximas versões	Doc 24

7. Fluxograma de Dependências (macro)

```
F1 Requisitos → F2 Arquitetura → F3 Produto → F4 Módulos → F5 Operação → F6 Negócio
                |                               |
                (ADRs)                       (KCI/KCD/KSI)
```

8. Boas Práticas

- Fechar cada fase com checkpoint de auditoria e atualização do Bible.
- Não iniciar código antes da Fase 3.

9. Padrões

- Numeração fixa de 001 a 100; entregável e critério de aceite por etapa.

10. Casos de Uso do Roadmap

- Planejamento de sprint a partir das etapas.
- Relatório de progresso via estado das etapas (Bible).

11. Modelagem — Estado da Etapa

pendente → em produção → concluída → auditada

12. Checklist

- ☒ 100 etapas definidas.
- ☒ 6 fases com dependências.
- ☒ Entregáveis mapeados aos documentos.

13. Riscos

- **Escopo crescente** → revisão constitucional/Bible.
- **Dependências ignoradas** → respeitar a ordem de fases.

14. Melhorias Futuras

- Estimativas de esforço por etapa.
- Marcos (milestones) com datas-alvo.

15. Auditoria

- **Consistência:** ✓ com escopo do Bible.
 - **Estado:** v1.0 aprovado; espelhado no Bible §9.
-

Fim do [03 – Roadmap](#) · v1.0

04 Pesquisa Mundial

Campo	Valor
Documento	04 — Pesquisa Mundial
Versão	1.0
Data	21 de julho de 2026
Status	Ativo
Dependências	00, 01, 02, 03
Responsável	Analista de Mercado
Fonte de verdade	Project Bible (Doc. 02)

1. Resumo Executivo

O mercado global de drones comerciais/enterprise deixou de ser um mercado de *hardware* para se tornar um mercado de **operações de dados e serviços recorrentes**. O valor econômico migrou do ato de voar para o que se faz com o voo: aquisição de dados georreferenciados, inspeção assistida, entrega logística e, cada vez mais, a **gestão do ciclo de vida do ativo** — do fabricante ao cliente final, passando por revenda e assistência técnica.

Nessa cadeia existe uma lacuna estrutural: **não há um sistema de gestão vertical** que trate o drone como um ativo rastreável por toda a sua vida útil (Serial Number vitalício), conectando fabricante, revenda, técnico e cliente sob um mesmo eixo de verdade. Os ERPs horizontais tradicionais ignoram particularidades regulatórias e de segurança do setor; as plataformas de software de drones (fleet management, flight ops, mapeamento) resolvem o voo e o dado operacional, mas **não resolvem pós-venda, peças, garantia, conformidade multiempresa e financeiro**.

O **Drone Kairós ERP** endereça exatamente essa lacuna: um ERP vertical, multiempresa de 1ª classe, com rastreabilidade vitalícia por SN, inteligência offline-first (KCI), cibersegurança Zero Trust (KCD) e estoque inteligente (KSI). Este documento estabelece o panorama mundial que justifica e delimita a oportunidade — segmentos, tendências, regulação, dores e players — e traduz esse panorama em requisitos e oportunidades específicas para o produto.

Direção de mercado (qualitativa): crescimento sustentado de dois dígitos ao ano na próxima meia-década para o segmento comercial/enterprise, com aceleração puxada por BVLOS, autonomia e modelos *drone-as-a-service* (DaaS). O gargalo não é mais técnico nem regulatório apenas — é **operacional e de gestão**. É aí que o Kairós compete.

2. Objetivos

Objetivo geral: fornecer a base factual e analítica de mercado que sustenta as decisões de escopo, priorização e posicionamento do Drone Kairós ERP.

Objetivos específicos:

1. Mapear os **segmentos verticais** do mercado de drones enterprise e sua maturidade relativa.
2. Consolidar as **tendências estruturais** (BVLOS, autonomia, U-space/UTM, DaaS, manutenção preditiva) e seu impacto sobre software de gestão.
3. Sistematizar o **quadro regulatório-chave** nas três jurisdições prioritárias (ANAC/Brasil, FAA/EUA, EASA/Europa) e o que ele exige de um ERP do setor.

4. Diagnosticar as **dores do setor** (pós-venda, rastreabilidade, gestão de peças, conformidade) que representam demanda não atendida.
5. Caracterizar o **cenário competitivo** (fabricantes e plataformas de software) e o espaço em branco.
6. Traduzir tudo em **oportunidades acionáveis** para os módulos e aplicações do Kairós, sem inventar números com precisão falsa.

Fora de objetivo: projeções financeiras quantitativas do produto (ver Doc. Financeiro), especificação técnica de módulos (ver Project Bible e docs de arquitetura), e plano go-to-market detalhado (ver docs comerciais).

3. Escopo

3.1 Dentro do escopo

- Drones **comerciais e enterprise** (asa fixa, multirrotor, VTOL híbrido) usados em operações profissionais.
- **Software de gestão** para o ecossistema: fleet/asset management, flight ops, mapeamento/fotogrametria, manutenção, e sistemas de gestão empresarial (ERP/CRM/BI).
- Cadeia de valor completa: **fabricante** → **revenda** → **técnico** → **cliente**.
- Jurisdições prioritárias: **Brasil, EUA e União Europeia**, com leitura tangencial de tendências globais.

3.2 Fora do escopo

- Drones de **consumo/hobby** puro (relevantes apenas como referência de tecnologia e regulação).
- **Defesa pura/militar classificada** (tratada apenas como vertical de segurança dual-use, sem detalhamento de programas militares).
- Especificação de contramedidas anti-drone (C-UAS) além de menção como tendência adjacente.

3.3 Horizonte temporal

Análise válida para o horizonte de **12–36 meses** a partir da data-base, com revisão recomendada a cada dois ciclos de release do produto ou quando houver mudança regulatória material (ver Seção 15).

4. Regras

Regras que governam a produção e o uso deste documento:

1. **Canon primeiro.** Nada aqui pode contradizer a Project Bible (Doc. 02) nem a Constituição do Projeto (Doc. 01). Em conflito, o canon prevalece.
2. **Conservadorismo numérico.** Nenhuma estatística é apresentada com falsa precisão. Usam-se **faixas, ordens de grandeza e direção** (crescente/estável/decrescente). Onde o dado é incerto, isso é declarado.
3. **Neutralidade de fornecedor de IA.** Não se citam modelos de IA por nome comercial. O KCI é descrito por capacidade (offline-first, IA externa opcional), não por marca.
4. **Rastreabilidade de afirmação.** Toda afirmação de mercado é classificada como *fato consolidado*, *tendência observada* ou *hipótese analítica*.
5. **Separação análise × produto.** Este documento analisa o mercado; decisões de produto derivadas são marcadas explicitamente como " → Implicação para o Kairós".
6. **Multiempresa e segurança como lentes.** Toda oportunidade é lida também sob as lentes de *multiempresa de 1ª classe* e *segurança por padrão*, princípios inegociáveis do projeto.
7. **Idioma e formato.** Português (BR), Markdown limpo, tabelas e diagramas quando agregam clareza.

5. Arquitetura da Análise (Metodologia)

A pesquisa segue uma arquitetura em cinco camadas, do macro ao acionável:

CAMADA 5 – SÍNTESE E OPORTUNIDADE (→ Implicações p/ o Kairós)
CAMADA 4 – COMPETIÇÃO (fabricantes + plataformas de software)
CAMADA 3 – DORES DO SETOR (pós-venda, peças, conformidade)
CAMADA 2 – FORÇAS (tendências + regulação)
CAMADA 1 – SEGMENTOS VERTICAIS (demanda por vertical)

Método de leitura por camada:

Camada	Pergunta central	Saída
1 — Segmentos	Onde está a demanda e qual sua maturidade?	Matriz de segmentos (Seção 11)
2 — Forças	O que move o mercado nos próximos 12–36 meses?	Tendências + quadro regulatório (Seções 8, 9)
3 — Dores	Que problema recorrente ninguém resolve bem?	Casos de uso (Seção 10)
4 — Competição	Quem já atua e onde está o espaço em branco?	Mapa competitivo (Seção 6)
5 — Síntese	O que o Kairós deve construir e por quê?	Oportunidades e checklist (Seções 10, 12)

Critério de qualificação de oportunidade (funil): uma oportunidade só é registrada se (a) resolve uma dor observada em ≥ 2 segmentos, **ou** (b) é exigida por regulação em ≥ 1 jurisdição prioritária, **ou** (c) explora espaço em branco competitivo defensável pelos módulos proprietários (KCI/KCD/KSI).

6. Diagramas

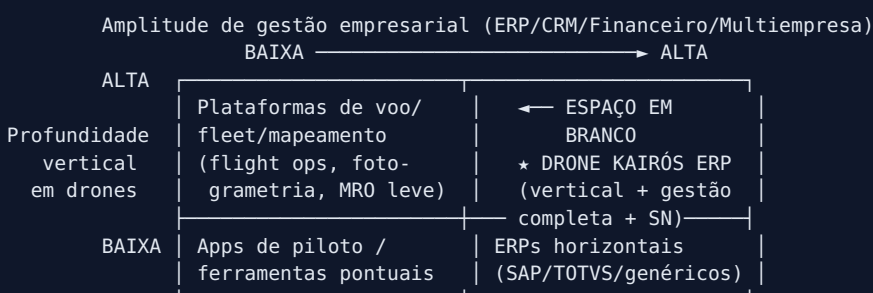
6.1 Cadeia de valor do ecossistema de drones (posicionamento do Kairós)

```

flowchart LR
    F[Fabricante] -->|produz + SN vitalício| R[Revenda / Distribuidor]
    R -->|vende + ativa garantia| C[Cliente / Operador]
    C -->|abre chamado / OS| T[Técnico / Assistência]
    T -->|peças + laudo| R
    F -->|homologação + firmware| C
    subgraph KAIROS[Drone Kairós ERP – eixo de verdade multiempresa]
        SN[(Serial Number rastreável vitalício)]
    end
    F -.registra.-> SN
    R -.movimenta.-> SN
    C -.opera.-> SN
    T -.mantém.-> SN

```

6.2 Mapa competitivo — espaço em branco



Leitura: o quadrante superior-direito — profundidade vertical e amplitude de gestão — está estruturalmente vago. Plataformas de software de drones não sobem para ERP; ERPs horizontais não descem para a especificidade do setor. É o território do Kairós.

6.3 Pilha funcional do setor (onde cada camada é atendida hoje)

Gestão empresarial: ERP · CRM · Financeiro · BI · Multiempresa	← lacuna vertical
Pós-venda: garantia · peças · OS · rastreabilidade por SN	← lacuna crítica
Operação de dados: mapeamento · inspeção · laudos	← bem atendido
Operação de voo: fleet · flight ops · UTM/U-space	← bem atendido
Hardware: drone · payload · SN · firmware	← commoditizando

7. Fluxogramas

7.1 Fluxo de decisão de compra do operador enterprise (jornada de demanda)

```

flowchart TD
    A[Necessidade operacional  
inspeção/agro/mapeamento] --> B{Comprar ou contratar  
DaaS?}
    B -->|Comprar| C[Seleção de hardware  
+ homologação regulatória]
    B -->|DaaS| D[Contrata prestador  
sem CAPEX]
    C --> E{Como gerir frota,  
garantia e conformidade?}
    E -->|Planilhas/ERP genérico| F[Dor: sem rastreabilidade  
sem pós-venda vertical]
    E -->|Plataforma vertical| G[Kairós: SN vitalício  
+ ERP + KSI + KCD]
    D --> H[Prestador precisa de  
gestão de frota + financeiro]
    H --> G
    F -- migração --> G
  
```

7.2 Ciclo de vida do ativo rastreado por SN (dor central que o Kairós resolve)

```
flowchart LR
    N[Nascimento do SN  
na fábrica] --> D[Distribuição  
revenda]
    D --> V[Venda + ativação  
de garantia]
    V --> O[Operação  
horas de voo / logs]
    O --> M{Manutenção}
    M -->|Preventiva/preditiva| MP[KSI aciona peça  
antes da falha]
    M -->|Corretiva| CO[OS + troca de peça  
+ laudo]
    MP --> O
    CO --> O
    O --> RC[Recall / boletim  
técnico por SN]
    RC --> O
    O --> EOL[Fim de vida /  
baixa / revenda usada]
    EOL -- histórico íntegro --> N
```

8. Boas Práticas (Tendências como práticas emergentes do setor)

As tendências estruturais do mercado, lidas como práticas que o software de gestão do setor precisa suportar:

8.1 BVLOS (Beyond Visual Line of Sight)

Voo além da linha visual é o divisor de águas econômico: transforma operações de nicho (uma aeronave, um piloto, um campo) em **operações escaláveis** (uma equipe, muitas aeronaves, área extensa). A liberação progressiva de BVLOS nas três jurisdições é a alavanca de crescimento mais importante do setor.

→ **Implicação para o Kairós:** BVLOS multiplica horas de voo por ativo, o que amplia a demanda por manutenção preditiva (KSI) e por registro rigoroso de conformidade e logs por SN (KCD/ERP).

8.2 Autonomia e operação one-to-many

Docas automáticas ("drone-in-a-box"), decolagem/pouso autônomos e um operador supervisionando múltiplas aeronaves. Reduz custo por voo e desloca o gargalo do piloto para a **gestão da frota e dos dados**.

→ **Implicação:** o eixo de valor migra para software de gestão — exatamente a camada do Kairós.

8.3 U-space / UTM (Unmanned Traffic Management)

Sistemas de gestão de tráfego não tripulado (U-space na Europa, UTM nos EUA, iniciativas em construção no Brasil) que orquestram identificação remota, geofencing dinâmico e separação de tráfego.

→ **Implicação:** o ERP não gerencia tráfego, mas deve **integrar-se** a provedores UTM/U-space e manter registros de conformidade (Remote ID, autorizações) vinculados ao SN e à empresa.

8.4 Drone-as-a-Service (DaaS)

Cliente compra o resultado (o dado, a inspeção, a entrega), não a aeronave. Cresce forte porque elimina CAPEX e complexidade regulatória do lado do cliente.

→ **Implicação:** o Kairós deve suportar o **modelo de prestador de serviço** (contratos, SLA, faturamento recorrente, alocação de frota por projeto) além do modelo de proprietário de ativo — multiempresa e financeiro tornam-se diferenciais.

8.5 Manutenção preditiva

Uso de dados de voo (horas, ciclos, telemetria) para antecipar falhas e programar troca de componentes antes da quebra. Reduz downtime e é crítico em BVLOS/autonomia.

→ **Implicação:** núcleo do **KSI** (estoque inteligente) somado ao histórico por SN. Diferencial competitivo direto.

8.6 Convergência de dados e edge/offline

Operações em campo com conectividade intermitente exigem processar e sincronizar dados no limite da rede.

→ **Implicação:** o **KCI offline-first** é resposta direta a essa realidade operacional; a IA externa é opcional e desacoplada.

9. Padrões (Quadro Regulatório-Chave)

Regulação é, no setor de drones, um **padrão de facto** que o software precisa incorporar. Síntese qualitativa das três jurisdições prioritárias.

9.1 Comparativo das jurisdições

Dimensão	Brasil (ANAC + DECEA + ANATEL)	EUA (FAA)	Europa (EASA + autoridades nacionais)
Estrutura	Regulação por classes/risco; registro de aeronave; coordenação de espaço aéreo com DECEA	Regras por peso e finalidade; regime específico para operação comercial	Modelo baseado em risco por categorias (aberta/específica/certificada)
Registro do ativo	Cadastro obrigatório da aeronave (identificação)	Registro obrigatório da aeronave	Registro do operador + identificação da aeronave
Identificação remota	Em evolução, alinhamento gradual a padrões globais	Remote ID exigido	Remote ID exigido nas categorias aplicáveis
BVLOS	Autorização caso a caso, com abertura progressiva	Migrando de waiver para regra mais estruturada	Enquadrado na categoria específica com avaliação de risco
Habilitação do operador	Requisitos por tipo de operação	Certificação de piloto remoto	Competência por categoria/subcategoria
Espectro/ telecom	Homologação de equipamento de radiofrequência	Conformidade de RF	Conformidade de RF e marcação de classe

Nota de conservadorismo: as regras evoluem rapidamente nas três regiões. A tabela captura **direção e natureza** dos requisitos, não o texto vigente de cada norma. Revalidar a cada ciclo (Seção 15).

9.2 O que a regulação exige de um ERP do setor

Padrões que o Kairós precisa suportar por design:

1. **Identidade persistente do ativo** — registro, número de série, identificação remota, associados ao SN vitalício.
2. **Trilha de conformidade por operação** — autorizações, categoria de risco, habilitação do operador, área de voo.
3. **Boletins e recalls por SN** — capacidade de notificar e rastrear ações obrigatórias do fabricante por aeronave.

4. **Segregação multijurisdicção** — a mesma plataforma operando empresas sob ANAC, FAA e EASA simultaneamente, com regras distintas por empresa/país.
5. **Retenção e auditabilidade de registros** — logs íntegros, versionados, exportáveis (reforço do KCD/Zero Trust).

→ **Implicação transversal:** conformidade não é um módulo isolado; é uma **propriedade multiempresa** do sistema. Cada empresa carrega seu perfil regulatório.

10. Casos de Uso (Dores do Setor traduzidas em oportunidade)

Cada caso parte de uma dor observada e termina na resposta do produto.

CU-01 — Pós-venda cego

Dor: fabricante e revenda perdem o rastro do ativo após a venda. Não sabem onde está, quanto voou, se foi mantido, se está em conformidade. Garantia vira disputa; NPS despenca. **Resposta Kairós:** SN vitalício + histórico de operação e manutenção + garantia ativável. O ativo nunca "some".

CU-02 — Rastreabilidade fragmentada

Dor: informação do ativo espalhada em planilhas, e-mails e sistemas desconexos entre fabricante, revenda e técnico. Nenhuma fonte única de verdade. **Resposta Kairós:** eixo de verdade multiempresa em torno do SN; cada elo (fabricante/revenda/técnico/cliente) enxerga sua fatia com permissões Zero Trust (KCD).

CU-03 — Gestão de peças reativa

Dor: peças críticas em falta quando a aeronave quebra; ou estoque parado de itens que não giram. Downtime caro, especialmente em operações BVLOS/autônomas. **Resposta Kairós:** KSI cruza horas de voo, ciclos e histórico por SN para prever demanda de peças e disparar reposição antes da falha (manutenção preditiva).

CU-04 — Conformidade manual e arriscada

Dor: conformidade regulatória gerida à mão, sem trilha auditável; risco de operar fora de norma e de não conseguir provar conformidade em fiscalização. **Resposta Kairós:** perfil regulatório por empresa (ANAC/FAA/EASA), trilha de conformidade por operação e registros auditáveis por design.

CU-05 — Prestador DaaS sem gestão de negócio

Dor: operador DaaS gerencia voo bem, mas fatura, contratos, SLA e alocação de frota por projeto no improviso. **Resposta Kairós:** ERP + CRM + Financeiro nativos, com frotalocável por projeto/cliente e faturamento recorrente.

CU-06 — Operação em campo sem sinal

Dor: técnico ou operador em zona rural/remota sem conectividade; dados de OS, laudo e leitura de SN presos até voltar à cobertura. **Resposta Kairós:** KCI offline-first — App Técnico e App Cliente operam offline e sincronizam depois, sem perda de integridade.

CU-07 — Recall/boletim técnico sem alcance

Dor: fabricante emite boletim de segurança e não consegue garantir que chegou a todas as unidades afetadas. **Resposta Kairós:** disparo de boletim/recall por faixa de SN, com confirmação de leitura e status de execução por aeronave.

Matriz dor × módulo

Caso	Dor	Módulo/App primário	Princípio reforçado
CU-01	Pós-venda cego	ERP + SN vitalício	Rastreabilidade
CU-02	Rastreabilidade fragmentada	Multiempresa + KCD	Multiempresa 1ª classe
CU-03	Peças reativas	KSI	Segurança/continuidade operacional
CU-04	Conformidade manual	ERP + KCD	Segurança por padrão
CU-05	DaaS sem gestão	ERP/CRM/Financeiro	Multiempresa
CU-06	Campo sem sinal	KCI (offline-first)	Arquitetura antes de código
CU-07	Recall sem alcance	ERP + App Cliente/Técnico	Rastreabilidade

11. Modelagem (Segmentação de Mercado)

11.1 Segmentos verticais — maturidade, drivers e demanda de gestão

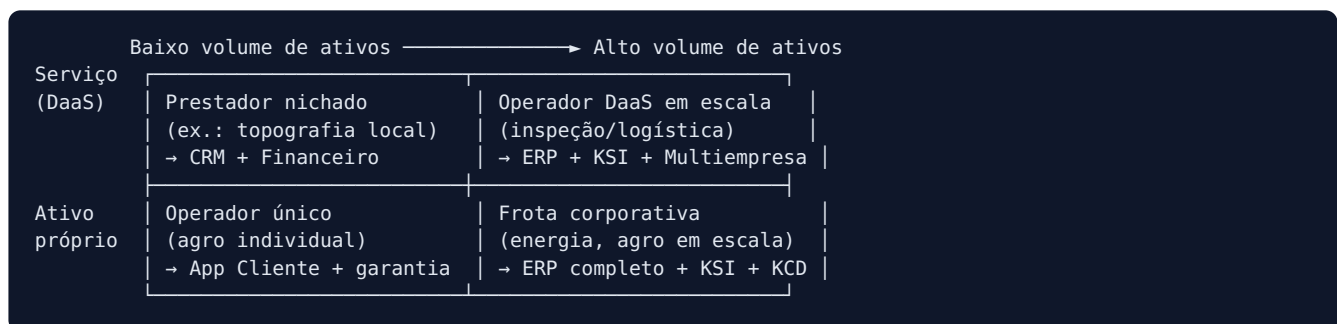
Vertical	Maturidade	Driver principal	Intensidade de pós-venda	Demanda por ERP vertical
Agro	Alta / crescente	Pulverização, sensoramento, produtividade	Alta (uso intenso, peças)	Muito alta
Inspeção de infraestrutura (energia, óleo & gás, telecom, ferrovia)	Alta	Segurança, custo vs. inspeção tripulada, BVLOS	Alta (operação contínua)	Muito alta
Mapeamento / topografia	Alta	Precisão, velocidade, LiDAR/ fotogrametria	Média	Alta
Logística / delivery	Emergente / crescente	Última milha, autonomia, docas	Muito alta (frota + uptime)	Muito alta
Segurança / defesa (dual-use)	Alta	Vigilância, resposta, fronteiras	Alta (disponibilidade crítica)	Alta (com KCD)
Mídia / audiovisual	Madura	Custo de produção, criatividade	Baixa/Média	Média

Leitura de priorização (hipótese analítica): agro, inspeção de infraestrutura e logística/delivery concentram a maior interseção entre volume de ativos, intensidade de pós-venda e necessidade de conformidade — logo, a maior demanda por um ERP vertical. Mapeamento e segurança são fortes secundários; mídia é oportunista.

11.2 Segmentação por papel na cadeia (persona × aplicação Kairós)

Papel	Dor dominante	Aplicação Kairós	Valor entregue
Fabricante	Perde rastro pós-venda; recall difícil	Painel Fabricante (web)	Visibilidade vitalícia por SN; boletins dirigidos
Revenda / distribuidor	Estoque, garantia, relacionamento	Painel Revenda (web) + KSI	Giro de estoque; garantia rastreável; CRM
Técnico / assistência	OS em campo, peças, laudo offline	App Técnico (Flutter) + KCI	Produtividade offline; histórico por SN
Cliente / operador	Frota, conformidade, custo	App Cliente (Flutter) + ERP	Controle de frota, conformidade, financeiro
Administrador da plataforma	Governança multiempresa	Portal Administrativo (web) + KCD	Zero Trust, isolamento por empresa, auditoria

11.3 Segmentação por modelo de negócio do operador



O Kairós, por ser multiempresa de 1ª classe, cobre os quatro quadrantes **na mesma instância**, adaptando o conjunto de módulos ativos por perfil de empresa.

11.4 Players relevantes (referência de mercado, sem endosso)

Fabricantes de hardware (referência de cadeia, não parceria pressuposta): atores globais consolidados em multirrotor comercial (ex.: DJI, Autel), players ocidentais com forte tração enterprise e autonomia (ex.: Skydio), e fabricantes históricos de asa fixa/consumo com braço profissional (ex.: Parrot), além de fabricantes especializados por vertical (agro, VTOL de longo alcance, docas autônomas).

Plataformas de software (categorias, não concorrentes diretos do ERP): fleet/flight management, mapeamento e fotogrametria, gestão de missão e análise de dados de inspeção, e provedores de UTM/U-space. **Nenhuma delas ocupa o quadrante de gestão empresarial vertical completa** (Seção 6.2) — é o espaço do Kairós.

Regra de canon: a menção a marcas serve para situar a cadeia de valor. O Kairós é **agnóstico de fabricante** e integra-se por SN, não depende de um OEM específico.

12. Checklist

Checklist de cobertura da pesquisa e de prontidão para uso pelas equipes de produto/arquitetura.

Cobertura analítica - [x] Segmentos verticais mapeados com maturidade e demanda de gestão - [x] Tendências estruturais (BVLOS, autonomia, UTM/U-space, DaaS, preditiva, offline) cobertas - [x] Quadro regulatório das três jurisdições prioritárias consolidado (qualitativo) - [x] Dores do setor traduzidas em casos de uso - [x] Cenário competitivo e espaço em branco identificados - [x] Oportunidades vinculadas a módulos/aplicações do Kairós

Aderência ao canon - [x] Sem contradição com Project Bible / Constituição - [x] Sem estatística de precisão falsa (uso de faixas/direção) - [x] Sem citação de modelo de IA por nome comercial - [x] Multiempresa e segurança aplicados como lentes transversais - [x] Rastreabilidade por SN tratada como eixo central

Prontidão de uso (handoff) - [] Revisado pela liderança de produto - [] Requisitos derivados incorporados ao backlog (Doc. Roadmap) - [] Perfis regulatórios por empresa refletidos na modelagem de dados - [] Data de próxima revalidação agendada (Seção 15)

13. Riscos

ID	Risco	Prob.	Impacto	Mitigação
R-01	Volatilidade regulatória invalida premissas (BVLOS, Remote ID mudam)	Alta	Alto	Perfil regulatório configurável por empresa; revalidação por ciclo; não hard-codar regras
R-02	Commoditização de hardware reduz margem e desloca poder para plataformas — favorável, mas exige velocidade	Média	Médio	Focar valor em software/gestão (tese central); acelerar módulos proprietários
R-03	Dependência de um OEM dominante distorce integrações	Média	Alto	Agnosticismo de fabricante por design; integração por SN; abstração de fornecedor
R-04	Entrada de player horizontal (grande ERP) na vertical	Baixa/Média	Alto	Defensabilidade via KCI/KCD/KSI e rastreabilidade vitalícia; profundidade vertical difícil de replicar
R-05	Plataforma de software de drones sobe para gestão	Média	Alto	Vantagem de ERP/Financeiro/Multiempresa nativos; velocidade e foco em pós-venda
R-06	Superestimar maturidade de logística/delivery (regulação ainda restringe)	Média	Médio	Priorizar agro e inspeção no curto prazo; delivery como aposta de médio prazo
R-07	Conectividade em campo pior que o previsto quebra UX	Média	Alto	KCI offline-first como requisito não-negociável, não como feature
R-08	Fragmentação de dados de terceiros (UTM, fabricantes) dificulta integração	Alta	Médio	Camada de integração desacoplada; contratos de dados versionados
R-09	Falsa precisão em projeções compromete credibilidade do documento	Baixa	Médio	Regra de conservadorismo numérico (Seção 4)
R-10	Requisitos de segurança/soberania de dados por jurisdição	Média	Alto	KCD Zero Trust; isolamento multiempresa; residência de dados configurável

14. Melhorias Futuras

Evoluções recomendadas para as próximas versões deste documento e para a inteligência de mercado do projeto:

- Instrumentar dados primários.** Substituir gradualmente faixas qualitativas por dados observados do próprio Kairós (uma vez em operação: horas de voo por SN, MTBF de peças, tempo de OS), fechando o ciclo pesquisa → produto → pesquisa.
- Observatório regulatório contínuo.** Rotina de monitoramento das mudanças em ANAC/DECEA, FAA e EASA, com alerta quando uma premissa da Seção 9 muda — alimentando o perfil regulatório configurável.
- Aprofundar verticais prioritárias.** Documentos-satélite dedicados a agro, inspeção e logística, com jornadas e requisitos específicos por vertical.
- Mapa de integrações.** Catálogo de provedores UTM/U-space, fabricantes e plataformas de dados com prioridade de integração e esforço estimado.

5. **Benchmark competitivo vivo.** Ficha por concorrente/categoria atualizada por ciclo, medindo avanço rumo ao quadrante de gestão vertical.
6. **Modelagem de expansão geográfica.** Além de BR/EUA/UE, avaliar jurisdições de crescimento (ex.: Oriente Médio, Ásia-Pacífico) sob a lente multiempresa/multijurisdição.
7. **Cenários de DaaS.** Modelar o impacto do avanço de DaaS sobre o mix de clientes do Kairós (proprietários vs. prestadores).

15. Auditoria

Controle de versão

Versão	Data	Autor	Mudança
1.0	21/07/2026	Analista de Mercado	Criação do documento — panorama mundial completo

Rastreabilidade de dependências

Documento	Papel nesta análise
00	Índice / estrutura documental
01 — Constituição do Projeto	Princípios inegociáveis (lentes de análise)
02 — Project Bible	Fonte de verdade sobre produto, módulos e escopo
03 — Roadmap	Destino dos requisitos derivados desta pesquisa

Classificação das afirmações (amostra de rastreabilidade)

Afirmação	Classe
Valor do setor migra de hardware para serviços/dados	Tendência observada
Quadrante de gestão vertical completa está vago	Hipótese analítica
BVLOS é a principal alavanca de crescimento	Tendência observada
Regras de ANAC/FAA/EASA seguem modelo baseado em risco	Fato consolidado (direção)
Agro/inspeção/logística lideram demanda por ERP vertical	Hipótese analítica

CrITÉRIOS de revalidação obrigatória - Mudança material em BVLOS ou Remote ID em qualquer jurisdição prioritária. - Entrada relevante de concorrente no quadrante de gestão vertical. - A cada **dois ciclos de release** do produto, no mínimo.

Próxima revisão recomendada: ao término do próximo ciclo de planejamento de produto ou mediante gatilho regulatório, o que ocorrer primeiro.

Assinatura de responsabilidade: Analista de Mercado — documento produzido sob as regras da Seção 4 e em conformidade com o canon do projeto (Docs. 01 e 02).

Fim do Documento 04 — Pesquisa Mundial · Drone Kairós ERP · v1.0

05 Benchmark

Documento: 05 – Benchmark **Versão:** 1.0 · **Data:** 21 de julho de 2026 · **Status:** Ativo **Dependências:** 00, 01, 02, 03, 04 **Responsável:** Analista de Produto

1. Resumo Executivo

Este documento estabelece o **benchmark competitivo** do Drone Kairós ERP contra três categorias de soluções que, individualmente, tocam partes do problema que o Kairós resolve de forma integrada:

- **(a) ERPs genéricos/enterprise** — SAP S/4HANA, Oracle NetSuite, TOTVS Protheus, Microsoft Dynamics 365. Fortes em finanças, fiscal e multiempresa; agnósticos de vertical e sem rastreabilidade de campo por Serial Number (SN).
- **(b) Field Service / pós-venda e manutenção** — ServiceMax, Fieldwire, IFS FSM, Salesforce Field Service. Fortes em ordens de serviço e força de campo; fracos em ERP financeiro completo e sem inteligência específica de drones.
- **(c) Gestão de frota/operação de drones** — DJI FlightHub 2, DroneDeploy, Auterion, Skydio Cloud, Airdata. Fortes em telemetria, missões e voo; não são ERPs, não gerenciam a cadeia comercial (fabricante → revenda → técnico → cliente) e não tratam ciclo de vida de peças com estoque inteligente.

Conclusão central: nenhuma solução do mercado cobre, num único produto, a combinação **vertical de drones + rastreabilidade vitalícia por SN + ERP/CRM/Financeiro/BI + apps de campo + API pública + os três módulos proprietários KCI (inteligência offline-first), KCD (cibersegurança Zero Trust) e KSI (estoque inteligente)**. O Kairós ocupa um espaço em branco (*white space*) na interseção das três categorias.

Nota metodológica: a avaliação usa capacidades publicamente descritas de cada categoria/produto no momento da redação. Onde uma capacidade depende de configuração, parceiro ou módulo pago, isso é sinalizado como **Parcial**. Não são atribuídos dados numéricos proprietários de terceiros.

2. Objetivos

1. Mapear o cenário competitivo real em que o Kairós será lançado e comparado.
2. Definir **critérios objetivos** de avaliação de funcionalidades relevantes ao ecossistema de drones.
3. Construir uma **matriz comparativa** rastreável e defensável.
4. Identificar **lacunas de mercado** exploráveis como diferencial.
5. Formalizar o **posicionamento diferenciado** do Kairós para uso em pitch, vendas e estratégia de produto.

3. Escopo

Dentro do escopo: - Comparação por capacidades funcionais e arquiteturas das 3 categorias. - Matriz de 12 critérios ponderados. - Análise de lacunas e posicionamento.

Fora do escopo: - Comparação de preço/licenciamento comercial (tratado no Doc 04 — Pesquisa de Mercado e no plano de negócio). - Benchmark de performance técnica (latência, throughput) — pertence aos documentos de arquitetura e testes. - Avaliação jurídica de conformidade de concorrentes.

4. Regras (Critérios de Avaliação)

Cada solução é avaliada em **12 critérios**. A escala de atendimento é padronizada:

Símbolo	Nível	Significado
●	Nativo	Recurso central, pronto para uso, coberto pelo produto-núcleo
◐	Parcial	Existe via módulo pago, parceiro, customização ou de forma limitada
○	Ausente	Não faz parte da proposta do produto

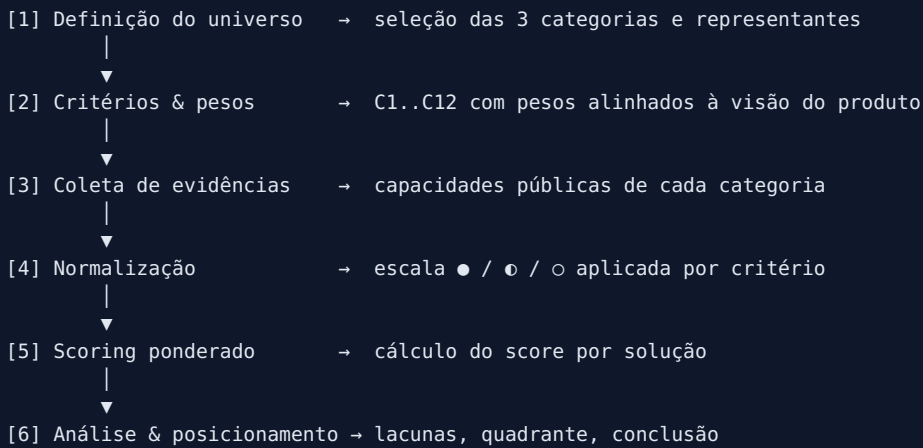
Critérios (C1–C12):

ID	Critério	Definição operacional	Peso
C1	Multiempresa (multi-tenant)	Isolamento e operação simultânea de fabricante, revendas e clientes	3
C2	Rastreabilidade vitalícia por SN	Histórico completo do ativo do "nascimento" ao descarte, por número de série	5
C3	Gestão de peças / estoque inteligente	Controle de peças com previsão de demanda e sugestão automática (KSI)	4
C4	Diagnóstico assistido por IA	Apoio à triagem/diagnóstico técnico com inteligência (KCI)	4
C5	Cibersegurança embarcada	Modelo Zero Trust integrado ao produto (KCD), não só perímetro de TI	5
C6	Apps de campo	Aplicativos para técnico e cliente (Flutter), com uso offline	3
C7	API pública	Integração aberta e documentada para terceiros	3
C8	BI / Analytics	Painéis e indicadores de decisão embarcados	2
C9	ERP Financeiro/Fiscal	Contas a pagar/receber, fiscal, contábil	3
C10	CRM / Pós-venda	Funil comercial, ordens de serviço, garantia	3
C11	Vertical de drones	Aderência nativa ao domínio (voo, componentes, ciclo de vida do drone)	5
C12	Offline-first	Operação plena sem conectividade contínua no campo	4

O **score ponderado** de cada solução é a soma dos pesos, atribuindo ● = 1,0 , ◐ = 0,5 , ○ = 0,0 × peso do critério. Score máximo teórico = soma dos pesos = **44**.

5. Arquitetura (Metodologia de Benchmark)

O benchmark segue um pipeline de 6 estágios, garantindo repetibilidade em versões futuras:



Princípios: - **Conservadorismo:** na dúvida entre ● e ◐, atribui-se ◐; entre ◐ e ○, atribui-se ○. Evita superestimar o Kairós. - **Rastreabilidade:** cada célula deve ser justificável pela proposta pública da categoria. - **Neutralidade de vertical:** ERPs e FSM são penalizados apenas onde a ausência é estrutural, não conjuntural.

6. Diagramas

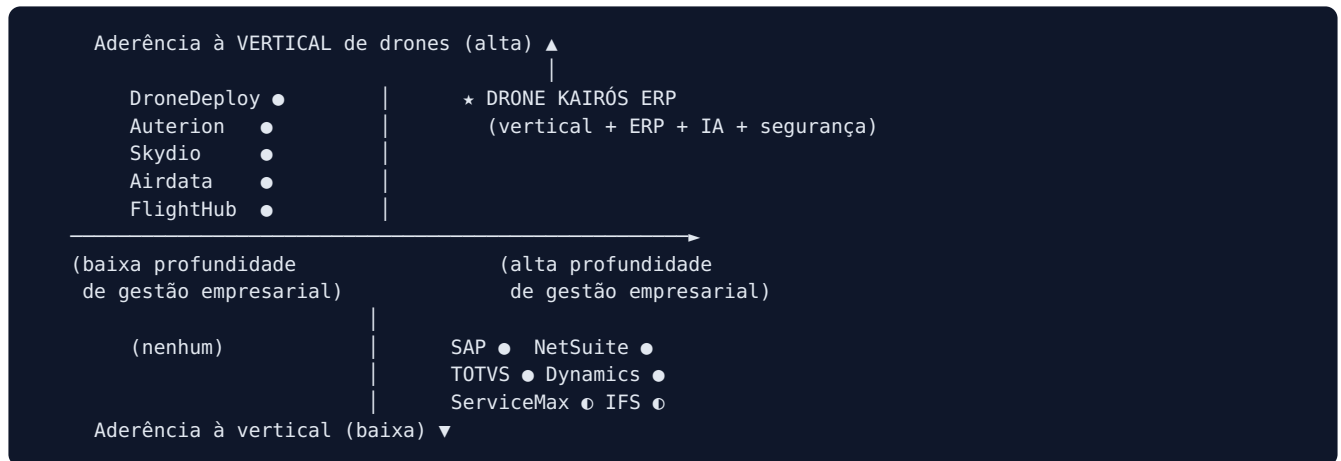
6.1 Posicionamento nas três categorias (interseção)

```

graph TD
    subgraph ERP["ERP Enterprise (SAP, NetSuite, TOTVS, Dynamics)"]
        E1[Financeiro/Fiscal]
        E2[Multiempresa]
    end
    subgraph FSM["Field Service (ServiceMax, IFS, Salesforce FS)"]
        F1[Ordens de Serviço]
        F2[Força de campo]
    end
    subgraph DRONE["Frota de Drones (FlightHub, DroneDeploy, Auterion, Skydio, Airdata)"]
        D1[Telemetria/Voo]
        D2[Missões/Mapas]
    end
    K((Drone Kairós ERP<br/>KCI · KCD · KSI<br/>SN vitalício))
    E1 --> K
    E2 --> K
    F1 --> K
    F2 --> K
    D1 --> K
    D2 --> K
    K -. "cobre a interseção<br/>que ninguém cobre" .-> K

```

6.2 Quadrante de posicionamento



O Kairós é o único ocupante do quadrante **superior-direito**: alta aderência à vertical e alta profundidade de gestão empresarial.

7. Fluxogramas

7.1 Fluxo de decisão do avaliador (como classificar uma célula)

```
flowchart TD
    A[Critério Cx para Solução Y] --> B{Recurso é central<br/>e pronto para uso?}
    B -- Sim --> C[● Nativo]
    B -- Não --> D{Existe via módulo pago,<br/>parceiro ou customização?}
    D -- Sim --> E[○ Parcial]
    D -- Não --> F[○ Ausente]
    C --> G[Registrar evidência]
    E --> G
    F --> G
    G --> H[Aplicar peso e somar ao score]
```

7.2 Jornada de valor comparada (ciclo de vida de um drone)

```
flowchart LR
    N[Fabricação<br/>+ SN gerado] --> V[Venda revenda]
    V --> C[Cliente ativa]
    C --> U[Uso/voo]
    U --> M[Manutenção]
    M --> P[Peça/estoque]
    P --> D[Descarte/recompra]
    classDef k fill:#0b3,stroke:#063,color:#fff
    N:::k
    D:::k
```

Leitura: ERPs cobrem bem "Venda" e "Financeiro"; FSM cobre "Manutenção"; plataformas de drone cobrem "Uso/voo". Somente o Kairós encadeia **todas** as etapas sob o mesmo SN, do "nascimento" ao descarte (rastreabilidade vitalícia).

8. Boas Práticas

- **Comparar por capacidade, não por marketing:** avaliar o que o produto faz nativamente, não o que a página de vendas promete.

- **Isolar a vertical:** um ERP genérico não deve ser "punido" por não ser de drones; deve ser avaliado pelo custo/ esforço de torná-lo vertical (customização = ◄, não ●).
- **Evidência antes de score:** nenhuma célula recebe ● sem justificativa funcional.
- **Revisão periódica:** re-executar o benchmark a cada release maior de concorrentes ou do Kairós (rastreado no Project Bible).
- **Separar núcleo de ecossistema:** capacidades entregues apenas por parceiros/integrações contam como ◄.

9. Padrões

- **Escala de avaliação:** ● / ◄ / ○ (Seção 4), imutável entre versões para permitir comparação temporal.
- **Nomenclatura de módulos proprietários:** KCI (inteligência offline-first), KCD (cibersegurança Zero Trust), KSI (estoque inteligente) — nomes fixos conforme Canon do projeto (Docs 01/02).
- **Formato de matriz:** linhas = soluções agrupadas por categoria; colunas = C1..C12; última coluna = score ponderado.
- **Padrão de referência de IA:** funcionalidades de inteligência são descritas por capacidade ("diagnóstico assistido", "previsão de demanda"), **sem citar modelos comerciais de IA por nome.**
- **Versionamento:** SemVer do documento; mudanças de critério exigem incremento maior (ex.: 1.0 → 2.0).

10. Casos de Uso

UC-01 — Comprador enterprise (revenda de médio porte) Precisa gerir estoque de peças, ordens de serviço e finanças e rastrear cada drone vendido. Com ERP genérico + FSM + planilha de SN, monta um "Frankenstein" de 3 sistemas. Com Kairós, resolve em uma plataforma. → *Argumento de consolidação.*

UC-02 — Fabricante que quer garantia rastreável Quer saber o histórico completo de cada unidade produzida (peças trocadas, falhas, técnico responsável). ERPs não têm SN de campo vitalício; plataformas de drone não têm cadeia comercial. → *Kairós é o único com C2 + C11 nativos.*

UC-03 — Técnico em campo sem conexão Faz diagnóstico numa fazenda sem sinal. FSMs em nuvem degradam offline; apps de voo não fazem OS. → *KCI offline-first (C12 + C4) é diferencial exclusivo.*

UC-04 — Cliente final acompanhando o ativo Quer ver garantia, histórico e status via app. Poucos concorrentes oferecem app de cliente ligado ao SN. → *App Flutter cliente + rastreabilidade.*

UC-05 — CISO avaliando risco Exige segurança embarcada Zero Trust, não apenas TLS de perímetro. → *KCD como recurso de produto (C5), raro no mercado.*

11. Modelagem — Matriz Comparativa (obrigatória)

Legenda: ● Nativo (1,0) · ◄ Parcial (0,5) · ○ Ausente (0,0). Score máximo = 44.

11.1 Categoria (a) — ERPs genéricos/enterprise

Solução	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	Score
SAP S/4HANA	●	◄	◄	◄	◄	◄	●	●	●	●	○	○	26,5
Oracle NetSuite	●	◄	◄	○	◄	◄	●	●	●	●	○	○	23,5
TOTVS Protheus	●	◄	◄	○	○	◄	●	◄	●	●	○	○	20,0
MS Dynamics 365	●	◄	◄	◄	◄	◄	●	●	●	●	○	○	26,5

Padrão da categoria: fortes em C1/C7/C8/C9/C10; **estruturalmente ausentes** em C11 (vertical de drones) e C12 (offline-first); rastreabilidade por SN (C2) existe como lote/série genérico, não vitalício de campo → ◄.

11.2 Categoria (b) — Field Service / manutenção e pós-venda

Solução	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	Score
ServiceMax	🟢	🟢	🟢	🟢	🟢	🟡	🟡	🟢	🟠	🟡	🟠	🟢	21,5
IFS FSM	🟢	🟢	🟢	🟢	🟢	🟡	🟡	🟡	🟢	🟡	🟠	🟢	23,5
Salesforce Field Service	🟢	🟢	🟠	🟢	🟢	🟡	🟡	🟡	🟠	🟡	🟠	🟢	20,5
Fieldwire	🟠	🟠	🟢	🟠	🟠	🟡	🟡	🟢	🟠	🟢	🟠	🟢	11,0

Padrão da categoria: fortes em C6/C7/C10; medianos em C4/C5; **ausentes** em C9 (ERP financeiro completo) e C11 (vertical de drones). Rastreabilidade de ativo por SN é parcial e não vitalícia.

11.3 Categoria (c) — Gestão de frota/operação de drones

[illegible]

Padrão da categoria: fortes em C11/C6/C7; **ausentes** em C3 (estoque de peças), C9 (financeiro) e C10 (CRM/pós-venda). São plataformas de **operação de voo**, não de gestão empresarial da cadeia.

11.4 Drone Kairós ERP

[illegible]

A pontuação máxima do Kairós reflete a **proposta de design** (todos os critérios são requisitos de produto, conforme Docs 01–03), não uma medição de produto já entregue. A entrega efetiva é rastreada pelo Roadmap (Doc 03) e pelos Quality Gates.

11.5 Consolidado por categoria (média de score)

Categoria	Score médio	C2 (SN vitalício)	C11 (vertical)	C3+C4+C5 (KSI/KCI/KCD)
(a) ERP enterprise	~24,1	●	○	fraco
(b) Field Service	~19,1	●	○	médio
(c) Frota de drones	~20,3	●	●	fraco
Kairós	44,0	●	●	forte (nativo)

12. Checklist

Cobertura do benchmark - [x] As três categorias foram representadas por soluções reconhecidas. - [x] 12 critérios definidos com pesos e definição operacional. - [x] Escala ● / ◐ / ○ aplicada de forma conservadora. - [x] Matriz comparativa completa (11.1–11.5). - [x] Lacunas de mercado identificadas (Seção 13/14). - [x] Posicionamento diferenciado formalizado.

Qualidade / integridade - [x] Nenhum dado numérico proprietário de terceiro inventado. - [x] Nenhum modelo comercial de IA citado por nome. - [x] Nomes de módulos (KCI/KCD/KSI) conforme Canon. - [x] Pontuação do Kairós explicitada como proposta de design, não medição.

13. Riscos

ID	Risco	Impacto	Prob.	Mitigação
R1	Concorrente de drones adicionar módulo de gestão (subir no quadrante)	Alto	Média	Aprofundar KCI/KCD/KSI e rastreabilidade vitalícia como fosso técnico
R2	ERP enterprise lançar vertical de drones via parceiro	Alto	Baixa	Velocidade de nicho + offline-first, difíceis de replicar em suíte genérica
R3	Viés de otimismo na auto-avaliação do Kairós	Médio	Média	Score = proposta de design; validação real via Quality Gates (Doc 03)
R4	CrITÉrios/pesos desatualizarem frente ao mercado	Médio	Média	Revisão a cada release maior; versionar documento
R5	Comparação percebida como injusta por não citar preço	Baixo	Média	Escopo explícito (Seção 3); preço tratado no Doc 04
R6	Capacidades de concorrentes mudarem após redação	Médio	Alta	Datar evidências; re-executar pipeline da Seção 5

14. Melhorias Futuras

- Lacunas de mercado identificadas (oportunidade do Kairós):** - **L1 — SN vitalício de campo:** nenhuma categoria oferece rastreabilidade do "nascimento ao descarte" ligada à cadeia comercial. → *Fosso do Kairós*. - **L2 — Estoque inteligente de peças em contexto de drones (KSI):** ausente nas plataformas de voo, genérico nos ERPs. - **L3 — Diagnóstico assistido offline (KCI):** FSMs degradam sem conexão; plataformas de voo não fazem OS. - **L4 — Cibersegurança embarcada Zero Trust como recurso de produto (KCD):** tratada como TI de perímetro pelos concorrentes, não como capacidade do produto. - **L5 — Cadeia completa fabricante → revenda → técnico → cliente** num só multi-tenant: fragmentada hoje entre 2–3 sistemas.
- Evolução do próprio benchmark:** - Adicionar score de **TCO/integração** (nº de sistemas necessários para igualar o Kairós). - Introduzir eixo de **maturidade de dados/IA** por concorrente. - Incluir avaliação de **conformidade regulatória de voo** por região. - Automatizar coleta de evidências e versionar a matriz em formato de dados (CSV/JSON) além de Markdown.
- Posicionamento diferenciado (síntese para pitch):**

O Drone Kairós ERP é a única plataforma que une **a profundidade de gestão de um ERP enterprise, a força de campo de um field service e a aderência vertical de uma plataforma de drones** — amarradas por **rastreabilidade vitalícia por Serial Number** e potencializadas pelos módulos proprietários **KCI, KCD e KSI**. Onde o mercado exige 3 sistemas, o Kairós entrega 1.

15. Auditoria

Versão	Data	Autor	Alteração
1.0	21/07/2026	Analista de Produto	Criação do benchmark: 3 categorias, 12 critérios, matriz comparativa completa, lacunas e posicionamento

Fontes de verdade (dependências): - **00** — Índice / Canon do projeto - **01** — Constituição do Projeto (missão, princípios) - **02** — Project Bible (definições de KCI/KCD/KSI, arquitetura de alto nível) - **03** — Roadmap (estado de entrega dos critérios) - **04** — Pesquisa de Mercado (contexto de preço/demanda)

Critérios de aceite deste documento: - Matriz comparativa presente e completa. ✓ - Três categorias analisadas com representantes reconhecidos. ✓ - Lacunas e posicionamento explícitos. ✓ - Sem dados falsos ou modelos de IA nomeados comercialmente. ✓

Próxima revisão programada: a cada release maior do Kairós ou de concorrente relevante (rastreado no Project Bible).

Fim do Documento 05 — Benchmark · Drone Kairós ERP · v1.0

06 Requisitos

Documento: 06 – Requisitos **Versão:** 1.0 · **Data:** 21 de julho de 2026 · **Status:** Ativo **Dependências:** 00 – Master Prompt, 01 – Constituição, 02 – Project Bible, 03 – Roadmap, 04 – Pesquisa de Mercado, 05 – Benchmark de Concorrentes **Responsável:** Engenharia de Requisitos

1. Resumo Executivo

Este documento é a **especificação de requisitos** do Drone Kairós ERP — a ponte formal entre a visão (docs 00–02), o plano (doc 03) e a arquitetura (docs 07+). Ele traduz o propósito constitucional ("plataforma de gestão definitiva do ecossistema de drones, com rastreabilidade vitalícia por Serial Number") em **personas**, **requisitos funcionais (RF)**, **requisitos não-funcionais (RNF)**, **casos de uso**, **priorização MoSCoW**, **definição de MVP** e **matriz de rastreabilidade**.

Cada requisito recebe um **identificador estável** (**RF-NNN**, **RNF-NNN**, **UC-NNN**), uma **prioridade MoSCoW** e um **vínculo de rastreabilidade** para o módulo/documento que o realizará. O documento é a fonte para o backlog (etapa 016), para os critérios de aceite (etapa 013) e para a validação de requisitos (etapa 020). Ele obedece integralmente às cláusulas pétreas da Constituição e ao escopo consolidado do Project Bible; qualquer contradição segue o Procedimento de Conflito (Bible §4).

2. Objetivos

- **Definir personas** dos cinco perfis (Fabricante, Revenda, Técnico de Campo, Cliente Final, Administrador da Plataforma) com objetivos, dores e tarefas.
- **Catalogar os Requisitos Funcionais (RF)** por módulo/perfil, com ID, ator, prioridade e MVP.
- **Catalogar os Requisitos Não-Funcionais (RNF)** por atributo de qualidade (segurança, performance, disponibilidade, escalabilidade, multiempresa, offline-first, usabilidade, conformidade), com métrica verificável.
- **Especificar os casos de uso principais** ponta a ponta: fluxo de Ordem de Serviço, cadastro de equipamento com SN, baixa de estoque e diagnóstico KCI.
- **Priorizar** o escopo por MoSCoW e **delimitar o MVP**.
- **Rastrear** cada requisito ao módulo/contexto e ao documento-alvo do roadmap.

3. Escopo

3.1 Abrangência

Aplica-se a toda a plataforma: as cinco aplicações (App Cliente, App Técnico, Painel Fabricante, Painel Revenda, Portal Administrativo), os serviços transversais (ERP, CRM, Financeiro, BI, API pública, multiempresa, histórico vitalício por SN) e os três módulos proprietários (**KCI** — inteligência offline-first; **KCD** — cibersegurança Zero Trust; **KSI** — estoque inteligente).

3.2 Fora de escopo deste documento

- Decisões de estilo arquitetural (monólito modular × microsserviços) → Doc 07.
- Modelagem física de dados e estratégia de SN → Doc 08.
- Contratos de API detalhados (OpenAPI) → Doc 09.
- Detalhamento interno dos módulos → Docs 13 (KCI), 14 (KCD), 15 (KSI).
- Conformidade legal por país → Doc 23.

Este documento define **o quê** (requisitos e critérios de aceite), não **o como** (realização técnica).

3.3 Definição do MVP

O MVP entrega o **fluxo de valor mínimo end-to-end** que prova a proposta: rastrear um equipamento por SN, executar uma Ordem de Serviço com diagnóstico assistido e dar baixa de peça no estoque — tudo sob multiempresa e segurança por padrão.

Inclui (MVP): 1. Identidade, acesso e perfis (RBAC) + isolamento multiempresa. 2. Cadastro de equipamento com SN e histórico vitalício (linha do tempo de eventos). 3. Ordem de Serviço ponta a ponta (abertura → diagnóstico → execução → baixa de peça → encerramento). 4. KSI base: itens, saldo por depósito e movimentação de entrada/saída com baixa vinculada à OS. 5. KCI base offline-first: diagnóstico por sintomas + base de conhecimento local + sugestão (RAG local); IA externa **opcional e desabilitada por padrão**. 6. App do Técnico (execução de OS offline) e App do Cliente (consulta de histórico do SN). 7. Painéis Fabricante e Revenda (cadastro e acompanhamento) e Portal Administrativo (governança mínima + auditoria). 8. KCD base: autenticação forte, criptografia, trilha de auditoria imutável, isolamento por tenant.

Adiado (pós-MVP): motor preditivo/generativo do KCI, RFID/NFC no KSI, reposição automática e Curva ABC, BI avançado, CRM completo, Financeiro completo, webhooks públicos, SOC/threat detection avançado, internacionalização plena.

4. Regras (Regras de Negócio que Restringem Requisitos)

Regras de negócio ([RN](#)) são invariantes de domínio que atravessam múltiplos requisitos. Todo RF deve respeitá-las.

ID	Regra de Negócio	Origem
RN-01	O Serial Number é único, imutável e vitalício ; identifica o equipamento em todos os contextos e tenants.	Constituição Art. III
RN-02	Nenhum dado cruza a fronteira do tenant sem autorização explícita e auditada (isolamento multiempresa).	Constituição Art. IV / Bible §5
RN-03	Nada acontece sem registro : toda operação relevante gera evento de auditoria imutável.	Constituição Art. II.3
RN-04	O histórico do equipamento é append-only ; eventos não são apagados, apenas anexados/estornados com rastro.	Constituição Art. V
RN-05	Baixa de estoque exige lastro : toda saída referencia origem (OS, venda ou ajuste justificado).	KSI / Financeiro
RN-06	Saldo de estoque nunca fica negativo sem bloqueio ou aprovação explícita.	KSI
RN-07	IA externa é opcional, desligada por padrão e nunca envia dado sensível sem consentimento do tenant.	KCI / Art. II.2
RN-08	Acesso segue menor privilégio (Zero Trust): toda requisição é autenticada e autorizada por perfil e tenant.	KCD
RN-09	O cliente tem direito de acesso ao histórico completo do seu equipamento e à portabilidade dos seus dados.	Constituição Art. V
RN-10	Operações offline são reconciliadas na sincronização com resolução determinística de conflitos.	KCI offline-first
RN-11	Encerramento de OS exige diagnóstico registrado, peças baixadas e assinatura/aceite quando aplicável.	OS
RN-12	Dados pessoais seguem minimização e finalidade ; retenção e expurgo conforme conformidade (Doc 23).	LGPD/GDPR

5. Arquitetura da Especificação (Organização dos Requisitos)

5.1 Taxonomia e esquema de identificação

Os requisitos são organizados por **contexto de domínio / módulo**, alinhados ao Mapa de Contextos (Bible §10). O identificador é estável e nunca reutilizado.

- **RF-NNN** — Requisito Funcional (o que o sistema faz).
- **RNF-NNN** — Requisito Não-Funcional (atributo de qualidade, com métrica).
- **UC-NNN** — Caso de Uso (comportamento observável de ponta a ponta).
- **RN-NN** — Regra de Negócio (invariante de domínio).

Faixas de numeração dos RF (por contexto):

Faixa	Contexto / Módulo	Perfis primários
RF-001...019	Identidade, Acesso e Multiempresa (Tenancy)	Todos / Administrador
RF-020...039	Cadastro de Equipamento (SN) e Histórico Vitalício	Fabricante / Revenda / Cliente
RF-040...059	Ordens de Serviço (OS)	Técnico / Revenda
RF-060...079	KSI — Estoque Inteligente	Revenda / Fabricante
RF-080...099	KCI — Inteligência e Diagnóstico	Técnico
RF-100...109	KCD — Cibersegurança	Administrador
RF-110...119	CRM	Revenda
RF-120...129	Financeiro	Revenda / Fabricante
RF-130...139	BI e Indicadores	Fabricante / Revenda / Admin
RF-140...149	API Pública e Integrações	Integrador / Admin
RF-150...159	Notificações, Documentos e Apps	Cliente / Técnico

5.2 Catálogo de Requisitos Funcionais (RF)

Legenda de prioridade: **M** = Must · **S** = Should · **C** = Could · **W** = Won't (agora). MVP: dentro / fora.

5.2.1 Identidade, Acesso e Multiempresa

ID	Requisito	Ator	MoSCoW	MVP
RF-001	Autenticar usuário com credenciais e MFA opcional/obrigatório por política do tenant.	Todos	M	
RF-002	Gerenciar perfis e papéis (RBAC) por perfil de negócio (Fabricante, Revenda, Técnico, Cliente, Admin).	Administrador	M	
RF-003	Isolar dados por tenant em todas as consultas e operações (multiempresa 1ª classe).	Sistema	M	
RF-004	Permitir que um usuário pertença a múltiplos tenants com contexto ativo selecionável.	Todos	S	
RF-005	Cadastrar e administrar empresas (tenants) : dados, hierarquia fabricante → revenda, status.	Administrador	M	
RF-006	Convidar e provisionar usuários por tenant com atribuição de papéis.	Administrador / Revenda	M	
RF-007	Registrar consentimentos e termos (privacidade, IA externa) por usuário/tenant.	Todos	S	
RF-008	Recuperar acesso (redefinição de senha, revogação de sessão) com auditoria.	Todos	M	
RF-009	Definir políticas de sessão (expiração, dispositivos confiáveis) por tenant.	Administrador	C	

5.2.2 Cadastro de Equipamento (SN) e Histórico Vitalício

ID	Requisito	Ator	MoSCoW	MVP
RF-020	Cadastrar equipamento gerando/registando Serial Number único e imutável (RN-01).	Fabricante	M	
RF-021	Vincular ao SN: modelo, lote, ficha técnica, componentes e data de fabricação .	Fabricante	M	
RF-022	Registrar transferência de posse/custódia (fabricante → revenda → cliente) mantendo o histórico.	Fabricante / Revenda	M	
RF-023	Manter linha do tempo vitalícia (append-only) de eventos do equipamento (RN-04).	Sistema	M	
RF-024	Consultar o histórico completo por SN (fabricação, vendas, OS, peças, diagnósticos).	Cliente / Técnico	M	
RF-025	Gerar etiqueta/QR Code do SN para identificação física do equipamento.	Fabricante / Revenda	S	
RF-026	Registrar garantia (início, vigência, cobertura) vinculada ao SN.	Fabricante / Revenda	S	
RF-027	Buscar equipamento por SN, QR Code, cliente, modelo ou lote.	Todos	M	
RF-028	Registrar recall/campanha de campo por lote/modelo, notificando afetados.	Fabricante	C	
RF-029	Exportar o dossiê do equipamento (portabilidade — RN-09).	Cliente	S	

5.2.3 Ordens de Serviço (OS)

ID	Requisito	Ator	MoSCoW	MVP
RF-040	Abrir OS vinculada a um SN, com sintomas, prioridade e solicitante.	Revenda / Técnico / Cliente	M	
RF-041	Atribuir OS a um técnico e agendar atendimento.	Revenda	M	
RF-042	Executar OS em campo offline , registrando diagnóstico, ações e evidências (fotos).	Técnico	M	
RF-043	Consumir peças na OS com baixa vinculada ao estoque (RN-05).	Técnico	M	
RF-044	Encerrar OS exigindo diagnóstico, peças baixadas e aceite quando aplicável (RN-11).	Técnico	M	
RF-045	Manter estados da OS (aberta → atribuída → em execução → aguardando peça → concluída → cancelada).	Sistema	M	
RF-046	Sincronizar OS executada offline com resolução de conflitos (RN-10).	Sistema	M	
RF-047	Registrar checklist e procedimentos por tipo de serviço/modelo.	Técnico	S	
RF-048	Gerar laudo/relatório de OS (PDF) para cliente e revenda.	Sistema	S	
RF-049	Vincular OS a garantia e classificar como cobertura/cobrança.	Revenda	S	
RF-050	Registrar SLA e prazos da OS e alertar atrasos.	Revenda	C	

5.2.4 KSI — Estoque Inteligente

ID	Requisito	Ator	MoSCoW	MVP
RF-060	Cadastrar itens/peças com código, unidade, custo e vínculo a modelos compatíveis.	Revenda / Fabricante	M	
RF-061	Controlar saldo por depósito/localização por tenant.	Revenda	M	
RF-062	Registrar movimentações (entrada, saída, transferência, ajuste) com lastro (RN-05).	Revenda / Técnico	M	
RF-063	Dar baixa de peça vinculada à OS de forma atômica e auditável.	Técnico / Sistema	M	
RF-064	Impedir saldo negativo sem bloqueio/aprovação (RN-06).	Sistema	M	
RF-065	Identificar itens por código de barras / QR Code .	Revenda	S	
RF-066	Identificar itens por RFID / NFC .	Revenda	C	
RF-067	Definir ponto de reposição e gerar sugestão de compra.	Revenda	S	
RF-068	Reposição automática e classificação Curva ABC .	Sistema	C	
RF-069	Previsão de demanda e integração com Financeiro/Compras.	Sistema	W	
RF-070	Realizar inventário/contagem com apuração de divergências.	Revenda	S	

5.2.5 KCI — Inteligência e Diagnóstico

ID	Requisito	Ator	MoSCoW	MVP
RF-080	Iniciar diagnóstico assistido a partir de sintomas e do histórico do SN.	Técnico	M	
RF-081	Operar offline-first : base de conhecimento e diagnóstico funcionam sem rede (RN-10).	Técnico	M	
RF-082	Sistema especialista : sugerir causas prováveis e procedimentos por regras.	Técnico	M	
RF-083	RAG local : recuperar trechos da Knowledge Base local relevantes ao caso.	Técnico	S	
RF-084	OCR / visão computacional para ler placas, SN e identificar defeitos em fotos.	Técnico	S	
RF-085	Motor preditivo : estimar falhas/manutenção com base no histórico.	Sistema	C	
RF-086	Motor generativo com IA externa opcional , desligada por padrão e com consentimento (RN-07).	Técnico	C	
RF-087	Realimentar a Knowledge Base com desfechos de OS (aprendizado).	Sistema	S	
RF-088	Registrar no histórico do SN o diagnóstico e a recomendação aplicados.	Sistema	M	

5.2.6 KCD — Cibersegurança

ID	Requisito	Ator	MoSCoW	MVP
RF-100	Manter trilha de auditoria imutável de eventos sensíveis (RN-03).	Sistema	M	
RF-101	Aplicar autorização Zero Trust por perfil e tenant em toda requisição (RN-08).	Sistema	M	
RF-102	Criptografar dados em trânsito e em repouso.	Sistema	M	
RF-103	Consultar logs e eventos de segurança no Portal Administrativo.	Administrador	S	
RF-104	Deteção de ameaças e alertas (SOC).	Administrador	C	
RF-105	Gerir backups e recuperação (DR) .	Administrador	S	
RF-106	Gerir segredos e chaves com rotação.	Administrador	S	

5.2.7 CRM, Financeiro, BI

ID	Requisito	Ator	MoSCoW	MVP
RF-110	Manter cadastro de clientes e vínculo aos equipamentos (SN).	Revenda	M	
RF-111	Registrar interações e atendimentos por cliente.	Revenda	S	
RF-112	Gerir oportunidades/pós-venda (renovação de garantia, upsell).	Revenda	C	
RF-120	Emitir documento financeiro de OS/venda (cobrança).	Revenda	S	
RF-121	Conciliar custos de peças consumidas com o Financeiro.	Revenda	C	
RF-122	Consolidar faturamento por tenant/período .	Revenda / Admin	C	
RF-130	Apresentar dashboards de OS, estoque e equipamentos por tenant.	Fabricante / Revenda	S	
RF-131	Indicadores de confiabilidade por modelo/lote (taxa de falha, MTBF).	Fabricante	C	
RF-132	Exportar relatórios (CSV/PDF) respeitando o isolamento de tenant.	Todos	S	

5.2.8 API Pública, Notificações, Documentos e Apps

ID	Requisito	Ator	MoSCoW	MVP
RF-140	Expor API pública (consulta de SN, OS, estoque) autenticada e por tenant.	Integrador	S	
RF-141	Emitir webhooks de eventos (OS criada/encerrada, movimentação).	Integrador	C	
RF-142	Gerir chaves/escopos de API por tenant.	Administrador	S	
RF-150	Enviar notificações (OS, garantia, recall) por push/e-mail.	Sistema	S	
RF-151	Anexar e versionar documentos (fotos, laudos, notas) a SN/OS.	Técnico / Revenda	M	
RF-152	App do Cliente : consultar equipamentos, histórico, garantia e abrir solicitação.	Cliente	M	
RF-153	App do Técnico : fila de OS, execução offline, captura de evidências e sincronização.	Técnico	M	
RF-154	Painel do Fabricante : cadastro de SN/lotos, confiabilidade e campanhas.	Fabricante	M	
RF-155	Painel da Revenda : clientes, OS, estoque e pós-venda.	Revenda	M	
RF-156	Portal Administrativo : tenants, usuários, auditoria e governança.	Administrador	M	

5.3 Catálogo de Requisitos Não-Funcionais (RNF)

Todo RNF tem **métrica verificável** e **meta**. As metas são de referência (v1.0) e serão refinadas em Doc 20 (testes).

5.3.1 Segurança (KCD)

ID	Requisito	Métrica / Meta	MoSCoW	MVP
RNF-001	Toda comunicação externa por TLS 1.2+ ; dados em repouso criptografados (AES-256).	100% dos canais/volumes	M	
RNF-002	Autorização Zero Trust em toda requisição (perfil + tenant), sem confiança implícita.	0 endpoints sem autorização	M	
RNF-003	Trilha de auditoria imutável e à prova de adulteração para eventos sensíveis.	100% dos eventos críticos	M	
RNF-004	Senhas com hash forte ; segredos fora do código; rotação de chaves.	0 segredos em repositório	M	
RNF-005	MFA disponível a todos os perfis e imponível por política de tenant.	Cobertura 100% dos perfis	S	
RNF-006	Registro e resposta a incidentes com detecção de anomalias (SOC).	MTTR alvo definido em Doc 14	C	

5.3.2 Performance

ID	Requisito	Métrica / Meta	MoSCoW	MVP
RNF-010	Tempo de resposta de operações interativas de leitura.	p95 < 300 ms (API)	M	
RNF-011	Tempo de resposta de operações de escrita/transação.	p95 < 800 ms	S	
RNF-012	Consulta de histórico por SN .	p95 < 500 ms para 10k eventos	S	
RNF-013	Diagnóstico KCI local (offline).	Resposta < 2 s no dispositivo	M	
RNF-014	Sincronização de OS offline ao reconectar.	100 OS < 30 s	S	

5.3.3 Disponibilidade e Resiliência

ID	Requisito	Métrica / Meta	MoSCoW	MVP
RNF-020	Disponibilidade dos serviços críticos (ERP/OS/SN).	≥ 99,9% mensal	M	
RNF-021	Degradação graciosa: apps operam offline sem perda de dados (RN-10).	0 perda em queda de rede	M	
RNF-022	RPO/RTO para recuperação de desastre.	RPO ≤ 15 min · RTO ≤ 4 h	S	
RNF-023	Idempotência e reprocessamento seguro de sincronização.	0 duplicidade após retry	M	

5.3.4 Escalabilidade

ID	Requisito	Métrica / Meta	MoSCoW	MVP
RNF-030	Suportar crescimento de tenants/equipamentos sem redesenho.	10k tenants · 10M SN (alvo)	M	
RNF-031	Escala horizontal dos serviços de aplicação.	Sem estado em sessão de app	S	
RNF-032	Preparação para multi-região (latência e residência de dados).	Definido em Doc 24	C	

5.3.5 Multiempresa (Tenancy)

ID	Requisito	Métrica / Meta	MoSCoW	MVP
RNF-040	Isolamento lógico total entre tenants em dados e em processamento (RN-02).	0 vazamento cross-tenant	M	
RNF-041	Todo registro persistido carrega identificador de tenant e é filtrado por padrão.	100% das entidades	M	
RNF-042	Personalização por tenant (marca, políticas) sem fork de código .	Config por tenant	S	

5.3.6 Offline-first

ID	Requisito	Métrica / Meta	MoSCoW	MVP
RNF-050	Apps (Técnico/Cliente) executam funções críticas offline e reconciliam depois.	OS e diagnóstico 100% offline	M	
RNF-051	Resolução determinística de conflitos na sincronização (RN-10).	Regra documentada e testável	M	
RNF-052	Fila local durável de operações pendentes, resistente a fechamento do app.	0 perda em kill do app	M	

5.3.7 Usabilidade e Acessibilidade

ID	Requisito	Métrica / Meta	MoSCoW	MVP
RNF-060	Cada perfil tem experiência dedicada e coerente (Constituição Art. III).	5 perfis atendidos	M	
RNF-061	Fluxo de OS em campo executável com poucos toques e uma mão.	≤ 5 passos para baixa de peça	S	
RNF-062	Conformidade de acessibilidade (contraste, leitura de tela).	WCAG 2.1 AA (alvo)	S	
RNF-063	Suporte a internacionalização (idioma, moeda, formato).	i18n desde a base	C	

5.3.8 Conformidade, Observabilidade e Manutenibilidade

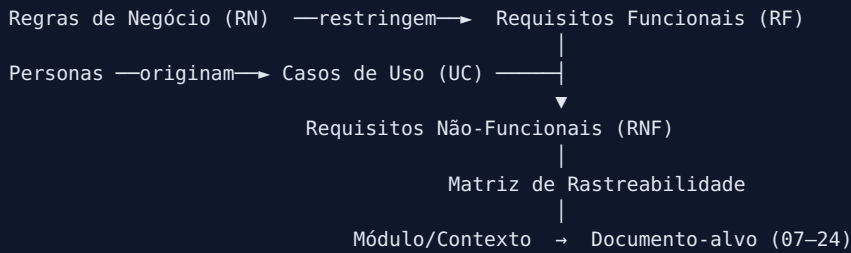
ID	Requisito	Métrica / Meta	MoSCoW	MVP
RNF-070	LGPD/GDPR: minimização, finalidade, consentimento, portabilidade e expurgo (RN-12).	Detalhado em Doc 23	M	
RNF-071	Consentimento explícito para uso de IA externa; nunca por padrão (RN-07).	Opt-in registrado	M	
RNF-072	Observabilidade: logs estruturados, métricas e tracing correlacionados.	Cobertura de serviços críticos	S	
RNF-073	Manutenibilidade: fronteiras DDD explícitas e contratos versionados.	Alinhado a Docs 07/09/10	M	
RNF-074	Portabilidade de dados do tenant/equipamento em formato aberto.	Export padronizado	S	

6. Diagramas

6.1 Perfis × Aplicações × Módulos

```
graph LR
    subgraph Perfis
        FAB[Fabricante]
        REV[Revenda]
        TEC[Técnico de Campo]
        CLI[Cliente Final]
        ADM[Administrador]
    end
    subgraph Apps
        PF[Painel Fabricante]
        PR[Painel Revenda]
        AT[App Técnico]
        AC[App Cliente]
        PA[Portal Admin]
    end
    subgraph Modulos
        SN[(Cadastro SN / Histórico)]
        OS[(Ordens de Serviço)]
        KSI[(KSI Estoque)]
        KCI[(KCI Inteligência)]
        KCD[(KCD Segurança)]
    end
    FAB --> PF --> SN
    REV --> PR --> OS
    REV --> PR --> KSI
    TEC --> AT --> OS
    TEC --> AT --> KCI
    CLI --> AC --> SN
    ADM --> PA --> KCD
    OS --> SN
    OS --> KSI
    KCI --> SN
    KCD -. protege .-> SN
    KCD -. protege .-> OS
```

6.2 Organização e rastreabilidade dos requisitos



7. Fluxogramas (Casos de Uso)

7.1 Fluxo de Ordem de Serviço (ponta a ponta)

```
flowchart TD
    A[Solicitação / sintoma no SN] --> B[Abrir OS - RF-040]
    B --> C[Atribuir a técnico e agendar - RF-041]
    C --> D[Técnico executa em campo OFFLINE - RF-042]
    D --> E[Diagnóstico KCI local - RF-080/081]
    E --> F{Precisa de peça?}
    F -- Sim --> G[Baixa de peça vinculada à OS - RF-043/063]
    G --> H{Saldo suficiente?}
    H -- Não --> I[Bloqueio/aprovação ou aguardando peça - RF-064/045]
    I --> J[Registrar ações e evidências]
    H -- Sim --> J
    F -- Não --> J
    J --> K[Encerrar OS: diagnóstico + aceite - RF-044]
    K --> L[Sincronizar e reconciliar - RF-046]
    L --> M[Anexar ao histórico do SN - RF-023/088]
    M --> N[Notificar cliente / gerar laudo - RF-150/048]
```

7.2 Cadastro de equipamento com SN

```
flowchart TD
    A[Fabricante inicia cadastro] --> B[Gerar/registrar SN único - RF-020 / RN-01]
    B --> C[Vincular modelo, lote, componentes - RF-021]
    C --> D[Emitir etiqueta/QR - RF-025]
    D --> E[Registrar garantia - RF-026]
    E --> F[Abrir linha do tempo vitalícia append-only - RF-023 / RN-04]
    F --> G[Transferir custódia fabricante→revenda→cliente - RF-022]
    G --> H[Disponível para consulta por SN - RF-024/027]
```

7.3 Baixa de estoque (vinculada à OS)

```
sequenceDiagram
    participant T as Técnico (App)
    participant OS as Serviço de OS
    participant KSI as Estoque (KSI)
    participant AUD as Auditoria (KCD)
    T->>OS: Consumir peça X na OS (RF-043)
    OS->>KSI: Solicitar baixa com lastro OS (RF-063 / RN-05)
    KSI->>KSI: Verificar saldo (RF-064 / RN-06)
    alt Saldo suficiente
        KSI->>OS: Baixa confirmada (atômica)
        OS->>AUD: Registrar evento imutável (RF-100 / RN-03)
        OS->>T: Peça baixada / atualizar OS
    else Saldo insuficiente
        KSI->>OS: Bloqueio
        OS->>T: Aguardando peça / aprovação (RF-045)
    end
    end
```

7.4 Diagnóstico KCI (offline-first)

```
flowchart TD
  A[Técnico abre diagnóstico na OS - RF-080] --> B[Coletar sintomas + histórico do SN - RF-024]
  B --> C[Sistema especialista: regras/causas prováveis - RF-082]
  C --> D[RAG local na Knowledge Base - RF-083]
  D --> E{IA externa habilitada e consentida?}
  E -- Não / padrão --> F[Recomendação local offline - RF-081 / RN-07]
  E -- Sim opt-in --> G[Enriquecer com motor generativo externo - RF-086]
  F --> H[Registrar diagnóstico no histórico do SN - RF-088]
  G --> H
  H --> I[Realimentar Knowledge Base ao encerrar OS - RF-087]
```

8. Boas Práticas (Engenharia de Requisitos)

- **Um requisito, uma ideia:** atômico, testável e sem ambiguidade ("deve", não "poderia").
- **Rastreável nos dois sentidos:** cada RF/RNF liga-se a uma origem (persona/regra) e a um destino (módulo/documento).
- **Critério de aceite explícito:** todo item Must tem condição de verificação (base do Doc 20).
- **Priorizar valor de negócio** com MoSCoW e revisar a cada onda do roadmap.
- **Não decidir arquitetura aqui:** requisitos descrevem comportamento e qualidade, não solução técnica.
- **Segurança e multiempresa como transversais:** presentes por padrão, não como itens opcionais.
- **Validar com stakeholders** antes de congelar (etapa 020) e registrar mudanças com versão.

9. Padrões (Padrão de Escrita de Requisitos)

9.1 Sintaxe padrão do RF

[ID] — Como [perfil/ator], o sistema **deve** [ação] [objeto] [condição/restrição], respeitando [RN aplicável].

Exemplo: RF-043 — Como Técnico, o sistema deve consumir peças na OS dando baixa atômica no estoque, respeitando RN-05.

9.2 Sintaxe padrão do RNF (mensurável)

[ID] — O sistema **deve** [atributo de qualidade] medido por [métrica] atingindo [meta] sob [contexto/carga].

9.3 Qualidade do requisito (INVEST + verificável)

Independente · Negociável · Valioso · Estimável · Pequeno · **Testável**. Todo requisito Must possui critério de aceite no formato **Dado / Quando / Então**.

9.4 Padrão de priorização — MoSCoW

Nível	Significado	Regra de uso
Must	Indispensável; sem ele não há produto viável.	Compõe o MVP.
Should	Importante, mas contornável no curto prazo.	Primeira onda pós-MVP.
Could	Desejável; entra se houver capacidade.	Backlog priorizado.
Won't (agora)	Fora do horizonte atual; reavaliar depois.	Registrado para futuro.

9.5 Ciclo de vida do requisito

proposto → aprovado → em construção → entregue → validado → obsoleto. IDs nunca são reutilizados; alterações incrementam a versão do documento e são anotadas na Auditoria.

10. Casos de Uso (Especificação)

UC-001 — Executar Ordem de Serviço

- **Ator principal:** Técnico de Campo · **Secundários:** Revenda, Cliente, Sistema.
- **Pré-condições:** OS atribuída; equipamento identificado por SN; app autenticado (pode estar offline).
- **Fluxo principal:** (1) Técnico abre a OS na fila; (2) confirma o SN; (3) inicia diagnóstico KCI local; (4) registra ações e evidências; (5) consome peças com baixa (UC-003); (6) encerra a OS com diagnóstico e aceite; (7) app enfileira sincronização.
- **Fluxos alternativos:** (A1) Sem peça em estoque → OS vai a "aguardando peça" (RF-045). (A2) Offline → operações ficam em fila local durável (RNF-052) e reconciliam ao reconectar (RF-046). (A3) Conflito na sincronização → resolução determinística (RNF-051).
- **Pós-condições:** OS concluída; histórico do SN atualizado (append-only); estoque debitado; evento auditado; cliente notificado.
- **Requisitos:** RF-040..046, RF-048, RF-063, RF-080, RF-100, RF-150; RNF-013, RNF-021, RNF-050..052.

UC-002 — Cadastrar equipamento com Serial Number

- **Ator principal:** Fabricante.
- **Pré-condições:** Usuário do tenant fabricante autenticado com permissão de cadastro.
- **Fluxo principal:** (1) informa modelo/lote/componentes; (2) sistema gera/registra SN único e imutável; (3) emite etiqueta/QR; (4) registra garantia; (5) abre a linha do tempo vitalícia.
- **Fluxos alternativos:** (A1) SN informado já existe → rejeição (RN-01). (A2) Lote em recall → marcação e notificação (RF-028).
- **Pós-condições:** Equipamento rastreável por SN; disponível para transferência de custódia.
- **Requisitos:** RF-020..027; RNF-040, RNF-041, RNF-070.

UC-003 — Dar baixa de peça no estoque

- **Ator principal:** Técnico (ou Revenda) · **Secundário:** Sistema/KSI, Auditoria.
- **Pré-condições:** Peça cadastrada; OS ou lastro válido; permissão no tenant.
- **Fluxo principal:** (1) seleciona peça e quantidade na OS; (2) KSI valida saldo; (3) baixa atômica com lastro; (4) evento auditado; (5) custo disponível para conciliação financeira.
- **Fluxos alternativos:** (A1) Saldo insuficiente → bloqueio/aprovação (RF-064/RN-06). (A2) Ajuste sem OS → exige justificativa (RN-05).
- **Pós-condições:** Saldo atualizado (nunca negativo sem aprovação); movimentação rastreável.
- **Requisitos:** RF-060..064; RNF-003, RNF-023, RNF-040.

UC-004 — Diagnóstico assistido pelo KCI

- **Ator principal:** Técnico.
- **Pré-condições:** OS aberta; base de conhecimento local presente no dispositivo.
- **Fluxo principal:** (1) coleta sintomas e histórico do SN; (2) sistema especialista sugere causas; (3) RAG local traz procedimentos; (4) técnico aplica recomendação; (5) diagnóstico é anexado ao histórico do SN.
- **Fluxos alternativos:** (A1) IA externa habilitada e consentida → enriquecimento generativo (RF-086/RN-07); caso contrário, tudo permanece local. (A2) Foto do defeito → OCR/visão computacional apoia a identificação (RF-084).
- **Pós-condições:** Recomendação registrada; Knowledge Base realimentada no encerramento.
- **Requisitos:** RF-080..088; RNF-013, RNF-050, RNF-071.

11. Modelagem — Personas e Matriz de Rastreabilidade

11.1 Personas

Persona 1 — Fabricante (Marina, Gerente de Produção/Qualidade)

- **Contexto:** indústria que produz drones e componentes; opera o Painel do Fabricante.
- **Objetivos:** garantir rastreabilidade de produção por SN; medir confiabilidade por modelo/lote; gerir garantias e campanhas de recall.
- **Dores:** falta de visibilidade do que acontece com o produto após a venda; recalls lentos e imprecisos; dados de falha dispersos entre revendas.
- **Tarefas-chave:** cadastrar SN/lotos (RF-020..021), acompanhar indicadores de falha (RF-131), abrir recall (RF-028), definir garantia (RF-026).
- **Requisitos associados:** RF-020..028, RF-130..132; RNF-030, RNF-040.

Persona 2 — Revenda (Rafael, Gerente de Loja/Assistência)

- **Contexto:** distribui, vende e faz pós-venda; opera o Painel da Revenda.
- **Objetivos:** vender e prestar assistência com eficiência; controlar estoque de peças; manter clientes satisfeitos e recorrentes.
- **Dores:** OS descontroladas e sem histórico; falta de peça no momento do serviço; retrabalho e prejuízo de garantia.
- **Tarefas-chave:** abrir/atribuir OS (RF-040/041), gerir estoque (RF-060..067), cadastro de clientes (RF-110), acompanhar pós-venda (RF-112).
- **Requisitos associados:** RF-040..050, RF-060..070, RF-110..122; RNF-011, RNF-040.

Persona 3 — Técnico de Campo (Tiago, Técnico de Manutenção)

- **Contexto:** atende em campo, muitas vezes **sem conectividade**; opera o App do Técnico.
- **Objetivos:** diagnosticar rápido e resolver na primeira visita; registrar tudo sem burocracia; trabalhar mesmo offline.
- **Dores:** sinal instável em campo; diagnósticos por tentativa e erro; retrabalho por falta de peça ou de histórico.
- **Tarefas-chave:** executar OS offline (RF-042/046), diagnóstico KCI (RF-080..088), baixa de peça (RF-043/063), evidências (RF-151).
- **Requisitos associados:** RF-040..048, RF-063, RF-080..088, RF-151..153; RNF-013, RNF-050..052, RNF-061.

Persona 4 — Cliente Final (Camila, Operadora de Drone)

- **Contexto:** possui/usa o equipamento; opera o App do Cliente.
- **Objetivos:** saber o estado e o histórico do seu equipamento; acionar suporte facilmente; ter garantia e transparência.
- **Dores:** não saber histórico de manutenções; abrir chamado é confuso; dúvida sobre garantia.
- **Tarefas-chave:** consultar histórico por SN (RF-024/152), verificar garantia (RF-026), abrir solicitação (RF-040), receber notificações (RF-150), exportar dossiê (RF-029).
- **Requisitos associados:** RF-024, RF-026, RF-029, RF-040, RF-150..152; RNF-060, RNF-070, RNF-074.

Persona 5 — Administrador da Plataforma (Alex, Admin/SecOps)

- **Contexto:** governa a plataforma multiempresa; opera o Portal Administrativo.
- **Objetivos:** garantir isolamento entre tenants, segurança e conformidade; auditar tudo; provisionar empresas e usuários.
- **Dores:** risco de vazamento cross-tenant; falta de trilha de auditoria; incidentes sem visibilidade.
- **Tarefas-chave:** gerir tenants/usuários (RF-005/006), auditar eventos (RF-103/100), políticas de segurança (RF-101..106), chaves de API (RF-142).
- **Requisitos associados:** RF-001..009, RF-100..106, RF-142, RF-156; RNF-001..006, RNF-040..042, RNF-070.

11.2 Matriz de Rastreabilidade (Requisito → Módulo/Contexto → Documento-alvo)

Requisitos	Módulo / Contexto (Bible §10)	Documento(s)-alvo	Etapa Roadmap
RF-001...009, RNF-002	Identidade & Acesso	Doc 07, Doc 14	036, 047
RF-003...005, RNF-040..042	Multiempresa / Tenancy	Doc 08	024, 037
RF-020...029, RN-01	Cadastro/Serial Number & Histórico	Doc 08	026, 038, 039
RF-040...050	Ordens de Serviço	Doc 07/10	040, 048
RF-060...070	Estoque (KSI)	Doc 15	072–075
RF-080...088	Inteligência (KCI)	Doc 13	061–067
RF-100...106, RNF-001..006	Segurança (KCD)	Doc 14	030, 053, 059, 068–071
RF-110...122	CRM / Financeiro	Docs 17/18	077
RF-130...132	BI	Doc 16	076
RF-140...142	API Pública	Doc 09	054, 055
RF-150...156	Notificações, Documentos e Apps	Docs 11/12	042–046, 051, 052
RNF-010...014, RNF-020..023	Performance / Disponibilidade	Doc 19, Doc 20	081, 085
RNF-030...032	Escalabilidade / Multi-região	Doc 24	098
RNF-050...052	Offline-first	Doc 12, Doc 13	045, 046
RNF-060...063	Usabilidade / Acessibilidade / i18n	Doc 11, Doc 23	033, 050, 096
RNF-070...074	Conformidade / Observabilidade / Manutenibilidade	Doc 19, Doc 23	019, 081, 097

Rastreabilidade reversa: cada documento-alvo, ao ser produzido, referencia de volta os RF/RNF que realiza, fechando o ciclo bidirecional (§8).

12. Checklist

- ☒ Cinco personas descritas (objetivos, dores, tarefas, requisitos).
- ☒ RF catalogados por módulo/perfil com ID, MoSCoW e MVP.
- ☒ RNF catalogados por atributo de qualidade com métrica e meta.
- ☒ Casos de uso principais especificados (OS, SN, baixa de estoque, diagnóstico KCI).
- ☒ Fluxogramas dos casos de uso (Mermaid/ASCII).
- ☒ Regras de negócio (RN) declaradas e vinculadas.
- ☒ Priorização MoSCoW aplicada.
- ☒ MVP delimitado (inclui/adia).
- ☒ Matriz de rastreabilidade requisito → módulo → documento.
- ☐ Validação formal com stakeholders (etapa 020) — pendente.

13. Riscos

Risco	Impacto	Mitigação
Escopo inflado (tudo vira Must)	MVP inviável, atraso	MoSCoW disciplinado; revisão por onda; corte defendido pelo valor.
Requisitos ambíguos/sem métrica	Retrabalho, disputa em aceite	Padrão §9 + critério Dado/Quando/Então em todo Must.
Vazamento cross-tenant	Quebra de confiança, legal	RNF-040/041 obrigatórios desde o design (RN-02).
Offline mal especificado	Perda de dados em campo	RNF-050..052 + resolução determinística de conflitos (RN-10).
Dependência de IA externa	Custo, privacidade, indisponibilidade	KCI offline-first; IA externa opt-in e desligada por padrão (RN-07).
Requisitos desalinhados do Bible	Contradição de fonte de verdade	Procedimento de Conflito (Bible §4); rastreabilidade §11.2.
Rastreabilidade não mantida	Requisito órfão sem realização	Matriz §11.2 revisada a cada documento produzido.

14. Melhorias Futuras

- Vincular cada RF a **casos de teste** (Doc 20) e a **user stories/tickets** do backlog.
- Adicionar **critérios de aceite Dado/Quando/Então** completos por requisito Must.
- Automatizar a **matriz de rastreabilidade** a partir de tags nos documentos.
- Detalhar **requisitos de conformidade por país** (Doc 23) e residência de dados (Doc 24).
- Especificar **NFR budgets** por serviço após a arquitetura (Doc 07/19).
- Evoluir personas com **pesquisa de campo** (jornadas, etapa 006).

15. Auditoria

- **Consistência com a Constituição:** ✓ SN vitalício (Art. III), multiempresa 1ª classe (Art. IV), segurança por padrão (Art. IV), offline-first (Art. II.4), direitos do usuário (Art. V).
- **Consistência com o Project Bible:** ✓ escopo consolidado (§5), glossário (§6) e mapa de contextos (§10) respeitados; sem contradições.
- **Consistência com o Roadmap:** ✓ produz os entregáveis das etapas 003, 004, 005, 007–011, 013, 014, 016, 017 e 018 do Doc 03.
- **Cobertura:** 5 personas · 70 RF · 32 RNF · 4 UC principais · 12 RN · matriz de rastreabilidade completa.
- **Pendências:** validação formal (etapa 020) e critérios de aceite detalhados (Doc 20).
- **Estado:** aprovado como **v1.0**; base para o backlog e para a Fase 2 (Arquitetura).

Fim do 06 – Requisitos · v1.0

07 Arquitetura

Documento: 07 – Arquitetura **Versão:** 1.0 · **Data:** 21 de julho de 2026 · **Status:** Ativo **Dependências:** 00 – Master Prompt, 01 – Constituição, 02 – Project Bible, 03 – Roadmap, 04 – Pesquisa de Mercado, 05 – Benchmark, 06 – Requisitos **Responsável:** Arquiteto de Software

1. Resumo Executivo

Este documento define a **arquitetura de referência** do Drone Kairós ERP: a plataforma completa de gestão do ecossistema de drones que conecta **fabricante** → **revenda** → **técnico** → **cliente** sob rastreabilidade vitalícia por **Serial Number (SN)** e isolamento **multiempresa (multi-tenant)** de primeira classe.

A decisão estrutural central é adotar um **monólito modular orientado a Domain-Driven Design (DDD)**, com fronteiras de contexto (*bounded contexts*) explícitas e comunicação interna por contratos, **evoluindo seletivamente para microserviços** apenas onde a extração seja justificada por escala, isolamento de falha, cadência de deploy independente ou requisito regulatório. Essa escolha equilibra velocidade de entrega nas fases iniciais do roadmap (Fase 3) com a capacidade de escalar por domínio nas fases avançadas (Fases 4–6), sem pagar antecipadamente o custo operacional de um sistema distribuído.

A arquitetura assenta sobre cinco pilares herdados da Constituição (Artigo IV): **arquitetura antes de código**, **DDD com fronteiras explícitas**, **multiempresa como requisito de primeira classe**, **segurança por padrão (Zero Trust via KCD)** e **contratos de API primeiro**. A esses somam-se **observabilidade nativa** e o registro de toda decisão relevante como **ADR**.

O modelo recomendado de multi-tenancy é **isolamento por linha com `tenant_id` (row-level) reforçado por Row-Level Security no banco**, com caminho de **promoção para schema-por-tenant ou banco-por-tenant** para clientes enterprise que exijam isolamento físico ou residência de dados. A comunicação é **híbrida**: síncrona (REST público, gRPC/in-process interno) para consultas e comandos com resposta imediata; assíncrona (eventos de domínio sobre um barramento) para propagação de fatos, integração entre contextos e desacoplamento temporal.

2. Objetivos

- Estabelecer a **visão arquitetural em níveis** (estilo C4: contexto, containers, componentes) como linguagem comum entre negócio e engenharia.
- Definir os **bounded contexts** do domínio e o **context map** que os relaciona.
- Fixar a **decisão de estilo arquitetural** (monólito modular → microserviços) com trade-offs explícitos e critérios de extração.
- Recomendar a **estratégia de multi-tenancy** e seu caminho de evolução.
- Definir os **padrões de comunicação** síncrona e assíncrona entre contextos.
- Integrar **segurança por design** (KCD/Zero Trust) e **observabilidade** desde o núcleo.
- Listar os **ADRs-chave** que esta arquitetura obriga a registrar.

3. Escopo

Coberto: a arquitetura lógica e de execução de toda a plataforma — ERP, CRM, Financeiro, BI, API pública, apps Flutter (cliente/técnico), painéis web (fabricante/revenda/admin) e os módulos proprietários **KCI** (inteligência offline-first), **KCD** (cibersegurança Zero Trust) e **KSI** (estoque inteligente). Inclui bounded contexts, context map, estilo arquitetural, multi-tenancy, comunicação, segurança por design e observabilidade em nível arquitetural.

Não coberto aqui (delegado): modelagem de dados detalhada e estratégia de SN → 08; contratos de API e mensageria em detalhe → 09; padrões de código e camadas → 10; detalhamento do KCI → 13; do KCD → 14; do KSI → 15; observabilidade operacional (stacks, dashboards) → 19. Este documento estabelece as **fronteiras e princípios**; os documentos citados aprofundam cada eixo.

4. Regras (Princípios Arquiteturais)

Princípios inegociáveis, subordinados à Constituição e ao Bible. Cada um é testável e serve de critério em revisão arquitetural (etapa 035).

#	Princípio	Regra prática	Origem
P1	Fronteiras explícitas	Todo código pertence a exatamente um bounded context; dependências entre contextos só por contrato público (API/evento), nunca por acesso direto a tabelas alheias.	Const. Art. IV
P2	Multiempresa por padrão	Nenhuma consulta cruza <code>tenant_id</code> ; isolamento é imposto por infraestrutura (RLS), não por disciplina do desenvolvedor.	Const. Art. IV
P3	API-first	O contrato (OpenAPI/Protobuf) é escrito e revisado antes da implementação.	Const. Art. IV
P4	Segurança por padrão	Zero Trust: autenticar e autorizar em cada salto; negar por omissão; criptografar em trânsito e em repouso.	KCD / Art. II.2
P5	Rastreabilidade	Toda mutação de estado relevante emite evento de domínio auditável.	Const. Art. II.3
P6	Offline-first onde crítico	Apps de campo (técnico) e KCI operam sem rede; sincronização é eventual e resolve conflitos de forma determinística.	Const. Art. II.4
P7	Observável por construção	Todo serviço expõe logs estruturados, métricas e tracing correlacionados por <code>trace_id</code> e <code>tenant_id</code> .	Doc 19
P8	Evolução justificada	Extrair um microserviço exige ADR com gatilho mensurável; não se distribui por moda.	Este doc \$5
P9	Idempotência e reprocessamento	Consumidores de eventos são idempotentes; comandos externos aceitam chave de idempotência.	Este doc \$5.5
P10	Dados de um contexto pertencem ao contexto	Sem banco compartilhado entre contextos como canal de integração; cada contexto é dono do seu esquema.	DDD

5. Arquitetura (o núcleo)

5.1 Estilo arquitetural: monólito modular → microserviços

O sistema nasce como **monólito modular**: um único artefato deployável (por camada de aplicação) internamente particionado em módulos que correspondem 1:1 aos bounded contexts. Cada módulo tem seu próprio pacote, seu esquema de dados lógico e uma **API interna explícita** (interface de aplicação). A comunicação entre módulos ocorre **in-process** por essas interfaces ou por **eventos de domínio** publicados num barramento — jamais por acesso direto ao armazenamento de outro módulo (P1, P10).

Essa disciplina cria um **monólito pronto para desmembrar**: como as fronteiras já são reais no código (não apenas convencionais), extrair um módulo para serviço próprio é uma operação de **empacotamento e transporte**, não de reescrita.

Camadas internas (por bounded context) — arquitetura hexagonal / ports & adapters:

```

flowchart TB
    subgraph BC["Bounded Context (módulo)"]
        direction TB
        API["Adaptador de Entrada<br/>(REST / gRPC / consumidor de evento)"]
        APP["Camada de Aplicação<br/>(casos de uso, orquestração, transações)"]
        DOM["Domínio<br/>(entidades, agregados, regras, eventos)"]
        INFRA["Adaptador de Saída<br/>(repositórios, publisher de eventos, gateways)"]
        API --> APP --> DOM
        APP --> INFRA
        INFRA -- implementa ports --> DOM
    end
    end
    DB["Esquema do contexto"]
    BUS["Barramento de Eventos"]
    INFRA --> DB
    INFRA --> BUS

```

Critérios objetivos de extração para microserviço (gatilhos de ADR — P8):

Gatilho	Sinal mensurável	Contexto candidato típico
Escala assimétrica	Um contexto consome > 40% dos recursos ou precisa escalar horizontalmente sozinho	BI, KCI (inferência)
Isolamento de falha	Falha no contexto ameaça disponibilidade do núcleo transacional	Integrações externas, API Pública
Cadência de deploy	Necessidade de release independente > 3× a do monólito	KCI, BI
Requisito regulatório	Residência/segregação de dados exigida por contrato ou lei	Tenancy enterprise, Financeiro
Perfil tecnológico distinto	Runtime/linguagem diferente (ex.: pipeline de visão computacional, modelos)	KCI
Fronteira de segurança	Superfície de ataque que se beneficia de isolamento de rede	KCD, API Pública

Trade-offs do estilo:

Critério	Monólito modular	Microserviços	Escolha Kairós
Velocidade inicial	Alta	Baixa (overhead de infra)	Monólito nas Fases 3–4
Custo operacional	Baixo	Alto (orquestração, malha, observabilidade distribuída)	Monólito enquanto possível
Escala independente	Limitada	Nativa	Extrair onde gatilho §5.1
Consistência transacional	Forte (transação local)	Eventual (saga)	Manter forte no núcleo (OS, Financeiro)
Isolamento de falha	Fraco	Forte	Extrair fronteiras de risco (API Pública, KCI)
Complexidade cognitiva	Concentrada	Distribuída	Reduzir com módulos bem selados
Refatoração de fronteiras	Barata (in-process)	Cara (contrato de rede)	Monólito permite corrigir fronteiras cedo

Candidatos naturais à extração precoce (já projetados como serviços desde a Fase 4): **KCI** (perfil de runtime distinto e cadência própria), **BI** (escala de leitura assimétrica) e **API Pública** (fronteira de segurança e isolamento de falha). O **núcleo transacional** (Identidade, Cadastro/SN, Ordens de Serviço, Estoque, Compras, Financeiro) permanece coeso pelo maior tempo possível, pois se beneficia de **consistência transacional forte**.

5.2 Bounded contexts

Doze contextos delimitados, agrupados por natureza. Cada um é dono do seu modelo e do seu esquema (P10).

#	Bounded Context	Responsabilidade nuclear	Natureza	Extração
1	Identidade & Acesso	Autenticação, usuários, papéis, permissões, sessões, MFA.	Núcleo/Genérico	Tardia
2	Tenancy	Ciclo de vida de empresas, planos, isolamento, provisionamento, contexto de tenant.	Suporte transversal	Tardia
3	Cadastro / Serial Number	Registro de equipamentos, identidade vitalícia por SN, genealogia, histórico de propriedade.	Núcleo (core domain)	Nunca (é o coração)
4	Ordens de Serviço (OS)	Diagnóstico, execução, ciclo de vida do serviço técnico, garantia.	Núcleo (core domain)	Tardia
5	Estoque / KSI	Itens, movimentações, saldos, curva ABC, reposição, códigos (barras/QR/RFID/NFC).	Núcleo	Média
6	Compras	Fornecedores, pedidos de compra, recebimento, integração com estoque.	Suporte	Média
7	Financeiro	Contas a pagar/receber, faturamento, fluxo de caixa, conciliação.	Núcleo	Média (regulatório)
8	CRM	Clientes, oportunidades, pós-venda, relacionamento, atendimento.	Suporte	Média
9	BI	Indicadores, dashboards, agregações analíticas, relatórios.	Genérico/Analítico	Precoce
10	Inteligência / KCI	Base de conhecimento, RAG local, sistema especialista, OCR/visão, preditivo/generativo, IA externa opcional.	Núcleo diferenciador	Precoce
11	Segurança / KCD	Zero Trust, autorização, auditoria, detecção de ameaças, criptografia, SOC, backup/DR.	Transversal	Precoce (isolamento)
12	API Pública	Gateway externo, contratos versionados, webhooks, chaves/quotas, integrações de parceiros.	Transversal/Borda	Precoce

Classificação por **Core Domain Chart**: **SN** e **OS** são o *core domain* (vantagem competitiva, rastreabilidade vitalícia); **KCI** é o *core* diferenciador; **KSI/KCD** são diferenciadores de suporte; **Identidade**, **Tenancy**, **BI** são genéricos/de suporte (candidatos a soluções consolidadas onde fizer sentido).

5.3 Multi-tenancy

Três estratégias possíveis, avaliadas:

Estratégia	Isolamento	Custo/densidade	Operação	Residência de dados	Uso recomendado
Row-level (<code>tenant_id</code> + RLS)	Lógico (imposto pelo banco)	Excelente (alta densidade)	Simples (1 esquema, 1 pipeline de migração)	Compartilhada	Padrão — maioria dos tenants
Schema por tenant	Médio (namespaces separados)	Bom (dezenas–centenas)	Migração multiplicada por tenant	Compartilhada (mesma instância)	Tenants médios com requisito de separação
Banco por tenant	Forte (físico)	Baixo (custo alto por tenant)	Complexa (N instâncias, N backups)	Dedicada / por região	Enterprise, regulados, residência exigida

Recomendação (a formalizar em ADR — etapa 024): row-level com `tenant_id` obrigatório em todas as tabelas de negócio, reforçado por Row-Level Security (RLS) no banco, de modo que o isolamento não dependa da disciplina do

desenvolvedor (P2). A cada requisição, o **contexto de tenant** é resolvido no gateway (a partir do token) e propagado por toda a cadeia; a sessão de banco é configurada com o `tenant_id` corrente, e a RLS impede qualquer vazamento entre empresas — inclusive em caso de bug de query.

O modelo prevê **caminho de promoção**: um tenant pode ser **migrado para schema dedicado ou banco dedicado** (mesmo código, adaptador de persistência distinto) quando o contrato exigir isolamento físico ou residência regional. Isso mantém a densidade e o custo baixos por padrão, sem fechar a porta ao isolamento forte.

```
flowchart LR
    REQ["Requisição<br/>(JWT com tenant_id)"] --> GW["API Gateway / KCD<br/>resolve contexto de tenant"]
    GW --> CTX["Tenant Context<br/>(propagado)"]
    CTX --> APP["Camada de Aplicação"]
    APP --> DBP["Estratégia do tenant"]
    DBP -->|padrão| RLS["Banco compartilhado<br/>RLS por tenant_id"]
    DBP -->|enterprise| SCH["Schema dedicado"]
    DBP -->|regulado| DEDI["Banco dedicado / região"]
```

5.4 Comunicação síncrona × assíncrona

Modelo **híbrido**, escolhido por natureza da interação:

Dimensão	Síncrona	Assíncrona
Protocolo	REST (externo, API Pública) · gRPC/in-process (interno)	Eventos de domínio sobre barramento
Quando usar	Comando/consulta que exige resposta imediata; leitura sob demanda	Propagação de fato consumado; integração entre contextos; trabalho diferido
Acoplamento	Temporal (chamador espera)	Desacoplado no tempo
Consistência	Forte (transação)	Eventual (por saga/projeção)
Falha	Propaga (timeout, circuit breaker)	Absorvida (fila, retry, DLQ)
Exemplo Kairós	Abrir OS, autenticar, consultar saldo de estoque	"OS concluída" → KSI baixa peça, Financeiro fatura, BI agrega

Regras de uso: - **Comandos que mudam estado dentro de um contexto** → transação local síncrona. - **Reações de outros contextos a um fato** → evento de domínio assíncrono (P5). Ex.: `OrdemDeServicoConcluida` dispara baixa de estoque (KSI), faturamento (Financeiro) e agregação (BI). - **Fluxos multi-contexto com invariantes** (ex.: reserva de peça → execução → faturamento) → **saga** orquestrada, com passos compensatórios; nunca transação distribuída de duas fases no núcleo. - **Idempotência** obrigatória em todo consumidor e comando externo (P9), via chave de idempotência e `event_id` deduplicado. - Onde consistência de leitura entre contextos importa, usar **CQRS** com projeções materializadas (especialmente BI e painéis).

5.5 Segurança por design (integração com KCD)

O **KCD** não é um módulo periférico: é a **camada de confiança Zero Trust** que atravessa toda requisição (P4). Detalhamento em 14; aqui, os pontos de integração arquitetural:

- **Autenticação forte** no gateway (tokens de curta duração, MFA, rotação de chaves); **autoriza-se em cada salto**, nunca só na borda.
- **Autorização por política** (RBAC + atributos de tenant/papel) avaliada na camada de aplicação de cada contexto — negar por omissão.
- **Isolamento de tenant** como controle de segurança (RLS), não apenas de dados (P2).
- **Auditoria imutável**: todo evento de domínio sensível alimenta a trilha de auditoria do KCD (append-only), correlacionada por `trace_id` e `tenant_id`.
- **Criptografia** em trânsito (TLS) e em repouso; segredos em cofre; PII segregada.
- **Superfícies isoladas**: API Pública e integrações externas ficam em fronteira de rede própria (candidatas à extração precoce), com quotas, rate limiting e detecção de anomalia.

- **Offline-first seguro** (P6): dados em cache no app do técnico são criptografados; a sincronização reautentica e revalida autorização.

5.6 Observabilidade

Três sinais correlacionados, nativos em cada contexto (P7); operacionalização em 19 :

- **Logs estruturados** (JSON) com `trace_id`, `tenant_id`, `context`, `user_id` — nunca PII em claro.
- **Métricas RED** (Rate, Errors, Duration) por caso de uso e por contexto, além de métricas de negócio (OS por status, giro de estoque, latência de inferência KCI).
- **Tracing distribuído** propagado por todos os saltos (síncronos e assíncronos — o `trace_id` viaja no envelope do evento), permitindo reconstruir uma saga ponta a ponta.
- **Correlação obrigatória por tenant**, para diagnosticar problemas isolando a empresa afetada sem violar isolamento.

6. Diagramas (C4 / Mermaid)

6.1 Nível 1 — Contexto de Sistema (C4)

```

flowchart TB
    Fab([Fabricante]):::actor
    Rev([Revenda]):::actor
    Tec([Técnico]):::actor
    Cli([Cliente]):::actor
    Adm([Administrador]):::actor
    Parc([Parceiro / Integrador]):::actor

    KAIROS["<b>Drone Kairós ERP</b><br/>Plataforma multiempresa do<br/>ecossistema de drones"]:::system

    IAX["Serviço de IA externa<br/>(opcional, via KCI)"]:::ext
    PAG["Gateways de Pagamento"]:::ext
    FISC["Serviços Fiscais / Regulatórios"]:::ext
    MSG["Notificações (e-mail/push/SMS)"]:::ext

    Fab --> KAIROS
    Rev --> KAIROS
    Tec --> KAIROS
    Cli --> KAIROS
    Adm --> KAIROS
    Parc --> API[API Pública] KAIROS

    KAIROS -.opcional.-> IAX
    KAIROS --> PAG
    KAIROS --> FISC
    KAIROS --> MSG

    classDef actor fill:#e8eef7,stroke:#33517a,color:#12233f;
    classDef system fill:#33517a,stroke:#1c2f4a,color:#fff;
    classDef ext fill:#eee,stroke:#999,color:#333;
  
```

6.2 Nível 2 — Containers (C4)

```
flowchart TB
    subgraph Clients["Clientes"]
        APPC["App Cliente<br/>(Flutter)"]
        APPT["App Técnico<br/>(Flutter, offline-first)"]
        WEBF["Painel Fabricante<br/>(Web)"]
        WEBR["Painel Revenda<br/>(Web)"]
        WEBA["Portal Admin<br/>(Web)"]
    end

    GW["API Gateway + KCD<br/>(Zero Trust, tenant context,<br/>rate limit, auditoria)"]:::edge

    subgraph Core["Núcleo (monólito modular)"]
        IDN["Identidade & Acesso"]
        TEN["Tenancy"]
        SN["Cadastro / Serial Number"]
        OS["Ordens de Serviço"]
        KSI["Estoque / KSI"]
        CMP["Compras"]
        FIN["Financeiro"]
        CRM["CRM"]
    end

    subgraph Extracted["Serviços extraídos / borda"]
        KCI["KCI – Inteligência<br/>(offline-first + IA opc.)"]
        BI["BI / Analytics"]
        PUB["API Pública + Webhooks"]
    end

    BUS["Barramento de Eventos"]:::bus
    DB["Bancos por contexto<br/>RLS por tenant_id"]:::db
    DW["Data store analítico"]:::db

    APPC --> GW
    APPT --> GW
    WEBF --> GW
    WEBR --> GW
    WEBA --> GW
    GW --> Core
    GW --> PUB
    Core --> BUS
    KCI --> BUS
    BI --> BUS
    PUB --> BUS
    Core --> DB
    BI --> DW
    Core -.eventos.-> BI

    classDef edge fill:#7a3333,stroke:#4a1c1c,color:#fff;
    classDef bus fill:#33517a,stroke:#1c2f4a,color:#fff;
    classDef db fill:#2f6f4f,stroke:#1c4a34,color:#fff;
```

6.3 Nível 3 — Componentes do contexto Ordens de Serviço (C4)

```
flowchart TB
    IN["Adaptador de Entrada<br/>(REST/gRPC + consumidores)"]
    subgraph APP["Camada de Aplicação (casos de uso)"]
        AbrirOS["AbrirOrdemDeServico"]
        Diag["RegistrarDiagnostics"]
        ExecOS["ExecutarServico"]
        FecharOS["ConcluirOrdemDeServico"]
    end
    subgraph DOM["Domínio"]
        AggOS["Agregado OrdemDeServico"]
        Pol["Políticas de Garantia"]
        Ev["Eventos: OSAberta,<br/>OSConcluida, PecaRequisitada"]
    end
    subgraph OUT["Adaptadores de Saída"]
        Repo["Repositório OS"]
        Pub["Publisher de Eventos"]
        GwSN["Gateway → Cadastro/SN"]
        GwKSI["Gateway → Estoque/KSI"]
    end
    IN --> APP
    APP --> AggOS
    AggOS --> Pol
    AggOS --> Ev
    APP --> Repo
    APP --> GwSN
    APP --> GwKSI
    Ev --> Pub
```

7. Fluxogramas

7.1 Fluxo transversal — Conclusão de Ordem de Serviço (síncrono + eventos)

```
sequenceDiagram
    autonumber
    participant T as App Técnico
    participant GW as Gateway/KCD
    participant OS as Ordens de Serviço
    participant BUS as Barramento
    participant KSI as Estoque/KSI
    participant FIN as Financeiro
    participant SN as Cadastro/SN
    participant BI as BI

    T->>GW: PATCH /os/{id}/concluir (JWT + tenant)
    GW->>GW: AuthN/AuthZ + resolve tenant
    GW->>OS: ConcluirOrdemDeServico
    OS->>OS: valida invariantes + transação local
    OS-->>GW: 200 (OS concluída)
    OS-->>BUS: OrdemDeServicoConcluida (evento)
    BUS-->>KSI: baixa peças utilizadas (idempotente)
    BUS-->>FIN: gera fatura / a receber
    BUS-->>SN: anexa evento ao histórico vitalício
    BUS-->>BI: atualiza indicadores
```


7.2 Fluxo — Sincronização offline-first do App Técnico

```
flowchart TD
    A["Técnico opera offline<br/>(KCI local + fila local)"] --> B{"Rede disponível?"}
    B -- Não --> A
    B -- Sim --> C["Reautenticar (KCD)"]
    C --> D["Enviar comandos enfileirados<br/>(com chave de idempotência)"]
    D --> E{"Conflito de versão?"}
    E -- Não --> F["Aplicar + confirmar"]
    E -- Sim --> G["Resolução determinística<br/>(regra de merge por campo/última escrita)"]
    G --> F
    F --> H["Baixar deltas do servidor"]
    H --> A
```

7.3 Fluxograma de decisão — Extrair um microserviço?

```
flowchart TD
    S["Contexto sob pressão"] --> Q1{"Alguns gatilhos §5.1<br/>é mensurável?"}
    Q1 -- Não --> KEEP1["Manter no monólito<br/>(reforçar módulo)"]
    Q1 -- Sim --> Q2{"Fronteira já é limpa<br/>(sem acesso a dados alheios)?"}
    Q2 -- Não --> FIX["Selar fronteira primeiro<br/>(barato, in-process)"]
    Q2 -- Sim --> Q3{"Custo operacional<br/>compensa o ganho?"}
    Q3 -- Não --> KEEP2[""]
    Q3 -- Sim --> ADR["Registrar ADR + extrair<br/>(contrato de rede versionado)"]
    FIX --> Q1
    ADR --> Q1
```

8. Boas Práticas

- **Selar antes de dividir:** endurecer fronteiras de módulo no monólito é o pré-requisito de qualquer extração; corrigir fronteira in-process é barato, corrigir em rede é caro.
- **Contrato antes de código** (P3): OpenAPI para o externo, Protobuf para gRPC interno, esquema de evento versionado para o barramento.
- **Eventos como fatos no passado**, nomeados no domínio (`OrdemDeServicoConcluida`), nunca comandos disfarçados.
- **Idempotência sempre** em consumidores e comandos externos; toda mensagem carrega `event_id` e `trace_id`.
- **tenant_id** nunca é opcional e nunca é parâmetro de query controlado pela aplicação — vem do contexto de tenant resolvido no gateway (P2).
- **Migrações versionadas** e compatíveis para frente; sem *schema* compartilhado como canal de integração (P10).
- **Feature flags** para desacoplar deploy de release e permitir rollout progressivo por tenant.
- **Toda decisão estrutural vira ADR** (Const. Art. VI.4).

9. Padrões (Táticos e Estratégicos de DDD)

Estratégicos: - *Bounded Context* + *Ubiquitous Language* por contexto (glossário no Bible). - *Context Mapping* com padrões de relacionamento explícitos (§11). - *Core Domain Chart* para priorizar investimento (SN, OS e KCI recebem o melhor esforço de modelagem). - *Anti-Corruption Layer (ACL)* em toda integração com serviços externos (IA externa, fiscal, pagamento) e entre contextos com modelos divergentes.

Táticos: - *Aggregates* com raiz e invariantes (ex.: `OrdemDeServico`, `Equipamento` por SN); transação nunca cruza agregado. - *Domain Events* como mecanismo de integração e auditoria (P5). - *Repositories* por agregado; *Value Objects* para conceitos imutáveis (SN, dinheiro, endereço). - *Application Services* orquestram casos de uso e transações; *Domain Services* para regras sem dono natural. - *CQRS* onde leitura e escrita divergem (BI, painéis, projeções). - *Saga* para consistência entre contextos com passos compensatórios. - *Outbox pattern* para publicação transacional confiável de eventos (evita evento perdido/duplicado na fronteira banco ↔ barramento).

10. Casos de Uso (Cenários Arquiteturais)

- **CA-1 · Rastreabilidade vitalícia:** ao vender, transferir ou reparar um equipamento, cada fato é anexado ao histórico do SN (contexto Cadastro/SN) via evento — reconstrói-se a vida inteira do drone independentemente de qual contexto originou o fato.
- **CA-2 · Diagnóstico offline:** técnico em campo sem rede usa o KCI local (RAG/sistema especialista) para diagnosticar; comandos ficam em fila idempotente e sincronizam ao reconectar (§7.2), sem perda e sem duplicação.
- **CA-3 · Escala do BI:** relatórios pesados não competem com o núcleo transacional porque o BI é extraído e lê de um *data store* próprio alimentado por eventos (CQRS) — escala de leitura sem impactar OS/Financeiro.
- **CA-4 · Tenant regulado:** cliente enterprise exige residência de dados; o mesmo código serve o tenant via **banco dedicado por região** (§5.3) — sem bifurcar a base de código.
- **CA-5 · Integração de parceiro:** integrador consome a API Pública versionada, isolada em fronteira de rede própria, com quota e webhooks — falha ou abuso do parceiro não degrada o núcleo (isolamento de falha).
- **CA-6 · IA externa opcional:** o KCI pode enriquecer diagnósticos com IA externa, sempre atrás de uma ACL e com *fallback* local — a plataforma nunca fica dependente nem inoperante sem o serviço externo.

11. Modelagem (Context Map)

Relacionamentos entre contextos com padrões de DDD estratégico: **U** = *Upstream*, **D** = *Downstream*, **ACL** = *Anti-Corruption Layer*, **OHS** = *Open Host Service*, **PL** = *Published Language*, **CF** = *Conformist*, **SK** = *Shared Kernel*.

```
flowchart LR
  IDN["Identidade & Acesso"]:::sk
  TEN["Tenancy"]:::sk
  SN["Cadastro / SN<br/>(CORE)"]:::core
  OS["Ordens de Serviço<br/>(CORE)"]:::core
  KSI["Estoque / KSI"]
  CMP["Compras"]
  FIN["Financeiro"]
  CRM["CRM"]
  BI["BI"]
  KCI["KCI (CORE)"]:::core
  KCD["KCD (Segurança)"]:::sec
  PUB["API Pública"]:::edge

  IDN -- "SK (contexto de segurança)" --> KCD
  TEN -- "SK (tenant_id em tudo)" --> SN
  TEN -- SK --> OS
  OS -- "U/D: consome SN" --> SN
  OS -- "eventos → baixa peça" --> KSI
  OS -- "eventos → fatura" --> FIN
  CMP -- "reabastece" --> KSI
  KSI -- "custo → " --> FIN
  CRM -- "cliente ↔ SN/OS" --> OS
  OS -- "eventos" --> BI
  FIN -- "eventos" --> BI
  KSI -- "eventos" --> BI
  KCI -- "ACL (IA externa opc.)" --> OS
  PUB -- "OHS + PL (contratos)" --> OS
  PUB -- "OHS + PL" --> SN
  KCD -- "Zero Trust atravessa tudo" --> PUB

  classDef core fill:#33517a,stroke:#1c2f4a,color:#fff;
  classDef sk fill:#5a5a7a,stroke:#33334a,color:#fff;
  classDef sec fill:#7a3333,stroke:#4a1c1c,color:#fff;
  classDef edge fill:#7a5a33,stroke:#4a3a1c,color:#fff;
```

Leitura do mapa: - **Tenancy** e **Identidade & Acesso** funcionam como *Shared Kernel* transversal — o `tenant_id` e o modelo de identidade são compartilhados de forma controlada por todos. - **Cadastro/SN** é *upstream* de quase tudo: OS, CRM e API Pública consomem sua identidade vitalícia. - **OS** é o grande orquestrador transacional, publicando eventos que KSI, Financeiro, SN e BI consomem. - **API Pública** expõe os contextos como *Open Host Service* com *Published Language* (contratos versionados) — parceiros nunca falam o modelo interno diretamente. - **KCI** integra-se via *ACL*,

isolando o núcleo de mudanças em provedores de IA externa. - **KCD** é o único que legitimamente atravessa todos os contextos, como plano de controle de segurança.

12. Checklist

- ☒ Visão C4 em três níveis (contexto, containers, componentes) definida.
- ☒ 12 bounded contexts delimitados e classificados (core/suporte/genérico).
- ☒ Context map com padrões estratégicos explícitos.
- ☒ Estilo arquitetural decidido (monólito modular → microsserviços) com gatilhos de extração mensuráveis.
- ☒ Estratégia de multi-tenancy recomendada (row-level + RLS) com caminho de promoção.
- ☒ Modelo de comunicação híbrido (síncrono/assíncrono) com regras de uso.
- ☒ Segurança por design (KCD/Zero Trust) integrada à arquitetura.
- ☒ Observabilidade nativa (logs/métricas/tracing correlacionados) especificada.
- ☒ ADRs-chave listados (§13/§15).
- ☐ Modelagem de dados detalhada → delegada ao 08.
- ☐ Contratos de API e mensageria detalhados → delegados ao 09.

13. Riscos

Risco	Impacto	Probab.	Mitigação
Fronteiras erodidas (módulos acessando dados alheios)	Alto — impede extração futura e quebra P1	Média	Testes de arquitetura que proíbem dependências ilegais; revisão de fronteira em cada PR
Monólito distribuído (extração prematura sem fronteira limpa)	Alto — pior dos dois mundos	Média	Gatilhos §5.1 obrigatórios; ADR antes de extrair
Vazamento entre tenants	Crítico — quebra P2 e confiança	Baixa	RLS no banco (não só na aplicação); testes de isolamento por tenant
Consistência eventual mal compreendida	Médio — bugs de estado entre contextos	Média	Sagas explícitas, outbox, idempotência (P9); documentar invariantes
Dependência de IA externa (KCI)	Médio — indisponibilidade externa	Média	ACL + fallback local offline-first (CA-6)
Explosão de custo operacional ao distribuir cedo	Médio	Média	Manter núcleo coeso; extrair só BI/KCI/API Pública primeiro
Auditoria incompleta	Alto — regulatório/confiança	Baixa	Eventos de domínio como fonte de auditoria (P5); trilha append-only no KCD

14. Melhorias Futuras

- **Malha de serviços (service mesh)** quando o número de serviços extraídos justificar mTLS e roteamento centralizados.
- **Event sourcing** para os contextos onde o histórico é o próprio ativo (Cadastro/SN, auditoria KCD) — avaliar após estabilização do modelo de eventos.
- **Multi-região ativa** para tenants regulados, com replicação e roteamento por residência de dados (Doc 24).
- **Plataforma de dados** (lakehouse) alimentando BI e o motor preditivo do KCI a partir do mesmo fluxo de eventos.
- **Contract testing automatizado** (consumidor/produtor) na API Pública e no barramento, como quality gate.
- **Chaos engineering** sobre as fronteiras de falha extraídas (API Pública, KCI) na Fase 5.

15. Auditoria

ADRs-chave a registrar (obrigatórios):

ADR	Decisão	Etapa roadmap
ADR-001	Estilo arquitetural: monólito modular DDD com extração seletiva	023
ADR-002	Multi-tenancy: row-level + RLS, com promoção a schema/banco dedicado	024
ADR-003	Identidade universal por Serial Number (estratégia de SN)	026
ADR-004	Comunicação híbrida: REST/gRPC síncrono + eventos assíncronos	029
ADR-005	Outbox + idempotência como padrão de publicação de eventos	029
ADR-006	Zero Trust (KCD) como plano de controle transversal	030
ADR-007	Observabilidade: correlação por <code>trace_id</code> + <code>tenant_id</code>	031
ADR-008	Extração precoce de KCI, BI e API Pública como serviços	027

Consistência: - **Com a Constituição:** ✓ atende Art. IV (arquitetura antes de código, DDD, multiempresa, segurança por padrão, API-first) e Art. VI.4 (decisões viram ADR). - **Com o Roadmap:** ✓ entrega as etapas 021–023 e alimenta 024–035; extração de KCI/BI alinhada à Fase 4. - **Pendências delegadas:** modelagem de dados ([08](#)), contratos/mensageria ([09](#)), padrões de código ([10](#)), detalhamento KCI/KCD/KSI ([13](#) / [14](#) / [15](#)), observabilidade operacional ([19](#)). - **Estado:** aprovado como v1.0; base para as etapas de fundação da Fase 2 e para os ADRs listados.

Fim do [07 – Arquitetura](#) · v1.0

08 Banco de Dados

Documento: 08 – Banco de Dados **Versão:** 1.0 · **Data:** 21 de julho de 2026 · **Status:** Ativo **Dependências:** 00 – Master Prompt, 01 – Constituição, 02 – Project Bible, 03 – Roadmap, 04 – Pesquisa de Mercado, 05 – Benchmark, 06 – Requisitos, 07 – Arquitetura **Responsável:** Engenharia de Dados / DBA

1. Resumo Executivo

Este documento define a **camada de persistência** do Drone Kairós ERP: a estratégia de SGBD, o modelo lógico das entidades-âncora, o desenho de multi-tenancy no banco, a estratégia de histórico vitalício e as políticas operacionais (índices, particionamento, retenção, integridade e migrações).

A decisão fundamental é **PostgreSQL 16+ como núcleo relacional transacional (OLTP)**, complementado por armazenamentos especializados **apenas onde o modelo relacional não é a melhor ferramenta**: séries temporais para telemetria/observabilidade, busca vetorial para o RAG do KCI, e objetos para documentos e mídias. O relacional permanece a **fonte de verdade** de todo dado de negócio.

Três pilares constitucionais guiam cada decisão desta camada (01 – Constituição, Artigos II e III):

1. **Multiempresa de primeira classe** → isolamento por `tenant_id` com **Row-Level Security (RLS)** aplicada por padrão, sem depender da disciplina da aplicação.
2. **Rastreabilidade total** → o **Serial Number (SN)** é a chave de negócio âncora e o **histórico vitalício** é garantido por um modelo híbrido (log append-only imutável + tabelas temporais + projeções).
3. **Segurança por padrão** → criptografia, princípio do menor privilégio, e nenhum dado sai de um tenant sem trilha de auditoria.

Recomendação central deste documento: núcleo PostgreSQL com RLS por `tenant_id` (isolamento *row-level*), SN como *business key* com identidade técnica em `UUID`, e histórico vitalício por **log de auditoria append-only + tabelas temporais (system-versioned via triggers)**, reservando *event sourcing* completo apenas aos contextos de OS e Estoque.

2. Objetivos

- Definir a **estratégia de SGBD** (relacional como núcleo; complementos NoSQL/time-series/vetorial justificados por caso de uso).
- Especificar o **modelo lógico** das oito entidades-âncora, com relacionamentos, cardinalidades e atributos principais.
- Formalizar a **estratégia de Serial Number** como chave de negócio e âncora de rastreabilidade transversal.
- Escolher e justificar a abordagem de **histórico vitalício** (temporal × event sourcing × append-only).
- Desenhar o **isolamento multi-tenant** no banco (schema × row-level, RLS, políticas e chaves).
- Estabelecer políticas de **índices, particionamento, retenção e arquivamento**.
- Fixar regras de **integridade, constraints, soft-delete e versionamento de registros**.
- Definir o processo de **migrações** de esquema (versionadas, reversíveis, com *quality gate*).

3. Escopo

Coberto: modelo lógico e físico do banco transacional; multi-tenancy; SN e rastreabilidade; histórico/auditoria; particionamento e retenção; constraints e migrações; DDL ilustrativa das tabelas centrais; interfaces de persistência dos complementos (time-series, vetorial, objetos).

Não coberto aqui (referências): contratos de API e eventos (09), padrões de código/ORM de acesso (10), estratégia de segurança/criptografia detalhada e Zero Trust (14), lógica interna do KCI/RAG (13), lógica de KSI (15), observabilidade e pipelines (19), estratégia de testes de dados (20), backup/DR operacional (14 / 19) e multi-região (24). Este documento fornece os **contratos de dados**; os documentos citados fornecem a operação.

4. Regras

#	Regra (inviolável nesta camada)	Origem
R-01	Toda tabela de negócio possui <code>tenant_id NOT NULL</code> e é protegida por política RLS.	Constituição, Art. IV (multiempresa)
R-02	Nenhuma linha cruza a fronteira de tenant sem trilha de auditoria explícita.	Art. II.3 / KCD
R-03	O Serial Number é imutável ; nunca é reusado, renomeado ou deletado fisicamente.	Art. III (SN universal)
R-04	Nada de DELETE físico em dado de negócio: usa-se soft-delete + auditoria. Purga só por política de retenção formal.	Art. II.3
R-05	Chave primária técnica é sempre UUID (v7, ordenável no tempo) ; a chave de negócio (ex.: SN) é <i>unique constraint</i> separada.	Padrão de dados
R-06	Todo evento relevante gera registro append-only imutável no log de auditoria (<code>audit_event</code>).	Art. II.3 / KCD
R-07	Migração de esquema é versionada, revisável e reversível ; proibido ALTER manual em produção.	Art. VI.3 (arquitetura antes de código)
R-08	Dados sensíveis são criptografados em repouso; PII segue minimização e política de retenção.	Art. V / LGPD
R-09	Colunas monetárias usam NUMERIC (nunca float); tempo é sempre TIMESTAMPTZ (UTC).	Padrão de dados
R-10	Toda FK entre entidades de negócio inclui <code>tenant_id</code> na relação (evita <i>cross-tenant join</i>).	R-01 + integridade

5. Arquitetura de Dados

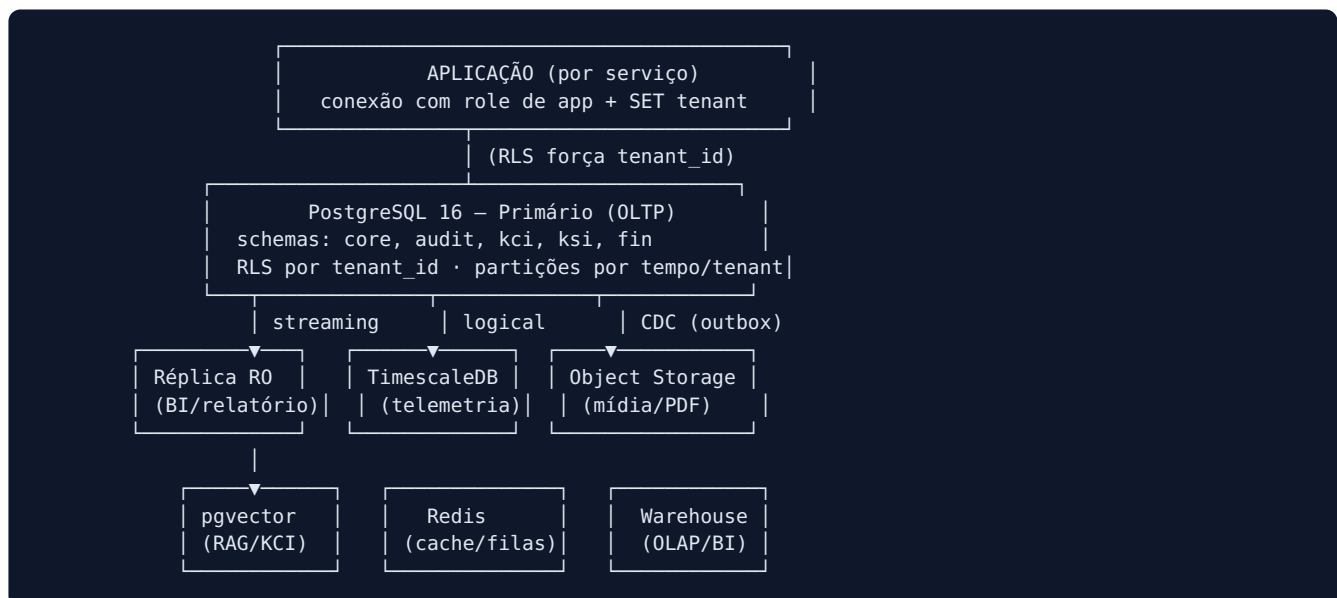
5.1 Estratégia de SGBD — Polyglot Persistence com núcleo relacional

O núcleo do sistema é **transacional e altamente relacional** (equipamentos, OS, estoque, financeiro exigem ACID, integridade referencial e consultas complexas). Portanto, **relacional é o núcleo**; NoSQL/time-series/vetorial entram como **complementos especializados**, nunca como fonte de verdade de negócio.

Camada	Tecnologia	Papel	Justificativa
Núcleo OLTP	PostgreSQL 16+	Fonte de verdade de todo dado de negócio.	ACID, RLS nativa, particionamento declarativo, JSONB , TIMESTAMPZ , extensões (pgvector, PostGIS), maturidade e ecossistema.
Séries temporais	TimescaleDB (extensão do Postgres) ou banco TSDB dedicado	Telemetria de voo/bateria, métricas de sensores, health-check de equipamentos.	Compressão nativa, <i>continuous aggregates</i> , retenção automática. Mantém compatibilidade SQL/Postgres.
Busca vetorial (KCI)	pgvector (extensão)	Embeddings do RAG local do KCI (Knowledge Base, manuais, histórico de diagnósticos).	Colocaliza vetor + metadado relacional no mesmo tenant; evita cópia de dados sensíveis. Escala para índice dedicado (Qdrant/Milvus) se o volume exigir.
Documentos/JSON flexível	JSONB no Postgres (ou DocumentDB se necessário)	Payloads semiestruturados: laudos de diagnóstico KCI, snapshots de configuração, respostas de dispositivos.	Evita explosão de tabelas EAV; indexável por GIN.
Objetos/Mídia	Object Storage (S3-compatível)	Fotos de OS, PDFs, firmwares, anexos.	O banco guarda apenas metadados e ponteiros (chave, hash, MIME), nunca o binário.
Cache / filas de leitura	Redis	Sessões, <i>rate limiting</i> , cache de projeções e <i>hot data</i> .	Latência; não é fonte de verdade (efêmero).
Analytics/BI	Réplica/Warehouse (ELT para colunar)	Dashboards BI (16), agregações pesadas.	Separa OLAP de OLTP; consome de réplica, nunca do primário transacional.

Princípio de fronteira: o dado nasce e é validado no núcleo relacional. Complementos recebem **projeções** ou **cópias derivadas**; se um complemento for perdido, ele é **reconstruível** a partir do núcleo + log de auditoria.

5.2 Topologia física (visão)



5.3 Multi-tenancy no banco — decisão

Três modelos foram avaliados:

Modelo	Isolamento	Custo operacional	Escala (nº tenants)	Veredito
Banco por tenant	Máximo	Muito alto (N bancos, N migrações, N backups)	Baixo (dezenas)	inviável para SaaS de escala
Schema por tenant	Alto	Alto (N schemas, migração fan-out)	Médio (centenas)	opção para <i>tenants enterprise</i> dedicados
Row-level (<code>tenant_id</code>) + RLS	Lógico forte (garantido pelo SGBD)	Baixo (1 esquema, 1 migração)	Alto (milhares)	padrão adotado

Decisão (ADR-0005, a formalizar): row-level com `tenant_id` + RLS nativa do PostgreSQL como padrão, com **schema dedicado** disponível como *tier* premium para clientes que exijam isolamento físico/compliance específico. RLS transforma o isolamento de "responsabilidade da aplicação" em **garantia do banco** — alinhado a "segurança por padrão".

Mecânica da RLS: - A aplicação conecta com uma *role* sem `BYPASSRLS` e executa `SET app.current_tenant = '<uuid>'` no início da transação (via *pool* consciente de tenant). - Cada tabela tem `POLICY` que filtra `tenant_id = current_setting('app.current_tenant')::uuid` para `SELECT/INSERT/UPDATE/DELETE`. - Jobs administrativos usam *role* separada com escopo auditado.

5.4 Estratégia de Serial Number (SN)

O SN é a **âncora de rastreabilidade vitalícia** (Constituição, Art. III). Regras de modelagem:

- **Identidade técnica × chave de negócio:** cada equipamento tem `id UUID` (PK técnica) e `serial_number` (business key, `UNIQUE` global — ver nota de unicidade). Chaves estrangeiras internas referenciam o `UUID`; integrações e humanos referenciam o SN.
- **Unicidade:** o SN é **globalmente único no fabricante** e, na prática Kairós, **único na plataforma**. A `UNIQUE` é sobre `(serial_number)` normalizado (uppercase, sem espaços) — pois o histórico do SN é **transversal e vitalício**, podendo atravessar tenants ao longo da vida (fabricante → revenda → cliente). O vínculo de **posse** varia no tempo; o SN não.
- **Imutabilidade:** `serial_number` nunca é alterado (constraint + trigger de bloqueio). Correção de cadastro errado gera **novo equipamento + evento de migração**, nunca `UPDATE` do SN.
- **Rastreabilidade transversal:** todas as entidades de evento (OS, movimentação de estoque, telemetria, laudo KCI, documento financeiro) carregam referência ao equipamento (via `equipamento_id`), permitindo montar a **linha do tempo completa por SN** com uma única âncora.
- **Custódia entre tenants:** a relação equipamento ↔ tenant é **temporal** (tabela `equipamento_custodia`), não um campo fixo. Isso preserva o histórico vitalício mesmo quando o drone muda de dono, respeitando RLS (cada tenant vê apenas o trecho da vida sob sua custódia; o fabricante e o admin veem a linha completa conforme política).

5.5 Histórico Vitalício — abordagens e recomendação

Requisito constitucional: **"nada acontece sem registro"** e **"direito ao histórico completo do equipamento"** (Art. II.3, Art. V). Comparação das abordagens:

Abordagem	O que é	Prós	Contras	Uso no Kairós
Tabelas temporais (system-versioned)	Cada tabela tem par <code>_history</code> com <code>valid_from/valid_to</code> ; toda alteração arquiva a versão anterior.	Consulta "como era em T"; transparente ao domínio; baixo atrito.	Não modela <i>intenção</i> (só estados); crescimento das history tables.	Padrão para entidades mestre (equipamento, item, empresa, perfil).
Event Sourcing	Estado é derivado de um fluxo imutável de eventos de domínio; projeções materializam o "agora".	Auditoria perfeita, <i>time-travel</i> , <i>replay</i> , CQRS natural.	Complexidade alta, versionamento de eventos, curva de aprendizado.	Seletivo — apenas OS e Movimentação de Estoque (contextos com fluxo de eventos rico).
Append-only Audit Log	Log central imutável de eventos técnicos (quem/o quê/quando/antes/depois), fora do domínio.	Universal, à prova de adulteração (hash-chain), independente do modelo.	Não substitui reconstrução de estado; é registro, não fonte primária de projeção.	Transversal e obrigatório para todas as entidades.

Recomendação (modelo híbrido em três camadas):

1. **audit_event** (append-only, imutável, encadeado por hash) — camada universal e obrigatória (R-06). Registra *toda* mutação relevante de *toda* entidade, com `hash_prev` / `hash_atual` (encadeamento estilo *blockchain* leve) para detecção de adulteração. É a espinha dorsal da rastreabilidade e da conformidade KCD.
2. **Tabelas temporais** (`*_history`) — para entidades mestre, viabilizando "estado em qualquer instante" sem custo cognitivo para o domínio.
3. **Event sourcing seletivo** — nos *bounded contexts* de **OS** e **Estoque**, onde a sequência de eventos é a regra de negócio (ex.: ciclo de vida da OS, movimentações que compõem o saldo).

A **linha do tempo por SN** é servida por uma *view*/projeção que une `audit_event` + eventos de OS + movimentações + telemetria, filtrando por `equipamento_id`.

6. Diagramas — Modelo ER (Mermaid)

```

erDiagram
    EMPRESA ||--o{ USUARIO : "emprega"
    EMPRESA ||--o{ PERFIL : "define"
    USUARIO }o--|| PERFIL : "possui"
    EMPRESA ||--o{ EQUIPAMENTO_CUSTODIA : "detém custódia"
    EQUIPAMENTO ||--o{ EQUIPAMENTO_CUSTODIA : "é custodiado por"
    EMPRESA ||--o{ ORDEM_SERVICO : "abre"
    EQUIPAMENTO ||--o{ ORDEM_SERVICO : "é objeto de"
    USUARIO ||--o{ ORDEM_SERVICO : "executa (técnico)"
    ORDEM_SERVICO ||--o{ OS_ITEM : "consome"
    ITEM ||--o{ OS_ITEM : "é aplicado em"
    EMPRESA ||--o{ ITEM : "cataloga"
    ITEM ||--o{ MOVIMENTACAO_ESTOQUE : "movimenta"
    EMPRESA ||--o{ MOVIMENTACAO_ESTOQUE : "registra"
    ORDEM_SERVICO ||--o{ MOVIMENTACAO_ESTOQUE : "origina"
    EMPRESA ||--o{ DOCUMENTO_FINANCEIRO : "emite/recebe"
    ORDEM_SERVICO ||--o{ DOCUMENTO_FINANCEIRO : "fatura"
    EMPRESA ||--o{ AUDIT_EVENT : "gera"
    USUARIO ||--o{ AUDIT_EVENT : "autor"

    EMPRESA {
        uuid id PK
        text razao_social
        text cnpj_tax_id UK
        text tier "row_level | schema_dedicado"
        timestampz criado_em
        boolean ativo
    }
    USUARIO {
        uuid id PK
        uuid tenant_id FK
        uuid perfil_id FK
        citext email UK
        text status
        timestampz criado_em
    }
    PERFIL {
        uuid id PK
        uuid tenant_id FK
        text nome
        jsonb permissoes
    }
    EQUIPAMENTO {
        uuid id PK
        text serial_number UK "imutável, âncora vitalícia"
        text modelo
        text fabricante
        date data_fabricacao
        text status_ciclo_vida
        timestampz criado_em
    }
    EQUIPAMENTO_CUSTODIA {
        uuid id PK
        uuid equipamento_id FK
        uuid tenant_id FK
        text tipo "fabricante|revenda|cliente"
        timestampz custodia_de
        timestampz custodia_ate "null = atual"
    }
    ORDEM_SERVICO {
        uuid id PK
        uuid tenant_id FK
        uuid equipamento_id FK
        uuid tecnico_id FK
        text numero UK
        text status
        jsonb diagnostico_kci
        timestampz aberta_em
        timestampz fechada_em
        timestampz deletado_em "soft-delete"
    }

```

```

ITEM {
  uuid id PK
  uuid tenant_id FK
  text sku UK
  text descricao
  text curva_abc
  numeric custo_medio
}
OS_ITEM {
  uuid id PK
  uuid tenant_id FK
  uuid os_id FK
  uuid item_id FK
  numeric quantidade
}
MOVIMENTACAO_ESTOQUE {
  uuid id PK
  uuid tenant_id FK
  uuid item_id FK
  uuid os_id FK "nullable"
  text tipo "entrada|saida|ajuste"
  numeric quantidade
  numeric saldo_apos
  timestampz ocorrido_em
}
DOCUMENTO_FINANCEIRO {
  uuid id PK
  uuid tenant_id FK
  uuid os_id FK "nullable"
  text tipo "receber|pagar|nf"
  numeric valor
  char moeda
  text status
  date vencimento
}
AUDIT_EVENT {
  uuid id PK
  uuid tenant_id FK
  uuid autor_id FK
  text entidade
  uuid entidade_id
  text acao
  jsonb antes
  jsonb depois
  text hash_prev
  text hash_atual
  timestampz ocorrido_em
}

```

7. Fluxogramas

7.1 Escrita transacional com RLS + auditoria

```

Requisição autenticada
├─ App abre transação → SET app.current_tenant = <tenant do token>
├─ INSERT/UPDATE em tabela de negócio
│   └─ RLS valida tenant_id —(falha)→ ERRO (linha invisível / rejeitada)
├─ Trigger AFTER escreve em audit_event (antes/depois + hash encadeado)
├─ (contexto OS/Estoque) publica evento no outbox → CDC → projeções
└─ COMMIT —(rollback)→ auditoria da transação falha junto (atômico)

```

7.2 Montagem da linha do tempo vitalícia por SN

```
Entrada: serial_number
├─ Resolve equipamento_id via UNIQUE(serial_number)
├─ Une por equipamento_id:
│   ├─ eventos de custódia (equipamento_custodia)
│   ├─ ordens de serviço + eventos de OS (event store)
│   ├─ movimentações de estoque relacionadas
│   ├─ laudos/diagnósticos KCI (JSONB / vetorial)
│   ├─ telemetria (TimescaleDB, agregada)
│   └─ audit_event (todas as mutações)
├─ Aplica RLS/visibilidade por perfil (cliente vê sua janela; fabricante/admin veem tudo)
└─ Ordena por ocorrido_em → TIMELINE VITALÍCIA
```

7.3 Ciclo de migração de esquema

```
Branch de migração → escreve migration UP + DOWN (versionada)
→ CI: aplica em banco efêmero + testa rollback + valida RLS/constraints
→ Revisão (DBA) → Quality Gate
→ Staging (expand) → Deploy código → Backfill → Contract (remove legado)
→ Produção (mesma sequência: expand → migrate → contract)
```

8. Boas Práticas

- **Expand-Migrate-Contract** para toda mudança de esquema (zero-downtime): primeiro adiciona o novo, migra dados, depois remove o antigo — nunca em um passo.
- **RLS "negar por padrão"**: políticas restritivas; role de app sem `BYPASSRLS`; teste automatizado que tenta vazar entre tenants em cada PR.
- **UUID v7** como PK (ordenável no tempo → melhor localidade de índice que v4, sem expor sequência como *bigserial*).
- **NUMERIC** para dinheiro, **TIMESTAMPZ** (UTC) para tempo, **citext** /normalização para e-mail e SN.
- **Índices dirigidos por consulta**: compostos iniciando por `tenant_id`; parciais para `deletado_em IS NULL`; GIN para `JSONB` /busca.
- **FKs sempre com `tenant_id` na relação** (R-10) e **`ON DELETE RESTRICT`** (soft-delete é a regra; cascata física é proibida em negócio).
- **Outbox pattern** para publicar eventos ao mesmo commit da escrita (consistência com [09 – API/Eventos](#)).
- **Conexões via pool consciente de tenant**; nunca reutilizar sessão sem resetar `app.current_tenant`.
- **Migrações idempotentes e reversíveis**; proibido DDL manual em produção (R-07).
- **Backup lógico + físico (PITR)**; testar restauração periodicamente (o backup não testado não existe).

9. Padrões

9.1 Convenções de nomenclatura

Objeto	Padrão	Exemplo
Tabela	<code>snake_case</code> , singular	<code>ordem_servico</code>
Coluna	<code>snake_case</code>	<code>serial_number</code> , <code>criado_em</code>
PK	<code>id</code> (UUID v7)	<code>id uuid</code>
FK	<code><entidade>_id</code>	<code>equipamento_id</code>
History	<code><tabela>_history</code>	<code>equipamento_history</code>
Índice	<code>ix_<tabela>_<colunas></code>	<code>ix_os_tenant_status</code>
Unique	<code>uq_<tabela>_<colunas></code>	<code>uq_equipamento_serial</code>
Constraint check	<code>ck_<tabela>_<regra></code>	<code>ck_mov_quantidade_positiva</code>
Schema	por contexto	<code>core</code> , <code>audit</code> , <code>ksi</code> , <code>fin</code> , <code>kci</code>

9.2 Colunas de controle (padrão em toda tabela de negócio)

Coluna	Tipo	Regra
<code>id</code>	<code>uuid</code>	PK, v7, <code>DEFAULT</code> gerado no banco
<code>tenant_id</code>	<code>uuid</code>	<code>NOT NULL</code> , alvo da RLS (R-01)
<code>criado_em</code>	<code>timestampz</code>	<code>NOT NULL DEFAULT now()</code>
<code>atualizado_em</code>	<code>timestampz</code>	atualizado por trigger
<code>versao</code>	<code>bigint</code>	<i>optimistic locking</i> (incrementa a cada update)
<code>deletado_em</code>	<code>timestampz</code>	<code>NULL</code> = ativo; preenchido = soft-delete (R-04)
<code>criado_por</code> / <code>atualizado_por</code>	<code>uuid</code>	rastreio de autoria

9.3 Soft-delete e versionamento

- **Soft-delete:** `UPDATE ... SET deletado_em = now()`; todas as *queries* e *views* de negócio filtram `deletado_em IS NULL` (índice parcial). Purga física somente por política de retenção (R-04) e sempre com evento de auditoria.
- **Versionamento otimista:** coluna `versao` no `WHERE` do `UPDATE`; conflito → erro 409 na API (`09`).
- **Versionamento histórico:** trigger *system-versioned* grava a versão anterior em `*_history` com `valid_from/valid_to`.

10. Casos de Uso

#	Caso	Como o modelo atende
UC-01	Cliente pede o histórico completo do drone (SN)	Resolve <code>equipamento_id</code> por SN → <i>timeline</i> (7.2) unindo custódia, OS, estoque, KCI, telemetria e <code>audit_event</code> .
UC-02	Drone é vendido de revenda para cliente	Fecha <code>custodia_ate</code> da custódia atual, abre nova custódia; SN e histórico permanecem intactos; evento auditado.
UC-03	Técnico fecha OS consumindo peças (offline-first)	OS + <code>os_item</code> + <code>movimentacao_estoque</code> (event-sourced) sincronizam ao reconectar; saldo recalculado; tudo auditado.
UC-04	Auditor investiga adulteração	Verifica o encadeamento <code>hash_prev</code> → <code>hash_atual</code> em <code>audit_event</code> ; quebra da cadeia denuncia manipulação.
UC-05	Tenant A tenta ler dado do tenant B	RLS bloqueia no SGBD independentemente de bug na aplicação; tentativa auditada (KCD).
UC-06	KCI busca casos similares para diagnóstico	Embedding da falha → busca por similaridade em <code>pgvector</code> dentro do tenant; resultados ancorados a OS/SN.
UC-07	BI agrega faturamento por período	Consulta a réplica RO / warehouse; nunca onera o primário OLTP.
UC-08	DPO atende pedido de exclusão (LGPD)	Anonimização de PII do usuário preservando integridade referencial e histórico do equipamento (dado de negócio ≠ PII).

11. Modelagem — Modelo Lógico Detalhado

11.1 Entidades-âncora (resumo)

Entidade	Papel	Chave de negócio	Histórico
Empresa (Tenant)	Organização isolada	<code>cnpj/tax_id</code>	temporal
Equipamento (SN)	Drone; âncora vitalícia	<code>serial_number</code>	temporal + audit + timeline
Usuário / Perfil	Identidade e permissões	<code>email</code> / nome do perfil	temporal + audit
Ordem de Serviço	Unidade de trabalho	<code>numero</code>	event sourcing
Peça / Item	Catálogo de estoque	<code>sku</code>	temporal
Movimentação de Estoque	Fluxo de saldo	(evento)	event sourcing
Documento Financeiro	Receber/pagar/NF	<code>numero_documento</code>	temporal + audit
Evento de Auditoria	Trilha imutável	(append-only)	é o histórico

11.2 DDL ilustrativa — `core.empresa` (tenant) e helper de RLS

```
-- Extensões base (uma vez por banco)
CREATE EXTENSION IF NOT EXISTS pgcrypto; -- gen_random_uuid / hashing
CREATE EXTENSION IF NOT EXISTS citext;   -- e-mail case-insensitive
-- (pg_uuidv7 ou geração v7 na app para PKs ordenáveis)

CREATE SCHEMA IF NOT EXISTS core;
CREATE SCHEMA IF NOT EXISTS audit;

CREATE TABLE core.empresa (
    id          uuid          PRIMARY KEY DEFAULT gen_random_uuid(),
    razao_social text          NOT NULL,
    cnpj_tax_id citext         NOT NULL,
    pais        char(2)       NOT NULL DEFAULT 'BR',
    tier         text          NOT NULL DEFAULT 'row_level'
                CHECK (tier IN ('row_level','schema_dedicado')),
    ativo       boolean       NOT NULL DEFAULT true,
    criado_em   timestampz    NOT NULL DEFAULT now(),
    atualizado_em timestampz  NOT NULL DEFAULT now(),
    versao      bigint        NOT NULL DEFAULT 1,
    deletado_em timestampz,
    CONSTRAINT uq_empresa_cnpj UNIQUE (cnpj_tax_id, pais)
);

-- Função de contexto de tenant (usada por todas as políticas RLS)
CREATE OR REPLACE FUNCTION core.current_tenant() RETURNS uuid
LANGUAGE sql STABLE AS $$
    SELECT NULLIF(current_setting('app.current_tenant', true), '')::uuid
$$;
```


11.3 DDL ilustrativa — core.equipamento (âncora SN) com RLS e imutabilidade

```
CREATE TABLE core.equipamento (
  id                uuid          PRIMARY KEY DEFAULT gen_random_uuid(),
  serial_number     text          NOT NULL,          -- BUSINESS KEY (imutável)
  modelo            text          NOT NULL,
  fabricante        text          NOT NULL,
  data_fabricacao   date,
  status_ciclo_vida text          NOT NULL DEFAULT 'ativo'
  CHECK (status_ciclo_vida IN
        ('fabricado','ativo','manutencao','baixado')),
  especificacoes    jsonb         NOT NULL DEFAULT '{}':jsonb,
  criado_em         timestampz    NOT NULL DEFAULT now(),
  atualizado_em      timestampz    NOT NULL DEFAULT now(),
  versao            bigint        NOT NULL DEFAULT 1,
  deletado_em       timestampz,
  -- SN normalizado e único na plataforma (vitalício, transversal a tenants)
  CONSTRAINT uq_equipamento_serial UNIQUE (serial_number),
  CONSTRAINT ck_equipamento_serial_norm
    CHECK (serial_number = upper(btrim(serial_number)))
);

-- Custódia temporal: o vínculo com o tenant varia no tempo; o SN não.
CREATE TABLE core.equipamento_custodia (
  id                uuid          PRIMARY KEY DEFAULT gen_random_uuid(),
  equipamento_id     uuid          NOT NULL REFERENCES core.equipamento(id),
  tenant_id         uuid          NOT NULL REFERENCES core.empresa(id),
  tipo              text          NOT NULL CHECK (tipo IN ('fabricante','revenda','cliente')),
  custodia_de       timestampz    NOT NULL DEFAULT now(),
  custodia_ate       timestampz,          -- NULL = custódia atual
  CONSTRAINT ck_custodia_intervalo CHECK (custodia_ate IS NULL OR custodia_ate > custodia_de)
);

-- No máximo uma custódia aberta por equipamento
CREATE UNIQUE INDEX uq_custodia_ativa
  ON core.equipamento_custodia (equipamento_id)
  WHERE custodia_ate IS NULL;

-- RLS: um tenant só enxerga equipamentos sob sua custódia (atual ou passada)
ALTER TABLE core.equipamento_custodia ENABLE ROW LEVEL SECURITY;
CREATE POLICY p_custodia_tenant ON core.equipamento_custodia
  USING (tenant_id = core.current_tenant())
  WITH CHECK (tenant_id = core.current_tenant());

-- Bloqueio de alteração do SN (imutabilidade – R-03)
CREATE OR REPLACE FUNCTION core.bloqueia_alteracao_sn() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
  IF NEW.serial_number <> OLD.serial_number THEN
    RAISE EXCEPTION 'Serial Number é imutável (id=%).', OLD.id;
  END IF;
  NEW.atualizado_em := now();
  NEW.versao := OLD.versao + 1;
  RETURN NEW;
END $$;
CREATE TRIGGER tg_equipamento_sn_imutavel
  BEFORE UPDATE ON core.equipamento
  FOR EACH ROW EXECUTE FUNCTION core.bloqueia_alteracao_sn();

-- Índices de acesso
CREATE INDEX ix_equipamento_ativo ON core.equipamento (id)
  WHERE deletado_em IS NULL;
CREATE INDEX ix_equipamento_especs ON core.equipamento USING gin (especificacoes);
```

11.4 DDL ilustrativa — `audit.audit_event` (append-only, hash-encadeado, particionado)

```
-- Log de auditoria imutável e particionado por mês (retenção/arquivamento)
CREATE TABLE audit.audit_event (
  id          uuid          NOT NULL DEFAULT gen_random_uuid(),
  tenant_id   uuid          NOT NULL,
  autor_id    uuid,
  entidade    text          NOT NULL,
  entidade_id uuid          NOT NULL,
  acao        text          NOT NULL CHECK (acao IN ('insert','update','delete','custom')),
  antes       jsonb,
  depois      jsonb,
  hash_prev   text,
  hash_atual  text          NOT NULL,
  ocorrido_em timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (id, ocorrido_em)
) PARTITION BY RANGE (ocorrido_em);

-- Partições mensais (criadas por automação; ex. julho/2026)
CREATE TABLE audit.audit_event_2026_07 PARTITION OF audit.audit_event
  FOR VALUES FROM ('2026-07-01') TO ('2026-08-01');

-- Imutabilidade: sem UPDATE/DELETE (apenas INSERT)
CREATE RULE audit_no_update AS ON UPDATE TO audit.audit_event DO INSTEAD NOTHING;
CREATE RULE audit_no_delete AS ON DELETE TO audit.audit_event DO INSTEAD NOTHING;

-- RLS para leitura por tenant (admin/KCD usam role auditada dedicada)
ALTER TABLE audit.audit_event ENABLE ROW LEVEL SECURITY;
CREATE POLICY p_audit_tenant ON audit.audit_event
  FOR SELECT USING (tenant_id = core.current_tenant());

CREATE INDEX ix_audit_entidade ON audit.audit_event (tenant_id, entidade, entidade_id, ocorrido_em);
```

As demais tabelas (`usuario`, `perfil`, `ordem_servico`, `item`, `movimentacao_estoque`, `documento_financeiro`) seguem o mesmo gabarito: `tenant_id` NOT NULL, RLS USING/WITH CHECK por `core.current_tenant()`, colunas de controle (§9.2), soft-delete e trigger de auditoria disparando em `audit.audit_event`.

11.5 Índices, Particionamento, Retenção e Arquivamento

Tema	Política
Índices	Compostos começando por <code>tenant_id</code> ; parciais em <code>deletado_em IS NULL</code> ; GIN para <code>JSONB</code> /full-text; <code>pgvector</code> HNSW/IVFFlat para embeddings; cobrir <i>queries</i> dos casos de uso da §10.
Particionamento	<code>audit_event</code> e telemetria: RANGE por tempo (mês). Tabelas muito grandes multi-tenant: considerar HASH por <code>tenant_id</code> ou LIST para <i>tenants enterprise</i> . Partições futuras criadas por automação (<code>pg_partman</code>).
Retenção	OLTP quente: dado ativo. Auditoria/telemetria: janela quente (ex. 12–18 meses) + arquivamento frio. Dado de negócio de SN: vitalício (nunca expira — Art. V), podendo ser movido para <i>cold storage</i> consultável.
Arquivamento	Partições antigas → DETACH → exportação para storage barato (Parquet/objeto) mantendo consultabilidade via <i>foreign data wrapper</i> ou warehouse.
PII (LGPD)	Retenção e minimização por finalidade; anonimização atende ao "direito ao esquecimento" sem apagar o histórico do equipamento (que é dado de negócio, não PII).

11.6 Integridade e Constraints (síntese)

- **Referencial:** todas as FKs **ON DELETE RESTRICT** (cascata física proibida em negócio); FKs compostas incluindo `tenant_id` onde aplicável.
- **Domínio:** **CHECK** para *enums* de status, quantidades positivas, intervalos temporais coerentes, moeda ISO-4217.
- **Unicidade:** **UNIQUE** sobre chaves de negócio normalizadas (SN, SKU, e-mail, número de OS/documento) — com escopo por tenant onde a chave é local (ex.: `uq_item_sku` sobre `(tenant_id, sku)`).
- **Concorrência:** **versao** (optimistic locking) em toda entidade mutável.

11.7 Migrações

- Ferramenta versionada (Flyway/Liigibase/Sqitch ou nativa do ORM) com arquivos `V<n>__descricao.sql` e `DOWN` correspondente.
- **Toda migração passa por CI:** aplica em banco efêmero, roda testes (incluindo *teste de vazamento entre tenants* e verificação de que RLS continua ativa), valida `UP` e `DOWN`.
- **Zero-downtime:** padrão *expand* → *migrate/backfill* → *contract* (§8).
- **Proibido** `ALTER` manual em produção (R-07); toda mudança é rastreável a um *commit* e a um ADR quando estrutural.

12. Checklist

- ☒ SGBD núcleo definido (PostgreSQL) e complementos justificados (time-series, vetorial, objetos, cache).
- ☒ Multi-tenancy decidido: **row-level + RLS** (schema dedicado como *tier premium*).
- ☒ SN modelado como *business key* imutável, com identidade técnica em UUID e custódia temporal.
- ☒ Histórico vitalício: híbrido **audit append-only + temporal + event sourcing seletivo**.
- ☒ Entidades-âncora modeladas com relacionamentos, cardinalidades e atributos.
- ☒ Diagrama ER (Mermaid) e DDL ilustrativa de 3 tabelas centrais entregues.
- ☒ Políticas de índice, particionamento, retenção e arquivamento definidas.
- ☒ Integridade, constraints, soft-delete e versionamento padronizados.
- ☒ Processo de migração (versionado, reversível, com *quality gate*) definido.
- ☐ ADR-0005 (multi-tenancy) e ADR-0006 (histórico vitalício) a formalizar.
- ☐ Benchmark de `pgvector` × índice dedicado (a executar na Fase 4, 13).

13. Riscos

#	Risco	Impacto	Mitigação
RK-01	Vazamento entre tenants por bug de aplicação.	Crítico (confiança)	RLS "negar por padrão" no SGBD + teste de vazamento em cada PR (R-01, UC-05).
RK-02	Crescimento descontrolado de auditoria/history.	Alto (custo/perf)	Particionamento por tempo + arquivamento frio + compressão (§11.5).
RK-03	Hot partition / skew por tenant muito grande.	Médio	HASH por tenant ou schema dedicado para <i>enterprise</i> (§5.3).
RK-04	Divergência de projeção (event sourcing) vs. estado.	Médio	Reprocessamento (<i>replay</i>) idempotente + reconciliação; núcleo reconstruível do log.
RK-05	Migração destrutiva em produção.	Alto	Expand-migrate-contract + <code>DOWN</code> testado + proibição de DDL manual (R-07).
RK-06	PII x histórico vitalício (conflito LGPD).	Alto (legal)	Separar PII de dado de negócio; anonimizar PII preservando SN/histórico (UC-08).
RK-07	Adulteração do log de auditoria.	Crítico	Encadeamento por hash + <code>RULE</code> anti-UPDATE/DELETE + WORM/cold storage (§11.4).
RK-08	Acoplamento excessivo aos complementos (vetorial/TSDB).	Médio	Manter relacional como fonte de verdade; complementos reconstruíveis (§5.1).

14. Melhorias Futuras

- Formalizar **ADR-0005** (multi-tenancy RLS) e **ADR-0006** (histórico vitalício híbrido).
- Avaliar `pg_partman` para automação total de partições e **Citus** para *sharding* horizontal na escala multi-região (24).

- **PITR + backups geo-redundantes** e testes automáticos de restauração (integra [14](#) / [19](#)).
- **Column-level encryption** para PII crítica e *tokenização* de identificadores sensíveis.
- **Data contracts** versionados entre núcleo e complementos (KCI/BI) via schema registry.
- **Catálogo de dados + lineage** automatizado para governança (integra [19](#)).
- Prova de conceito de **event sourcing** completo em OS/Estoque com *snapshotting* e *projections* materializadas.

15. Auditoria

- **Consistência constitucional:** ✓ alinhado aos Artigos II (rastreabilidade, autonomia), III (SN universal), IV (multiempresa, segurança por padrão) e V (histórico e privacidade).
 - **Consistência com o Bible:** ✓ cobre as oito entidades-âncora e os módulos KCI/KCD/KSI conforme [02](#) §5, [§11](#).
 - **Rastreabilidade de roadmap:** entrega as etapas **024** (multi-tenancy), **025** (modelagem de dados) e **026** (estratégia de SN) da Fase 2 ([03](#)).
 - **Pendências abertas:** ADR-0005/0006; benchmark [pgvector](#); políticas finais de retenção por país ([23](#)).
 - **Estado:** aprovado como **v1.0**; fornece os contratos de dados para [09](#) (API/eventos), [10](#) (padrões de código), [13](#) (KCI), [14](#) (KCD), [15](#) (KSI) e [19](#) (operação).
-

Fim do [08](#) – Banco de Dados · v1.0

09 Microsserviços

Documento: 09 – Microsserviços **Versão:** 1.0 · **Data:** 21 de julho de 2026 · **Status:** Ativo **Dependências:** 00, 01, 02, 03, 04, 05, 06, 07, 08 **Responsável:** Arquiteto de Sistemas

1. Resumo Executivo

Este documento define a **arquitetura de microsserviços** do Drone Kairós ERP, derivada diretamente dos *bounded contexts* estabelecidos no canon do projeto (DDD, API-first, event-driven onde justificado, multiempresa). A estratégia central é **evolutiva**: não nascemos como constelação de serviços, e sim como um **monólito modular** com fronteiras internas rígidas (um módulo por bounded context, banco lógico segregado por schema), do qual extraímos serviços apenas quando um driver mensurável exige — escala independente, cadência de deploy divergente, isolamento de falha ou fronteira de time.

A comunicação segue dois planos: **síncrono** (REST para o mundo externo e parceiros da API Pública; gRPC para chamadas internas de baixa latência e alto volume) e **assíncrono** (broker de eventos como espinha dorsal para desacoplamento temporal, com eventos de domínio versionados como `EquipamentoRegistrado`, `OSConcluida`, `EstoqueBaixado`). A consistência entre serviços é garantida por **outbox pattern** na publicação, **idempotência** no consumo e **sagas** (orquestração para fluxos críticos de negócio, coreografia para reações simples). A borda é protegida por um **API Gateway** e dois **BFFs** (web e mobile). A malha adota **service mesh** para mTLS, descoberta, e políticas de resiliência (circuit breaker, retry com backoff, timeout, bulkhead), e **tracing distribuído** (W3C Trace Context / OpenTelemetry) como requisito não-funcional obrigatório.

O documento entrega o **catálogo de serviços**, o **catálogo de eventos**, contratos de exemplo (OpenAPI e Protobuf), diagramas Mermaid da malha, o fluxograma de uma saga (`Compra → Recebimento → Estoque`), padrões de integração/resiliência, checklist de prontidão para extração, riscos e trilha de auditoria.

2. Objetivos

- **01.** Traduzir os 12 bounded contexts em uma decomposição de serviços com responsabilidade única e propriedade exclusiva de dados (*database-per-service* lógico).
- **02.** Padronizar contratos: REST/OpenAPI 3.1 na borda, gRPC/Protobuf no interior, com versionamento explícito e compatibilidade retroativa.
- **03.** Estabelecer o backbone assíncrono: broker, convenção de tópicos, esquema de eventos de domínio e governança de schema.
- **04.** Garantir consistência distribuída sem 2PC: outbox, idempotência, sagas.
- **05.** Definir a camada de operação: API Gateway, BFFs, service discovery, service mesh e resiliência.
- **06.** Tornar a observabilidade distribuída (tracing, métricas, logs correlacionados) um requisito de arquitetura, não um adendo.
- **07.** Definir a **estratégia de evolução** e os gatilhos objetivos de extração de serviço.

3. Escopo

No escopo: decomposição em serviços; contratos síncronos e assíncronos; gateway/BFF; broker e eventos de domínio; consistência (saga/outbox/idempotência); descoberta, mesh e resiliência; tracing distribuído; roadmap monólito → serviços.

Fora do escopo (referência a outros documentos): modelagem de dados por contexto (Doc 06), segurança/IAM detalhada e KCD (Doc 07), infraestrutura/IaC e topologia de clusters (Doc 08), pipelines de CI/CD (Doc de operação),

modelo de tenancy físico (Doc de Tenancy). Aqui tratamos apenas das **interfaces e do comportamento distribuído** desses módulos.

4. Regras

- **R1. Contrato primeiro.** Nenhum serviço é implementado antes do contrato (OpenAPI/proto) revisado e versionado no repositório de contratos.
- **R2. Propriedade de dados.** Cada dado tem **um** serviço dono. Nenhum serviço acessa a tabela de outro; integração só por API ou evento.
- **R3. Sem banco compartilhado.** Proibido *shared database*. No monólito modular, isolamento por schema; ao extrair, banco físico próprio.
- **R4. Assíncrono por padrão** para propagação de fatos de domínio; síncrono apenas quando há necessidade de resposta imediata do chamador.
- **R5. Tenant sempre presente.** Todo request e todo evento carrega `tenant_id`; ausência é erro de contrato.
- **R6. Idempotência obrigatória** em todo consumidor de eventos e em endpoints de escrita não naturalmente idempotentes (via `Idempotency-Key`).
- **R7. Compatibilidade retroativa.** Mudança quebra-contrato exige nova versão maior; a antiga é mantida durante a janela de depreciação.
- **R8. Correlação ponta-a-ponta.** `traceparent` (W3C) propagado em REST, gRPC e cabeçalhos de mensagem, sem exceção.
- **R9. Falha isolada.** Toda chamada de saída de serviço tem timeout, retry com jitter e circuit breaker definidos; sem *defaults* infinitos.
- **R10. Extração justificada.** Um serviço só é extraído do monólito com um driver documentado (ver §14 e checklist §12).

5. Arquitetura (malha de serviços)

5.1 Camadas lógicas

1. **Borda (Edge):** API Gateway (roteamento, authN/Z, rate limit, TLS termination) + BFF Web + BFF Mobile.
2. **Serviços de domínio:** um serviço (ou módulo, na fase monólito) por bounded context.
3. **Backbone assíncrono:** broker de eventos + registry de schema + conectores de outbox.
4. **Plano de dados:** um armazenamento por serviço (PostgreSQL como padrão OLTP; motor de busca/analytics para BI/KCI conforme Doc 06/08).
5. **Plano de malha:** service mesh (sidecars) para mTLS, discovery, políticas de tráfego e telemetria.
6. **Observabilidade:** coletor OpenTelemetry → backend de tracing, métricas e logs.

5.2 Mapeamento bounded context → serviço

Bounded Context	Serviço	Responsabilidade nuclear	Dados que possui	Interface principal
Identidade & Acesso	<code>iam-service</code>	AuthN, tokens, papéis, permissões, sessões	Usuários, credenciais, roles, escopos	REST (externo) + gRPC (introspecção interna)
Tenancy	<code>tenancy-service</code>	Ciclo de vida de empresas/tenants, planos, isolamento	Tenants, assinaturas, configuração por tenant	REST + eventos
Cadastro / Serial Number	<code>registry-service</code>	Cadastro de equipamentos/drones e serial number único	Equipamentos, seriais, modelos, fabricantes	REST + gRPC + eventos
Ordens de Serviço	<code>workorder-service</code>	Ciclo de vida da OS (abertura → execução → conclusão)	Ordens de serviço, itens, apontamentos, status	REST + eventos + saga
Estoque / KSI	<code>inventory-service</code>	Posição de estoque, movimentações, KSI (índice de estoque)	Itens, saldos, reservas, movimentações	gRPC (interno) + eventos
Compras	<code>procurement-service</code>	Requisições, cotações, pedidos de compra, recebimento	Requisições, cotações, pedidos, fornecedores	REST + eventos + saga
Financeiro	<code>finance-service</code>	Contas a pagar/receber, lançamentos, conciliação	Títulos, lançamentos, centros de custo	REST + eventos
CRM	<code>crm-service</code>	Clientes, contatos, oportunidades, atendimento	Contas, contatos, funil, interações	REST + eventos
BI	<code>bi-service</code>	Agregações, dashboards, indicadores	Modelos analíticos (read models)	REST (consulta) — consome eventos
Inteligência / KCI	<code>kci-service</code>	Índice de inteligência/insights, recomendações	Features, scores, modelos	gRPC + eventos
Segurança / KCD	<code>kcd-service</code>	Índice de conformidade/defesa, auditoria de segurança	Eventos de segurança, achados, políticas	eventos + REST (relatórios)
API Pública	<code>public-api-service</code>	Fachada externa para parceiros, quotas, chaves	Chaves de API, contratos de parceiro, quotas	REST (público)

Na **fase monólito modular**, cada linha acima é um **módulo** com pacote e schema próprios; na fase de serviços, torna-se um *deployable* independente. O contrato entre eles é idêntico em ambos os casos — muda apenas o transporte (chamada in-process vs. rede).

5.3 Diagrama da malha

```
flowchart TB
    subgraph Clientes
        W[Web App]
        M[Mobile App]
        P[Parceiros / Integradores]
    end

    subgraph Borda
        GW[API Gateway]
        BW[BFF Web]
        BM[BFF Mobile]
    end

    W --> BW
    M --> BM
    P --> GW
    BW --> GW
    BM --> GW

    subgraph Dominio[Serviços de Domínio]
        IAM[iam-service]
        TEN[tenancy-service]
        REG[registry-service]
        OS[workorder-service]
        INV[inventory-service]
        PROC[procurement-service]
        FIN[finance-service]
        CRM[crm-service]
        BI[bi-service]
        KCI[kci-service]
        KCD[kcd-service]
        PUB[public-api-service]
    end

    GW --> IAM
    GW --> REG
    GW --> OS
    GW --> PROC
    GW --> FIN
    GW --> CRM
    GW --> PUB
    PUB --> REG
    PUB --> OS

    subgraph Async[Backbone de Eventos]
        BROKER[(Event Broker)]
        SR[Schema Registry]
    end

    REG -. publica .-> BROKER
    OS -. publica .-> BROKER
    INV -. publica .-> BROKER
    PROC -. publica .-> BROKER
    FIN -. publica .-> BROKER
    BROKER -. consome .-> INV
    BROKER -. consome .-> FIN
    BROKER -. consome .-> BI
    BROKER -. consome .-> KCI
    BROKER -. consome .-> KCD

    subgraph Observ[Observabilidade]
        OTEL[OpenTelemetry Collector]
        TRACE[Tracing/Métricas/Logs]
    end

    Dominio -. telemetria .-> OTEL --> TRACE
```


6. Diagramas

6.1 Contexto de borda (Gateway + BFF)

```
flowchart LR
    subgraph Frontends
        WEB[SPA Web]
        APP[App Mobile]
    end
    WEB -->|GraphQL/REST agregado| BFFW[BFF Web]
    APP -->|REST enxuto/otimizado| BFFM[BFF Mobile]
    BFFW --> GW[API Gateway]
    BFFM --> GW
    GW -->|JWT introspection| IAM[iam-service]
    GW -->|roteamento + rate limit| SVCS[Serviços de Domínio]
    classDef edge fill:#0b7285,color:#fff;
    class GW,BFFW,BFFM edge;
```

O **BFF** existe para desacoplar a experiência de cada canal do contrato dos serviços de domínio: o BFF Web agrega múltiplas chamadas em uma resposta rica; o BFF Mobile enxuga payloads, reduz *round-trips* e adapta a modelos de tela pequenos. Nenhum BFF possui dados próprios — é *stateless* e orquestrador.

6.2 Comunicação síncrona vs. assíncrona

```
flowchart TB
    A[procurement-service] -->|gRPC: ReservarItens| B[inventory-service]
    A -->|evento: PedidoCompraAprovado| BR[(Broker)]
    BR --> FIN[finance-service]
    BR --> BI[bi-service]
    note1[Síncrono quando o chamador precisa da resposta agora]
    note2[Assíncrono para propagar fato consumado a N interessados]
```

6.3 Fluxo de um evento de domínio (outbox → broker → consumidores)

```
sequenceDiagram
    participant S as workorder-service
    participant DB as DB (tabela + outbox)
    participant R as Outbox Relay
    participant B as Broker
    participant I as inventory-service
    participant F as finance-service

    S->>DB: TX { grava OS + insere evento OSConcluida na outbox }
    DB-->>S: commit
    R->>DB: poll outbox (não publicados)
    R->>B: publica OSConcluida (chave = tenant_id:os_id)
    R->>DB: marca como publicado
    B->>I: OSConcluida (baixa peças consumidas)
    B->>F: OSConcluida (gera título a receber)
```

7. Fluxogramas (uma saga)

Saga: Compra → Recebimento → Estoque → Financeiro (orquestração, com compensação).

```

flowchart TD
    START([Requisição de Compra aprovada]) --> 01[Orquestrador Saga cria PedidoCompra]
    01 --> S1[procurement: emite PedidoCompra]
    S1 -->|ok| S2[inventory: reserva capacidade de recebimento]
    S2 -->|ok| S3[Recebimento físico registrado]
    S3 --> S4[inventory: dá entrada no estoque -> EstoqueEntrada]
    S4 -->|ok| S5[finance: provisiona conta a pagar]
    S5 -->|ok| DONE([Saga concluída])

    S2 -->|falha| C1[compensa: cancela PedidoCompra]
    S4 -->|falha| C2[compensa: estorna reserva + cancela pedido]
    S5 -->|falha| C3[compensa: estorna entrada de estoque + notifica]
    C1 --> ABORT1([ABORT([Saga abortada])])
    C2 --> ABORT2([ABORT])
    C3 --> ABORT3([ABORT])

```

Máquina de estados da saga (orquestrador):

```

stateDiagram-v2
    [*] --> PEDIDO_CRIADO
    PEDIDO_CRIADO --> RECEBIMENTO_RESERVADO: reservaOk
    PEDIDO_CRIADO --> ABORTADA: reservaFalha / compensa
    RECEBIMENTO_RESERVADO --> ESTOQUE_ENTRADO: entradaOk
    RECEBIMENTO_RESERVADO --> ABORTADA: entradaFalha / compensa
    ESTOQUE_ENTRADO --> FINANCEIRO_PROVISIONADO: provisaoOk
    ESTOQUE_ENTRADO --> ABORTADA: provisaoFalha / compensa
    FINANCEIRO_PROVISIONADO --> CONCLUIDA
    CONCLUIDA --> [*]
    ABORTADA --> [*]

```

Regra de ouro da saga: cada passo tem uma **ação** e uma **compensação** semanticamente reversível. Não há rollback distribuído; há compensação de negócio. O orquestrador persiste o estado da saga (também via outbox) para sobreviver a reinícios.

8. Boas Práticas

- **Serviço = capacidade de negócio**, não entidade CRUD. `workorder-service` cobre o ciclo da OS, não apenas a tabela.
- **Contratos pequenos e estáveis**; dados internos livres para evoluir. Nunca vaze o modelo interno no contrato.
- **Comandos vs. eventos**: comando é intenção direcionada a **um** serviço (imperativo, pode falhar); evento é fato consumado, no passado, para **N** interessados.
- **Eventos são fatos, não notificações RPC disfarçadas**. `EstoqueBaixado` descreve o que ocorreu; não instrui ninguém a fazer algo.
- **Payload de evento auto-suficiente + versionado**. Inclua `event_id`, `tenant_id`, `occurred_at`, `version`. Evite *chatty callbacks* para reidratar dados.
- **Idempotência sempre**: consumidores deduplicam por `event_id`; endpoints de escrita aceitam `Idempotency-Key`.
- **Timeouts curtos e explícitos**; retry só em erros transitórios e idempotentes; *backoff* com jitter.
- **Falhe rápido na borda, seja resiliente no interior**. Circuit breaker protege o chamador de um dependente doente.
- **Versione desde o v1**. `/api/v1/...` na borda; `package ... v1` no proto.
- **Observabilidade como código**: `traceparent` propagado, spans nomeados por operação de negócio, atributos com `tenant_id` e `service.name`.
- **Segurança por padrão**: mTLS entre serviços (mesh), JWT validado na borda, *least privilege* por escopo de token.

9. Padrões (integração/resiliência)

9.1 Integração

Padrão	Quando usar	Observação
Request/Response (REST)	Borda externa, parceiros, leituras interativas	OpenAPI 3.1, JSON, v na rota
Request/Response (gRPC)	Interno, baixa latência, alto volume (ex.: inventory.ReservarItens)	Protobuf, HTTP/2, streaming quando útil
Event Notification	Propagar fato de domínio	Payload magro + link; ou payload completo se auto-suficiente
Event-Carried State Transfer	Consumidor precisa de dados sem chamar de volta	Reduz acoplamento temporal; cuidado com tamanho
CQRS + read model	BI, KCI, telas de leitura pesada	Read models materializados por consumo de eventos
API Composition / BFF	Agregar N serviços para uma tela	No BFF, nunca no cliente
Outbox	Publicar evento atômico com a transação de negócio	Elimina <i>dual-write</i>
Saga (orquestração)	Fluxo multi-serviço crítico (compra, faturamento)	Orquestrador central, estado persistido
Saga (coreografia)	Reações simples encadeadas	Sem orquestrador; cuidado com ciclos

9.2 Resiliência

Padrão	Objetivo	Parâmetro de referência
Timeout	Evitar espera infinita	p.ex. 500 ms–2 s por chamada interna
Retry + backoff/jitter	Absorver falhas transitórias	máx. 3 tentativas, só em erro idempotente
Circuit Breaker	Isolar dependente doente	abre em >50% erro/janela; half-open para sondar
Bulkhead	Conter esgotamento de recursos	pools separados por dependência
Rate limiting / Throttling	Proteger contra abuso e picos	por tenant e por chave de API
Fallback / Degradação graciosa	Manter valor parcial	cache, valor default, resposta reduzida
Dead Letter Queue (DLQ)	Não perder mensagem envenenada	inspeção + reprocesso manual/automático

```
flowchart LR
    Caller[procurement-service] -->|timeout 800ms + retry x3| CB{Circuit Breaker}
    CB -->|fechado| Inv[inventory-service]
    CB -->|aberto| FB[Fallback: fila de reserva assíncrona]
    Inv -->|erro persistente| DLQ[(DLQ)]
```

10. Casos de Uso

UC-01 — Registro de equipamento com serial único. Operador cadastra drone → [registry-service](#) valida unicidade do serial, persiste e publica [EquipamentoRegistrado](#). [kci-service](#) e [bi-service](#) atualizam índices; [crm-service](#) associa ao cliente. Idempotência garante que um retry do cliente não gere duplicidade (chave = tenant + serial).

UC-02 — Conclusão de OS baixando estoque. Técnico conclui OS → [workorder-service](#) grava e emite [OSConcluida](#) (via outbox). [inventory-service](#) consome e gera [EstoqueBaixado](#) para as peças; [finance-service](#)

gera título a receber. Se `inventory` falhar, a OS permanece concluída (fato imutável) e o consumo é reprocessado da DLQ — consistência eventual.

UC-03 — Compra ponta-a-ponta (saga). Requisição aprovada dispara a saga do §7. Falha no provisionamento financeiro compensa a entrada de estoque e o pedido, deixando o sistema em estado íntegro de negócio.

UC-04 — Consulta agregada no app mobile. App pede "resumo do drone X" → BFF Mobile compõe `registry` (dados do equipamento) + `workorder` (últimas OS) + `inventory` (peças associadas) em uma resposta enxuta, com um único `traceparent` cobrindo as três chamadas.

UC-05 — Parceiro externo via API Pública. Integrador consulta status de OS → `public-api-service` valida chave/quota, delega a `workorder-service` por gRPC interno e responde REST versionado, sem expor contratos internos.

11. Modelagem (catálogo de serviços + eventos)

11.1 Catálogo de serviços (síntese)

Serviço	Transporte de entrada	Publica eventos	Consume eventos	Store
<code>iam-service</code>	REST + gRPC	<code>UsuarioCriado</code> , <code>PapelAlterado</code>	—	PostgreSQL
<code>tenancy-service</code>	REST	<code>TenantProvisionado</code> , <code>PlanoAlterado</code>	—	PostgreSQL
<code>registry-service</code>	REST + gRPC	<code>EquipamentoRegistrado</code> , <code>SerialBaixado</code>	<code>TenantProvisionado</code>	PostgreSQL
<code>workorder-service</code>	REST	<code>OSAberta</code> , <code>OSConcluida</code> , <code>OSCancelada</code>	<code>EquipamentoRegistrado</code>	PostgreSQL
<code>inventory-service</code>	gRPC	<code>EstoqueEntrada</code> , <code>EstoqueBaixado</code> , <code>EstoqueReservado</code>	<code>OSConcluida</code> , <code>PedidoCompraRecebido</code>	PostgreSQL
<code>procurement-service</code>	REST	<code>PedidoCompraAprovado</code> , <code>PedidoCompraRecebido</code>	<code>EstoqueBaixado</code> (reposição)	PostgreSQL
<code>finance-service</code>	REST	<code>TituloGerado</code> , <code>TituloLiquidado</code>	<code>OSConcluida</code> , <code>PedidoCompraAprovado</code>	PostgreSQL
<code>crm-service</code>	REST	<code>ClienteCriado</code> , <code>OportunidadeGanha</code>	<code>EquipamentoRegistrado</code> , <code>OSConcluida</code>	PostgreSQL
<code>bi-service</code>	REST (query)	—	(todos, para read models)	Analytics store
<code>kci-service</code>	gRPC	<code>InsightGerado</code>	<code>EquipamentoRegistrado</code> , <code>OSConcluida</code> , <code>EstoqueBaixado</code>	Feature store
<code>kcd-service</code>	REST (relatórios)	<code>AlertaSegurancaGerado</code>	(eventos de segurança/auditoria)	PostgreSQL
<code>public-api-service</code>	REST (público)	<code>ChamadaExternaRegistrada</code>	—	PostgreSQL

11.2 Catálogo de eventos de domínio

Evento	Produtor	Consumidores	Chave de partição	Payload (essencial)
EquipamentoRegistrado	registry	crm, kci, bi	tenant_id:equipamento_id	equipamento_id, serial, modelo, cliente_id
OSConcluida	workorder	inventory, finance, crm, kci	tenant_id:os_id	os_id, equipamento_id, itens[], técnico_id
EstoqueBaixado	inventory	procurement, bi, kci	tenant_id:item_id	item_id, quantidade, saldo_atual, origem
EstoqueEntrada	inventory	bi, finance	tenant_id:item_id	item_id, quantidade, pedido_id
PedidoCompraAprovado	procurement	finance, inventory, bi	tenant_id:pedido_id	pedido_id, fornecedor_id, itens[], total
PedidoCompraRecebido	procurement	inventory, finance	tenant_id:pedido_id	pedido_id, itens_recebidos[]
TituloGerado	finance	bi, crm	tenant_id:titulo_id	titulo_id, tipo, valor, vencimento
AlertaSegurancaGerado	kcd	iam, bi	tenant_id:alerta_id	alerta_id, severidade, contexto

11.3 Envelope de evento (contrato canônico)

Todo evento trafega no envelope abaixo (CloudEvents-like), com **schema versionado** no registry:

```
{
  "event_id": "0f9c2b8e-4c3a-4d2e-9f1a-1b2c3d4e5f60",
  "type": "OSConcluida",
  "version": "1.0",
  "tenant_id": "tnt_00421",
  "occurred_at": "2026-07-21T14:03:11Z",
  "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
  "producer": "workorder-service",
  "data": {
    "os_id": "os_88213",
    "equipamento_id": "eqp_00713",
    "itens": [{ "item_id": "itm_501", "quantidade": 2 }],
    "tecnico_id": "usr_3391"
  }
}
```

11.4 Contrato REST (trecho OpenAPI 3.1 — workorder-service)

```
openapi: 3.1.0
info:
  title: Workorder Service API
  version: "1.0.0"
servers:
  - url: https://api.dronekairos.com/workorder/v1
paths:
  /ordens-servico/{osId}/conclusao:
    post:
      operationId: concluirOrdemServico
      summary: Conclui uma Ordem de Serviço e dispara baixa de estoque
      parameters:
        - name: osId
          in: path
          required: true
          schema: { type: string }
        - name: X-Tenant-Id
          in: header
          required: true
          schema: { type: string }
        - name: Idempotency-Key
          in: header
          required: true
          schema: { type: string, format: uuid }
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/ConclusaoOSRequest'
      responses:
        '200':
          description: OS concluída
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/OrdemServico'
        '409':
          description: OS já concluída (conflito idempotente)
components:
  schemas:
    ConclusaoOSRequest:
      type: object
      required: [tecnicoId, itensConsumidos]
      properties:
        tecnicoId: { type: string }
        itensConsumidos:
          type: array
          items:
            type: object
            required: [itemId, quantidade]
            properties:
              itemId: { type: string }
              quantidade: { type: integer, minimum: 1 }
    OrdemServico:
      type: object
      properties:
        osId: { type: string }
        status: { type: string, enum: [ABERTA, EM_EXECUCAO, CONCLUIDA, CANCELADA] }
        concluidaEm: { type: string, format: date-time }
```

11.5 Contrato interno gRPC (trecho Protobuf — `inventory-service`)

```
syntax = "proto3";
package dronekairos.inventory.v1;

service InventoryService {
  // Reserva itens de forma idempotente (chave = reservation_id).
  rpc ReservarItens(ReservarItensRequest) returns (ReservarItensResponse);
  rpc ConfirmarBaixa(ConfirmarBaixaRequest) returns (ConfirmarBaixaResponse);
}

message ItemQuantidade {
  string item_id = 1;
  int32 quantidade = 2;
}

message ReservarItensRequest {
  string tenant_id = 1;
  string reservation_id = 2; // idempotência
  string origem_id = 3;      // ex.: os_id ou pedido_id
  repeated ItemQuantidade itens = 4;
}

message ReservarItensResponse {
  enum Status { OK = 0; INSUFICIENTE = 1; JA_RESERVADO = 2; }
  Status status = 1;
  repeated ItemQuantidade faltantes = 2;
}
```

11.6 Versionamento de API

- **REST:** versão maior na rota (`/v1`, `/v2`). Mudanças aditivas (novos campos opcionais) não sobem versão; mudanças quebra-contrato sim. Janela de depreciação mínima anunciada via cabeçalho `Deprecation` + `Sunset`.
- **gRPC/proto:** pacote versionado (`...v1`); campos só adicionados (nunca renumerados/reusados); campos removidos viram `reserved`.
- **Eventos:** `version` no envelope; consumidores toleram campos desconhecidos (tolerant reader). Mudança quebra-contrato = novo `type` ou nova versão maior, coexistindo durante a migração.

12. Checklist

Prontidão de um serviço (Definition of Ready para extração/deploy):

- [] Contrato OpenAPI/proto publicado, revisado e versionado no repo de contratos.
- [] Propriedade de dados exclusiva; nenhum acesso cruzado a schema alheio.
- [] `tenant_id` presente em todo request, evento e span.
- [] Publicação de eventos via **outbox** (sem dual-write).
- [] Consumidores **idempotentes** (dedupe por `event_id`); DLQ configurada.
- [] Timeout, retry (só idempotente) e circuit breaker definidos para cada dependência.
- [] mTLS ativo no mesh; token JWT validado; escopos mínimos.
- [] `traceparent` propagado; spans nomeados por operação de negócio; métricas RED expostas.
- [] Health checks (`/healthz`, `/readyz`) e sinais de liveness/readiness.
- [] Migração de dados e *feature flag* de corte prontas (para extração do módulo).
- [] Runbook de falha e reprocesso de DLQ documentado.
- [] Testes de contrato (consumer-driven) verdes.

13. Riscos

ID	Risco	Prob.	Impacto	Mitigação
RS-01	Decomposição prematura (nano-serviços) antes de fronteiras claras	Alta	Alto	Começar monólito modular; extrair só com driver (§14)
RS-02	Transação distribuída / dual-write inconsistente	Média	Alto	Outbox + saga + idempotência; proibir 2PC
RS-03	Acoplamento por evento mal versionado quebra consumidores	Média	Alto	Schema registry + tolerant reader + versionamento
RS-04	Falha em cascata por dependência síncrona lenta	Média	Alto	Circuit breaker, timeout, bulkhead, fallback
RS-05	Mensagem envenenada trava partição	Média	Médio	DLQ + limite de retry + alerta
RS-06	Vazamento entre tenants	Baixa	Crítico	<code>tenant_id</code> obrigatório, testes de isolamento, mTLS
RS-07	Observabilidade insuficiente para depurar fluxo distribuído	Média	Alto	Tracing obrigatório desde o v1; correlação ponta-a-ponta
RS-08	Explosão operacional (muitos serviços, poucos SREs)	Média	Médio	Mesh + plataforma interna + automação; extração gradual
RS-09	Contrato interno vaza para cliente externo	Baixa	Médio	BFF/API Pública como fachada; nunca expor gRPC interno
RS-10	Divergência de read models (BI/KCI) por evento perdido	Média	Médio	Entrega ao menos uma vez + idempotência + reprocesso

14. Melhorias Futuras

- **Estratégia de evolução (norteadora):** 1. **Fase 0 — Monólito modular:** um deployable, módulos por bounded context, schemas isolados, contratos internos já definidos, outbox já em uso para eventos. 2. **Fase 1 — Primeiras extrações:** extrair serviços com maior pressão (candidatos naturais: `inventory-service` por volume, `iam-service` por isolamento de segurança, `bi-service` / `kci-service` por carga analítica assimétrica). 3. **Fase 2 — Malha completa:** service mesh, discovery dinâmico, BFFs dedicados, API Pública endurecida. 4. **Fase 3 — Otimização:** CQRS onde compensar, event sourcing seletivo em contextos de auditoria (KCD), autoscaling por serviço.
- **Gatilhos objetivos de extração:** necessidade de escala independente; cadência de deploy divergente; isolamento de falha crítico; fronteira de time (Conway); requisito regulatório de isolamento.
- **Evoluções técnicas:** *schema registry* com validação em CI; testes de contrato consumer-driven no pipeline; *chaos engineering* para validar resiliência; *service catalog* interno; *event sourcing* seletivo para KCD/auditoria; gateway com *canary* e *A/B* por tenant.

15. Auditoria

Item	Descrição
Rastreabilidade	Cada serviço mapeia 1:1 a um bounded context do canon (§5.2); cada evento tem produtor e consumidores declarados (§11.2).
Trilha técnica	<code>trace_id</code> no envelope de evento e <code>traceparent</code> em REST/gRPC permitem reconstruir qualquer fluxo distribuído ponta-a-ponta.
Log de auditoria de negócio	<code>kcd-service</code> e <code>finance-service</code> retêm eventos imutáveis; ações sensíveis geram <code>AlertaSegurancaGerado</code> .
Governança de contratos	Toda mudança de contrato passa por revisão, versionamento e teste de compatibilidade antes do merge.
Conformidade multiempresa	<code>tenant_id</code> obrigatório e verificado em borda, mesh e persistência; testes de isolamento no checklist (§12).
Versão do documento	v1.0 — 21 de julho de 2026. Alterações futuras registradas no controle de versão do repositório de documentação.
Aprovação	Arquiteto de Sistemas (responsável); dependências 00–08 verificadas e coerentes com o PROJECT CANON.

Nota de encerramento. Este documento estabelece o *contrato arquitetural* de microsserviços do Drone Kairós ERP. Ele é deliberadamente **evolutivo**: as fronteiras aqui descritas valem tanto para o monólito modular inicial quanto para a malha de serviços futura — o que muda é o transporte, não o contrato. Nenhuma extração de serviço deve ocorrer sem driver documentado e checklist (§12) satisfeito.

10 Backend

Campo	Valor
Documento	10 — Backend
Versão	1.0
Data	21 de julho de 2026
Status	Ativo
Dependências	00, 01, 02, 03, 04, 05, 06, 07, 08, 09
Responsável	Engenharia de Backend
Fonte de verdade	Project Bible (Doc. 02) · Constituição (Doc. 01)

1. Resumo Executivo

Este documento define o **padrão de engenharia de backend** do Drone Kairós ERP: a linguagem e o framework de referência, a arquitetura interna de cada serviço, os padrões táticos de Domain-Driven Design (DDD), a estratégia de segurança e isolamento multiempresa, o tratamento de erros, os padrões de resiliência e transações, e a disciplina de testes e qualidade. É a fonte de verdade para *como* escrevemos serviços — complementa (não substitui) os documentos de arquitetura de solução (visão macro), de dados e de API.

O backend do Kairós é composto por **serviços de domínio autônomos** (ERP, CRM, Financeiro, BI, API Pública, e os módulos proprietários **KCI** — inteligência offline-first, **KCD** — cibersegurança Zero Trust, **KSI** — estoque inteligente), organizados por *bounded context*. Cada serviço é internamente estruturado segundo **Arquitetura Hexagonal (Ports & Adapters)** combinada com **Clean Architecture**, isolando o núcleo de domínio de qualquer detalhe de infraestrutura. A comunicação é **API-first** (contratos antes de código) e **event-driven onde justificado**, com o padrão **Transactional Outbox** garantindo consistência entre estado local e eventos publicados.

Recomendação de stack padrão: **Java 21 + Spring Boot 3.x** como padrão corporativo para os serviços de domínio transacionais (ERP, CRM, Financeiro, API Pública), pela maturidade do ecossistema enterprise, força em regras fiscais/financeiras, talento abundante no Brasil e suporte de longo prazo. **Go 1.22+** é adotado como stack secundária, autorizada apenas para serviços de alta vazão, baixa latência ou footprint reduzido — notadamente **ingestão de telemetria**, **sincronização offline-first do KCI** e *edge/gateways*. Todo desvio da stack padrão exige justificativa arquitetural registrada (ADR).

Princípios não-negociáveis herdados da Constituição: arquitetura antes de código; segurança por padrão; contratos de API primeiro; observabilidade nativa; **multiempresa como cidadã de primeira classe** — o **tenant** vive no contexto de segurança e é aplicado em todas as camadas, nunca opcional.

2. Objetivos

Objetivo geral: padronizar a construção de serviços de backend de forma que qualquer time possa criar, evoluir e operar um serviço do Kairós com previsibilidade, segurança e qualidade uniformes.

Objetivos específicos:

1. **Recomendar** linguagem e framework de referência, com trade-offs explícitos e critérios de exceção.
2. **Definir** a arquitetura interna do serviço (camadas, portas, adaptadores, inversão de dependência).

3. **Codificar** os padrões táticos de DDD (agregados, repositórios, serviços de domínio, eventos de domínio) e o uso de CQRS onde couber.
4. **Estabelecer** a estratégia de segurança: autenticação OIDC/JWT, autorização RBAC/ABAC, isolamento multiempresa e validação de entrada.
5. **Padronizar** o tratamento de erros e a propagação de resultados.
6. **Garantir** resiliência e integridade transacional (Outbox, idempotência, retries, circuit breaker).
7. **Instituir** a pirâmide de testes e as portas de qualidade (quality gates).
8. **Prover** uma estrutura de projeto/pastas de referência, reutilizável entre serviços.

Fora de objetivo: topologia de deploy/infraestrutura (Doc. de DevOps/Infra), modelagem física de dados por módulo (Doc. de Dados), especificação de endpoints públicos (Doc. de API), e frontend/apps de campo.

3. Escopo

Dentro do escopo:

- Padrão de linguagem/framework e critérios de exceção.
- Arquitetura interna dos serviços de domínio (Hexagonal + Clean).
- Padrões táticos DDD e CQRS.
- Segurança aplicada no serviço (authN/authZ, tenant, validação).
- Tratamento de erros, resiliência, transações, mensageria (Outbox/idempotência).
- Testes automatizados e qualidade de código.
- Estrutura de pastas de referência.

Fora do escopo (referenciado em outros documentos):

- Malha de serviços, ingress, service mesh, CI/CD, Kubernetes → Doc. Infra/DevOps.
- Esquemas de banco por agregado, particionamento, retenção → Doc. Dados.
- Design de endpoints, versionamento externo, SDKs públicos → Doc. API Pública.
- Modelagem estratégica de *bounded contexts* e *context map* → Doc. Arquitetura de Solução.

Aplicabilidade: obrigatório para todos os serviços de backend de todos os módulos (KCI/KCD/KSI e ERP/CRM/Financeiro/BI/API Pública).

4. Regras

Regras normativas. Chaves **DEVE** (obrigatório), **NÃO DEVE** (proibido), **DEVERIA** (recomendado forte).

4.1 Arquitetura

- **R-A01 — DEVE** isolar o domínio: nenhuma classe de `domain/` importa framework, ORM, HTTP ou driver de banco.
- **R-A02 — DEVE** apontar todas as dependências para dentro (regra da dependência da Clean Architecture): infraestrutura depende de domínio, nunca o inverso.
- **R-A03 — DEVE** expor a lógica de negócio somente por **casos de uso** (application services); adaptadores de entrada não contêm regra de negócio.
- **R-A04 — NÃO DEVE** vazar entidades de persistência (JPA/entities) para além da camada de infraestrutura; a fronteira usa DTOs/comandos/objetos de domínio.

4.2 Multiempresa (Tenancy)

- **R-T01 — DEVE** resolver o `tenant_id` a partir do token de segurança (claim), nunca de parâmetro livre do cliente.
- **R-T02 — DEVE** propagar o `tenant_id` no contexto de execução (security context + contexto assíncrono) até o repositório.

- **R-T03 — DEVE** aplicar filtro de tenant de forma automática e obrigatória na camada de persistência (row-level ou schema-per-tenant conforme Doc. Dados). Consulta sem tenant é um defeito, não uma opção.
- **R-T04 — NÃO DEVE** existir endpoint, job ou consumidor de evento que opere sem tenant resolvido (exceção: rotinas de plataforma, explicitamente marcadas e auditadas).

4.3 Segurança

- **R-S01 — DEVE** autenticar via OIDC/OAuth2 com validação de assinatura e claims do JWT (issuer, audience, expiração).
- **R-S02 — DEVE** autorizar por RBAC como base e ABAC para regras contextuais (tenant, propriedade do recurso, escopo).
- **R-S03 — DEVE** validar toda entrada na borda (sintática) e no domínio (invariantes de negócio). Nunca confiar no cliente.
- **R-S04 — NÃO DEVE** registrar segredos, tokens ou PII em log. Mascaramento obrigatório.
- **R-S05 — DEVE** negar por padrão (deny-by-default): ausência de permissão explícita = negado.

4.4 Confiabilidade

- **R-C01 — DEVE** usar o padrão **Transactional Outbox** para publicar eventos que precisam ser consistentes com a mudança de estado.
- **R-C02 — DEVE** tornar consumidores de eventos e endpoints de escrita **idempotentes** (chave de idempotência / dedupe).
- **R-C03 — DEVE** retornar erros num formato padronizado (RFC 7807 *Problem Details*) e nunca expor stack trace ao cliente.
- **R-C04 — DEVERIA** aplicar timeouts, retries com backoff+jitter e circuit breaker em toda chamada de saída.

4.5 Qualidade

- **R-Q01 — DEVE** ter cobertura mínima de 80% de linhas no núcleo de domínio e aplicação; domínio testado sem mocks de infraestrutura.
- **R-Q02 — DEVE** passar em análise estática, verificação de dependências (regra da dependência) e SAST/SCA antes do merge.
- **R-Q03 — NÃO DEVE** fazer merge com portas de qualidade vermelhas.

5. Arquitetura (interna do serviço)

5.1 Escolha de linguagem/framework

Avaliamos quatro candidatos maduros para serviços transacionais enterprise. A avaliação pondera: adequação a ERP/Financeiro (regras fiscais, transações, precisão decimal), ecossistema DDD/segurança, disponibilidade de talento (Brasil), performance/custo, e maturidade operacional.

Critério	Java 21 / Spring Boot 3	.NET 8 / ASP.NET Core	Node 20 / NestJS	Go 1.22
Maturidade enterprise / ERP	• Altíssima	• Alta	• Média	• Média
Ecosistema DDD/segurança	• (Spring Security, Modulith)	• (identidade, EF)	• (bibliotecas fragmentadas)	• (montagem manual)
Precisão financeira/decimal	• <code>BigDecimal</code> nativo	• <code>decimal</code> nativo	◦ <code>number</code> = risco; precisa lib	• precisa lib/inteiros
Talento no Brasil	• Abundante	• Bom	• Abundante	• Escasso sênior
Performance/latência	• Boa (JIT/GraalVM p/ boot)	• Muito boa	• I/O bom, CPU fraco	• Excelente
Footprint / edge	• Médio (nativo via GraalVM)	• Médio	• Médio	• Mínimo
Concorrência massiva	• (virtual threads / Loom)	• (async/await)	• (event loop, 1 core)	• (goroutines)
Custo de manutenção a longo prazo	• LTS previsível	• LTS previsível	• churn de libs	• simples

Decisão (ADR-BE-001):

- **Padrão corporativo — Java 21 + Spring Boot 3.x** para serviços de domínio transacionais: ERP, CRM, Financeiro, BI (camada de serviço), API Pública. Justificativa: precisão decimal nativa (`BigDecimal`), Spring Security/Spring Modulith para DDD e segurança, *virtual threads* (Project Loom) para concorrência sem complexidade reativa, e o maior pool de talento sênior enterprise no Brasil. LTS e previsibilidade reduzem risco de longo prazo num ERP.
- **Stack secundária autorizada — Go 1.22+** para serviços de alta vazão / baixa latência / footprint reduzido: ingestão de telemetria de frota, endpoints de sincronização offline-first do **KCI**, gateways de borda e o plano de dados do **KCD**. Justificativa: goroutines, binários estáticos e consumo de memória mínimo em edge.
- **Exceções** exigem ADR aprovado. Node/NestJS e .NET permanecem como *fallback* estratégico documentado, não como padrão. `number` de JavaScript é explicitamente proibido para valores monetários.

A recomendação padroniza sem uniformizar à força: um ERP financeiro e um coletor de telemetria têm perfis distintos. A regra é **um padrão forte (Java/Spring) e um caminho de exceção estreito e justificado (Go)**.

5.2 Estilo arquitetural: Hexagonal + Clean

Cada serviço adota **Ports & Adapters (Hexagonal)** com as fronteiras da **Clean Architecture**. O núcleo (domínio + aplicação) é agnóstico de tecnologia; tudo que é "detalhe" (HTTP, banco, mensageria, provedor de identidade) é um **adaptador** conectado por meio de **portas** (interfaces).

As quatro camadas:

1. **Domínio (`domain`)** — o coração. Agregados, entidades, objetos de valor, eventos de domínio, serviços de domínio, exceções de negócio e **portas de saída** (interfaces de repositório). Zero dependência de framework.
2. **Aplicação (`application`)** — orquestra casos de uso (comandos/consultas), define **portas de entrada** (interfaces de caso de uso), coordena transações e publica eventos. Depende apenas do domínio.
3. **Infraestrutura (`infrastructure`)** — adaptadores de saída: implementações JPA de repositórios, publicador de eventos (Outbox), clientes HTTP, provedor de identidade, cache. Depende de domínio e aplicação (implementa as portas).
4. **Interface / Entrada (`interfaces`)** — adaptadores de entrada: controllers REST/gRPC, consumidores de mensageria, jobs agendados. Traduzem protocolo em comandos/consultas. Depende de aplicação.

Inversão de dependência (DIP): a aplicação declara `RepositorioDeOrdemServico` (porta); a infraestrutura fornece `RepositorioDeOrdemServicoJpa` (adaptador). A composição (injeção) ocorre no *bootstrap* (Spring `@Configuration`), o único ponto que conhece todas as camadas.

```

flowchart TB
    subgraph Entrada["Adaptadores de Entrada (interfaces)"]
        REST[Controller REST]
        MSG[Consumidor de Eventos]
        JOB[Job Agendado]
    end
    end
    subgraph Core["Núcleo agnóstico de tecnologia"]
        subgraph App["Aplicação (casos de uso)"]
            UC[Command/Query Handlers]
            PIN[[Porta de Entrada]]
            POUT[[Porta de Saída]]
        end
        end
        subgraph Dom["Domínio"]
            AGG[Agregados & VOs]
            DS[Serviços de Domínio]
            EV[Eventos de Domínio]
        end
        end
    end
    subgraph Saida["Adaptadores de Saída (infrastructure)"]
        JPA[Repositório JPA]
        OUT[Publicador Outbox]
        IDP[Cliente OIDC]
        CACHE[Cache]
    end
    end

    REST --> PIN
    MSG --> PIN
    JOB --> PIN
    PIN --> UC
    UC --> AGG
    UC --> DS
    AGG --> EV
    UC --> POUT
    POUT -.implementado por.-> JPA
    POUT -.implementado por.-> OUT
    POUT -.implementado por.-> IDP
    POUT -.implementado por.-> CACHE

    classDef core fill:#eef,stroke:#446;
    class App,Dom,UC,PIN,POUT,AGG,DS,EV core;

```

Regra da dependência (setas sempre apontam para o núcleo):

```

flowchart LR
    I[interfaces] --> A[application]
    INF[infrastructure] --> A
    INF --> D[domain]
    A --> D
    D --> nada["(nada – puro)"]
    classDef pure fill:#dfd,stroke:#282;
    class D pure;

```

5.3 Anatomia de um serviço (visão de execução)

```
flowchart LR
    Client([Cliente / App de Campo]) -->|HTTPS + JWT| GW[API Gateway]
    GW --> SVC
    subgraph SVC [Serviço de Domínio Kairós]
        F1[Filtro AuthN OIDC] --> F2[Resolvedor de Tenant]
        F2 --> F3[Filtro AuthZ RBAC/ABAC]
        F3 --> CTRL[Controller]
        CTRL --> HDL[Use Case Handler]
        HDL --> AGGR[Agregado]
        HDL --> REPO[(Repositório)]
        HDL --> OB[[Tabela Outbox]]
    end
    end
    REPO --> DB[(PostgreSQL por tenant)]
    OB --> REL[Relay Outbox] --> BROKER{{Broker de Eventos}}
    BROKER --> OTHER[Outros serviços / BI / KSI]
```

6. Diagramas

6.1 Mapa de contêineres lógicos do backend

```
flowchart TB
    subgraph Edge [Borda]
        GW[API Gateway / OIDC]
    end
    subgraph Dominios [Serviços de Domínio]
        ERP[ERP · Java/Spring]
        CRM[CRM · Java/Spring]
        FIN[Financeiro · Java/Spring]
        PUB[API Pública · Java/Spring]
        BI[BI · Java/Spring]
    end
    subgraph Proprietarios [Módulos Proprietários]
        KCI[KCI Sync · Go]
        KCD[KCD Zero Trust · Go]
        KSI[KSI Estoque · Java/Spring]
    end
    BROKER{{Event Broker}}
    IDP[(Provedor de Identidade OIDC)]

    GW --> ERP & CRM & FIN & PUB & BI & KCI & KSI
    GW -.valida token.-> IDP
    ERP <--> BROKER
    FIN <--> BROKER
    KSI <--> BROKER
    CRM <--> BROKER
    KCI <--> BROKER
    BI --> BROKER
    KCD -.políticas.-> GW
```

6.2 Diagrama de classes — padrões táticos DDD (exemplo: Ordem de Serviço)

```
classDiagram
class OrdemDeServico {
    <<AggregateRoot>>
    +OrdemId id
    +TenantId tenantId
    +DroneSN drone
    +StatusOS status
    +List-ItemOS~ itens
    +abrir(...)
    +adicionarPeca(...)
    +concluir(...)
    -garantirInvariantes()
}
class ItemOS {
    <<Entity>>
    +ItemId id
    +Quantidade qtd
}
class DroneSN {
    <<ValueObject>>
    +String serial
    +validar()
}
class Dinheiro {
    <<ValueObject>>
    +BigDecimal valor
    +Moeda moeda
    +somar(Dinheiro)
}
class OrdemConcluida {
    <<DomainEvent>>
    +OrdemId id
    +Instant em
}
class RepositorioOrdem {
    <<Port>>
    +salvar(OrdemDeServico)
    +porId(OrdemId, TenantId)
}
OrdemDeServico "1" *-- "n" ItemOS
OrdemDeServico --> DroneSN
ItemOS --> Dinheiro
OrdemDeServico ..> OrdemConcluida : emite
RepositorioOrdem ..> OrdemDeServico : persiste
```


6.3 Sequência — comando de escrita com Outbox

```
sequenceDiagram
    autonumber
    participant C as Controller
    participant H as ConcluirOSHHandler
    participant A as Agregado OrdemDeServico
    participant R as RepositorioOrdem
    participant O as OutboxRepository
    participant TX as Transação
    participant RL as Outbox Relay
    participant B as Broker

    C->>H: ConcluirOSCommand(tenant, osId, idemKey)
    activate TX
    H->>R: porId(osId, tenant)
    R-->>H: OrdemDeServico
    H->>A: concluir()
    A-->>H: evento OrdemConcluida
    H->>R: salvar(ordem)
    H->>O: gravar(evento) [mesma TX]
    deactivate TX
    Note over TX: commit atômico (estado + evento)
    RL->>O: ler pendentes
    RL->>B: publicar OrdemConcluida
    RL->>O: marcar publicado
```

7. Fluxogramas

7.1 Fluxo de uma requisição HTTP de escrita (borda → domínio)

```
flowchart TD
    A[Requisição HTTP + JWT] --> B{Assinatura/claims válidos?}
    B -- Não --> E401[401 Unauthorized]
    B -- Sim --> C[Resolver tenant do claim]
    C --> D{Tenant ativo?}
    D -- Não --> E403[403 Forbidden]
    D -- Sim --> V{Validação sintática do payload}
    V -- Falha --> E422[422 Problem Details]
    V -- OK --> Z{Idempotência: chave já vista?}
    Z -- Sim --> R200[Retorna resultado anterior]
    Z -- Não --> AUTHZ{RBAC/ABAC permite?}
    AUTHZ -- Não --> E403b[403 Forbidden]
    AUTHZ -- Sim --> UC[Executar caso de uso]
    UC --> INV{Invariantes de domínio OK?}
    INV -- Não --> E409[409/422 erro de negócio]
    INV -- OK --> TX[Persistir + gravar Outbox na mesma TX]
    TX --> OK[200/201 + Location]
```

7.2 Fluxo de tratamento de erro e resultado

```
flowchart TD
    S[Executa caso de uso] --> T{Tipo de falha}
    T -- Regra de negócio --> BN[DomainException]
    T -- Entrada inválida --> VE[ValidationException]
    T -- Não encontrado --> NF[NotFoundException]
    T -- Concorrência --> CC[OptimisticLockException]
    T -- Dependência externa --> DE[Resilience: retry/circuit]
    BN --> MAP[Handler Global @RestControllerAdvice]
    VE --> MAP
    NF --> MAP
    CC --> MAP
    DE -->|esgotou tentativas| MAP
    MAP --> PD[ProblemDetail RFC 7807 + traceId]
    PD --> LOG[Log estruturado sem PII/segredos]
    LOG --> RESP[Resposta padronizada ao cliente]
```

8. Boas Práticas

1. **Domínio primeiro, framework depois.** Modele agregados e invariantes antes de escolher anotações. O domínio deve compilar sem Spring.
2. **Objetos de valor para tudo que tem regra.** `DroneSN`, `Dinheiro`, `TenantId`, `CNPJ`, `Email` — imutáveis, auto-validados, sem *primitive obsession*.
3. **Agregados pequenos.** Uma transação = um agregado. Consistência entre agregados é *eventual*, via eventos de domínio.
4. **Casos de uso finos.** Um handler = uma intenção de negócio. Sem lógica de negócio em controller ou repositório.
5. **Fronteiras explícitas.** DTO na entrada/saída, comando/consulta na aplicação, agregado no domínio, entity JPA só na infraestrutura. Nunca cruze essas fronteiras com o mesmo objeto.
6. **Tenant é invariante de plataforma.** Toda consulta e escrita carrega tenant. Trate consulta sem tenant como bug de segurança.
7. **Idempotência por padrão em escritas.** Chave de idempotência no cabeçalho; dedupe persistido.
8. **Erros são de negócio ou técnicos.** Exceções de negócio são de primeira classe e mapeadas para respostas claras; exceções técnicas nunca vazam detalhes.
9. **Observabilidade nativa.** Logs estruturados (JSON) com `traceId`, `tenantId`, `userId`; métricas por caso de uso; *tracing* distribuído propagado nos eventos.
10. **Imutabilidade e funções puras no domínio.** Facilita teste e raciocínio; efeitos colaterais só nas bordas.
11. **Migrações versionadas** (Flyway/Liquibase). Nada de schema implícito por ORM em produção.
12. **Feature flags** para desacoplar deploy de release, especialmente em módulos regulatórios (fiscal).

9. Padrões

9.1 Padrões táticos de DDD

Agregado (raiz + invariantes). A raiz é a única porta de entrada de escrita; protege invariantes; emite eventos de domínio.

```
// domain/os/OrdemDeServico.java - sem qualquer dependência de framework
public final class OrdemDeServico {
    private final OrdemId id;
    private final TenantId tenantId;
    private final DroneSN drone;
    private StatusOS status;
    private final List<ItemOS> itens = new ArrayList<>();
    private final List<EventoDeDominio> eventos = new ArrayList<>();

    public static OrdemDeServico abrir(TenantId tenant, DroneSN drone) {
        var os = new OrdemDeServico(OrdemId.novo(), tenant, drone, StatusOS.ABERTA);
        os.registrar(new OrdemAberta(os.id, tenant, Instant.now()));
        return os;
    }

    public void concluir() {
        if (status != StatusOS.EM_EXECUCAO)
            throw new DomainException("OS só pode ser concluída a partir de EM_EXECUCAO");
        if (itens.isEmpty())
            throw new DomainException("OS sem itens não pode ser concluída");
        this.status = StatusOS.CONCLUIDA;
        registrar(new OrdemConcluida(id, tenantId, Instant.now()));
    }

    private void registrar(EventoDeDominio e) { eventos.add(e); }
    public List<EventoDeDominio> puxarEventos() {
        var copia = List.copyOf(eventos); eventos.clear(); return copia;
    }
    // getters de leitura; sem setters públicos
}
```

Objeto de Valor (imutável, auto-validado).

```
public record Dinheiro(BigDecimal valor, Moeda moeda) {
    public Dinheiro {
        Objects.requireNonNull(valor); Objects.requireNonNull(moeda);
        if (valor.scale() > moeda.casasDecimais())
            throw new DomainException("Escala inválida para " + moeda);
    }
    public Dinheiro somar(Dinheiro outro) {
        if (!moeda.equals(outro.moeda)) throw new DomainException("Moedas divergentes");
        return new Dinheiro(valor.add(outro.valor), moeda);
    }
}
```

Repositório como porta (interface no domínio, implementação na infraestrutura).

```
// domain/os/RepositorioOrdem.java (PORTA de saída)
public interface RepositorioOrdem {
    Optional<OrdemDeServico> porId(OrdemId id, TenantId tenant);
    void salvar(OrdemDeServico ordem);
}
```

```
// infrastructure/persistence/RepositorioOrdemJpa.java (ADAPTADOR)
@Repository
class RepositorioOrdemJpa implements RepositorioOrdem {
    private final JpaOrdemDao dao;
    private final OrdemMapper mapper;

    @Override public Optional<OrdemDeServico> porId(OrdemId id, TenantId tenant) {
        return dao.findByIdAndTenantId(id.valor(), tenant.valor()).map(mapper::paraDominio);
    }
    @Override public void salvar(OrdemDeServico ordem) {
        dao.save(mapper.paraEntity(ordem));
    }
}
```

Serviço de Domínio (lógica que não pertence a um único agregado — ex.: cálculo de garantia cruzando drone + peça + política).

```
public class PoliticaDeGarantia {
    public boolean cobre(DroneSN drone, Peca peca, LocalDate data) { /* regra pura */ }
}
```

Evento de Domínio — expressa fato consumado; publicado via Outbox (ver 9.5).

9.2 CQRS onde couber

CQRS é aplicado **seletivamente**, não como dogma. Comandos passam pelo agregado (consistência forte). Consultas de leitura pesada (BI, listagens, dashboards) usam **modelos de leitura** dedicados, projetados a partir de eventos, evitando sobrecarregar o modelo transacional.

```
flowchart LR
    subgraph Escrita[Lado Comando]
        CMD[Command] --> HDLC[Command Handler] --> AGGR[Agregado] --> WDB[(Modelo transacional)]
        AGGR --> EVT[Eventos]
    end
    subgraph Leitura[Lado Consulta]
        Q[Query] --> HDLQ[Query Handler] --> RDB[(Read Model)]
    end
    EVT --> PROJ[Projetor] --> RDB
```

Diretriz: usar CQRS *sem* event sourcing por padrão (read models atualizados por projetores). Event sourcing é considerado apenas onde a trilha de eventos é o requisito (ex.: rastreabilidade vitalícia por SN no KSI) — decisão por ADR.

9.3 Segurança

Autenticação (OIDC/JWT). O gateway e cada serviço validam o token (assinatura via JWKS, `iss`, `aud`, `exp`). O serviço confia no token, não em cabeçalhos arbitrários.

```
@Configuration
@EnableWebSecurity
class SecurityConfig {
    @Bean SecurityFilterChain chain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
            .authorizeHttpRequests(a -> a
                .requestMatchers("/actuator/health", "/actuator/info").permitAll()
                .anyRequest().authenticated() // deny-by-default
            )
            .oauth2ResourceServer(o -> o.jwt(j -> j.jwtAuthenticationConverter(conv())));
        http.addFilterAfter(new TenantContextFilter(), BearerTokenAuthenticationFilter.class);
        return http.build();
    }
}
```

Isolamento multiempresa (tenant no contexto de segurança). O `tenant_id` vem do claim, é colocado num contexto de escopo de requisição e aplicado automaticamente na persistência (ex.: filtro Hibernate por tenant / RLS no PostgreSQL).

```
public final class TenantContext {
    private static final ThreadLocal<TenantId> ATUAL = new ThreadLocal<>();
    public static void definir(TenantId t) { ATUAL.set(t); }
    public static TenantId obrigatorio() {
        var t = ATUAL.get();
        if (t == null) throw new SecurityException("Tenant não resolvido"); // R-T04
        return t;
    }
    public static void limpar() { ATUAL.remove(); }
}
```

```
// Filtro que resolve o tenant do JWT e ativa RLS na conexão
class TenantContextFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
        throws ServletException, IOException {
        var auth = (JwtAuthenticationToken) SecurityContextHolder.getContext().getAuthentication();
        var tenant = new TenantId(auth.getToken().getClaimAsString("tenant_id"));
        try {
            TenantContext.definir(tenant); // R-T02
            chain.doFilter(req, res);
        } finally { TenantContext.limpar(); }
    }
}
```

Persistência com RLS: cada conexão executa `SET app.tenant_id = ?` no início da transação; políticas Row-Level Security do PostgreSQL garantem que nenhuma linha de outro tenant seja lida/escrita, mesmo diante de um bug de aplicação — defesa em profundidade (R-T03).

Autorização (RBAC + ABAC). RBAC define papéis (`OS_TECNICO`, `FIN_APROVADOR`); ABAC refina por atributos (o recurso pertence ao tenant? o usuário é dono? valor abaixo do limite de alçada?).

```
@PreAuthorize("hasRole('FIN_APROVADOR')") // RBAC
public ResultadoAprovacao aprovar(AprovarPagamentoCommand cmd, Jwt jwt) {
    // ABAC contextual
    if (cmd.valor().valor().compareTo(alcada(jwt)) > 0)
        throw new AccessDeniedException("Valor acima da alçada do aprovador");
    ...
}
```

Validação de entrada em duas camadas. Sintática na borda (Bean Validation) + semântica/invariante no domínio (nunca só na borda).

```
public record AbrirOSRequest(
    @NotBlank String serialDrone,
    @NotNull @Size(min = 1) List<@Valid ItemRequest> itens) {}
```

9.4 Tratamento de erros e resultados

Erros de negócio são **explícitos**; a resposta é padronizada em **RFC 7807 (Problem Details)** com `traceId`. Duas abordagens coexistem: exceções de domínio (mapeadas por um handler global) e, em fluxos onde a falha é esperada e frequente, um tipo `Result<T>` para evitar controle de fluxo por exceção.

```
@RestControllerAdvice
class HandlerGlobalDeErros {
    @ExceptionHandler(DomainException.class)
    ProblemDetail negocio(DomainException ex) {
        var pd = ProblemDetail.forStatusAndDetail(HttpStatus.UNPROCESSABLE_ENTITY, ex.getMessage());
        pd.setType(URI.create("https://erros.kairos/negocio"));
        pd.setProperty("traceId", MDC.get("traceId"));
        return pd; // nunca expõe stack trace (R-C03)
    }
    @ExceptionHandler(NotFoundException.class)
    ProblemDetail naoEncontrado(NotFoundException ex) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
    }
    @ExceptionHandler(AccessDeniedException.class)
    ProblemDetail negado(AccessDeniedException ex) {
        return ProblemDetail.forStatus(HttpStatus.FORBIDDEN); // sem detalhar o porquê
    }
}
```

```
// Result<T> para falhas esperadas (ex.: importação em lote)
public sealed interface Result<T> permits Ok, Falha {
    record Ok<T>(T valor) implements Result<T> {}
    record Falha<T>(String codigo, String mensagem) implements Result<T> {}
}
```

9.5 Resiliência e transações (Outbox + Idempotência)

Transactional Outbox. O evento é gravado na **mesma transação** da mudança de estado. Um *relay* separado publica no broker e marca como enviado — garante que estado e evento nunca divergem (R-C01).

```
@Service
class ConcluirOSHandler {
    private final RepositorioOrdem repo;
    private final OutboxRepository outbox;

    @Transactional
    public void handle(ConcluirOSCommand cmd) {
        var tenant = TenantContext.obrigatorio();
        var os = repo.porId(cmd.osId(), tenant)
            .orElseThrow(() -> new NotFoundException("OS não encontrada"));
        os.concluir(); // invariantes no domínio
        repo.salvar(os);
        os.puxarEventos().forEach(e -> // mesma TX (R-C01)
            outbox.gravar(OutboxMessage.de(e, tenant)));
    }
}
```

```
flowchart LR
    TX["TX[estado + outbox]"] --> P["P[Tabela outbox: PENDENTE]"]
    P --> RELAY["RELAY[Relay poll/CDC]"]
    RELAY --> B["B[Broker]"]
    RELAY --> M["M[MARCA ENVIADO]"]
    B --> CONS["CONS[Consumidor idempotente]"]
    CONS --> DED["DED{Chave já processada?}"]
    DED -- Sim --> SKIP["SKIP[Ignora]"]
    DED -- Não --> APL["APL[Aplica + registra chave]"]
```

Idempotência. Escritas expostas aceitam [Idempotency-Key](#); consumidores de eventos usam a [messageId](#) como chave de dedupe persistida (R-C02).

```
@Transactional
public Resposta processar(String idempotencyKey, Comando cmd) {
    if (idem.existe(idempotencyKey))
        return idem.resultadoAnterior(idempotencyKey); // não reexecuta
    var resp = executar(cmd);
    idem.registrar(idempotencyKey, resp);
    return resp;
}
```

Resiliência de chamadas de saída (timeout + retry com backoff/jitter + circuit breaker):

```
@CircuitBreaker(name = "fiscalSefaz", fallbackMethod = "enfileirarParaReenvio")
@Retry(name = "fiscalSefaz")
@TimeLimiter(name = "fiscalSefaz")
public CompletableFuture<Autorizacao> transmitirNfe(Nfe nfe) { ... }
```

9.6 Padrões de teste

Pirâmide: muitos testes de unidade de domínio (sem infraestrutura), testes de caso de uso com dublês nas portas, e testes de integração com dependências reais via **Testcontainers**.

```

flowchart TB
    E2E[Contrato/E2E · poucos] --> INT[Integração Testcontainers · alguns]
    INT --> UC[Casos de uso · muitos]
    UC --> UNI[Unidade de domínio · muitíssimos]
    style UNI fill:#dfd,stroke:#282
    style E2E fill:#fdd,stroke:#822

```

```

// Teste de domínio: rápido, puro, sem Spring
class OrdemDeServicoTest {
    @Test void nao_conclui_sem_itens() {
        var os = OrdemDeServico.abrir(TenantId.de("t1"), DroneSN.de("SN-123"));
        assertThatThrownBy(os::concluir).isInstanceOf(DomainException.class);
    }
}

```

```

// Teste de integração com PostgreSQL real + RLS
@SpringBootTest
@Testcontainers
class RepositorioOrdemJpaIT {
    @Container static PostgreSQLContainer<?> pg = new PostgreSQLContainer<>("postgres:16");
    @Test void isola_por_tenant() { /* grava t1, consulta como t2, espera vazio */ }
}

```

Qualidade de código: análise estática (SpotBugs/Checkstyle/PMD), verificação da regra da dependência (ArchUnit), cobertura mínima (JaCoCo $\geq 80\%$ no núcleo), SAST/SCA no pipeline, revisão obrigatória por par.

```

// ArchUnit: proíbe domínio de depender de framework/infra (R-A01)
@ArchTest static final ArchRule dominio_puro =
    noClasses().that().resideInAPackage("..domain..")
        .should().dependOnClassesThat()
            .resideInAnyPackage("..infrastructure..", "org.springframework..", "jakarta.persistence..");

```

10. Casos de Uso

10.1 Concluir Ordem de Serviço (comando transacional + evento)

Ator: Técnico de campo (papel [OS_TECNICO](#)). **Pré:** OS em execução, tenant resolvido. **Fluxo:** requisição autenticada → tenant do claim → RBAC/ABAC → handler [ConcluiROSHandler](#) → agregado valida invariantes → persiste + grava Outbox na mesma TX → evento [OrdemConcluida](#) publicado → **KSI** dá baixa de peças, **Financeiro** gera faturamento. Consistência entre serviços é eventual, orquestrada por eventos.

10.2 Aprovar pagamento com alçada (RBAC + ABAC)

Ator: Aprovador financeiro ([FIN_APROVADOR](#)). Autorização em duas etapas: papel (RBAC) e limite de alçada + tenant do recurso (ABAC). Acima da alçada → 403 com Problem Details, sem vazar regra sensível.

10.3 Importação de telemetria de frota (stack Go, alta vazão)

Ator: Gateway de borda. Serviço em **Go** recebe lotes de telemetria, valida e persiste em store otimizado para série temporal, emitindo eventos agregados para o **BI**. Exemplo de exceção justificada à stack padrão (ADR): perfil de altíssima vazão/baixa latência, sem regra financeira.

10.4 Sincronização offline-first (KCI)

Ator: App de campo sem conectividade. Registra mutações locais com chave de idempotência; ao reconectar, envia o *log* de mutações ao serviço **KCI**; o backend aplica com dedupe e resolve conflitos por política (last-writer-wins por campo ou merge de domínio). Idempotência (R-C02) é o que torna o *replay* seguro.

10.5 Consulta de dashboard (CQRS — read model)

Ator: Gestor. Consulta atendida por **read model** projetado de eventos (não toca o modelo transacional), garantindo latência baixa em listagens e agregações do BI.

11. Modelagem (estrutura de camadas/pastas)

Estrutura de referência para um serviço Java/Spring, organizada por **camada e feature**. A regra da dependência é fisicamente visível nos pacotes.

```
servico-ordens/
├── build.gradle.kts
├── src/main/java/kairos/ordens/
│   ├── OrdensApplication.java           # bootstrap (composição)
│   ├── domain/                          # NÚCLEO — sem framework
│   │   ├── os/
│   │   │   ├── OrdemDeServico.java      # Aggregate Root
│   │   │   ├── ItemOS.java              # Entity
│   │   │   ├── StatusOS.java
│   │   │   ├── OrdemId.java              # Value Object (id)
│   │   │   ├── RepositorioOrdem.java     # PORTA de saída
│   │   │   └── eventos/OrdemConcluida.java
│   │   ├── shared/
│   │   │   ├── TenantId.java Dinheiro.java DroneSN.java
│   │   │   ├── DomainException.java EventoDeDominio.java
│   │   └── servico/PoliticaDeGarantia.java # Domain Service
│   ├── application/                      # CASOS DE USO
│   │   ├── os/
│   │   │   ├── ConcluirOSCommand.java
│   │   │   ├── ConcluirOSHandler.java    # implementa porta de entrada
│   │   │   └── port/in/ConcluirOSUseCase.java
│   │   └── port/out/OutboxRepository.java # PORTA de saída
│   ├── infrastructure/                   # ADAPTADORES DE SAÍDA
│   │   ├── persistence/
│   │   │   ├── RepositorioOrdemJpa.java
│   │   │   ├── OrdemEntity.java OrdemMapper.java
│   │   │   ├── OutboxEntity.java OutboxRepositoryJpa.java
│   │   │   └── rls/TenantConnectionCustomizer.java
│   │   ├── messaging/OutboxRelay.java
│   │   ├── security/SecurityConfig.java TenantContextFilter.java
│   │   └── config/BeansConfig.java        # injeção/composição
│   ├── interfaces/                       # ADAPTADORES DE ENTRADA
│   │   ├── rest/OrdemController.java dto/AbrirOSRequest.java
│   │   ├── rest/HandlerGlobalDeErros.java
│   │   └── messaging/OrdemEventListener.java
├── src/main/resources/
│   ├── application.yml
│   └── db/migration/V1_schema.sql V2_outbox.sql V3_rls_policies.sql
└── src/test/java/kairos/ordens/
    ├── domain/OrdemDeServicoTest.java    # unidade pura
    ├── application/ConcluirOSHandlerTest.java # portas dubladas
    └── infrastructure/RepositorioOrdemJpaIT.java # Testcontainers
```

Mapa camada → responsabilidade:

Pasta	Depende de	Contém	Não pode conter
<code>domain</code>	nada	agregados, VOs, eventos, portas de saída, serviços de domínio	Spring, JPA, HTTP
<code>application</code>	<code>domain</code>	comandos/consultas, handlers, portas de entrada	JPA, controllers
<code>infrastructure</code>	<code>domain</code> , <code>application</code>	repositórios JPA, Outbox relay, segurança, clientes	regra de negócio
<code>interfaces</code>	<code>application</code>	controllers, listeners, DTOs, mapeadores de erro	regra de negócio, JPA

12. Checklist

Arquitetura - [] Domínio compila sem framework; ArchUnit valida a regra da dependência. - [] Regra de negócio apenas em agregados/serviços de domínio; handlers finos. - [] Portas (interfaces) no núcleo; adaptadores na infraestrutura. - [] Sem vazamento de entities JPA para fora da infraestrutura.

Multiempresa - [] `tenant_id` resolvido do JWT, nunca do corpo/parâmetro. - [] Propagação de tenant no contexto (sync e assíncrono). - [] Filtro/RLS de tenant obrigatório na persistência; teste de isolamento verde.

Segurança - [] OIDC/JWT validado (assinatura, iss, aud, exp). - [] RBAC base + ABAC contextual; deny-by-default. - [] Validação sintática (borda) + invariantes (domínio). - [] Sem PII/segedos em log; erros sem stack trace.

Confiabilidade - [] Outbox na mesma transação da mudança de estado. - [] Idempotência em escritas e consumidores. - [] Problem Details (RFC 7807) com `traceld`. - [] Timeout + `retry(backoff/jitter)` + circuit breaker nas chamadas de saída.

Qualidade - [] Cobertura $\geq 80\%$ no núcleo; testes de domínio sem mocks de infra. - [] Integração com Testcontainers. - [] SAST/SCA e análise estática verdes; revisão por par. - [] Migrações versionadas (Flyway/Liquibase).

Observabilidade - [] Logs estruturados com `traceld/tenantId/userId`. - [] Métricas por caso de uso; tracing propagado nos eventos.

13. Riscos

ID	Risco	Prob.	Impacto	Mitigação
RB-01	Vazamento entre tenants por consulta sem filtro	Baixa	Crítico	RLS no PostgreSQL (defesa em profundidade) + teste de isolamento obrigatório (R-T03)
RB-02	Divergência entre estado e eventos publicados	Média	Alto	Transactional Outbox + relay idempotente (R-C01/R-C02)
RB-03	Erosão da arquitetura (domínio importando framework)	Média	Alto	ArchUnit no pipeline como quality gate bloqueante
RB-04	Precisão monetária incorreta	Baixa	Crítico	<code>BigDecimal</code> / <code>decimal</code> obrigatórios; <code>number</code> JS proibido; VO <code>Dinheiro</code>
RB-05	Escassez de talento sênior em stack secundária (Go)	Média	Médio	Restringir Go a casos justificados; padronizar libs; formação interna
RB-06	Excesso de CQRS/event sourcing sem necessidade	Média	Médio	CQRS seletivo; event sourcing só por ADR onde a trilha é requisito
RB-07	Reprocessamento não-idempotente corrompe estado	Média	Alto	Idempotency-Key + dedupe por messageId persistido
RB-08	Falha de dependência externa (SEFAZ/fiscal) derruba fluxo	Média	Alto	Circuit breaker + fila de reenvio + fallback assíncrono
RB-09	Segredos/PII em logs	Baixa	Alto	Mascaramento, revisão de logs, política R-S04
RB-10	Acoplamento síncrono excessivo entre serviços	Média	Médio	Preferir eventos; consistência eventual; contratos versionados

14. Melhorias Futuras

1. **Compilação nativa (GraalVM)** dos serviços de borda/edge para reduzir *cold start* e footprint, aproximando o custo operacional do Go sem trocar a linguagem.
2. **Spring Modulith** para reforçar limites de módulo e verificar dependências entre *bounded contexts* dentro de um mesmo processo antes de extrair microsserviços.
3. **Outbox via CDC** (Change Data Capture, ex.: Debezium) substituindo *polling*, reduzindo latência de publicação de eventos.
4. **SDK interno de plataforma** (starter Spring) encapsulando tenant, segurança, Outbox, idempotência e observabilidade — reduz *boilerplate* e padroniza por construção.
5. **Contract testing** (consumer-driven) entre produtores e consumidores de eventos, versionando esquemas em *schema registry*.
6. **Testes de segurança automatizados** de isolamento multiempresa como suíte dedicada no pipeline.
7. **Event sourcing seletivo** para a rastreabilidade vitalícia por SN (KSI), onde a trilha de eventos é o próprio ativo de negócio.
8. **Políticas OPA/Rego** para externalizar decisões ABAC complexas do KCD, versionadas como código.

15. Auditoria

Rastreabilidade de decisões (ADR): toda decisão estrutural deste documento é registrada como ADR versionado no repositório (</docs/adr/>), incluindo ADR-BE-001 (stack padrão Java/Spring + Go secundário) e ADRs de exceção por serviço.

Trilha de auditoria de execução: cada serviço emite eventos de auditoria (quem, o quê, quando, tenant) para ações sensíveis (aprovações financeiras, mudanças de configuração, acessos administrativos), com `traceId` correlacionável ao log estruturado e ao tracing distribuído. Registros de auditoria são imutáveis e retidos conforme política de compliance (Doc. Segurança).

Portas de qualidade verificáveis (quality gates): o pipeline bloqueia merge se qualquer item falhar — ArchUnit (regra da dependência), cobertura mínima, SAST/SCA, teste de isolamento de tenant, análise estática. O estado das portas é auditável por build.

Conformidade com o cânone: este documento não contradiz a Constituição (Doc. 01) nem o Project Bible (Doc. 02); reforça multiempresa de 1ª classe, segurança por padrão, contratos-primeiro e observabilidade.

Controle de versões do documento:

Versão	Data	Autor	Mudança
1.0	21/07/2026	Engenharia de Backend	Emissão inicial — padrão de backend, stack, arquitetura, DDD, segurança, resiliência e testes.

Fim do Documento 10 — Backend · Drone Kairós ERP.

11 Frontend

Documento: 11 – Frontend **Versão:** 1.0 · **Data:** 21 de julho de 2026 · **Status:** Ativo **Dependências:** 00, 01, 02, 03, 04, 05, 06, 07, 08, 09, 10 **Responsável:** Engenharia Frontend / UX

1. Resumo Executivo

Este documento define a **arquitetura, o design system e os padrões de engenharia frontend** dos painéis web do Drone Kairós ERP. O escopo cobre exclusivamente as **superfícies web** — **Painel do Fabricante, Painel da Revenda, Portal Administrativo e telas de BI**. Os aplicativos de campo (técnico, piloto, cliente) são **Flutter** e estão especificados em documento separado; este documento não os cobre, mas define os contratos de UX compartilhados (tokens, i18n, terminologia) que ambos os mundos consomem.

Recomendação de stack: **React 18+ com TypeScript estrito**, empacotado por **Vite**, roteado por **TanStack Router** (type-safe) e com **TanStack Query** para data fetching/cache de estado de servidor. A justificativa detalhada está na seção 5; em resumo, o React oferece o maior ecossistema enterprise, a melhor maturidade de TypeScript, a maior disponibilidade de talento no mercado brasileiro e a flexibilidade arquitetural que a natureza multiempresa e white-label do Kairós exige.

Pilares inegociáveis (herdados do Project Canon):

1. **Multiempresa de 1ª classe** — o **tenant** é um contexto de primeira ordem na UI: tema, dados, permissões e navegação derivam dele.
2. **Segurança por padrão** — rotas protegidas por papel, *nenhuma* decisão de autorização confiada apenas ao cliente, tokens de curta duração e isolamento de tenant explícito.
3. **i18n-ready** — nenhuma string *hard-coded*; pt-BR como padrão, en-US e es-ES como alvos imediatos.
4. **Acessibilidade** — meta **WCAG 2.2 nível AA** em todas as telas de produção.

O documento entrega diagramas Mermaid, exemplos de estrutura de componentes, o mapa de telas por perfil e um checklist de auditoria acionável.

2. Objetivos

1. **Recomendar e justificar** a stack web de forma defensável (React vs. Angular vs. Vue).
2. Especificar um **Design System** versionado — tokens, biblioteca de componentes, temas, white-label e padrões de UI.
3. Definir a **arquitetura frontend**: modularização por domínio, avaliação de micro-frontends, gerenciamento de estado, data fetching e cache.
4. Padronizar os **três painéis por perfil** (fabricante, revenda, admin) e o modelo de navegação.
5. Estabelecer o modelo de **autenticação/autorização na UI** com rotas protegidas por papel e contexto de tenant.
6. Garantir **acessibilidade (WCAG 2.2 AA)** e **internacionalização** por construção.
7. Definir metas e técnicas de **performance** (code splitting, SSR/CSR, orçamento de bundle).
8. Definir a **estratégia de testes** (unitário, componente, integração, E2E, visual, a11y).

3. Escopo

3.1 Dentro do escopo

- Painéis web SPA: Fabricante, Revenda, Portal Administrativo.
- Telas de BI embarcadas (dashboards, relatórios, exportações).
- Design System web (pacote `@kairos/ui`) e biblioteca de tokens (`@kairos/tokens`).
- Camada de autenticação/autorização de UI e o contexto de tenant.
- Estratégia de i18n, a11y, performance e testes das superfícies web.

3.2 Fora do escopo (referenciado, não detalhado)

Item	Onde é tratado
Apps de campo (técnico/piloto/cliente)	Documento Mobile/Flutter (separado)
Contratos de API, GraphQL/REST, paginação	Documento 08 — API/Backend
Modelo de identidade, papéis, escopos OAuth	Documento 07 — Segurança & IAM
Modelo de dados e SN (Serial Number)	Documento 06 — Domínio/Dados
Design visual de marca (logo, ilustração)	Guia de Marca

3.3 Perfis (personas) das superfícies web

- **Fabricante** — gestão de catálogo, produção, homologação, garantia, rede de revendas, BI de fabricação.
- **Revenda** — vendas, CRM, ordens de serviço, estoque local, faturamento, BI comercial.
- **Administrador da plataforma (Kairós)** — operação multiempresa, provisionamento de tenants, auditoria, feature flags, suporte, observabilidade de negócio.

4. Regras

Regras normativas. Palavras-chave conforme RFC 2119 (**DEVE**, **NÃO DEVE**, **PODE**).

#	Regra
R-01	Todo texto visível ao usuário DEVE vir da camada de i18n. Strings <i>hard-coded</i> NÃO DEVEM passar no CI (regra de lint).
R-02	Nenhuma tela DEVE renderizar dados de outro tenant. O <code>tenantId</code> DEVE ser derivado do token/sessão, nunca de parâmetro de URL manipulável sem revalidação no servidor.
R-03	Decisões de autorização na UI (ocultar/mostrar) são conveniência , não segurança. O backend DEVE reautorizar toda ação.
R-04	Todo componente interativo DEVE ser operável por teclado e ter nome acessível (WCAG 2.2 AA).
R-05	Todo acesso a dados de servidor DEVE passar pela camada TanStack Query (sem <code>fetch</code> solto em componentes).
R-06	Tokens de design DEVEM ser a única fonte de cor, espaçamento, tipografia e raio. Valores mágicos em CSS NÃO DEVEM ser usados.
R-07	Segredos, chaves e tokens de longa duração NÃO DEVEM residir no bundle nem no <code>localStorage</code> . Tokens de acesso DEVEM ser de curta duração; refresh via cookie <code>HttpOnly</code> + <code>SameSite</code> .
R-08	Toda rota de dados sensíveis DEVE ser <i>lazy-loaded</i> e protegida por <i>guard</i> de papel + tenant.
R-09	O orçamento de performance (seção 7) é um <i>gate</i> de CI. Regressões acima do limite DEVEM bloquear o merge.
R-10	Todo componente do Design System DEVE ter história (Storybook), teste de a11y e teste de snapshot visual antes de ser publicado.
R-11	Erros de rede/servidor DEVEM ser tratados com <i>error boundary</i> de rota e estado de UI explícito (loading/empty/error/success).
R-12	O tema do tenant (white-label) DEVE ser aplicado via CSS custom properties em tempo de execução, sem rebuild.

5. Arquitetura (Frontend)

5.1 Recomendação de stack — comparação

Avaliação dos três frameworks candidatos sob os critérios que importam para o Kairós: ecossistema enterprise, maturidade de TypeScript, adequação a multiempresa/white-label, disponibilidade de talento (BR) e flexibilidade arquitetural.

Critério (peso)	React 18+	Angular 17+	Vue 3
Ecossistema enterprise (25%)	Excelente — maior universo de libs (data grid, charts, forms, RBAC)	Muito bom — framework "baterias inclusas", opinativo	Bom — ecossistema sólido, menor que React
Maturidade TypeScript (20%)	Excelente — TS de 1ª classe, tipos maduros	Excelente — TS nativo/obrigatório	Muito bom — TS melhorou muito no v3
Multiempresa / white-label (15%)	Excelente — theming via context + CSS vars trivial	Bom — mais cerimônia (DI, módulos)	Muito bom — theming simples
Talento no mercado (BR) (15%)	Excelente — maior pool de contratação	Médio — pool menor, mais sênior	Médio — pool crescente, menor
Flexibilidade arquitetural (10%)	Excelente — não opinativo, compõe micro-frontends	Médio — opinativo, mais rígido	Muito bom — flexível
Curva de aprendizado (10%)	Boa	Íngreme	Suave
Performance / bundle (5%)	Muito boa (com Vite + code splitting)	Boa (bundle maior)	Muito boa
Veredito	Recomendado	Alternativa forte para times já Angular	Alternativa leve

Decisão: React 18+ com TypeScript estrito.

Justificativa resumida:

- **Ecossistema enterprise e contratação** — o React tem o maior conjunto de bibliotecas maduras para os problemas concretos do Kairós (data grids densos para estoque/produção, gráficos de BI, formulários complexos de garantia/OS, virtualização de listas de SN) e o maior pool de talentos no Brasil, reduzindo risco de projeto.
- **Flexibilidade para multiempresa/white-label** — theming em tempo de execução via Context + CSS custom properties é idiomático e barato; não exige rebuild por tenant.
- **Arquitetura evolutiva** — o React não nos amarra a uma opinião única; permite começar como monólito modular e extrair micro-frontends se e quando a escala exigir (seção 5.4).
- **Angular** seria a escolha se o time já fosse majoritariamente Angular (opinatividade reduz decisões), mas impõe mais cerimônia para white-label e tem pool de contratação menor. **Vue** é tecnicamente excelente e mais leve, porém com ecossistema enterprise e mercado de talentos menores — risco maior para um ERP de longo ciclo.

5.2 Stack de referência

Camada	Escolha	Papel
Linguagem	TypeScript (strict)	Segurança de tipos ponta a ponta
UI	React 18+	Biblioteca de renderização
Build/Dev	Vite	Bundler rápido, HMR, code splitting
Roteamento	TanStack Router	Rotas type-safe, guards, <i>lazy routes</i>
Estado de servidor	TanStack Query	Fetch, cache, revalidação, retry
Estado de cliente	Zustand	Estado global leve (UI, preferências)
Formulários	React Hook Form + Zod	Forms performáticos + validação/inferência de tipos
Estilo	CSS Modules + Design Tokens (CSS vars)	Escopo local + theming em runtime
Componentes base	Radix UI (headless)	Acessibilidade nativa, sem opinião visual
i18n	react-i18next + ICU MessageFormat	Traduções, pluralização, formatação
Data grid	TanStack Table	Tabelas densas, virtualizadas
Gráficos/BI	Visx / ECharts	Visualizações de BI
Testes	Vitest, Testing Library, Playwright, axe-core	Unit/componente/E2E/a11y
Documentação viva	Storybook	Catálogo do Design System

5.3 Arquitetura de aplicação — monólito modular por domínio

A recomendação de **partida** é um **monólito modular** (uma SPA por painel, ou uma SPA única com *feature areas* por perfil), organizado por **domínio de negócio**, não por tipo de arquivo. Isso maximiza velocidade inicial, coesão e simplicidade, mantendo fronteiras claras para futura extração.

```

graph TD
  subgraph Shell["App Shell (React + TanStack Router)"]
    AUTH["Auth & Tenant Context"]
    NAV["Navegação por perfil"]
    THEME["Theme Provider (white-label)"]
    I18N["i18n Provider"]
  end

  subgraph Domains["Módulos de domínio (feature-sliced)"]
    CAT["Catálogo & Produção<br/>(Fabricante)"]
    WAR["Garantia & Homologação"]
    SALES["Vendas & CRM<br/>(Revenda)"]
    OS["Ordens de Serviço"]
    INV["Estoque (KSI)"]
    ADMIN["Operação Multiempresa<br/>(Admin)"]
    BI["BI & Relatórios"]
  end

  subgraph Shared["Camada compartilhada"]
    UI["@kairos/ui (Design System)"]
    TOKENS["@kairos/tokens"]
    API["@kairos/api-client<br/>(TanStack Query hooks)"]
    UTILS["@kairos/utils (i18n, format, rbac)"]
  end

  Shell --> Domains
  Domains --> Shared
  API --> Backend["API Kairós (doc 08)"]

```

Estrutura de pastas (feature-sliced por domínio):

```

src/
  app/
    # shell: providers, router, layouts
    providers/
      AuthProvider.tsx
      TenantProvider.tsx
      ThemeProvider.tsx
      QueryProvider.tsx
    router/
      routes.tsx      # árvore de rotas type-safe
      guards.ts       # role + tenant guards
    layouts/
      ManufacturerLayout.tsx
      ResellerLayout.tsx
      AdminLayout.tsx
  domains/
    catalog/
      components/
      hooks/          # useProducts, useProductMutation (TanStack Query)
      api/            # endpoints do domínio
      types/
      routes.tsx      # sub-rotas lazy do domínio
      index.ts        # API pública do módulo (barrel controlado)
    warranty/
    sales/
    service-orders/
    inventory/
    admin/
    bi/
  shared/
    ui/              # re-export de @kairos/ui
    lib/             # rbac, format, i18n helpers
    config/          # feature flags, env
  main.tsx

```

Regras de fronteira: um domínio **PODE** importar de `shared/` e da sua própria pasta; **NÃO DEVE** importar do interior de outro domínio (só do `index.ts` público). Enforced por ESLint (`import/no-restricted-paths` + `boundaries`).

5.4 Micro-frontends — avaliação

Avaliamos micro-frontends (MFE) via Module Federation. **Decisão: não adotar no MVP; reavaliar por gatilho.**

Aspecto	Monólito modular (recomendado agora)	Micro-frontends
Velocidade inicial	Alta	Baixa (overhead de infra)
Complexidade operacional	Baixa	Alta (versionamento, contratos, orquestração)
Deploy independente por time	Não	Sim
Consistência de Design System	Trivial	Exige governança forte de versões
Adequação ao tamanho de time atual	Ótima	Excessiva

Gatilhos que justificariam extrair MFEs (qualquer um sustentado):

1. Mais de ~3 times de produto autônomos tocando o mesmo repositório com conflitos frequentes.
2. Necessidade de ciclos de release independentes por painel (ex.: BI evoluindo em cadência própria).
3. Um painel com stack divergente justificada.

Quando ocorrer, o candidato natural a primeira extração é **BI & Relatórios** (fronteira mais isolada) via **Module Federation** com o App Shell como host. Até lá, mantemos **um design system compartilhado** e **fronteiras de domínio disciplinadas** — o que torna a futura extração barata.

5.5 Gerenciamento de estado

Separação explícita entre **estado de servidor** e **estado de cliente** — a principal decisão de arquitetura de estado.

```
graph LR
  subgraph Server["Estado de servidor (TanStack Query)"]
    Q1["Cache por queryKey<br/>[tenantId, domínio, filtros]"]
    Q2["Revalidação / staleTime"]
    Q3["Mutations + invalidation"]
  end
  subgraph Client["Estado de cliente (Zustand)"]
    C1["UI: sidebar, modais, tema"]
    C2["Preferências do usuário"]
    C3["Wizard/rascunho local"]
  end
  subgraph Local["Estado local (useState/RHF)"]
    L1["Formulários (React Hook Form)"]
    L2["Estado efêmero de componente"]
  end
  Q1 -->|dados| UI[Componentes]
  C1 --> UI
  L1 --> UI
```

- **Estado de servidor** → **TanStack Query**. *Nunca* duplicar dados de servidor em store global. A **queryKey** **DEVE** incluir o **tenantId** como primeiro segmento para isolar cache entre tenants (evita vazamento de cache ao trocar de contexto).
- **Estado de cliente global** → **Zustand**. Apenas UI e preferências (tema selecionado, sidebar colapsada, locale).
- **Estado local** → **useState** / **React Hook Form**. Formulários e efêmeros.

5.6 Data fetching e cache

```
sequenceDiagram
    participant C as Componente
    participant Q as TanStack Query
    participant A as api-client
    participant BE as API Kairós
    C->>Q: useProducts({ status })
    Q-->>C: cache (stale?) → render imediato
    Q->>A: fetch (se stale/miss)
    A->>BE: GET /v1/products (Bearer + X-Tenant)
    BE-->>A: 200 + dados
    A-->>Q: normaliza + valida (Zod)
    Q-->>C: re-render com dados frescos
    Note over Q: invalidação em mutations relacionadas
```

Diretrizes:

- **queryKey** canônica: `[tenantId, 'domínio', 'entidade', params]`.
- **staleTime** por natureza do dado: catálogo (min. 5–10 min), dados operacionais/OS (30–60 s), BI (configurável).
- **Mutations** disparam `invalidateQueries` das chaves afetadas; usar *optimistic updates* onde o UX exige resposta instantânea (ex.: mudar status de OS).
- **Cliente HTTP** (`@kairos/api-client`) injeta `Authorization` e cabeçalho de tenant, trata refresh de token de forma transparente e valida payloads com `Zod` (defesa contra drift de contrato).
- **Paginação/virtualização** para listas densas (SNs, estoque): cursor-based + TanStack Virtual.

6. Diagramas

6.1 Contexto de sistema (C4 — nível 1)

```
graph TB
    Fab["Fabricante"] --> WEB
    Rev["Revenda"] --> WEB
    Adm["Admin Kairós"] --> WEB
    WEB["Painéis Web Kairós<br/>(React SPA)"] --> BFF["BFF / API Gateway"]
    BFF --> CORE["Serviços Core (doc 08)"]
    BFF --> IAM["IAM / Auth (doc 07)"]
    WEB -. tema/tenant .-> CFG["Config de Tenant"]
    Field["Apps Flutter (campo)"] --> BFF
```

6.2 Componentes do frontend (C4 — nível 3)

```
graph TD
    subgraph SPA["React SPA"]
        Router["TanStack Router"]
        Guards[" Guards (role+tenant)"]
        Providers["Providers: Auth · Tenant · Theme · i18n · Query"]
        Features["Domínios (feature-sliced)"]
        DS["Design System @kairos/ui"]
        ApiC["api-client (Query hooks + Zod)"]
    end
    Router --> Guards --> Features
    Providers --> Router
    Features --> DS
    Features --> ApiC
    ApiC --> Net["HTTP (Bearer + X-Tenant-Id)"]
```

6.3 Camadas de theming (white-label)

```
graph LR
  Base["Tokens base (@kairos/tokens)"] --> Semantic["Tokens semânticos<br/>(color.bg.surface, color.action.primary)"]
  Semantic --> TenantTheme["Override por tenant<br/>(CSS custom properties em runtime)"]
  TenantTheme --> Components["Componentes (@kairos/ui)"]
  Mode["Modo claro/escuro"] --> Semantic
```

7. Fluxogramas

7.1 Bootstrap da aplicação (auth + tenant + tema)

```
flowchart TD
  Start([Carrega SPA]) --> HasSession{Sessão válida?}
  HasSession -- Não --> Login[Redireciona /login]
  Login --> Auth[Autentica via IAM]
  Auth --> HasSession
  HasSession -- Sim --> LoadTenant[Resolve tenant do token]
  LoadTenant --> MultiTenant{Usuário tem >1 tenant?}
  MultiTenant -- Sim --> Picker[Tenant Switcher]
  MultiTenant -- Não --> ApplyTheme
  Picker --> ApplyTheme[Aplica tema white-label do tenant]
  ApplyTheme --> LoadPerms[Carrega papéis/permisões]
  LoadPerms --> Route[Resolve rota inicial por perfil]
  Route --> Render([Renderiza painel])
```

7.2 Guard de rota protegida

```
flowchart TD
  Nav([Navega para rota]) --> Authn{Autenticado?}
  Authn -- Não --> ToLogin[/login]
  Authn -- Sim --> Tenant{Tenant no contexto?}
  Tenant -- Não --> ToPicker[Tenant Switcher]
  Tenant -- Sim --> Role{Papel autorizado p/ rota?}
  Role -- Não --> Forbidden[403 - Acesso negado]
  Role -- Sim --> Lazy[Carrega chunk lazy do domínio]
  Lazy --> Boundary[Error/Suspense boundary]
  Boundary --> Show([Renderiza tela])
```

7.3 Ciclo de mutação com invalidação de cache

```
flowchart LR
  Action([Usuário confirma ação]) --> Validate[Zod valida form]
  Validate -- inválido --> Errors[Exibe erros i18n]
  Validate -- válido --> Optimistic[Update otimista opcional]
  Optimistic --> Mutate[mutation → API]
  Mutate -- erro --> Rollback[Rollback + toast erro]
  Mutate -- ok --> Invalidate[invalidateQueries afetadas]
  Invalidate --> Refetch[Refetch em background]
  Refetch --> Done([UI consistente])
```

8. Boas Práticas

8.1 Engenharia

- **TypeScript strict** em todo o repositório; **any** proibido por lint (exceções justificadas por comentário).
- **Composição sobre herança**; componentes pequenos e puros; lógica em hooks reutilizáveis.

- **Separação apresentação vs. dados** — componentes "burros" recebem props; hooks concentram acesso a dados.
- **Barreils controlados** (`index.ts`) como única superfície pública de cada domínio.
- **Feature flags** (`@kairos/config`) para lançamento gradual e kill-switch.
- **Error boundaries por rota** + estados de UI explícitos (loading/empty/error/success) em toda tela.

8.2 UX

- Estados vazios com ação de saída (nunca uma tela em branco).
- Feedback em ≤ 100 ms para toda interação (skeletons, otimismo).
- Formulários com validação inline, mensagens acionáveis e preservação de rascunho.
- Consistência de terminologia via glossário i18n compartilhado com o Flutter.

8.3 Segurança de UI

- Nunca confiar em ocultação de elementos como controle de acesso (R-03).
- Sanitizar toda renderização de HTML dinâmico; CSP estrita; sem `dangerouslySetInnerHTML` sem sanitização.
- Tokens de acesso em memória; refresh via cookie `HttpOnly`.
- Isolamento de cache por `tenantId` na `queryKey` (evita vazamento entre tenants).

9. Padrões (Design System + Código)

9.1 Design System — visão

O Design System Kairós é distribuído como **pacotes versionados** (SemVer), consumidos por todos os painéis web:

- `@kairos/tokens` — tokens de design (fonte única de verdade), gerados via **Style Dictionary** para CSS custom properties, TS e (export) para Flutter.
- `@kairos/ui` — biblioteca de componentes React (headless Radix + tokens).
- `@kairos/icons` — iconografia SVG.
- Documentação viva em **Storybook** (R-10).

9.2 Tokens de design (3 camadas)

```
graph LR
  A["1. Primitivos<br>color.blue.500 · space.4 · font.size.14"] --> B["2. Semânticos<br>color.action.primary · color.bg.surface · color.text.default"]
  B --> C["3. Componente<br>button.primary.bg · input.border.focus"]
  C --> D["Override por tenant (runtime)"]
```

Exemplo (tokens semânticos, formato agnóstico):

```
{
  "color": {
    "action": { "primary": { "value": "{color.brand.600}" } },
    "bg": { "surface": { "value": "{color.neutral.0}" },
           "muted": { "value": "{color.neutral.50}" } },
    "text": { "default": { "value": "{color.neutral.900}" },
             "inverse": { "value": "{color.neutral.0}" } },
    "feedback": { "danger": { "value": "{color.red.600}" },
                  "success": { "value": "{color.green.600}" } }
  },
  "space": { "4": { "value": "16px" } },
  "radius": { "md": { "value": "8px" } }
}
```

Renderizados como CSS custom properties consumidas por todos os componentes:

```
:root {
  --color-action-primary: #1E5EFF;
  --color-bg-surface: #FFFFFF;
  --color-text-default: #101828;
  --space-4: 16px;
  --radius-md: 8px;
}
/* Override por tenant, injetado em runtime — sem rebuild */
[data-tenant="acme-drones"] {
  --color-action-primary: #0B7A57; /* verde da marca do tenant */
}
/* Modo escuro */
[data-theme="dark"] {
  --color-bg-surface: #101828;
  --color-text-default: #F2F4F7;
}
```

9.3 Temas e white-label multiempresa

```
flowchart TD
  Login[Login] --> Resolve[Resolve tenant]
  Resolve --> Fetch[Fetch GET /tenant/:id/branding]
  Fetch --> Brand{Branding custom?}
  Brand -- Sim --> Inject["Injeta CSS vars<br/>data-tenant no root"]
  Brand -- Não --> Default["Tema Kairós padrão"]
  Inject --> Mode[Aplica modo claro/escuro]
  Default --> Mode
  Mode --> Render[Render com tema efetivo]
```

- **White-label por tenant:** logo, cor primária, cor de destaque, raio, densidade — todos como override de tokens semânticos aplicados em runtime via `data-tenant`.
- **Sem rebuild por tenant (R-12):** o branding é dado, não build artifact.
- **Modo claro/escuro** ortogonal ao white-label.
- **Fallback** garantido para o tema Kairós padrão.
- **Acessibilidade:** validação automática de contraste (AA) sobre a paleta do tenant no pipeline de onboarding — um branding que reprove no contraste é sinalizado.

9.4 Biblioteca de componentes

Organização por **atomic design** pragmático:

Nível	Exemplos
Primitivos	<code>Button</code> , <code>Input</code> , <code>Select</code> , <code>Checkbox</code> , <code>Badge</code> , <code>Icon</code>
Compostos	<code>FormField</code> , <code>DataTable</code> , <code>Combobox</code> , <code>DatePicker</code> , <code>Tabs</code> , <code>Dialog</code> , <code>Toast</code>
Padrões	<code>PageHeader</code> , <code>FilterBar</code> , <code>EntityTable</code> , <code>DetailDrawer</code> , <code>Wizard</code> , <code>StatCard</code>
Layouts	<code>AppShell</code> , <code>SidebarNav</code> , <code>TenantSwitcher</code> , <code>SplitView</code>

Anatomia de um componente do DS:

```
packages/ui/src/Button/
  Button.tsx           # implementação (Radix Slot + tokens)
  Button.types.ts      # props tipadas
  Button.module.css    # estilos via CSS vars (tokens)
  Button.stories.tsx   # Storybook (variantes, estados)
  Button.test.tsx      # Testing Library + axe (ally)
  index.ts             # export público
```

Exemplo (assinatura + uso):

```
// Button.types.ts
export interface ButtonProps
  extends React.ButtonHTMLAttributes<HTMLButtonElement> {
  variant?: 'primary' | 'secondary' | 'ghost' | 'danger';
  size?: 'sm' | 'md' | 'lg';
  loading?: boolean;
  iconStart?: React.ReactNode;
  asChild?: boolean; // composição via Radix Slot
}

// Uso em um domínio
<Button variant="primary" loading={isSubmitting} iconStart={<Icon name="save" />}>
  {t('common.actions.save')}
</Button>
```

9.5 Padrões de UI recorrentes

- **Tela de listagem (Entity List):** `PageHeader` + `FilterBar` + `EntityTable` (TanStack Table, virtualizada) + paginação + ações em massa. Estados loading/empty/error padronizados.
- **Tela de detalhe:** `DetailDrawer` ou `SplitView`, abas por aspecto (dados, histórico, garantia, SN).
- **Formulário/Wizard:** React Hook Form + Zod, validação inline, autosave de rascunho.
- **Dashboard/BI:** grid de `StatCard` + gráficos (Visx/ECharts) + filtros globais com sincronização em URL.
- **Sincronização de estado na URL:** filtros, paginação e abas persistem em query params (deep-link e compartilhamento).

10. Casos de Uso

CU-01 — Login e resolução de tenant (multiempresa)

Ator: qualquer usuário. Fluxo: autentica → sistema resolve tenant(s) do token → se múltiplos, `TenantSwitcher` → aplica tema white-label → carrega permissões → roteia ao painel do perfil. **Regra:** cache é isolado por `tenantId`; trocar de tenant limpa/reparticiona o cache (R-02).

CU-02 — Fabricante cadastra produto e vincula série (SN)

Ator: Fabricante. Fluxo: Catálogo → "Novo produto" (Wizard RHF+Zod) → geração/associação de faixa de SN → salva (mutation otimista) → invalida lista. **A11y:** wizard navegável por teclado, foco gerenciado entre passos.

CU-03 — Revenda abre Ordem de Serviço

Ator: Revenda. Fluxo: OS → "Nova OS" → busca cliente/equipamento por SN (combobox assíncrono) → registra defeito/garantia → status otimista → notifica. **Autorização:** rota protegida por papel `reseller.service`.

CU-04 — Admin provisiona novo tenant e branding

Ator: Admin Kairós. Fluxo: Portal Admin → Tenants → "Novo tenant" → configura branding (cores, logo) → **validação automática de contraste AA** → publica → tenant recebe tema em runtime sem deploy.

CU-05 — Análise de BI comercial

Ator: Revenda/Fabricante. Fluxo: BI → dashboard com filtros globais (período, região) sincronizados na URL → drill-down → exporta (CSV/PDF). **Performance:** dados via TanStack Query com `staleTime` configurável e virtualização de tabelas.

CU-06 — Usuário troca idioma

Ator: qualquer. Fluxo: preferências → seleciona locale (pt-BR/en-US/es-ES) → `react-i18next` recarrega bundle de mensagens (lazy) → formatação de datas/números/moeda segue o locale (Intl). Preferência persistida por usuário.

11. Modelagem — Mapa de Telas por Perfil

11.1 Painel do Fabricante

```
graph TD
  MF["Painel Fabricante"] --> MF1[Dashboard]
  MF --> MF2["Catálogo & Produtos"]
  MF2 --> MF2a[Lista de produtos]
  MF2 --> MF2b["Detalhe / SN / BOM"]
  MF --> MF3["Produção & Lotes"]
  MF --> MF4["Homologação"]
  MF --> MF5["Garantia & RMA"]
  MF --> MF6["Rede de Revendas"]
  MF --> MF7["BI de Fabricação"]
  MF --> MF8[Configurações]
```

11.2 Painel da Revenda

```
graph TD
  MR["Painel Revenda"] --> MR1[Dashboard]
  MR --> MR2["Vendas & CRM"]
  MR2 --> MR2a[Clientes]
  MR2 --> MR2b[Oportunidades]
  MR --> MR3["Ordens de Serviço"]
  MR --> MR4["Estoque local (KSI)"]
  MR --> MR5["Faturamento"]
  MR --> MR6["Garantia (abertura)"]
  MR --> MR7["BI Comercial"]
  MR --> MR8[Configurações]
```

11.3 Portal Administrativo

```
graph TD
  MA["Portal Admin"] --> MA1[Dashboard operacional]
  MA --> MA2["Tenants & Provisionamento"]
  MA2 --> MA2a["Branding / White-label"]
  MA --> MA3["Usuários & Papéis (IAM)"]
  MA --> MA4["Feature Flags"]
  MA --> MA5["Auditoria & Logs"]
  MA --> MA6["Suporte / Impersonação"]
  MA --> MA7["Observabilidade de negócio"]
  MA --> MA8["Configurações da plataforma"]
```

11.4 Matriz tela × perfil × permissão (extrato)

Área / Tela	Fabricante	Revenda	Admin	Papel(is) exigido(s)
Dashboard				*,viewer
Catálogo/Produtos	(CRUD)	(leitura)		manufacturer.catalog
Produção & Lotes				manufacturer.production
Vendas & CRM				reseller.sales
Ordens de Serviço				reseller.service
Estoque (KSI)				inventory.manage
Garantia/RMA	(gestão)	(abertura)		warranty.*
Tenants/Branding				admin.tenant
Feature Flags				admin.flags
Auditoria				admin.audit

Legenda: acesso pleno somente leitura sem acesso.

11.5 Modelo de navegação

- **Sidebar por perfil** (itens derivados de papéis + feature flags): itens sem permissão nem aparecem (R-03 — conveniência, revalidado no servidor).
- **Top bar:** `TenantSwitcher` (se multi-tenant), seletor de idioma, notificações, perfil.
- **Breadcrumbs** + deep-link por URL em todas as telas de listagem/detalhe.

12. Autenticação, Autorização e Contexto de Tenant na UI

Consolida a seção 5 do foco de autenticação/autorização. Fonte de verdade de identidade: **doc 07 — Segurança & IAM**.

12.1 Modelo

```
graph TD
  A[Login OIDC/OAuth2 PKCE] --> B[Access token curto em memória]
  A --> C["Refresh via cookie HttpOnly+SameSite"]
  B --> D[AuthProvider]
  D --> E[TenantProvider: tenantId do token]
  E --> F[PermissionProvider: papéis/escopos]
  F --> G[Route Guards]
  G --> H["role + tenant OK?"]
  H -- Sim --> I[Render rota lazy]
  H -- Não --> J[403 / redirect]
```


12.2 Rotas protegidas por papel (exemplo)

```
// router/guards.ts
export function requireRole(roles: Role[]): RouteGuard {
  return ({ context }) => {
    const { isAuthenticated, tenantId, hasAnyRole } = context.auth;
    if (!isAuthenticated) return redirect({ to: '/login' });
    if (!tenantId) return redirect({ to: '/select-tenant' });
    if (!hasAnyRole(roles)) return redirect({ to: '/403' });
  };
}

// routes.tsx – rota lazy protegida
const productionRoute = createRoute({
  path: '/production',
  beforeLoad: requireRole(['manufacturer.production']),
  component: lazy(() => import('@domains/catalog/routes/Production')),
});
```

12.3 Componente de autorização declarativa

```
// <Can> – conveniência de UI (não substitui autorização no servidor, R-03)
<Can role="reseller.service" fallback=<EmptyNoAccess />>
  <NewServiceOrderButton />
</Can>
```

12.4 Regras de tenant

- **tenantId** deriva do **token**; parâmetros de URL de tenant são revalidados no servidor (R-02).
- Troca de tenant **reparticiona o cache** (queryKeys prefixadas por **tenantId**).
- Ações de impersonação (Admin) exibem **banner persistente** e são auditadas.

13. Acessibilidade (WCAG) e Internacionalização

13.1 Acessibilidade — meta WCAG 2.2 AA

Diretriz	Implementação
Perceptível	Contraste AA (validado no pipeline, inclusive temas de tenant); texto alternativo; não depender só de cor
Operável	Navegação por teclado completa; foco visível; skip links; sem armadilhas de foco
Compreensível	Labels e erros associados; linguagem clara; ordem de leitura lógica
Robusto	HTML semântico + ARIA via Radix; nomes acessíveis em todo controle

- **Ferramentas:** Radix (a11y nativa), **axe-core** em testes de componente e E2E, **eslint-plugin-jsx-ally**, teste manual com leitor de tela nas telas críticas.
- **Gate de CI:** violações axe de severidade *serious/critical* bloqueiam merge (R-10).

13.2 Internacionalização

- **react-i18next** + **ICU MessageFormat** (pluralização, gênero, seleção).
- **Intl** para datas, números e moeda por locale.
- Locales-alvo: **pt-BR** (padrão), **en-US**, **es-ES**.
- **Bundles de tradução lazy** por locale e por domínio.
- **RTL-ready** por construção (uso de propriedades lógicas CSS — **margin-inline**, **padding-block**), mesmo sem idioma RTL imediato.
- **Sem strings hard-coded** (R-01, enforced por lint). Chaves namespaced: **domínio.contexto.chave**.

- **Glossário compartilhado** com o app Flutter para consistência de terminologia.

```
// pluralização + interpolação (ICU)
t('inventory.items.count', { count }); // "1 item" / "5 itens"
// formatação por locale
new Intl.NumberFormat(locale, { style: 'currency', currency: 'BRL' }).format(v);
```

14. Performance

14.1 Estratégia de renderização — CSR com SSR seletivo

Superfície	Estratégia	Justificativa
Painéis autenticados (Fabricante/Revenda/Admin)	CSR (SPA)	Apps ricos atrás de login; SEO irrelevante; interatividade alta
Páginas públicas (login, marketing, status)	SSR/SSG (se aplicável)	TTFB e SEO
BI pesado	CSR + virtualização + streaming de dados	Volume de dados

O Kairós é um **ERP autenticado**: SSR não traz SEO e adiciona complexidade de infra. Adotamos **CSR com Vite**, reservando SSR/SSG apenas para superfícies públicas. Caso um framework SSR (Next/Remix) seja adotado depois, a fronteira de domínio já preparada torna a migração incremental.

14.2 Técnicas

- **Code splitting por rota e por domínio** (lazy routes) — cada painel/domínio é um chunk separado.
- **Tree-shaking** e imports granulares (evitar barrels pesados no runtime).
- **Prefetch** de rotas prováveis (hover/idle) via TanStack Router.
- **Virtualização** de listas densas (SN, estoque) com TanStack Virtual.
- **Memoização** criteriosa e `React.lazy` / `Suspense`.
- **Imagens** responsivas + `loading="lazy"`; ícones como SVG sprite.
- **Cache HTTP** e revalidação alinhados ao `staleTime` do Query.

14.3 Orçamento de performance (gate de CI — R-09)

Métrica	Alvo
LCP (p75)	≤ 2,5 s
INP (p75)	≤ 200 ms
CLS	≤ 0,1
Bundle inicial (gzip) do shell	≤ 200 KB
Chunk de domínio (gzip)	≤ 150 KB
Time to Interactive (mid-tier)	≤ 3,5 s

Medição contínua via **Lighthouse CI** + **web-vitals** em RUM (Real User Monitoring), segmentado por tenant.

15. Testes de Frontend

15.1 Pirâmide de testes

```
graph TD
  E2E["E2E (Playwright)<br/>fluxos críticos por perfil – poucos"] --> INT
  INT["Integração / Componente<br/>(Testing Library) – muitos"] --> UNIT
  UNIT["Unitário (Vitest)<br/>hooks, utils, rbac, zod – base"]
  VIS["Visual (Storybook + snapshot)"] --> INT
  A11Y["A11y (axe-core)"] --> INT
```

Camada	Ferramenta	Cobre
Unitário	Vitest	Hooks, utils, guards RBAC, schemas Zod, tokens
Componente/Integração	Testing Library + MSW	Componentes do DS e telas com API mockada
Acessibilidade	axe-core (jest-axe)	Violações WCAG por componente/tela
Visual	Storybook + snapshot (Chromatic/Playwright)	Regressão visual e temas (inclui white-label)
E2E	Playwright	Fluxos críticos: login, tenant switch, CU-02..CU-05
Contrato	MSW + tipos do api-client	Alinhamento com contratos da API (doc 08)

15.2 Metas e gates

- Cobertura mínima: **80%** em `shared/` e domínios críticos (garantia, OS, tenant).
- **E2E** roda por perfil (fabricante/revenda/admin) contra ambiente efêmero com dados semeados.
- **a11y** e **visual** são gates de publicação do DS (R-10).
- **MSW** garante testes determinísticos sem depender do backend real.

15.3 Exemplo — teste de componente com a11y

```
test('Button é acessível e responde ao clique', async () => {
  const onClick = vi.fn();
  const { container } = render(
    <Button variant="primary" onClick={onClick}>{'Salvar'}</Button>
  );
  expect(await axe(container)).toHaveNoViolations();
  await userEvent.click(screen.getByRole('button', { name: 'Salvar' }));
  expect(onClick).toHaveBeenCalledTimes(1);
});
```

16. Checklist

Stack & arquitetura - [] React 18+ com TypeScript strict configurado - [] Vite + TanStack Router (rotas type-safe e lazy) - [] TanStack Query como única camada de estado de servidor (R-05) - [] Fronteiras de domínio enforced por lint (`boundaries`)

Design System - [] `@kairos/tokens` gerado (Style Dictionary), 3 camadas - [] `@kairos/ui` publicado com Storybook, testes de a11y e visuais (R-10) - [] White-label por tenant em runtime, com validação de contraste AA (R-12) - [] Modo claro/escuro

Auth & tenant - [] Guards por papel + tenant em todas as rotas sensíveis (R-08) - [] Tokens de acesso em memória; refresh via cookie HttpOnly (R-07) - [] Cache isolado por `tenantId` (R-02) - [] `<Can>` como conveniência; autorização revalidada no servidor (R-03)

A11y & i18n - [] Zero strings hard-coded (lint) (R-01) - [] pt-BR/en-US/es-ES com bundles lazy - [] axe sem violações serious/critical (gate) - [] Navegação por teclado e foco visível em todas as telas (R-04)

Performance - [] Code splitting por rota/domínio - [] Orçamento de bundle e Web Vitals como gate de CI (R-09) - [] RUM segmentado por tenant

Testes - [] Vitest + Testing Library + MSW + Playwright configurados - [] Cobertura ≥ 80% em shared/ e domínios críticos - [] E2E por perfil

17. Riscos

#	Risco	Prob.	Impacto	Mitigação
RK-01	Vazamento de cache entre tenants ao trocar de contexto	Média	Alto	<code>queryKey</code> prefixada por <code>tenantId</code> ; reparticionar cache no switch; teste E2E de troca de tenant
RK-02	Branding de tenant reprovando contraste (a11y)	Média	Médio	Validação automática de contraste AA no onboarding; fallback ao tema padrão
RK-03	Crescimento do bundle degradando performance	Alta	Médio	Orçamento como gate de CI; code splitting; auditoria periódica
RK-04	Drift entre contrato de API e frontend	Média	Alto	Validação Zod no api-client; testes de contrato; tipos gerados
RK-05	Adoção prematura de micro-frontends aumentando complexidade	Baixa	Alto	Gatilhos objetivos (5.4); manter monólito modular até atingi-los
RK-06	Autorização confiada apenas ao cliente	Baixa	Crítico	R-03 normativo; revalidação no servidor; testes de segurança
RK-07	Escassez de talento na stack	Baixa	Médio	React = maior pool no BR; padrões documentados; DS reduz curva
RK-08	Inconsistência de terminologia web × Flutter	Média	Médio	Glossário i18n compartilhado; tokens exportados para ambos
RK-09	Regressão visual não detectada	Média	Médio	Snapshots visuais (Storybook) como gate do DS

18. Melhorias Futuras

1. **Extração de micro-frontend de BI** via Module Federation quando os gatilhos da seção 5.4 forem atingidos.
2. **SSR/streaming seletivo** (Next/Remix) para superfícies públicas e first paint de dashboards pesados.
3. **Editor visual de branding** self-service para tenants (preview em tempo real + validação AA).
4. **Modo offline parcial** para painéis web (alinhado ao KCI offline-first do ecossistema).
5. **Design tokens dinâmicos por densidade** (compacto/confortável) e temas de alto contraste.
6. **Geração de tipos ponta a ponta** (OpenAPI/GraphQL → TS) para eliminar drift (RK-04).
7. **Teste de performance sintético por tenant** com dados representativos.
8. **Copiloto de UX** (busca/ações assistidas) sobre a camada de dados existente.

19. Auditoria

19.1 Rastreabilidade

Requisito do canon	Onde é atendido
Multiempresa 1ª classe	5.3, 5.5, 9.3, 12.4; regras R-02, R-12
Segurança por padrão	8.3, 12; regras R-02, R-03, R-07, R-08
i18n-ready	13.2; regra R-01
Acessibilidade	13.1; regras R-04, R-10
Painéis: Fabricante/Revenda/Admin + BI	5.3, 10, 11
Mobile Flutter fora de escopo (doc separado)	3.2

19.2 Critérios de aceite do documento

- [x] Stack recomendada com comparação justificada (React vs Angular vs Vue) — seção 5.1
- [x] Design System: tokens, componentes, temas, white-label — seção 9
- [x] Arquitetura: modularização, micro-frontends avaliados, estado, data fetching, cache — seção 5
- [x] Padrões dos painéis por perfil e navegação — seções 10, 11
- [x] Auth/authz na UI com rotas por papel e contexto de tenant — seção 12
- [x] Acessibilidade (WCAG) e i18n — seção 13
- [x] Performance (code splitting, SSR/CSR) — seção 14
- [x] Testes de frontend — seção 15
- [x] Diagramas Mermaid e exemplos de estrutura de componentes — seções 5, 6, 7, 9, 12, 15

19.3 Controle de versão

Versão	Data	Autor	Alteração
1.0	21 de julho de 2026	Engenharia Frontend / UX	Versão inicial aprovada

19.4 Cadência de revisão

- **Trimestral** ou a cada mudança relevante de stack/contrato de API.
- Revisores: Engenharia Frontend, UX, Segurança (seção 12), Arquitetura (seção 5).

Fim do documento 11 — Frontend · Drone Kairós ERP.

12 Mobile

Documento: 12 – Mobile **Versão:** 1.0 · **Data:** 21 de julho de 2026 · **Status:** Ativo **Dependências:** 00, 01, 02, 03, 04, 05, 06, 07, 08, 09, 10, 11 **Responsável:** Engenharia Mobile

1. Resumo Executivo

Este documento especifica a **camada mobile** do Drone Kairós ERP: dois aplicativos nativos construídos em **Flutter**, publicados a partir de um mesmo *monorepo* e compartilhando um núcleo de domínio comum, porém com escopos operacionais deliberadamente distintos.

- **App do Cliente** — canal de relacionamento e transparência. Permite ao cliente final acompanhar seus equipamentos por **Serial Number (SN)**, consultar o histórico de serviços, abrir solicitações, receber notificações de garantia e visualizar o ciclo de vida do ativo. Predominantemente *read-heavy*, com sincronização oportunista.
- **App do Técnico** — ferramenta de produção em campo. Executa **ordens de serviço (OS)**, aciona o **diagnóstico assistido pelo módulo KCI** (inteligência offline-first, capaz de rodar localmente no dispositivo), realiza **baixa de estoque via KSI**, captura evidências (foto, assinatura, código de barras/QR, RFID/NFC quando disponível) e opera de forma **offline-first**, pois a conectividade em campo é intermitente por natureza.

Os três pilares de projeto do Kairós são inegociáveis também no mobile: **offline-first onde crítico**, **segurança por padrão** (armazenamento seguro, biometria, tokens de curta duração) e **multiempresa** (isolamento de dados por *tenant* propagado até o dispositivo).

A recomendação arquitetural central deste documento é a adoção de **Riverpod** como solução de gerência de estado e injeção de dependências, sobre uma arquitetura em camadas (*Clean Architecture* pragmática) organizada por *features*, com um **motor de sincronização baseado em fila de operações (outbox)**, versionamento por vetor lógico e resolução de conflitos determinística. A justificativa comparativa contra BLoC encontra-se na Seção 5.2.

2. Objetivos

1. Definir uma **arquitetura Flutter única** e reaproveitável para os dois apps, minimizando divergência de código e custo de manutenção.
2. Garantir **operação offline-first** robusta no App do Técnico, com sincronização confiável, idempotente e auditável.
3. Especificar a **integração local com o módulo KCI** para diagnóstico assistido sem dependência de rede.
4. Padronizar o consumo de **recursos de campo** do dispositivo (câmera, código de barras/QR, OCR, RFID/NFC, geolocalização, assinatura).
5. Estabelecer a **postura de segurança no dispositivo**: armazenamento seguro, biometria, ciclo de vida de tokens e proteção multiempresa.
6. Definir a estratégia de **push notifications** transacionais e operacionais.
7. Delimitar com clareza o **escopo de cada app** (Cliente vs. Técnico) para evitar vazamento de responsabilidade.
8. Formalizar a estratégia de **testes automatizados e CI/CD mobile**.

3. Escopo

Dentro do escopo: - Arquitetura de aplicação Flutter (camadas, modularização, navegação, estado). - Persistência local, motor de sincronização e resolução de conflitos. - Integração *on-device* com KCI e baixa de estoque via KSI. - Recursos

de hardware de campo e captura de evidências. - Segurança do dispositivo e ciclo de vida de credenciais. - Notificações push e *deep links*. - Estratégia de testes e pipeline de CI/CD para Android e iOS.

Fora do escopo (tratado em outros documentos): - Contratos de API e *backend for frontend* — Doc. 08 (Integrações/API). - Especificação interna dos modelos e regras do KCI/KSI/KCD — Docs. de módulos proprietários. - Modelagem de dados canônica do servidor — Doc. 06. - Design system visual e *tokens* de UI — Doc. de Design (referência cruzada).

4. Regras

#	Regra	Racional
R1	Toda escrita crítica em campo passa primeiro pela fila local (outbox) e só depois sincroniza.	Garante offline-first e não perda de dados.
R2	Toda operação de escrita carrega um clientOperationId (UUID v4) gerado no dispositivo.	Idempotência de ponta a ponta; evita duplicidade em <i>retries</i> .
R3	Nenhum dado sensível trafega ou repousa fora do armazenamento seguro (Keychain/Keystore).	Segurança por padrão.
R4	O tenantId (empresa) é anexado a todo registro local e requisição.	Isolamento multiempresa até o dispositivo.
R5	Tokens de acesso têm vida curta; o refresh é silencioso e protegido por biometria após inatividade.	Reduz janela de exposição.
R6	O App do Cliente não executa baixa de estoque, diagnóstico KCI de execução, nem fechamento de OS.	Separação de escopo (Seção 7 do foco).
R7	Conflitos de sincronização seguem política determinística por tipo de entidade (Seção 7).	Consistência previsível e auditável.
R8	Todo evento relevante gera trilha de auditoria local espelhada ao servidor na sincronização.	Rastreabilidade (Seção 15).
R9	O binário é submetido a <i>checks</i> de integridade e <i>pinning</i> de certificado em produção.	Defesa contra MITM e adulteração.
R10	Recursos de hardware degradam graciosamente quando ausentes (ex.: sem NFC → fallback QR).	Heterogeneidade de aparelhos de campo.

5. Arquitetura (mobile)

5.1 Organização do monorepo e camadas

Adota-se um **monorepo** gerenciado com **melos**, com pacotes compartilhados e dois *apps runners*. O código de domínio e infraestrutura é comum; a divergência ocorre apenas na camada de apresentação e no conjunto de *features* habilitadas.

```
kairos_mobile/
├─ apps/
│  ├─ cliente/          # runner do App do Cliente
│  └─ tecnico/         # runner do App do Técnico
├─ packages/
│  ├─ core_domain/     # entidades, value objects, contratos de repositório
│  ├─ core_data/       # implementações: DB local, API, sync engine
│  ├─ core_security/   # secure storage, biometria, token manager
│  ├─ core_sync/       # outbox, conflict resolver, connectivity
│  ├─ core_ui/         # design system, widgets compartilhados
│  ├─ feature Equipamentos/
│  ├─ feature ordens Serviço/
│  ├─ feature kci diagnostico/
│  ├─ feature estoque ksi/
│  └─ feature captura_campo/ # câmera, barcode, OCR, NFC, assinatura, GPS
└─ melos.yaml
```

As camadas seguem uma **Clean Architecture pragmática** (sem cerimônia excessiva), com fluxo de dependência sempre apontando para dentro:

```
flowchart TB
    subgraph Apresentacao["Apresentação (por feature)"]
        UI["Widgets / Screens"]
        NOT["Notifiers (Riverpod)"]
    end
    subgraph Dominio["Domínio (core_domain)"]
        UC["Use Cases"]
        ENT["Entidades + Value Objects"]
        REPO["Contratos de Repositório"]
    end
    subgraph Dados["Dados (core_data / core_sync)"]
        REPOIMPL["Repositórios (impl)"]
        LOCAL["Fonte Local (Drift)"]
        REMOTE["Fonte Remota (Dio)"]
        SYNC["Sync Engine + Outbox"]
    end
    UI --> NOT --> UC --> REPO
    UC --> ENT
    REPO -.implementado por.-> REPOIMPL
    REPOIMPL --> LOCAL
    REPOIMPL --> REMOTE
    REPOIMPL --> SYNC
    SYNC --> LOCAL
    SYNC --> REMOTE
```

5.2 Decisão: Riverpod vs. BLoC

Ambas são soluções maduras. A recomendação é **Riverpod (v2, com geração de código)** pelos motivos abaixo, ponderados para o perfil do Kairós (offline-first, muitos estados assíncronos e derivados, injeção de dependências como cidadão de primeira classe).

Critério	Riverpod	BLoC
Injeção de dependência	Nativa (providers substituem <code>get_it</code>)	Requer solução externa
Estado assíncrono/derivado	<code>AsyncValue</code> + <code>select</code> de forma idiomática	Verboso; muitos estados manuais
Boilerplate	Baixo (com <code>riverpod_generator</code>)	Alto (events + states por caso)
Testabilidade	<code>ProviderContainer</code> + overrides triviais	Boa, porém mais cerimônia
Reatividade a conectividade/sync	Providers compostos e auto-invalidação	Streams manuais entre BLoCs
Curva para eventos disciplinados	Menos "trilhos" explícitos	Fluxo evento → estado muito explícito

Conclusão: Riverpod. A disciplina de eventos que o BLoC impõe é substituída por convenções de *Notifier* + *Use Cases*, e o ganho em composição de estado assíncrono é decisivo para a natureza offline-first. Onde um fluxo exigir

rastreabilidade de eventos forte (ex.: máquina de estados da OS), usa-se um *StateNotifier* modelado explicitamente como máquina de estados.

Exemplo de *Notifier* de sincronização com estado derivado:

```
part 'sync_controller.g.dart';

@riverpod
class SyncController extends _$SyncController {
  @override
  SyncState build() {
    // Observa conectividade e pendências da outbox de forma reativa.
    final online = ref.watch(connectivityProvider).valueOrNull ?? false;
    final pendentes = ref.watch(outboxCountProvider).valueOrNull ?? 0;
    return SyncState(
      online: online,
      pendentes: pendentes,
      fase: pendentes == 0 ? SyncFase.emDia : SyncFase.pendente,
    );
  }

  Future<void> sincronizarAgora() async {
    if (!state.online) return;
    state = state.copyWith(fase: SyncFase.sincronizando);
    final resultado = await ref.read(syncEngineProvider).drenarOutbox();
    state = state.copyWith(
      fase: resultado.temErro ? SyncFase.erro : SyncFase.emDia,
      ultimoErro: resultado.erro,
    );
  }
}
```

5.3 Navegação

Navegação declarativa com [go_router](#), com rotas tipadas, *guards* de autenticação/biometria e suporte a *deep links* (usados por push notifications). Cada *feature* expõe suas rotas; o app *runner* compõe a árvore.

```
final router = GoRouter(
  initialLocation: '/',
  redirect: (context, state) {
    final sessao = ref.read(sessaoProvider);
    if (!sessao.autenticado) return '/login';
    if (sessao.exigeBiometria) return '/desbloqueio';
    return null; // segue
  },
  routes: [
    GoRoute(path: '/login', builder: (_, __) => const LoginScreen()),
    GoRoute(path: '/desbloqueio', builder: (_, __) => const BiometriaScreen()),
    // Deep link de push: kairos://os/{id}
    GoRoute(
      path: '/os/:id',
      builder: (_, s) => OrdemServicoScreen(id: s.pathParameters['id']!),
    ),
  ],
);
```

6. Diagramas

6.1 Contexto dos dois apps

```
flowchart LR
    subgraph Dispositivos
        AC["App do Cliente\n(read-heavy)"]
        AT["App do Técnico\n(offline-first)"]
    end
    subgraph Backend["Backend Kairós"]
        BFF["API / BFF"]
        KCIsrv["KCI (servidor)"]
        KSIsrv["KSI (estoque)"]
        KCD["KCD (Zero Trust)"]
        DB["Núcleo de Dados"]
    end
    AC -->|HTTPS + token| BFF
    AT -->|HTTPS + token| BFF
    AT -. modelo empacotado .-> KCiloc["KCI local\n(on-device)"]
    BFF --> KCIsrv
    BFF --> KSIsrv
    BFF --> DB
    BFF --> KCD
    KCIsrv -->|atualização de modelo| AT
```

6.2 Camadas de dados no dispositivo (App do Técnico)

```
flowchart TB
    UIT["UI / Notifiers"] --> RepoT["Repositórios"]
    RepoT --> DBloc["Drift / SQLite\n(dados de trabalho)"]
    RepoT --> Outbox["Outbox\n(operações pendentes)"]
    RepoT --> Secure["Secure Storage\n(tokens, chaves)"]
    Outbox --> Engine["Sync Engine"]
    Engine -->|drena| APIrem["API Remota (Dio)"]
    Engine --> Resolver["Conflict Resolver"]
    KCIPkg["Pacote KCI local"] --> UIT
    KCIPkg --> DBloc
```

6.3 Máquina de estados da Ordem de Serviço

```
stateDiagram-v2
    [*] --> Atribuida
    Atribuida --> EmDeslocamento: técnico aceita
    EmDeslocamento --> EmExecucao: check-in (GPS)
    EmExecucao --> DiagnosticoKCI: solicita diagnóstico
    DiagnosticoKCI --> EmExecucao: laudo aplicado
    EmExecucao --> AguardandoPeca: KSI sem saldo local
    AguardandoPeca --> EmExecucao: peça disponibilizada
    EmExecucao --> Concluida: assinatura + evidências
    Concluida --> Sincronizada: outbox drenada
    Sincronizada --> [*]
```

7. Fluxogramas (sincronização offline)

7.1 Modelo de sincronização

O modelo é **offline-first com outbox transacional**. Toda mutação é gravada localmente dentro de uma transação que também enfileira uma operação na *outbox*. A UI é atualizada imediatamente (leitura otimista da base local). Um *engine* drena a fila quando há conectividade, com *backoff* exponencial e *jitter*.

```

flowchart TD
    A["Usuário executa ação\n(ex.: fecha OS)"] --> B["Transação local"]
    B --> C["Grava entidade em Drift"]
    B --> D["Enfileira operação na Outbox\n(clientOperationId, tenantId, versão)"]
    C --> E["UI atualiza (otimista)"]
    D --> F{"Há conectividade?"}
    F -- Não --> G["Aguarda\n(listener de conectividade)"]
    G --> F
    F -- Sim --> H["Sync Engine drena Outbox\n(ordem causal por entidade)"]
    H --> I{"Resposta do servidor"}
    I -- 2xx --> J["Marca operação como confirmada\naplica versão do servidor"]
    I -- 409 conflito --> K["Conflict Resolver\n(política por entidade)"]
    I -- 5xx / timeout --> L["Backoff exponencial + jitter\nreenfileira"]
    K --> M{"Resolução"}
    M -- auto --> J
    M -- requer humano --> N["Marca 'conflito pendente'\nnotifica usuário"]
    L --> F

```

7.2 Resolução de conflitos

Cada operação carrega uma **versão** (contador monotônico do servidor) e um `clientOperationId`. O servidor rejeita com **409** quando a versão base do cliente está defasada. A política é **por tipo de entidade**:

Entidade	Política	Racional
OS — campos de execução (notas, checklist)	<i>Merge</i> por campo (LWW por campo com timestamp)	Campos independentes raramente colidem.
OS — transição de estado	<i>Server-wins</i> + reprocessamento local	Estado é autoridade do servidor (regras de negócio).
Baixa de estoque (KSI)	<i>Append-only</i> idempotente por <code>clientOperationId</code>	Movimentos são fatos imutáveis; nunca sobrescrever.
Evidências (foto/assinatura)	<i>Append-only</i> (nunca conflita)	Anexos são aditivos.
Cadastro de cliente (App Cliente)	<i>Client-wins</i> com aprovação assíncrona	Cliente edita seus próprios dados.

Trecho ilustrativo do resolvidor:

```

Future<ResolucaoConflito> resolver(
    OperacaoPendente op,
    RespostaConflito servidor,
) async {
    switch (op.tipoEntidade) {
        case TipoEntidade.movimentoEstoque:
            // Idempotente: se o servidor já registrou este clientOperationId,
            // trata como sucesso (não reaplica).
            return servidor.jaProcessado(op.clientOperationId)
                ? ResolucaoConflito.confirmandoIdempotente
                : ResolucaoConflito.reenviar;

        case TipoEntidade.transicaoOS:
            // Server-wins: adota o estado do servidor e reprocessa localmente.
            await _repo.aplicarEstadoServidor(servidor.estadoAtual);
            return ResolucaoConflito.resolvidoServidor;

        case TipoEntidade.campoOS:
            // Merge por campo com Last-Write-Wins por timestamp de edição.
            final merged = _mesclarCampos(op.payload, servidor.payloadAtual);
            await _repo.aplicarMerge(merged);
            return ResolucaoConflito.mesclado;

        default:
            return ResolucaoConflito.exigeIntervencao;
    }
}

```

7.3 Estados de conexão

Modelam-se quatro estados expostos à UI, evitando o binário simplista *online/offline*:

```
stateDiagram-v2
    [*] --> Offline
    Offline --> Conectando: rede detectada
    Conectando --> Online: handshake + token válido
    Online --> Sincronizando: outbox não vazia
    Sincronizando --> Online: outbox drenada
    Online --> Offline: rede perdida
    Sincronizando --> Offline: rede perdida (retoma depois)
    Conectando --> Offline: falha de handshake
```

A UI reflete cada estado com um indicador não intrusivo (ex.: chip de status) e **nunca bloqueia** o técnico por ausência de rede.

8. Boas Práticas

- **Imutabilidade de modelos** com `freezed`; entidades de domínio sem dependência de framework.
- **Idempotência sempre**: nenhuma operação de escrita sem `clientId`.
- **Transações locais** englobando `escrita de dado + enfileiramento na outbox` (nunca uma sem a outra).
- **AsyncValue** para todo estado assíncrono, com tratamento explícito de `loading/error/data` na UI.
- **Feature-first**: cada *feature* é um pacote com seus próprios providers, rotas e testes.
- **Degradação graciosa de hardware**: checar disponibilidade de NFC/OCR antes de oferecer o recurso.
- **Observabilidade**: logs estruturados, *crash reporting* e métricas de sincronização (latência de drenagem, taxa de conflito).
- **Sem segredos no binário**: chaves e endpoints sensíveis via *secure storage*/configuração remota assinada.
- **Acessibilidade**: alvos de toque $\geq 48dp$, contraste adequado — relevante para uso em campo com luvas e sol.
- **Internacionalização** desde o início (`intl`), mesmo com pt-BR como *default*.

9. Padrões

9.1 Persistência local

Drift (SQLite tipado) como banco local, por oferecer *migrations*, consultas reativas (`Stream`) e transações. Tabelas centrais: `entidades_sincronizaveis`, `outbox`, `evidencias`, `trilha_auditoria`.

```
class Outbox extends Table {
    TextColumn get clientId => text(); // UUID v4 (PK)
    TextColumn get tenantId => text();
    TextColumn get tipoEntidade => text();
    TextColumn get entidadeId => text();
    IntColumn get versaoBase => integer();
    TextColumn get payloadJson => text();
    IntColumn get tentativas => integer().withDefault(const Constant(0));
    DateTimeColumn get criadoEm => dateTime();
    TextColumn get status => text().withDefault(const Constant('pendente'));

    @override
    Set<Column> get primaryKey => {clientId};
}
```

9.2 Cliente HTTP

Dio com *interceptors* para: injeção de token, `tenantId`, *retry* com *backoff*, *refresh* de token silencioso e **certificate pinning**. Erros de rede são classificados (recuperável vs. definitivo) para orientar a outbox.

9.3 Padrões de código

- **Result/Either** para erros de domínio (sem exceções para fluxo de controle esperado).
- **Repository** como fronteira; *Use Cases* orquestram regras; *Notifiers* apenas coordenam UI.
- **Convenção de nomes** em português no domínio de negócio (OS, estoque, equipamento) para alinhamento com o *ubiquitous language* do Kairós.

9.4 Integração com KCI local (App do Técnico)

O KCI é **offline-first** e **pode rodar localmente**. No dispositivo, o KCI é distribuído como um **pacote de inferência empacotado** (modelo + regras + base de conhecimento versionada), atualizável pelo servidor. O App do Técnico invoca o KCI para **diagnóstico assistido** sem depender de rede: dados do equipamento (SN, sintomas, leituras) entram, e o KCI devolve um **laudo sugerido** com hipóteses ranqueadas, peças prováveis e procedimento recomendado.

```
sequenceDiagram
    participant T as Técnico (UI)
    participant D as Diagnóstico (Notifier)
    participant K as KCI local
    participant DB as Base local
    participant KSI as KSI local (saldo)
    T->>D: Solicita diagnóstico (SN, sintomas)
    D->>DB: Carrega histórico do equipamento (SN)
    D->>K: Inferência local (sintomas + histórico)
    K-->>D: Laudo ranqueado + peças prováveis
    D->>KSI: Verifica saldo local das peças
    KSI-->>D: Disponibilidade
    D-->>T: Laudo + ações sugeridas (offline)
    Note over D,DB: Confirmação do técnico gera<br/>operações na outbox (auditável)
```

```
class KciDiagnosticoService {
    final KciEngineLocal _engine;          // motor de inferência empacotado
    final EquipamentoRepository _equip;

    Future<Laudo> diagnosticar({
        required String serialNumber,
        required List<Sintoma> sintomas,
        List<Leitura> leituras = const [],
    }) async {
        final historico = await _equip.historicoPorSerial(serialNumber);
        // Inferência 100% local – não requer conectividade.
        final resultado = await _engine.inferir(
            EntradaDiagnostico(
                serialNumber: serialNumber,
                sintomas: sintomas,
                leituras: leituras,
                historico: historico,
            ),
        );
        return Laudo(
            hipoteses: resultado.hipotesesRanqueadas,
            pecasProvaveis: resultado.pecas,
            procedimento: resultado.procedimentoSugerido,
            confianca: resultado.confianca,
            versaoModelo: _engine.versaoModelo, // para auditoria
        );
    }
}
```

Observação de governança: o laudo é **sugestão assistida**. A decisão e o fechamento da OS são sempre confirmados pelo técnico, e a `versaoModelo` é registrada na trilha de auditoria para rastreabilidade.

9.5 Recursos de campo

Padronizados na *feature* `feature_captura_campo`, com detecção de capacidade e *fallback*:

Recurso	Pacote/abordagem	Fallback
Câmera / foto de evidência	<code>camera</code> / <code>image_picker</code>	—
Código de barras / QR	<code>mobile_scanner</code>	Entrada manual do SN
OCR (placas, etiquetas)	<i>plugin OCR on-device</i>	Foto + digitação
RFID / NFC	<code>nfc_manager</code> (se disponível)	QR / código de barras
Geolocalização (check-in)	<code>geolocator</code>	Check-in manual com aviso
Assinatura do cliente	<code>signature</code> (canvas → PNG)	Foto de documento assinado

```
Future<String?> lerSerialNumber() async {
  // Preferência: NFC se o aparelho suportar; senão, escaneia QR/barras.
  if (await NfcManager.instance.isAvailable()) {
    final tag = await _lerTagNfc();
    if (tag != null) return tag.serial;
  }
  return _abrirLeitorCodigo(); // mobile_scanner
}
```

Todas as capturas viram **evidências append-only**, associadas à OS e enfileiradas para upload (nunca bloqueiam o fechamento offline).

9.6 Segurança no dispositivo

Consolidada em `core_security`, alinhada ao KCD (Zero Trust):

- **Armazenamento seguro** via `flutter_secure_storage` (Keychain no iOS, Keystore/EncryptedSharedPreferences no Android) para tokens, `tenantId` e chave de banco.
- **Banco local criptografado** (SQLCipher via Drift) com chave guardada no *secure storage*.
- **Biometria** (`local_auth`) exigida após inatividade e para operações sensíveis (ex.: liberar dados de outro cliente no App Cliente é bloqueado; no App Técnico, ações de estoque de alto valor).
- **Tokens de curta duração** com *refresh* silencioso; *logout* limpa segredos e apaga cache sensível.
- **Certificate pinning** e *checks* de integridade (root/jailbreak detection) em produção.

```
class TokenManager {
  final FlutterSecureStorage _cofre;

  Future<void> salvar(SessaoTokens t) async {
    await _cofre.write(key: 'access', value: t.access);
    await _cofre.write(key: 'refresh', value: t.refresh);
  }

  Future<String?> accessValido() async {
    final access = await _cofre.read(key: 'access');
    if (access == null || _expirado(access)) {
      return _refreshSilencioso(); // usa refresh; pode exigir biometria
    }
    return access;
  }

  Future<void> encerrar() async => _cofre.deleteAll();
}
```

9.7 Push notifications

Notificações push transacionais e operacionais via serviço de mensageria multiplataforma (FCM/APNs), com *routing* por *deep link* e respeito ao `tenantId`.

App	Exemplos de push
Cliente	Serviço concluído, garantia a vencer, atualização de status da solicitação, novo laudo disponível.
Técnico	Nova OS atribuída, reprogramação, peça liberada no KSI, atualização de modelo KCI disponível.

O *payload* traz um `deepLink` (ex.: `kairos://os/123`) e `tenantId`; o app valida o *tenant* ativo antes de navegar. Tokens de push são registrados por dispositivo/usuário e revogados no *logout*.

10. Casos de Uso (fluxo do técnico em campo)

Cenário: técnico atende uma OS de manutenção de drone em zona rural, **sem sinal de dados**.

```
sequenceDiagram
    participant T as Técnico
    participant App as App Técnico
    participant DB as Base local
    participant KCI as KCI local
    participant KSI as KSI local
    participant OB as Outbox
    T->>App: Abre OS atribuída (offline)
    App->>DB: Carrega OS + histórico do SN
    T->>App: Escaneia SN (QR/NFC)
    T->>App: Registra sintomas + fotos
    App->>KCI: Diagnóstico local
    KCI-->>App: Laudo (hipóteses + peças)
    T->>App: Confirma peça e substitui
    App->>KSI: Baixa de estoque (local)
    KSI-->>App: Saldo atualizado localmente
    App->>DB: Grava execução + evidências
    App->>OB: Enfileira operações (idempotentes)
    T->>App: Coleta assinatura do cliente
    T->>App: Fecha OS (offline)
    Note over App,OB: Ao recuperar rede, Sync Engine<br/>drena a Outbox e concilia versões
    App-->>T: Status "Sincronizado" quando concluído
```

Passos detalhados: 1. **Check-in** com geolocalização (armazenado localmente com timestamp). 2. **Identificação do equipamento** por SN (NFC → QR → manual). 3. **Diagnóstico KCI local**, retornando laudo e peças prováveis. 4. **Baixa de estoque KSI local** (movimento *append-only*, idempotente). 5. **Captura de evidências** (fotos, OCR de etiqueta, assinatura). 6. **Fechamento da OS offline** com transição de estado registrada. 7. **Sincronização diferida** quando a rede retorna, com conciliação de conflitos e confirmação de status.

Todas as etapas funcionam integralmente offline; a rede só é necessária para **propagar** o resultado.

11. Modelagem (módulos dos apps)

11.1 Módulos por app

```
flowchart LR
    subgraph Compartilhado["Núcleo compartilhado"]
        CD["core_domain"]
        CDA["core_data"]
        CS["core_security"]
        CSY["core_sync"]
        CUI["core_ui"]
    end
    subgraph Cliente["App do Cliente"]
        FE1["Meus Equipamentos (SN)"]
        FH1["Histórico de Serviços"]
        FS1["Solicitações"]
        FR1["Relacionamento / Notificações"]
    end
    subgraph Tecnico["App do Técnico"]
        F02["Ordens de Serviço"]
        FK2["Diagnóstico KCI"]
        FE2["Estoque KSI"]
        FC2["Captura de Campo"]
    end
    Compartilhado --> Cliente
    Compartilhado --> Tecnico
```

11.2 Escopo comparado (App Cliente vs. App Técnico)

Capacidade	App do Cliente	App do Técnico
Acompanhar equipamentos por SN	Sim (leitura)	Sim (contexto da OS)
Histórico de serviços	Sim (próprios)	Sim (do equipamento atendido)
Abrir solicitação/chamado	Sim	Não (recebe como OS)
Executar/fechar OS	Não	Sim
Diagnóstico KCI (execução)	Não	Sim (local)
Baixa de estoque KSI	Não	Sim
Captura (foto, assinatura, barcode, NFC, GPS)	Limitada (foto de solicitação)	Completa
Offline-first crítico	Parcial (leitura em cache)	Total
Notificações	Relacionamento/garantia	Operacionais
Perfil de dados	<i>Read-heavy</i>	<i>Write-heavy</i>

11.3 Entidade sincronizável (contrato base)

```
@freezed
class EntidadeSincronizavel with _$EntidadeSincronizavel {
  const factory EntidadeSincronizavel({
    required String id,
    required String tenantId,
    required int versao, // versão do servidor (conflito)
    required DateTime atualizadoEm,
    required EstadoSync estadoSync, // emDia | pendente | conflito
  }) = _EntidadeSincronizavel;
}

enum EstadoSync { emDia, pendente, conflito }
```


12. Checklist

Arquitetura - [] Monorepo `melos` com pacotes `core_*` e `features` isoladas. - [] Riverpod v2 com geração de código configurado. - [] `go_router` com `guards` de sessão/biometria e `deep links`.

Offline-first - [] Drift com SQLCipher e migrations versionadas. - [] Outbox transacional com `clientOperationId` em toda escrita. - [] Sync Engine com `backoff + jitter` e listener de conectividade. - [] Conflict Resolver por tipo de entidade implementado e testado. - [] Quatro estados de conexão refletidos na UI.

KCI / KSI - [] Pacote KCI local empacotado e versionado; atualização assinada. - [] `versaoModelo` registrada em auditoria. - [] Baixa KSI `append-only` idempotente.

Campo - [] Câmera, barcode/QR, OCR, NFC, GPS e assinatura com `fallbacks`. - [] Evidências `append-only` enfileiradas para upload.

Segurança - [] Secure storage para tokens/chave de DB. - [] Biometria por inatividade e operações sensíveis. - [] Certificate pinning e `root/jailbreak detection` em produção. - [] `tenantId` propagado em todo dado e requisição.

Push - [] Registro/revogação de token por dispositivo. - [] `Deep link` + validação de `tenant` antes de navegar.

Qualidade - [] Cobertura de testes por camada (Seção 14). - [] Pipeline CI/CD com `lint`, testes, `build` assinado e distribuição.

13. Riscos

#	Risco	Prob.	Impacto	Mitigação
RK1	Perda de dados de campo por falha antes da sincronização	Média	Alto	Outbox transacional + persistência imediata; nada só em memória.
RK2	Duplicidade de baixa de estoque em <code>retries</code>	Média	Alto	Idempotência por <code>clientOperationId</code> ; movimento <code>append-only</code> .
RK3	Conflitos de versão frequentes em OS colaborativas	Baixa	Médio	Merge por campo + server-wins em estado; alerta de conflito.
RK4	Modelo KCI local desatualizado gerando laudo pobre	Média	Médio	Atualização assinada e versionada; <code>versaoModelo</code> auditada; laudo é sugestão.
RK5	Heterogeneidade de aparelhos (sem NFC/OCR)	Alta	Baixo	Degradação graciosa com <code>fallbacks</code> .
RK6	Vazamento de dados multiempresa no dispositivo	Baixa	Crítico	<code>tenantId</code> obrigatório, DB criptografado, limpeza no <code>logout</code> .
RK7	Comprometimento de token em aparelho perdido	Média	Alto	Tokens curtos, biometria, revogação remota, <code>wipe</code> de segredos.
RK8	Crescimento da outbox em offline prolongado	Média	Médio	Compactação de operações, limites e alerta de backlog.
RK9	Divergência de código entre os dois apps	Média	Médio	Núcleo compartilhado + <code>features</code> ; CI que testa ambos os <code>runners</code> .

14. Melhorias Futuras

- **Sincronização delta/CRDT** para campos textuais colaborativos, reduzindo conflitos manuais.
- **Modo kiosk/rugged** otimizado para coletores industriais no App do Técnico.

- **Aprendizado incremental do KCI** com *feedback* do técnico (laudo aceito/rejeitado) enviado para reforço no servidor.
- **Realidade aumentada** para guiar procedimentos de reparo sobre a peça física.
- **Wear OS / watch companion** para notificações de OS.
- **Predição de peças** integrando KSI + histórico do SN para pré-carregar estoque no veículo.
- **Compartilhamento seguro de laudo** com o cliente via App do Cliente (*deep link*).

15. Auditoria

Toda ação relevante gera um **evento de auditoria local** (tabela `trilha_auditoria`), espelhado ao servidor na sincronização, garantindo rastreabilidade mesmo para operações realizadas offline.

Campos mínimos do evento:

Campo	Descrição
<code>eventoId</code>	UUID do evento.
<code>tenantId</code>	Empresa (isolamento multiempresa).
<code>usuarioId</code>	Autor da ação.
<code>tipoEvento</code>	Ex.: <code>os.fechada</code> , <code>estoque.baixa</code> , <code>kci.diagnostico</code> , <code>login.biometria</code> .
<code>entidadeRef</code>	SN / OS / movimento referenciado.
<code>ocorridoEm</code>	Timestamp do dispositivo (offline).
<code>sincronizadoEm</code>	Timestamp de conciliação no servidor.
<code>versaoModeloKci</code>	Quando aplicável, versão do modelo usado.
<code>hashCadeia</code>	Encadeamento (hash do evento anterior) para <i>tamper-evidence</i> .

```
Future<void> registrar(EventoAuditoria e) async {
  final anterior = await _repo.ultimoHash(e.tenantId);
  final comHash = e.copyWith(
    hashCadeia: sha256Hex('${anterior ?? ''}|${e.serialize()}' ),
  );
  await _repo.inserir(comHash); // local, append-only
  await _outbox.enfileirar(comHash); // espelha ao servidor
}
```

Princípios de auditoria:

- **Append-only local:** eventos nunca são editados ou apagados no dispositivo.
- **Encadeamento por hash** (*tamper-evidence*): adulteração quebra a cadeia.
- **Conciliação idempotente:** o servidor deduplica por `eventoId`.
- **Cobertura:** autenticação/biometria, transições de OS, movimentos de estoque, diagnósticos KCI, captura de evidências e eventos de sincronização.
- **Retenção e privacidade** seguem a política multiempresa; dados sensíveis auditados por referência, não por cópia.

Rastreabilidade documental: este documento depende dos Docs. 00–11 e é a fonte de verdade para a engenharia dos aplicativos móveis do Drone Kairós ERP. Alterações de escopo entre App do Cliente e App do Técnico exigem revisão conjunta com Produto e Segurança (KCD).

13 Kairós Core Intelligence (KCI)

Campo	Valor
Documento	13 — Kairós Core Intelligence (KCI)
Versão	1.0
Data	21 de julho de 2026
Status	Ativo
Dependências	00 a 12
Responsável	Especialista IA/ML

1. Resumo Executivo

O **Kairós Core Intelligence (KCI)** é o motor inteligente proprietário do **Drone Kairós ERP**, concebido para transformar o diagnóstico técnico de drones em um processo assistido, rastreável e reproduzível. O KCI é o cérebro analítico que suporta o técnico de campo e o técnico de bancada na jornada de reparo: da abertura da Ordem de Serviço (OS) ao encerramento com laudo técnico fundamentado.

O princípio fundador do KCI é ser **offline-first**. Todo o fluxo crítico de diagnóstico — reconhecimento de códigos (OCR), inspeção visual (Visão Computacional), raciocínio por regras (Sistema Especialista), previsão de falhas (Motor Preditivo), recuperação de conhecimento (RAG local sobre banco vetorial) e geração de laudos (Motor Generativo) — opera integralmente em ambiente local, sem dependência de conectividade. A integração com **IA externa é estritamente opcional**, acionada apenas como enriquecimento, sob guardrails, anonimização e com fallback determinístico garantido.

O KCI mantém um **histórico vitalício por Serial Number (SN)**: cada drone possui uma linha do tempo perpétua que agrega OS, telemetria, intervenções, peças substituídas, laudos e feedback de resultado. Essa memória longitudinal alimenta o Motor Preditivo e o ciclo de **Aprendizado (feedback loop)**, tornando o motor progressivamente mais preciso à medida que a base cresce.

O fluxo de referência do módulo é:

Diagnóstico → OCR → Visão Computacional → Sistema Especialista → Motor Preditivo → Motor Generativo → Knowledge Base → RAG Local → Aprendizado.

Os três pilares que governam toda decisão de projeto são: **offline-first** (autonomia operacional sem nuvem), **segurança por padrão** (dados protegidos em repouso e em trânsito) e **privacidade** (minimização, anonimização e controle do titular). O KCI adota **human-in-the-loop** como regra inegociável: nenhuma recomendação de alto impacto é executada sem validação humana explícita.

Este documento especifica visão, objetivos, escopo, governança de IA, arquitetura, pipelines, fluxogramas de diagnóstico, casos de uso, modelagem de dados, checklists, riscos, evolução e auditoria do KCI.

2. Objetivos

2.1 Objetivo Geral

Prover um motor de inteligência técnica **offline-first** capaz de assistir o diagnóstico, a manutenção preditiva e a geração de laudos de drones, elevando a taxa de acerto no primeiro reparo (*first-time-fix*) e reduzindo o tempo médio de diagnóstico, com total rastreabilidade e explicabilidade.

2.2 Objetivos Específicos

ID	Objetivo	Métrica de Sucesso (KPI)
OBJ-01	Assistir o diagnóstico técnico de drones com sugestões fundamentadas	Taxa de first-time-fix $\geq 85\%$
OBJ-02	Operar 100% do fluxo crítico offline	0 dependências externas obrigatórias no caminho crítico
OBJ-03	Reduzir tempo médio de diagnóstico	Redução $\geq 40\%$ vs. baseline manual
OBJ-04	Gerar laudos técnicos padronizados e auditáveis	100% dos laudos com trilha de evidências
OBJ-05	Antecipar falhas via manutenção preditiva	Antecipação ≥ 30 dias em componentes críticos
OBJ-06	Manter explicabilidade em toda recomendação	100% das recomendações com justificativa rastreável
OBJ-07	Aprender continuamente com o feedback do técnico	Melhoria mensurável de acurácia a cada ciclo de retreino
OBJ-08	Garantir privacidade e segurança por padrão	0 vazamentos; 100% dos dados sensíveis anonimizados antes de saída externa

2.3 Não-Objetivos (o que o KCI não é)

- Não é um sistema de decisão autônoma: **não executa reparos nem aprova ações críticas sozinho**.
- Não substitui o julgamento do técnico habilitado; **assiste**, não decide em seu lugar.
- Não depende de IA externa para funcionar; a IA externa é **enriquecimento opcional**.
- Não é um repositório de dados pessoais de clientes além do estritamente necessário à OS.

3. Escopo

3.1 Dentro do Escopo

- Motor de diagnóstico assistido com Sistema Especialista (regras e árvores de decisão).
- **OCR** para leitura de Serial Numbers, etiquetas, códigos de peça, QR/barras e telas de erro.
- **Visão Computacional** para inspeção de danos físicos, desgaste, corrosão, hélices, chassi e placas.
- **Motor Preditivo** de manutenção baseado em histórico vitalício por SN e telemetria.
- **Motor Generativo** local para laudos, recomendações e sumários de OS.
- **Knowledge Base** técnica versionada e curada.
- **RAG local** com banco vetorial embarcado para recuperação aumentada offline.
- **Ciclo de Aprendizado** com feedback do técnico e resultado do reparo.
- Integração com **App do Técnico**, **Ordens de Serviço** e **Telemetria**.
- Camada opcional de **IA externa** com guardrails, anonimização e fallback.

3.2 Fora do Escopo

- Controle de voo, pilotagem ou firmware embarcado do drone.
- Operação de e-commerce, faturamento e logística (tratados em módulos próprios).
- Treinamento de modelos de fundação do zero (o KCI usa modelos locais leves e ajustes incrementais).

3.3 Interfaces de Fronteira

Interface	Direção	Conteúdo
App do Técnico	Bidirecional	Captura de imagens/áudio/códigos, exibição de diagnóstico, coleta de feedback
Ordens de Serviço (OS)	Bidirecional	Contexto da OS, laudo gerado, ações recomendadas
Telemetria	Entrada	Séries temporais de voo, logs de erro, contadores de uso
Histórico por SN	Bidirecional	Linha do tempo vitalícia do ativo
IA Externa (opcional)	Saída/Entrada	Consulta anonimizada e resposta de enriquecimento

4. Regras (Governança de IA)

A governança de IA do KCI é normativa e vinculante. As regras abaixo têm precedência sobre qualquer otimização de desempenho ou conveniência.

4.1 Princípios Normativos

ID	Regra	Categoria
GOV-01	Offline-first é obrigatório: o caminho crítico nunca pode exigir conectividade	Disponibilidade
GOV-02	Toda recomendação deve ser explicável (evidências + regra/fonte rastreável)	Explicabilidade
GOV-03	Ações de alto impacto exigem human-in-the-loop (aprovação humana)	Controle
GOV-04	Nenhum dado sensível sai do ambiente local sem anonimização e consentimento	Privacidade
GOV-05	IA externa é opcional e sempre tem fallback local determinístico	Resiliência
GOV-06	Confiança (<i>confidence</i>) abaixo do limiar → escalar para humano, nunca autodecidir	Qualidade
GOV-07	Viés deve ser monitorado por segmentos (modelo de drone, fabricante, tipo de dano)	Equidade
GOV-08	Toda inferência é registrada em trilha de auditoria imutável	Auditabilidade
GOV-09	Versões de modelo, Knowledge Base e regras são versionadas e reproduzíveis	Governança
GOV-10	O técnico pode sempre sobrepôr (override) a sugestão do KCI, com registro	Autonomia humana

4.2 Limiares e Níveis de Confiança

Faixa de Confiança	Comportamento do KCI
≥ 0,90 (Alta)	Sugere com destaque; requer confirmação do técnico
0,70 – 0,89 (Média)	Sugere com alternativas ranqueadas; solicita validação explícita
0,50 – 0,69 (Baixa)	Apresenta hipóteses e pede mais evidências (nova foto, nova leitura)
< 0,50 (Insuficiente)	Não sugere diagnóstico; encaminha para análise humana / IA externa opcional

4.3 Matriz de Guardrails da IA Externa

Guardrail	Descrição
Anonimização	Remoção de SN real, dados de cliente e identificadores antes do envio; uso de pseudônimos
Minimização	Enviar apenas o mínimo necessário para o enriquecimento pretendido
Consentimento	Uso externo só ocorre com política/consentimento habilitado na OS
Fallback	Timeout ou indisponibilidade → volta ao resultado local sem degradar o fluxo
Validação de saída	Resposta externa é tratada como sugestão não confiável até validação por regras locais
Não-persistência sensível	Respostas externas não sobrescrevem a Knowledge Base sem curadoria humana

5. Arquitetura (do Motor)

5.1 Visão de Camadas

O KCI é organizado em cinco camadas lógicas, todas executáveis em ambiente local (edge/estação de reparo), com uma camada externa estritamente opcional.

```

flowchart TB
    subgraph L1["Camada de Captura"]
        A1[App do Técnico]
        A2[Sensores / Telemetria]
        A3[Câmera / Scanner]
    end
    subgraph L2["Camada de Percepção"]
        B1[OCR]
        B2[Visão Computacional]
    end
    subgraph L3["Camada de Raciocínio"]
        C1[Sistema Especialista]
        C2[Motor Preditivo]
        C3[RAG Local]
    end
    subgraph L4["Camada de Geração"]
        D1[Motor Generativo]
    end
    subgraph L5["Camada de Conhecimento e Dados"]
        E1[(Knowledge Base)]
        E2[(Banco Vetorial Local)]
        E3[(Histórico Vitalício por SN)]
    end
    subgraph LX["Camada Externa Opcional"]
        X1[{IA Externa com Guardrails}]
    end

    A1 --> B1
    A3 --> B1
    A3 --> B2
    A1 --> B2
    A2 --> C2
    B1 --> C1
    B2 --> C1
    C1 --> C3
    C3 --> E2
    C1 --> C2
    C2 --> E3
    C1 --> D1
    C2 --> D1
    C3 --> D1
    E1 --> E2
    D1 -. enriquecimento .-> X1
    X1 -. fallback local .-> D1
    D1 --> E3

```

5.2 Componentes do Fluxo de Referência

5.2.1 Knowledge Base (Base de Conhecimento Técnico)

Repositório curado e versionado do conhecimento de reparo: manuais, procedimentos, catálogos de peças, códigos de erro por fabricante/modelo, boletins técnicos, soluções validadas e laudos anteriores anonimizados. Estruturada em documentos atômicos ("cards de conhecimento") com metadados (modelo, componente, sintoma, versão, fonte, data de validade). É a fonte de verdade para o RAG e para a manutenção de regras do Sistema Especialista.

5.2.2 RAG Local (Recuperação Aumentada Offline)

Camada de recuperação semântica que indexa a Knowledge Base em um **banco vetorial local (embeddings)** embarcado. Dado um contexto de diagnóstico (sintomas + evidências), o RAG recupera os *cards* mais relevantes por similaridade vetorial, com re-ranqueamento por metadados (modelo/componente). O resultado alimenta o Motor Generativo com **grounding** verificável, reduzindo alucinação e garantindo citação de fonte. Opera 100% offline; o índice é atualizado em background quando a Knowledge Base muda.

5.2.3 Sistema Especialista (Regras / Árvores de Decisão)

Motor simbólico determinístico que aplica **regras de produção** e **árvores de decisão** para converter sintomas e evidências em hipóteses de diagnóstico ranqueadas. Encapsula o conhecimento heurístico do técnico sênior de forma auditável ("se código E13 + hélice trincada → suspeita de desbalanceamento do rotor"). É a espinha dorsal explicável do

KCI: cada conclusão aponta a cadeia de regras acionada. Prioritário sobre componentes probabilísticos quando há regra determinística aplicável.

5.2.4 OCR e Visão Computacional

- **OCR:** extrai texto estruturado de etiquetas, Serial Numbers, códigos de peça, QR/códigos de barras e telas de erro capturadas pelo App. Normaliza e valida SN contra o cadastro (checagem de formato e dígito).
- **Visão Computacional:** classifica e localiza danos em imagens — trincas, corrosão, superaquecimento, desgaste de hélice, conectores danificados, componentes ausentes. Retorna *bounding boxes*, rótulo de defeito e score de confiança. Modelos locais leves otimizados para inferência em estação/edge.

5.2.5 Motor Preditivo (Manutenção Preditiva)

Analisa o **histórico vitalício por SN** e a **telemetria** (horas de voo, ciclos de bateria, vibração, temperatura, contadores de erro) para estimar a probabilidade e a janela temporal de falhas futuras. Emprega modelos de séries temporais e detecção de anomalias, produzindo *health scores* por componente e alertas de manutenção antecipada. Realimenta a OS com recomendações preventivas.

5.2.6 Motor Generativo (Laudos / Recomendações)

Sintetiza, a partir do diagnóstico (Sistema Especialista), da previsão (Motor Preditivo) e do conhecimento recuperado (RAG local), um **laudo técnico estruturado** e recomendações de ação em linguagem natural clara. Opera **grounded** nas evidências recuperadas — cada afirmação do laudo referencia sua fonte (regra, card de KB, imagem, telemetria). Gera rascunho; o técnico revisa e aprova (human-in-the-loop).

5.2.7 Aprendizado (Feedback Loop)

Captura o desfecho real de cada OS (diagnóstico confirmado/refutado, peça que efetivamente resolveu, tempo de reparo, override do técnico) e o incorpora à Knowledge Base e aos conjuntos de treino. Fecha o ciclo: melhora regras, re-treina modelos de visão/predição e ajusta ranqueamento do RAG. Retreinos são versionados, avaliados e promovidos apenas após validação (ver Seção 15).

5.3 Tabela de Componentes

Componente	Tipo	Execução	Explicável	Entrada	Saída
Knowledge Base	Simbólico/Dados	Local	Sim	Curadoria	Cards de conhecimento
RAG Local	Vetorial/Semântico	Local	Sim (cita fonte)	Contexto + query	Trechos relevantes
Sistema Especialista	Simbólico	Local	Sim (cadeia de regras)	Sintomas/evidências	Hipóteses ranqueadas
OCR	Percepção	Local	Parcial	Imagem	Texto/código
Visão Computacional	Percepção	Local	Parcial (heatmap)	Imagem	Defeito + score
Motor Preditivo	Probabilístico	Local	Parcial (features)	Histórico/telemetria	Health score/janela
Motor Generativo	Generativo	Local	Sim (grounded)	Diagnóstico + RAG	Laudo/recomendação
Aprendizado	Pipeline	Local (batch)	Sim (versionado)	Feedback/desfecho	Modelos/regras v+1
IA Externa	Generativo	Externo (opcional)	Não confiável	Consulta anonimizada	Sugestão a validar

6. Diagramas (Pipeline)

6.1 Pipeline End-to-End de Diagnóstico

```
flowchart LR
    START([Abertura da OS]) --> DIAG[Diagnóstico Inicial]
    DIAG --> OCR[OCR: SN, códigos, erros]
    OCR --> SNCHK{SN válido?}
    SNCHK -- Não --> RECAP[Solicitar nova captura]
    RECAP --> OCR
    SNCHK -- Sim --> HIST[Carregar Histórico Vitalício por SN]
    HIST --> CV[Visão Computacional: inspeção de danos]
    CV --> SE[Sistema Especialista: regras/árvores]
    SE --> PRED[Motor Preditivo: risco futuro]
    PRED --> RAG[RAG Local: recuperar conhecimento]
    RAG --> CONF{Confiança >= limiar?}
    CONF -- Não --> EXT[IA Externa opcional / Escalar humano]
    EXT --> GEN
    CONF -- Sim --> GEN[Motor Generativo: laudo + recomendações]
    GEN --> HITL{Técnico aprova?}
    HITL -- Ajusta/Rejeita --> FEED[Registrar override]
    HITL -- Aprova --> OSUP[Atualizar OS + Laudo]
    OSUP --> FEED[Feedback Loop: desfecho]
    FEED --> LEARN[(Aprendizado: KB + retreino)]
    LEARN --> END([Encerramento])
```

6.2 Pipeline de Dados e Indexação (RAG / Knowledge Base)

```
flowchart TB
    KB[(Knowledge Base curada)] --> CHUNK[Fragmentação em cards atômicos]
    CHUNK --> EMB[Geração de embeddings local]
    EMB --> IDX[(Banco Vetorial Local)]
    Q[Contexto de diagnóstico] --> QEMB[Embedding da query]
    QEMB --> SRCH[Busca por similaridade]
    IDX --> SRCH
    SRCH --> RERANK[Re-ranqueamento por metadados]
    RERANK --> CTX[Contexto fundamentado]
    CTX --> GEN[Motor Generativo]
```

6.3 Decisão de Uso da IA Externa (Opcional)

```
flowchart TB
    R[Resultado local do KCI] --> C1{Confiança suficiente?}
    C1 -- Sim --> USE[Usar resultado local]
    C1 -- Não --> C2{Política externa habilitada + consentimento?}
    C2 -- Não --> ESC[Escalar para humano]
    C2 -- Sim --> C3{Conectividade disponível?}
    C3 -- Não --> FB[Fallback local + escalar humano]
    C3 -- Sim --> ANON[Anonimizar + minimizar dados]
    ANON --> CALL[Consultar IA Externa com timeout]
    CALL --> C4{Resposta válida no prazo?}
    C4 -- Não --> FB
    C4 -- Sim --> VALID[Validar saída por regras locais]
    VALID --> MERGE[Mesclar como sugestão a confirmar]
    MERGE --> USE
```

7. Fluxogramas (Diagnóstico)

7.1 Fluxo de Diagnóstico Assistido (Técnico)

```
flowchart TD
    A[Técnico abre OS no App] --> B[Captura foto da etiqueta/SN]
    B --> C[OCR lê e valida SN]
    C --> D[KCI carrega histórico vitalício]
    D --> E[Técnico descreve sintoma / captura imagens do defeito]
    E --> F[Visão Computacional classifica dano]
    F --> G[Sistema Especialista gera hipóteses ranqueadas]
    G --> H[RAG local anexa procedimentos e peças]
    H --> I[Motor Preditivo sinaliza riscos correlatos]
    I --> J[Motor Generativo redige laudo com evidências]
    J --> K{Técnico valida?}
    K -- Sim --> L[Executa reparo conforme laudo]
    K -- Não --> M[Ajusta hipótese / adiciona evidência]
    M --> G
    L --> N[Registra peça/ação que resolveu]
    N --> O[Feedback alimenta Aprendizado]
```

7.2 Árvore de Decisão do Sistema Especialista (Exemplo)

```
flowchart TD
    S[Sintoma reportado] --> Q1{Há código de erro via OCR?}
    Q1 -- Sim --> Q2{Código na Knowledge Base?}
    Q2 -- Sim --> R1[Aplicar regra do código: hipótese primária]
    Q2 -- Não --> Q3[Buscar similares via RAG]
    Q3 -- Sim --> Q4{Dano visível na Visão Computacional?}
    Q4 -- Sim --> Q5{Componente estrutural ou eletrônico?}
    Q5 -- Estrutural --> R2[Hipótese: chassi/hélice/conector]
    Q5 -- Eletrônico --> R3[Hipótese: placa/bateria/ESC]
    Q4 -- Não --> Q6{Telemetria com anomalia?}
    Q6 -- Sim --> R4[Hipótese preditiva por padrão anômalo]
    Q6 -- Não --> R5[Solicitar mais evidências]
    R1 --> RANK1[RANKEAR por confiança]
    R2 --> RANK2[RANK]
    R3 --> RANK3[RANK]
    R4 --> RANK4[RANK]
```

8. Boas Práticas

- **Grounding obrigatório:** o Motor Generativo só afirma o que pode citar (regra, card de KB, imagem ou telemetria). Sem fonte, não afirma.
- **Determinístico primeiro:** preferir o Sistema Especialista quando existir regra aplicável; usar componentes probabilísticos como complemento, não substituto.
- **Modelos locais leves:** priorizar modelos quantizados e otimizados para inferência em edge/estação, com orçamento de latência definido.
- **Confiança sempre visível:** exibir o score e a faixa de confiança ao técnico em toda sugestão.
- **Coleta de feedback de baixo atrito:** capturar desfecho com um toque (confirmou / não confirmou / qual peça resolveu).
- **Anonimização por padrão** antes de qualquer saída externa; nunca enviar SN real ou dados de cliente.
- **Fallback testado:** exercitar periodicamente o caminho offline e o fallback da IA externa.
- **Versionamento de tudo:** modelo, Knowledge Base, regras e índice vetorial versionados e reproduzíveis.
- **Idempotência de pipeline:** reprocessar a mesma evidência deve produzir o mesmo resultado (dado a mesma versão).
- **Observabilidade:** métricas de acurácia, latência, taxa de override e drift monitoradas continuamente.

9. Padrões

9.1 Padrões de Arquitetura e IA

Padrão	Aplicação no KCI
RAG (Retrieval-Augmented Generation)	Grounding do Motor Generativo sobre a Knowledge Base local
Sistema Especialista baseado em regras	Diagnóstico determinístico e explicável
Embeddings + Busca Vetorial	Recuperação semântica offline
Human-in-the-loop	Aprovação obrigatória em ações de alto impacto
Circuit Breaker / Fallback	Degradação graciosa quando IA externa falha
Event Sourcing	Histórico vitalício por SN como sequência de eventos
Feature Store	Consolidação de features de telemetria para o Motor Preditivo
Model Registry	Versionamento e promoção controlada de modelos
Data Minimization	Envio externo restrito ao mínimo anonimizado

9.2 Padrões de Interface (Contratos)

Contrato	Formato	Observação
Evento de captura	Estruturado (imagem + metadados)	Origem: App do Técnico
Resultado de percepção	Rótulo + score + região	OCR / Visão Computacional
Hipótese de diagnóstico	Lista ranqueada + cadeia de regras	Sistema Especialista
Laudo técnico	Documento estruturado + citações	Motor Generativo
Registro de feedback	Desfecho + peça resolutive + override	Aprendizado

9.3 Convenções

- Serial Number como chave primária do histórico vitalício.
- Cada inferência recebe um identificador de correlação para trilha de auditoria.
- Toda saída ao usuário inclui: conclusão, evidências, confiança e fonte.

10. Casos de Uso

10.1 UC-01 — Diagnóstico de Drone com Código de Erro

Ator: Técnico de bancada. **Pré-condição:** OS aberta. 1. Técnico fotografa a tela de erro; OCR extrai o código. 2. Sistema Especialista mapeia o código para hipótese primária. 3. RAG local anexa procedimento de reparo e peças compatíveis. 4. Motor Generativo redige laudo com passos e evidências. 5. Técnico valida, executa e registra desfecho.

10.2 UC-02 — Inspeção Visual de Dano Físico

Ator: Técnico de campo. 1. Técnico captura imagens do drone; Visão Computacional localiza trinca em hélice. 2. Sistema Especialista correlaciona com desbalanceamento de rotor. 3. Motor Preditivo verifica histórico de vibração do SN. 4. Laudo recomenda substituição e recalibração.

10.3 UC-03 — Manutenção Preditiva Antecipada

Ator: Gestor de manutenção. 1. Motor Preditivo detecta degradação de bateria por ciclos e temperatura. 2. Emite alerta com janela estimada de falha. 3. OS preventiva é sugerida antes da falha em campo.

10.4 UC-04 — Enriquecimento por IA Externa (Opcional)

Ator: Técnico. **Condição:** confiança local baixa + consentimento habilitado. 1. KCI anonimiza o contexto e consulta a IA externa com timeout. 2. Resposta é validada por regras locais e apresentada como sugestão a confirmar. 3. Em falha/timeout, fallback local + escalonamento humano.

10.5 UC-05 — Operação 100% Offline

Ator: Técnico em local sem conectividade. 1. Todo o fluxo (OCR → Visão → Especialista → Preditivo → RAG → Generativo) executa localmente. 2. Feedback é armazenado e sincronizado ao reconectar.

10.6 Tabela-Resumo

UC	Componentes-chave	Human-in-the-loop	IA Externa
UC-01	OCR, Especialista, RAG, Generativo	Sim	Não
UC-02	Visão, Especialista, Preditivo	Sim	Não
UC-03	Preditivo, Histórico SN	Sim (gestor)	Não
UC-04	Todos + externa	Sim	Sim (opcional)
UC-05	Todos locais	Sim	Não

11. Modelagem (Componentes + Dados)

11.1 Modelo de Componentes

```
flowchart LR
    APP[App do Técnico] --> KCI
    OS[Ordens de Serviço] <--> KCI
    TEL[Telemetria] --> KCI
    subgraph KCI[Kairós Core Intelligence]
        PERC[Percepção: OCR + Visão]
        REAS[Raciocínio: Especialista + Preditivo + RAG]
        GENC[Geração: Motor Generativo]
        LEARN[Aprendizado]
    end
    KCI <--> SN[(Histórico Vitalício por SN)]
    KCI --> KBC[(Knowledge Base)]
    KCI --> VEC[(Banco Vetorial)]
```

11.2 Entidades de Dados

Entidade	Descrição	Chave
Ativo (Drone)	Registro do equipamento	Serial Number
Evento de Histórico	Item da linha do tempo vitalícia	id_evento + SN
Ordem de Serviço	Contexto de reparo	id_OS
Captura	Imagem/áudio/código coletado	id_captura
Inferência	Resultado de um componente do KCI	id_inferência (correlação)
Card de Conhecimento	Unidade da Knowledge Base	id_card + versão
Vetor	Embedding de um card	id_vetor
Laudo	Documento gerado	id_laudo + id_OS
Feedback	Desfecho do reparo	id_feedback + id_OS
Modelo	Versão de modelo/regra em produção	id_modelo + versão

11.3 Modelo Entidade-Relacionamento (Simplificado)

```
erDiagram
    ATIVO ||--o{ EVENTO_HISTORICO : possui
    ATIVO ||--o{ ORDEM_SERVICO : gera
    ORDEM_SERVICO ||--o{ CAPTURA : contem
    CAPTURA ||--o{ INFERENCIA : produz
    INFERENCIA }o--o{ CARD_CONHECIMENTO : referencia
    CARD_CONHECIMENTO ||--|| VETOR : indexado_por
    ORDEM_SERVICO ||--|| LAUDO : encerra_com
    ORDEM_SERVICO ||--o{ FEEDBACK : recebe
    FEEDBACK }o--|| MODELO : realimenta
    ATIVO {
        string serial_number PK
        string modelo
        string fabricante
    }
    ORDEM_SERVICO {
        string id_OS PK
        string serial_number FK
        string status
    }
    LAUDO {
        string id_laudo PK
        string id_OS FK
        float confianca
    }
```

11.4 Ciclo de Vida do Dado no Aprendizado

```
flowchart LR
    FB[Feedback do reparo] --> CUR[Curadoria humana]
    CUR --> DS[Conjunto de treino versionado]
    DS --> TR[Treino/ajuste local]
    TR --> EVAL{Avaliação passa?}
    EVAL -- Não --> DESC[Descartar versão]
    EVAL -- Sim --> REG[(Model Registry)]
    REG --> PROMO[Promoção controlada]
    PROMO --> PROD[Produção]
    PROD --> FB
```

12. Checklist

12.1 Checklist de Prontidão (Go-Live)

- ☐ Fluxo crítico validado 100% offline (sem chamadas externas obrigatórias).
- ☐ OCR valida SN com checagem de formato e dígito.
- ☐ Visão Computacional calibrada para os defeitos-alvo com limiar de confiança definido.
- ☐ Sistema Especialista com regras versionadas e cadeia de explicação ativa.
- ☐ Motor Preditivo com features de telemetria consolidadas por SN.
- ☐ RAG local com índice vetorial atualizado e citação de fonte funcionando.
- ☐ Motor Generativo com grounding obrigatório e revisão humana no fluxo.
- ☐ Guardrails da IA externa (anonimização, minimização, fallback, timeout) testados.
- ☐ Trilha de auditoria imutável registrando toda inferência.
- ☐ Feedback loop coletando desfecho e alimentando retreino.

12.2 Checklist por Inferência

- ☐ Conclusão apresentada com confiança e faixa.
- ☐ Evidências e fontes anexadas.
- ☐ Confiança abaixo do limiar → escalonamento humano.
- ☐ Identificador de correlação registrado.
- ☐ Override do técnico habilitado e registrado.

12.3 Checklist de Privacidade e Segurança

- ☐ Dados sensíveis criptografados em repouso e em trânsito.
 - ☐ Anonimização antes de qualquer saída externa.
 - ☐ Consentimento verificado para uso externo.
 - ☐ Retenção mínima e política de expurgo aplicada.
 - ☐ Controle de acesso por papel (técnico, gestor, auditor).
-

13. Riscos

ID	Risco	Prob.	Impacto	Mitigação
R-01	Alucinação do Motor Generativo	Média	Alto	Grounding obrigatório via RAG + human-in-the-loop
R-02	Dependência indevida de IA externa	Baixa	Alto	Fallback local determinístico; externa opcional
R-03	Viés por sub-representação de modelos/danos	Média	Médio	Monitoramento por segmento + curadoria da KB
R-04	Drift de dados/telemetria	Média	Médio	Detecção de drift + retreino versionado
R-05	Vazamento de dados sensíveis	Baixa	Muito alto	Anonimização, criptografia, minimização
R-06	OCR/Visão com baixa qualidade de captura	Média	Médio	Reprocesso guiado + solicitação de nova captura
R-07	Excesso de confiança do técnico na sugestão	Média	Alto	Exibir confiança/limitações; override sempre disponível
R-08	Base de conhecimento desatualizada	Média	Médio	Versão e data de validade nos cards + revisão periódica
R-09	Latência inaceitável em edge	Baixa	Médio	Modelos leves/quantizados + orçamento de latência
R-10	Feedback loop enviesado (aprender erro)	Baixa	Alto	Curadoria humana antes de promover retreino

14. Melhorias Futuras

ID	Melhoria	Benefício Esperado
MF-01	Diagnóstico multimodal unificado (imagem + texto + telemetria)	Maior acurácia contextual
MF-02	Aprendizado federado entre estações preservando privacidade	Ganho coletivo sem centralizar dados sensíveis
MF-03	Explicações visuais (heatmaps) na inspeção de danos	Mais confiança e didática ao técnico
MF-04	Detecção proativa de anomalias em tempo quase real	Antecipação de falhas em campo
MF-05	Assistente conversacional offline de reparo	Suporte guiado passo a passo
MF-06	Auto-curadoria assistida da Knowledge Base	Redução do esforço de manutenção da KB
MF-07	Simulação de cenários de manutenção (what-if)	Melhor planejamento preventivo
MF-08	Calibração adaptativa de limiares por tipo de drone	Menos falsos positivos/negativos

15. Auditoria

15.1 Princípios de Auditoria

Toda decisão do KCI é **rastreável, reproduzível e revisável**. A trilha de auditoria é imutável e correlaciona captura, inferências, versões de modelo/KB/regras, confiança, fontes citadas, decisão humana (aprovação/override) e desfecho.

15.2 Itens Auditáveis

Item	O que é registrado
Inferência	Componente, entrada, saída, confiança, versão de modelo, correlação
Fonte	Cards de KB e evidências citadas no laudo
Decisão humana	Aprovação, ajuste ou override, com identificação e timestamp
IA externa	Se usada, dados anonimizados enviados, resposta, validação local
Retreino	Conjunto de dados, avaliação, promoção/descartes de versão
Privacidade	Consentimento, anonimização aplicada, acessos ao dado

15.3 Fluxo de Trilha de Auditoria

```
flowchart LR
    IN[Captura/Entrada] --> INF[Inferência do KCI]
    INF --> LOG[Log de Auditoria Imutável]
    HUM[Decisão Humana] --> LOG
    EXT[Uso de IA Externa] --> LOG
    OUT[Laudos/Recomendação] --> LOG
    FBK[Desfecho/Feedback] --> LOG
    LOG --> REV[Revisão / Conformidade]
    REV --> IMP[Melhoria de Governança]
```

15.4 Métricas de Governança Auditadas

Métrica	Objetivo
Taxa de override do técnico	Detectar desalinhamento entre KCI e prática
Acurácia por segmento	Monitorar viés por modelo/fabricante/dano
Taxa de grounding com fonte válida	Garantir explicabilidade
Taxa de fallback da IA externa	Medir resiliência offline
Drift de features de telemetria	Antecipar necessidade de retraining
Incidentes de privacidade	Meta: zero

15.5 Cadência de Revisão

- **Contínua:** métricas em tempo real (confiança, latência, override).
- **Mensal:** revisão de viés por segmento e qualidade da Knowledge Base.
- **Por retraining:** avaliação obrigatória antes de promoção de modelo.
- **Trimestral:** auditoria de privacidade e conformidade dos guardrails.

Nota de conformidade ao Project Canon: o KCI é offline-first, com IA externa estritamente opcional, sob guardrails, anonimização e fallback. O fluxo de referência (Diagnóstico → OCR → Visão Computacional → Sistema Especialista → Motor Preditivo → Motor Generativo → Knowledge Base → RAG Local → Aprendizado), o histórico vitalício por Serial Number e a integração com App do Técnico e Ordens de Serviço são respeitados integralmente. Princípios preservados: offline-first, segurança por padrão e privacidade.

14 Kairós Cyber Defense (KCD)

Documento: 14 – Kairós Cyber Defense (KCD) **Versão:** 1.0 · **Data:** 21 de julho de 2026 · **Status:** Ativo
Dependências: 00, 01, 02, 03, 04, 05, 06, 07, 08, 09, 10, 11, 12, 13 **Responsável:** Especialista em Cibersegurança (CISO/Arquiteto de Segurança)

1. Resumo Executivo

O **Kairós Cyber Defense (KCD)** é o módulo proprietário de cibersegurança do **Drone Kairós ERP** — não um acessório, mas uma camada de produto **projetada desde o design** (*security by design*) e integrada ao ciclo de vida completo da plataforma. O KCD materializa a doutrina **Zero Trust** do projeto: nenhuma rede, dispositivo, usuário, serviço ou requisição é confiável por origem; toda interação é **verificada continuamente**, autorizada sob **menor privilégio** e observada sob a premissa de **"assume breach"** (assuma que a violação já ocorreu).

O KCD atravessa o fluxo canônico do produto — **Segurança** → **Firewall** → **Criptografia** → **Auditoria** → **SOC** → **Zero Trust** → **Logs** → **Backups** → **Threat Detection** — e o converte em controles operacionais concretos: identidade forte (MFA, OIDC) e autorização granular (RBAC/ABAC); criptografia em repouso e em trânsito com gestão de chaves (KMS/HSM); **isolamento multiempresa** rígido por *tenant*; centro de operações de segurança (SOC) com **logging centralizado/SIEM**, detecção por regras e por anomalia, e resposta a incidentes; **backups, continuidade e Disaster Recovery** com RPO/RTO definidos; *hardening*, gestão de vulnerabilidades e **SDLC seguro** (SAST/DAST/SCA); e **auditoria contínua** com trilhas imutáveis para conformidade **LGPD/GDPR**.

Por operar um ERP que ingere **telemetria e ordens de serviço (OS) de drones**, o KCD adiciona controles específicos de **integridade e proveniência de dados** de dispositivos de campo: autenticação de dispositivo, assinatura de payload, anti-*replay* e validação de plausibilidade de telemetria. O resultado é uma postura defensiva **em profundidade** (*defense in depth*), auditável, alinhada a **NIST CSF 2.0**, **ISO/IEC 27001:2022** e **OWASP ASVS/SAMM**, e mensurável por indicadores (MTTD/MTTR, cobertura de MFA, conformidade de patch).

Os princípios inegociáveis do projeto sustentam todo o documento: **segurança por padrão**, **confiança é direito** (do cliente e do titular de dados) e **auditoria contínua**.

2. Objetivos

- **01.** Estabelecer o **modelo Zero Trust** operacional do produto: verificação contínua de identidade e postura, menor privilégio, microssegmentação e mentalidade *assume breach*.
- **02.** Definir a **gestão de identidade e acesso (IAM)**: autenticação forte (MFA, OIDC/OAuth2), autorização (RBAC + ABAC), ciclo de vida de identidades e gestão de segredos.
- **03.** Padronizar a **criptografia** em repouso e em trânsito e a **gestão de chaves** (hierarquia, rotação, custódia, envelope encryption).
- **04.** Garantir o **isolamento multiempresa** e a proteção de dados por *tenant* em todas as camadas (aplicação, dados, mensageria, observabilidade).
- **05.** Operacionalizar o **SOC**: logging centralizado/SIEM, detecção de ameaças (regras + anomalia), *threat intelligence* e resposta a incidentes.
- **06.** Definir **backups, continuidade de negócio (BCP) e Disaster Recovery (DR)** com metas de **RPO/RTO** por classe de dado.
- **07.** Institucionalizar **hardening, gestão de vulnerabilidades** e o **SDLC seguro** (SAST, DAST, SCA, revisão, *secrets scanning*).
- **08.** Assegurar **auditoria contínua e conformidade** (LGPD/GDPR) com **trilhas imutáveis** e evidências verificáveis.
- **09.** Proteger a **integridade dos dados de drones** (telemetria/OS): proveniência, assinatura, anti-*replay* e validação semântica.

- **O10.** Tornar a segurança **mensurável**: KPIs/KRIs, metas de SLA de segurança e melhoria contínua.

3. Escopo

No escopo: arquitetura de defesa em camadas do ERP; políticas de segurança; modelo Zero Trust; IAM (autenticação, autorização, identidades, segredos); criptografia e KMS; isolamento multiempresa; SOC/SIEM, detecção e resposta a incidentes; backup/BCP/DR; hardening e gestão de vulnerabilidades; SDLC seguro; auditoria e conformidade; controles específicos de telemetria/OS de drones.

Fora do escopo (referência a outros documentos): modelagem de dados por *bounded context* (Doc 06); arquitetura macro e microsserviços (Docs 07, 09); infraestrutura/IaC, topologia de clusters e rede física (Doc 08); pipelines de CI/CD e entrega (Docs de operação); firmware embarcado do drone (fora do perímetro do ERP — o KCD trata os dados **recebidos**, não o *firmware*). Aqui o KCD define **como** proteger, detectar, responder e comprovar; a implementação de cada módulo referenciado segue seu documento próprio.

Interfaces com outros documentos: consome os *bounded contexts* (Doc 06), a malha de serviços e o `tenant_id` obrigatório em todo request/evento (Doc 09), e a plataforma de infraestrutura/observabilidade (Doc 08).

4. Regras (Políticas de Segurança)

- **R1. Zero Trust por padrão.** Nenhuma requisição é autorizada por origem de rede; toda requisição é autenticada, autorizada e registrada. *Deny by default*.
- **R2. Menor privilégio.** Todo sujeito (usuário, serviço, *job*) recebe o mínimo de permissões necessário, com acesso *just-in-time* (JIT) e *just-enough-access* (JEA) para operações privilegiadas.
- **R3. MFA obrigatório.** Acesso humano a qualquer console administrativo, ambiente produtivo ou dado sensível exige **MFA resistente a phishing** (WebAuthn/FIDO2 preferencial).
- **R4. Tenant sempre presente e verificado.** Todo request, evento, log e registro de dado carrega `tenant_id`; o KCD **valida** o *tenant* em cada camada. Acesso *cross-tenant* é negado por padrão e auditado.
- **R5. Criptografia universal.** Dados em trânsito com **TLS 1.3** (mTLS interno); dados em repouso cifrados (AES-256-GCM); segredos nunca em texto claro, nunca no código.
- **R6. Segredos gerenciados.** Nenhum segredo em repositório, imagem, variável de ambiente estática ou log. Uso obrigatório do cofre (Vault/KMS) com rotação e *lease* curto.
- **R7. Tudo é logado, nada é adulterável.** Eventos de segurança e trilhas de auditoria são centralizados, correlacionados por `traceparent` e **imutáveis** (WORM/append-only, *hash chaining*).
- **R8. Assume breach.** Presume-se comprometimento: segmentação, contenção lateral, detecção contínua e *playbooks* de resposta são requisitos, não opções.
- **R9. Integridade de dados de campo.** Telemetria/OS de drones só é aceita com **dispositivo autenticado**, **payload assinado**, *nonce*/timestamp válido (*anti-replay*) e passando na validação de plausibilidade.
- **R10. Privacidade desde a concepção.** Minimização, finalidade, retenção limitada e base legal (LGPD/GDPR) são requisitos de design de qualquer feature que trate dado pessoal.
- **R11. Mudança segura.** Nenhum artefato vai a produção sem *gates* de segurança verdes (SAST/DAST/SCA, *secrets scan*, revisão) e *supply chain* verificada (SBOM, assinatura de imagem).
- **R12. Resposta com prazo.** Todo incidente tem classificação de severidade, SLA de resposta, responsável e registro pós-morte; violações de dados pessoais têm rito de notificação (ANPD/autoridade).
- **R13. Rotação e expiração.** Chaves, certificados, tokens e credenciais têm ciclo de vida com expiração e rotação automáticas; nada é "permanente".
- **R14. Verificabilidade.** Todo controle deve produzir **evidência auditável**; um controle sem evidência é considerado inexistente.

5. Arquitetura (Defesa em Camadas)

5.1 Princípio: Defense in Depth sob Zero Trust

O KCD organiza controles em **camadas concêntricas**, de modo que a falha de uma camada seja contida pela seguinte. Cada camada tem responsabilidade, controles e evidências próprias. O plano de controle Zero Trust (*Policy Decision Point* — PDP) é consultado por *Policy Enforcement Points* (PEP) distribuídos (gateway, service mesh, aplicação).

#	Camada	Responsabilidade nuclear	Controles KCD	Evidência
L0	Governança	Políticas, risco, conformidade	Políticas §4, matriz de risco §13, RACI	Política versionada, ata de risco
L1	Perímetro / Borda	Filtrar e proteger a borda	WAF, API Gateway, rate limit, DDoS, TLS 1.3	Logs de WAF/gateway
L2	Rede / Segmentação	Microsegmentação, mTLS	Service mesh, NetworkPolicy, <i>deny-all</i> base	Políticas de rede, telemetria mTLS
L3	Identidade (IAM)	Quem é e o que pode	OIDC, MFA, RBAC/ABAC, PDP/OPA	Logs de authN/authZ
L4	Aplicação	Código seguro, validação	Input validation, OWASP ASVS, CSRF/XSS/SQLi guards	Relatórios SAST/DAST
L5	Dados	Confidencialidade/integridade	Cripto em repouso, isolamento por <i>tenant</i> , mascaramento	Chaves KMS, testes de isolamento
L6	Segredos / Chaves	Custódia de material sensível	Vault/KMS, HSM, rotação, envelope encryption	Registro de rotação, ACLs do cofre
L7	Observabilidade / SOC	Detectar e responder	SIEM, regras+anomalia, SOAR, <i>playbooks</i>	Alertas, tickets, pós-mortes
L8	Continuidade	Sobreviver e restaurar	Backups, DR, RPO/RTO, testes de restore	Relatórios de restore/DR drill
L9	Borda de Campo (Drone)	Integridade de telemetria/OS	Auth de dispositivo, assinatura, anti-replay	Verificação de assinatura, quarentena

5.2 Planos de controle e execução (Zero Trust)

- **PDP (Policy Decision Point)**: motor central de decisão de autorização (ex.: OPA/Rego) que avalia identidade + postura + contexto (dispositivo, risco, hora, *tenant*) e retorna *permit/deny*.
- **PEP (Policy Enforcement Point)**: pontos que **impõem** a decisão — API Gateway (borda), sidecars da malha (leste-oeste), *middleware* da aplicação (nível de recurso/registro).
- **PIP (Policy Information Point)**: fontes de contexto — provedor de identidade (IdP/OIDC), *risk engine*, inventário de dispositivos, *threat intel*.
- **PAP (Policy Administration Point)**: onde políticas são escritas, versionadas e revisadas (GitOps de políticas).

5.3 Fluxo canônico do produto mapeado em controles

Etapa do fluxo canônico	Camada KCD	Controle principal
Segurança	L0/L3	Política + identidade
Firewall	L1/L2	WAF, segmentação, <i>deny-all</i>
Criptografia	L5/L6	TLS 1.3 + AES-256 + KMS
Auditoria	L7	Trilha imutável
SOC	L7	SIEM + SOAR
Zero Trust	L3 (PDP/PEP)	Verificação contínua
Logs	L7	Logging centralizado
Backups	L8	Backup + DR (RPO/RTO)
Threat Detection	L7/L9	Regras + anomalia + integridade de campo

6. Diagramas

6.1 Arquitetura de Defesa em Camadas

```
flowchart TB
    subgraph EXT["Mundo Externo"]
        U["Usuários / Parceiros"]
        D["Drones (Telemetria / OS)"]
        A["APIs de Terceiros"]
    end

    subgraph L1["L1 · Borda"]
        WAF["WAF + Anti-DDoS"]
        GW["API Gateway (PEP)\nTLS 1.3 · Rate Limit"]
        IG["Ingestão de Campo (PEP)\nAuth de Dispositivo"]
    end

    subgraph CTRL["Plano de Controle Zero Trust"]
        IDP["IdP / OIDC (PIP)"]
        PDP["PDP · OPA/Rego"]
        RISK["Risk Engine (PIP)"]
        VAULT["Vault / KMS / HSM"]
    end

    subgraph L2["L2/L3 · Malha + Identidade"]
        MESH["Service Mesh\nmTLS · deny-all"]
        SVC["Serviços de Domínio (PEP)\nRBAC/ABAC por recurso"]
    end

    subgraph L5["L5/L6 · Dados + Segredos"]
        DB["Dados por Tenant\nAES-256 em repouso"]
        SEC["Segredos (envelope enc.)"]
    end

    subgraph L7["L7 · SOC"]
        SIEM["SIEM / Logs imutáveis"]
        DET["Detecção: Regras + Anomalia"]
        SOAR["SOAR / Playbooks"]
    end

    subgraph L8["L8 · Continuidade"]
        BKP["Backups cifrados"]
        DR["DR · RPO/RT0"]
    end

    U --> WAF --> GW
    A --> GW
    D --> IG
    GW -->|verifica| PDP
    IG -->|verifica| PDP
    PDP --- IDP
    PDP --- RISK
    GW --> MESH --> SVC
    IG --> MESH
    SVC -->|chaves| VAULT
    SVC --> DB
    VAULT --- SEC
    MESH -.logs.-> SIEM
    SVC -.logs.-> SIEM
    GW -.logs.-> SIEM
    IG -.logs.-> SIEM
    SIEM --> DET --> SOAR
    DB --> BKP --> DR
    SOAR -.contenção.-> MESH
```

6.2 Modelo Zero Trust — Decisão de Acesso (PDP/PEP)

```
sequenceDiagram
    participant C as Cliente (Usuário/Serviço)
    participant PEP as PEP (Gateway/Mesh/App)
    participant IDP as IdP (OIDC)
    participant PDP as PDP (OPA)
    participant PIP as PIP (Risk/Device/Tenant)
    participant R as Recurso (Dado do Tenant)

    C->>PEP: Requisição + token
    PEP->>IDP: Validar token (assinatura, exp, aud)
    IDP-->>PEP: Claims (sub, tenant, roles, amr)
    PEP->>PDP: Autorizar? (sujeito, ação, recurso, contexto)
    PDP->>PIP: Postura (device, risco, hora, tenant match)
    PIP-->>PDP: Sinais de contexto
    PDP-->>PEP: Permit / Deny (+ obrigações: MFA step-up, log)
    alt Permit
        PEP->>R: Encaminha com escopo mínimo + tenant_id
        R-->>C: Resposta
    else Deny / Step-up
        PEP-->>C: 403 ou desafio MFA
    end
    PEP-->>PDP: Registro imutável da decisão (auditoria)
```

6.3 Isolamento Multiempresa (Tenancy)

```
flowchart LR
    subgraph REQ["Requisição"]
        T["JWT com tenant_id + roles"]
    end
    subgraph APP["Camada de Aplicação"]
        MW["Tenant Guard (middleware)\nvalida tenant do token vs. recurso"]
        RLS["Row-Level Security\npolicy: tenant_id = current_tenant"]
    end
    subgraph DATA["Camada de Dados"]
        S1["Schema/RLS Tenant A\nchave de dados A"]
        S2["Schema/RLS Tenant B\nchave de dados B"]
    end
    T --> MW
    MW --> RLS
    RLS --> S1
    RLS --> S2
    MW -. cross-tenant .-> S2
```

6.4 Integridade de Telemetria/OS de Drone

```
flowchart TB
    DRONE["Drone / Gateway de Campo"] -->|payload + assinatura + nonce + ts| IG["Ingestão KCD (PEP)"]
    IG --> A1{"Dispositivo autenticado?\n(mTLS / device cert)"}
    A1 -- Não --> Q["Quarentena + alerta SOC"]
    A1 -- Sim --> A2{"Assinatura válida?\n(chave do dispositivo)"}
    A2 -- Não --> Q
    A2 -- Sim --> A3{"Nonce/timestamp\nnão-replay?"}
    A3 -- Não --> Q
    A3 -- Sim --> A4{"Plausibilidade\n(GPS, faixa, taxa)?"}
    A4 -- Não --> Q
    A4 -- Sim --> OK["Aceita · marca proveniência\ne integridade verificada"]
    OK --> STORE["Armazena por tenant\ncom hash de integridade"]
    Q --> SIEM["SIEM / Detecção"]
```

7. Fluxogramas (Resposta a Incidente)

7.1 Ciclo de Resposta a Incidentes (NIST SP 800-61)

```
flowchart TB
    START(["Sinal / Alerta"]) --> TRIAGE["Triagem no SOC\nnormaliza, correlaciona, deduplica"]
    TRIAGE --> CLASS{"É incidente real?"}
    CLASS -- Falso positivo --> TUNE["Ajusta regra / fecha\nfeedback ao SIEM"]
    CLASS -- Sim --> SEV{"Severidade?"}
    SEV -- S1 Crítico --> WAR["Aciona War Room\n+ liderança + jurídico"]
    SEV -- S2/S3 --> ONCALL["Aciona on-call de segurança"]
    WAR --> CONT
    ONCALL --> CONT["Contenção\nisola tenant/host/conta\nrevoga tokens/chaves"]
    CONT --> ERAD["Erradicação\nremove acesso, patch, rebuild"]
    ERAD --> REC["Recuperação\nrestaura de backup verificado\nmonitora reincidência"]
    REC --> PII{"Dado pessoal\nafetado?"}
    PII -- Sim --> NOTIF["Rito LGPD/GDPR\nnotifica ANPD + titulares\n(prazos legais)"]
    PII -- Não --> POST
    NOTIF --> POST["Post-mortem sem culpa\nlições, ações, atualiza playbook"]
    TUNE --> END(["Encerra + evidência"])
    POST --> END
```

7.2 Severidade, SLA e Papéis

Severidade	Definição	SLA de resposta	Acionamento	Comunicação
S1 — Crítico	Vazamento de dados, <i>ransomware</i> , indisponibilidade multi-tenant, comprometimento de root/KMS	Reconhecer ≤ 15 min · Conter ≤ 1 h	War Room, CISO, Jurídico, Exec	Cliente + ANPD conforme LGPD
S2 — Alto	Comprometimento de conta privilegiada, <i>exploit</i> ativo em 1 <i>tenant</i>	Reconhecer ≤ 30 min · Conter ≤ 4 h	On-call + Líder de segurança	Cliente afetado
S3 — Médio	Tentativa contida, <i>misconfig</i> explorável, anomalia relevante	Reconhecer ≤ 2 h · Conter ≤ 24 h	On-call	Interna
S4 — Baixo	Ruído, varredura, política menor	Reconhecer ≤ 1 dia	Fila do SOC	Interna

8. Boas Práticas

- **Segurança por padrão:** *secure defaults* em todo módulo — TLS on, MFA on, *deny-all* de base, portas fechadas, mínimo de escopo.
- **Menor privilégio sempre:** contas de serviço dedicadas por serviço; acesso humano privilegiado via **JIT** com aprovação e expiração; sem contas compartilhadas.
- **Nunca confie na entrada:** validação e *sanitização* de todo *input*; *output encoding*; *parametrized queries*; *allowlist* > *denylist*.
- **Segredos fora do código:** cofre para todas as credenciais; *pre-commit hooks* e *secrets scanning* no pipeline; rotação automática.
- **Rotação e expiração curtas:** tokens de vida curta + *refresh*; certificados com renovação automática; chaves com rotação programada.
- **Imutabilidade operacional:** infraestrutura imutável, *rebuild* em vez de *patch in place* quando viável; imagens assinadas e verificadas.
- **Observabilidade acionável:** todo alerta tem *runbook*; sem alerta órfão; correlação por **transparent** de ponta a ponta.
- **Menos é mais superfície:** remover o desnecessário (serviços, portas, dependências, permissões) reduz risco mais do que adicionar controles.

- **Falhar fechado:** em dúvida de autorização ou indisponibilidade do PDP, **negar** (com *fail-safe* documentado para disponibilidade crítica).
- **Privacidade por padrão:** minimização de dado, mascaramento em não-produção, retenção com expurgo automático.
- **Treine o humano:** *awareness* anti-*phishing*, exercícios de mesa (*tabletop*), *red team/purple team* periódicos.
- **Documente a evidência:** cada controle gera artefato auditável; o que não é comprovável não conta.

9. Padrões (Frameworks de Referência)

Framework	Uso no KCD	Aplicação principal
NIST CSF 2.0	Espinha dorsal de governança	Funções Govern, Identify, Protect, Detect, Respond, Recover mapeadas às camadas §5
NIST SP 800-207	Modelo Zero Trust	Definição de PDP/PEP/PIP/PAP §5.2
NIST SP 800-61	Resposta a incidentes	Ciclo §7.1
NIST SP 800-53 / 800-63	Controles / identidade	Baseline de controles e níveis de autenticação (AAL)
ISO/IEC 27001:2022	SGSI	Anexo A (93 controles) como base do §11 e §15
ISO/IEC 27017 / 27018	Nuvem / PII em nuvem	Controles multiempresa e proteção de dados pessoais
ISO 22301	Continuidade de negócio	BCP/DR §11.6
OWASP ASVS 4.x	Verificação de app	Gates de aplicação §5 L4
OWASP SAMM / Top 10 / API Top 10	Maturidade e riscos	SDLC seguro §11.5
OWASP CycloneDX / SLSA	<i>Supply chain</i> / SBOM	Integridade da cadeia §11.5
CIS Benchmarks	Hardening	Baselines de SO, container, k8s, DB §11
MITRE ATT&CK	Deteção baseada em TTPs	Mapeamento de regras do SIEM §11.4
LGPD (Lei 13.709) / GDPR	Privacidade e conformidade	Base legal, direitos do titular, notificação §11.8

10. Casos de Uso

CU-01 — Login administrativo com MFA e step-up

Administrador de *tenant* acessa o console. OIDC valida credenciais; o PDP exige **MFA resistente a phishing** (WebAuthn). Ao tentar operação sensível (exportar dados pessoais), o *risk engine* detecta novo dispositivo e força **step-up** adicional. Toda a decisão é registrada em trilha imutável.

CU-02 — Tentativa de acesso cross-tenant

Um token do Tenant A tenta ler uma OS do Tenant B (via manipulação de ID). O *Tenant Guard* compara `tenant_id` do token com o do recurso; RLS no banco impede a leitura; o KCD retorna 403, gera alerta de **acesso cross-tenant** (indicador de IDOR/BOLA) e abre triagem no SOC.

CU-03 — Injeção detectada por WAF + SAST prévio

Requisição com *payload* de SQLi chega ao gateway; o WAF bloqueia. O SAST/DAST no pipeline já havia coberto o padrão; a aplicação usa *prepared statements*. O evento alimenta a detecção de anomalia (pico de 403 por IP) e dispara *rate limit* adaptativo.

CU-04 — Telemetria de drone adulterada (replay)

Um atacante reenvia um pacote de telemetria capturado. A ingestão do KCD valida assinatura (ok) mas detecta **nonce/timestamp reutilizado** → rejeita, coloca origem em quarentena e alerta o SOC (possível MITM/replay). O dado não contamina relatórios do *tenant*.

CU-05 — Credencial vazada em commit

Desenvolvedor comita acidentalmente uma chave. O *secrets scanning* no *pre-commit* e no CI bloqueia o *merge*; o KCD dispara **rotação automática** da credencial exposta e registra o incidente S3, com pós-morte e melhoria de *hook*.

CU-06 — Ransomware / recuperação

Deteção de cifragem anômala de arquivos em um serviço. *Playbook S1*: isola o host, revoga tokens/chaves, aciona War Room. Recuperação via **backup imutável** verificado (fora do alcance do atacante, *air-gapped/WORM*), restaurando dentro do RTO. Dado pessoal afetado → rito LGPD.

CU-07 — Direito do titular (LGPD/GDPR)

Titular solicita exclusão de dados. O KCD executa fluxo de *data subject request*: localiza dados por *tenant*, aplica expurgo/anonimização, preserva o mínimo legal, e gera **evidência auditável** do atendimento dentro do prazo legal.

CU-08 — Deteção por anomalia comportamental

Conta de usuário passa a acessar volumes atípicos de OS em horário incomum a partir de novo país (*impossible travel*). O modelo de anomalia eleva o *risk score*; o PDP força *step-up* e, na falha, suspende a sessão e abre incidente S2.

11. Modelagem (Controles por Domínio)

11.1 Zero Trust

Controle	Descrição	Padrão	Evidência
ZT-1 Verificação contínua	Reavaliação de sessão/postura a cada requisição sensível	800-207	Logs de decisão do PDP
ZT-2 Menor privilégio	RBAC/ABAC + JIT/JEA	800-53 AC	Matriz de acesso, aprovações JIT
ZT-3 Microsegmentação	mTLS + <i>deny-all</i> leste-oeste	800-207	NetworkPolicy, telemetria de malha
ZT-4 Assume breach	Contenção lateral, honeytokens	ATT&CK	Alertas de movimento lateral

11.2 IAM (Identidade e Acesso)

Controle	Descrição	Padrão	Evidência
IAM-1 OIDC/OAuth2	Federação de identidade, tokens JWT assinados	800-63	Config do IdP, validação de tokens
IAM-2 MFA phishing-resistant	WebAuthn/FIDO2 para acesso privilegiado	800-63 AAL2/3	Cobertura de MFA (%)
IAM-3 RBAC	Papéis por <i>bounded context</i> e <i>tenant</i>	27001 A.5.15	Definição de papéis, revisão de acesso
IAM-4 ABAC	Atributos (tenant, sensibilidade, contexto) no PDP	NIST ABAC	Políticas Rego
IAM-5 Ciclo de identidade	<i>Joiner/Mover/Leaver</i> , <i>deprovisioning</i> automático	27001 A.5.16	Logs de provisionamento
IAM-6 Gestão de segredos	Cofre, rotação, <i>lease</i> curto	27001 A.8.24	Registro de rotação, ACLs
IAM-7 Acesso privilegiado (PAM)	JIT, <i>break-glass</i> auditado	800-53 AC-6	Trilha de sessão privilegiada

11.3 Criptografia e Chaves

Controle	Descrição	Padrão	Evidência
CR-1 TLS em trânsito	TLS 1.3, mTLS interno, HSTS	27001 A.8.24	Scan de configuração TLS
CR-2 Cripto em repouso	AES-256-GCM em DB, objeto e backup	27001 A.8.24	Config de cifragem
CR-3 Hierarquia de chaves	<i>Root</i> (HSM) → KEK → DEK, <i>envelope encryption</i>	NIST 800-57	Diagrama e políticas de KMS
CR-4 Rotação	Rotação programada e sob comprometimento	800-57	Log de rotação
CR-5 Chave por tenant	Isolamento criptográfico entre <i>tenants</i>	27018	Mapa chave ↔ tenant
CR-6 Gestão de certificados	Emissão/renovação automática, revogação	27001 A.8.24	Inventário de certs, expiração

11.4 SOC, Logging e Detecção

Controle	Descrição	Padrão	Evidência
SOC-1 Logging centralizado	Coleta normalizada, correlação por trace	27001 A.8.15	Cobertura de fontes de log
SOC-2 Logs imutáveis	WORM/append-only + <i>hash chaining</i>	27001 A.8.15	Verificação de integridade
SOC-3 Detecção por regras	Regras mapeadas a MITRE ATT&CK	ATT&CK	Catálogo de regras
SOC-4 Detecção por anomalia	Baselines comportamentais (UEBA)	NIST Detect	Alertas de anomalia
SOC-5 Threat intelligence	IOCs, enriquecimento, <i>feeds</i>	800-53	Fontes de TI integradas
SOC-6 SOAR / playbooks	Automação de contenção e triagem	800-61	Playbooks versionados
SOC-7 Métricas	MTTD, MTTR, cobertura	CSF Detect/Respond	Painel de KPIs

11.5 SDLC Seguro e Vulnerabilidades

Controle	Descrição	Padrão	Evidência
SDL-1 SAST	Análise estática no CI	ASVS/SAMM	Relatórios por build
SDL-2 DAST	Teste dinâmico em ambiente de teste	ASVS	Relatórios DAST
SDL-3 SCA / dependências	Varredura de libs, CVEs, licenças	OWASP Dep.	Inventário + alertas
SDL-4 Secrets scanning	<i>Pre-commit</i> + CI	27001 A.8.24	Bloqueios registrados
SDL-5 SBOM + assinatura	CycloneDX, imagens assinadas, SLSA	SLSA	SBOM por release
SDL-6 Gestão de vulnerabilidades	<i>Scan</i> , priorização (CVSS/EPSS), SLA de correção	800-40	Painel de vulns + SLA
SDL-7 Hardening	CIS Benchmarks (SO, container, k8s, DB)	CIS	Score CIS

11.6 Continuidade e DR

Controle	Descrição	Padrão	Evidência
BC-1 Backups cifrados	3-2-1, cifrados, por <i>tenant</i>	27001 A.8.13	Job de backup + status
BC-2 Imutabilidade	Backups WORM/air-gapped anti-ransomware	ISO 22301	Config de imutabilidade
BC-3 Teste de restore	Restauração verificada periodicamente	22301	Relatório de restore
BC-4 DR / RPO-RTO	Metas por classe de dado (§11.6.1)	22301	DR drill
BC-5 Plano de continuidade	BCP com papéis e comunicação	22301	BCP versionado

11.6.1 Metas RPO/RTO por classe de dado

Classe de dado	Exemplo	RPO	RTO
Crítico transacional	OS, financeiro, identidade	≤ 5 min	≤ 1 h
Operacional	Estoque, cadastros	≤ 1 h	≤ 4 h
Telemetria de drone	Séries de voo/sensor	≤ 15 min	≤ 2 h
Analítico / BI	Data warehouse	≤ 24 h	≤ 24 h
Logs / auditoria	Trilhas	≤ 5 min	≤ 2 h

11.7 Isolamento Multiempresa

Controle	Descrição	Padrão	Evidência
MT-1 Tenant guard	Middleware valida <i>tenant</i> do token vs. recurso	27017	Testes de isolamento
MT-2 RLS / schema	Row-Level Security ou schema por <i>tenant</i>	27018	Políticas de RLS
MT-3 Chave por tenant	Isolamento criptográfico	27018	Mapa de chaves
MT-4 Isolamento de observabilidade	Logs/métricas segregados por <i>tenant</i>	27017	Config do SIEM
MT-5 Testes anti-cross-tenant	Testes automatizados de vazamento	ASVS	Suíte de testes

11.8 Auditoria, Privacidade e Conformidade

Controle	Descrição	Padrão	Evidência
AU-1 Trilha imutável	Quem, o quê, quando, de onde; <i>append-only</i>	27001 A.8.15	Trilha verificável
AU-2 Base legal LGPD/GDPR	Finalidade, consentimento/base, minimização	LGPD art. 7/GDPR art. 6	Registro de tratamento (RoPA)
AU-3 Direitos do titular	Acesso, correção, exclusão, portabilidade	LGPD/GDPR	Fluxo de DSR + prazos
AU-4 Notificação de incidente	Rito ANPD/autoridade	LGPD art. 48	Registro de notificação
AU-5 Retenção e expurgo	Ciclo de vida do dado, expurgo automático	27001 A.5.34	Política de retenção
AU-6 DPIA/RIPD	Avaliação de impacto para tratamentos de risco	GDPR art. 35	RIPD arquivado

11.9 Segurança de Dados de Drones (Telemetria/OS)

Controle	Descrição	Padrão	Evidência
DR-1 Autenticação de dispositivo	mTLS / certificado por drone/gateway	800-207	Inventário de dispositivos
DR-2 Assinatura de payload	Assinatura por chave do dispositivo	800-57	Verificação de assinatura
DR-3 Anti-replay	<i>Nonce</i> + <i>timestamp</i> + janela	ASVS	Rejeições registradas
DR-4 Plausibilidade	Validação semântica (GPS, faixas, taxas)	—	Regras de validação
DR-5 Proveniência/integridade	Hash de integridade + linhagem por <i>tenant</i>	800-53 SI	Selo de proveniência
DR-6 Quarentena	Isolamento de dado suspeito antes do uso	800-61	Fila de quarentena

12. Checklist

Zero Trust & IAM - [] MFA resistente a phishing habilitado para todo acesso privilegiado - [] PDP central (OPA) em produção com políticas versionadas (GitOps) - [] RBAC + ABAC aplicados; nenhuma permissão "*" sem justificativa - [] Acesso privilegiado somente via JIT com expiração e aprovação - [] `tenant_id` validado em gateway, malha e aplicação

Criptografia & Segredos - [] TLS 1.3 em toda borda; mTLS interno na malha - [] AES-256 em repouso (DB, objeto, backup); chave por *tenant* - [] Hierarquia root(HSM) → KEK → DEK com *envelope encryption* - [] Rotação automática de chaves, certificados e tokens - [] Zero segredos no código/imagem/env; cofre + *secrets scanning*

SOC, Logs & Detecção - [] Logging centralizado cobrindo todas as fontes críticas - [] Trilhas imutáveis (WORM/*hash chain*) verificadas - [] Regras mapeadas a MITRE ATT&CK + detecção por anomalia - [] Playbooks SOAR para S1–S4 testados - [] Painel de MTTD/MTTR ativo

SDLC & Vulnerabilidades - [] SAST, DAST, SCA e *secrets scan* como *gates* obrigatórios - [] SBOM (CycloneDX) e imagens assinadas por release - [] SLA de correção por severidade (CVSS/EPSS) cumprido - [] Baselines CIS aplicados e medidos

Continuidade & DR - [] Backups 3-2-1 cifrados e imutáveis - [] RPO/RTO definidos por classe e testados - [] DR drill executado no período; *runbook* atualizado

Multiempresa & Privacidade - [] Testes automatizados anti-cross-tenant no CI - [] RoPA e bases legais LGPD/GDPR documentadas - [] Fluxo de direitos do titular operante com prazos - [] Retenção e expurgo automáticos configurados

Drones - [] Autenticação de dispositivo + assinatura de payload - [] Anti-replay e validação de plausibilidade ativos - [] Quarentena de dados suspeitos antes do uso analítico

13. Riscos

ID	Risco	Prob.	Impacto	Exposição	Mitigação (controle)
RK-01	Vazamento cross-tenant por falha de isolamento (BOLA/IDOR)	Média	Crítico	Alta	MT-1..MT-5, testes anti-cross-tenant, RLS + chave por <i>tenant</i>
RK-02	Comprometimento de conta privilegiada	Média	Crítico	Alta	IAM-2 (MFA), IAM-7 (PAM/JIT), UEBA (SOC-4)
RK-03	Ransomware com destruição de backups	Baixa	Crítico	Média	BC-2 (imutável/air-gap), BC-3 (restore testado), CU-06
RK-04	Segredo vazado em repositório/imagem	Média	Alto	Alta	SDL-4, IAM-6, rotação automática
RK-05	Vulnerabilidade em dependência (<i>supply chain</i>)	Alta	Alto	Alta	SDL-3, SDL-5 (SBOM/assinatura), SLA de patch
RK-06	Telemetria adulterada/replay contaminando dados	Média	Alto	Alta	DR-1..DR-6 (auth, assinatura, anti-replay, plausibilidade)
RK-07	Exposição de dado pessoal / não conformidade LGPD	Média	Crítico	Alta	AU-2..AU-6, minimização, DPIA
RK-08	Indisponibilidade de PDP causando <i>fail-open</i>	Baixa	Alto	Média	PDP HA, <i>fail-closed</i> padrão, cache de decisão seguro
RK-09	Deteção tardia (MTTD alto)	Média	Alto	Alta	SOC-1..SOC-7, cobertura de log, <i>tuning</i> contínuo
RK-10	Erro de configuração de nuvem (<i>misconfig</i>)	Alta	Alto	Alta	SDL-7 (CIS), <i>policy as code</i> , <i>scanning</i> de IaC
RK-11	<i>Phishing</i> /engenharia social	Alta	Alto	Alta	MFA phishing-resistant, <i>awareness</i> , <i>tabletop</i>
RK-12	Insider malicioso/negligente	Baixa	Crítico	Média	Menor privilégio, trilha imutável, segregação de funções

Escala: Prob./Impacto ∈ {Baixa, Média, Alta / Baixo, Alto, Crítico}. Exposição = combinação — reavaliada trimestralmente no comitê de risco.

14. Melhorias Futuras

- **MF-1. Zero Trust adaptativo com risco contínuo:** *risk engine* com ML para *step-up* dinâmico e sessões de confiança contínua (CARTA).
- **MF-2. Passwordless amplo:** expansão de WebAuthn/passkeys para todos os perfis, redução de senhas a zero.
- **MF-3. Confidential computing:** processamento em enclaves (TEE) para dados de *tenants* de alta sensibilidade.
- **MF-4. Deteção por UEBA/ML avançada:** modelos por *tenant* e por perfil de drone; redução de MTTD.
- **MF-5. Cripto-agilidade e pós-quântica:** inventário de algoritmos e migração planejada para PQC (NIST FIPS 203/204).
- **MF-6. SOAR ampliado:** automação de contenção *end-to-end* (revogação, isolamento, *rebuild*) com aprovação humana no *loop* apenas para S1.
- **MF-7. Attestation de dispositivos de campo:** *remote attestation* do gateway do drone para elevar confiança na proveniência.
- **MF-8. Chaos security / purple team contínuo:** exercícios automatizados de resiliência e deteção.
- **MF-9. Data lineage completo:** linhagem criptográfica ponta-a-ponta da telemetria à decisão de negócio.
- **MF-10. Paineis de postura por tenant:** *security posture* self-service para o cliente (transparência = confiança é direito).

15. Auditoria

15.1 Modelo de auditoria contínua

O KCD adota **auditoria contínua**: os controles não são verificados apenas em janelas anuais, mas **monitorados por evidência automatizada** (*compliance as code*). Cada controle de §11 emite artefatos que alimentam painéis de conformidade mapeados a ISO 27001 (Anexo A), NIST CSF 2.0 e LGPD.

15.2 Trilha de auditoria (imutável)

- **Conteúdo mínimo por evento**: identidade do sujeito, ação, recurso, `tenant_id`, resultado (permit/deny), origem (IP/dispositivo), `traceparent`, *timestamp* confiável.
- **Integridade**: *append-only* com *hash chaining* (cada registro encadeia o hash do anterior) e selagem periódica; armazenamento WORM.
- **Segregação**: trilhas de auditoria isoladas dos operadores auditados (o SOC não pode apagar o próprio rastro).
- **Retenção**: conforme classe (mín. legal LGPD/setorial), com expurgo controlado e comprovável.

15.3 Cadência e responsáveis

Atividade	Frequência	Responsável	Saída
Revisão de acessos (<i>access review</i>)	Trimestral	Donos de sistema + Segurança	Evidência de recertificação
<i>Pentest</i> externo	Semestral	Terceiro independente	Relatório + plano de correção
DR drill / teste de restore	Semestral	Infra + Segurança	Relatório de RPO/RTO real
Revisão de políticas (§4)	Anual ou por mudança	CISO	Política versionada
Auditoria ISO 27001	Anual	Auditor certificado	Certificado / não conformidades
<i>Tabletop</i> de incidente	Semestral	SOC + liderança	Lições e ajuste de <i>playbook</i>
Revisão de risco (§13)	Trimestral	Comitê de Risco	Matriz atualizada
Verificação de integridade de trilhas	Contínua (automática)	KCD/SIEM	Alerta em caso de quebra

15.4 Métricas de auditoria (KPIs/KRIs)

Métrica	Meta
Cobertura de MFA (privilegiado)	100%
Cobertura de log de fontes críticas	≥ 98%
MTTD (tempo médio de detecção)	≤ 30 min (S1/S2)
MTTR (tempo médio de resposta)	≤ 4 h (S2)
Conformidade de patch crítico no SLA	≥ 95%
Backups com restore verificado	100% no período
Achados de auditoria em atraso	0 críticos
Testes anti-cross-tenant no CI	100% verdes

15.5 Encerramento

O KCD só é considerado **conforme** quando cada controle possui **evidência viva** (R14). A ausência de evidência equivale à ausência do controle. Este documento é revisado a cada mudança arquitetural relevante (Docs [07 / 08 / 09](#)), a cada incidente S1/S2 e, no mínimo, anualmente — sustentando os três princípios do projeto: **segurança por padrão**, **confiança é direito** e **auditoria contínua**.

Nota de dependências: este documento consome e referencia os artefatos 00 – 13 . Alterações em modelagem de dados (06), arquitetura (07), infraestrutura (08) ou microsserviços (09) exigem reavaliação dos controles correspondentes em §11.

15 Kairós Smart Inventory (KSI)

Documento: 15 – Kairós Smart Inventory (KSI) **Versão:** 1.0 · **Data:** 21 de julho de 2026 · **Status:** Ativo
Dependências: 00 – Master Prompt, 01 – Constituição, 02 – Project Bible, 03 – Roadmap, 04 – Pesquisa de Mercado, 05 – Benchmark, 06 – Requisitos, 07 – Arquitetura, 08 – Modelagem de Dados, 09 – Microserviços, 10 – Padrões de Código, 11 – API, 12 – Frontend/Apps, 13 – KCI, 14 – KCD **Responsável:** Especialista Supply Chain / Gestão de Estoque

1. Resumo Executivo

O **KSI — Kairós Smart Inventory** é o módulo proprietário de **estoque inteligente** do Drone Kairós ERP. Ele governa a posição física e contábil de todos os itens que circulam no ecossistema fabricante → revenda → técnico → cliente: peças de reposição, componentes, insumos de campo, kits de manutenção e os próprios equipamentos (drones) enquanto ativos de estoque antes da venda. Sua promessa central é **rastreabilidade vitalícia ponta a ponta** — cada peça que sai do estoque pode ser correlacionada ao **Serial Number (SN)** do equipamento em que foi aplicada e à **Ordem de Serviço (OS)** que a consumiu — combinada com **reposição preditiva** que antecipa a falta antes que ela pare uma manutenção em campo.

O KSI implementa o fluxo de referência canônico do projeto: **Estoque** → **Código de Barras** → **QR Code** → **RFID** → **NFC** → **Compras** → **Reposição Automática** → **Curva ABC** → **Previsão** → **Integração Financeira**. Na prática, isso significa que o técnico identifica a peça por leitura ótica (barcode/QR) ou por radiofrequência (RFID/NFC quando o hardware está disponível) diretamente pelo **app do técnico (Flutter, offline-first via KCI)**; o sistema dá **baixa automática** ao concluir a OS, atualiza saldos e reservas em tempo real, reclassifica a **curva ABC**, recalcula o **ponto de pedido** com base em **previsão de demanda** e, quando o saldo projetado cruza o gatilho, **sugere ou dispara automaticamente uma ordem de compra**. Ao recebimento, a valorização de estoque é lançada no **Financeiro** por **custo médio ponderado ou PEPS**, mantendo a contabilidade de estoque sempre conciliada.

Arquiteturalmente, o KSI é o **inventory-service** (bounded context **Estoque / KSI** do Doc 09), dono exclusivo dos dados de itens, saldos, reservas e movimentações, integrando-se por eventos de domínio (**EstoqueBaixado**, **PontoDePedidoAtingido**, **ItemRecebido**) e por chamadas gRPC internas de baixa latência. É **multiempresa** por construção: todo saldo, movimento e política de reposição é qualificado por **tenant_id**, com isolamento reforçado por Row-Level Security (Doc 07).

Este documento entrega: as regras de negócio de estoque, a arquitetura interna do módulo, os diagramas de domínio, os **fluxogramas de baixa automática e de reposição preditiva**, as fórmulas de classificação ABC / estoque de segurança / ponto de pedido / lote econômico, os casos de uso operacionais, o **modelo de dados resumido do estoque**, o checklist de prontidão, os riscos e a trilha de auditoria.

2. Objetivos

- **01.** Manter uma **posição de estoque única e confiável** (saldo físico, disponível, reservado, em trânsito) por item, lote, número de série, depósito e localização, com atualização em tempo real e conciliação contínua.
- **02.** Garantir **identificação automática** do item no ponto de uso via código de barras, QR Code, RFID e NFC, capturada pelo app do técnico, minimizando digitação e erro humano.
- **03.** Assegurar **rastreabilidade ponta a ponta**: peça (item + lote/SN) ↔ SN do equipamento ↔ OS ↔ técnico ↔ cliente, em ambos os sentidos (rastros para frente e para trás).
- **04.** Executar **baixa automática** de peças ao concluir a OS, com reserva prévia na abertura/planejamento e estorno em caso de cancelamento.
- **05.** Classificar o portfólio por **Curva ABC** (e critérios complementares XYZ/criticidade) para priorizar políticas de controle, contagem e reposição.

- **O6.** Prover **previsão de demanda e reposição preditiva** — estoque de segurança, ponto de pedido e lote de reposição calculados a partir de histórico, lead time e variabilidade.
- **O7.** Integrar-se a **Compras**: gerar sugestão de compra, converter em ordem de compra (OC) e conciliar o **recebimento** contra a OC (three-way match).
- **O8.** Integrar-se ao **Financeiro**: valorizar o estoque por **custo médio ponderado ou PEPS**, lançar provisões (obsolescência, perdas) e refletir o CMV (custo da mercadoria/peça vendida ou aplicada).
- **O9.** Suportar **inventário cíclico** por classe ABC, com bloqueio/ajuste auditável e sem parar a operação.
- **O10.** Ser **multiempresa** e **offline-first** (o técnico opera sem rede; a sincronização reconcilia via KCI).

3. Escopo

No escopo (coberto por este documento): - Cadastro de itens (SKU), unidades de medida e conversões, itens controlados por lote e/ou por número de série. - Estrutura física de estoque: múltiplas empresas, depósitos, localizações (endereço), zonas. - Saldos e estados de quantidade: físico, disponível, reservado, em trânsito, em quarentena/inspeção, bloqueado. - Reservas (hard/soft) para OS, venda e transferência. - Identificação automática: barcode (1D), QR Code (2D), RFID (UHF/HF) e NFC, incluindo captura mobile. - Movimentações: entrada (recebimento/devolução/ajuste positivo), saída (consumo/venda/ajuste negativo), transferência (entre depósitos/localizações/empresas) e baixa automática por OS. - Inventário cíclico e geral, contagem e ajuste. - Rastreabilidade e genealogia (peça ↔ SN ↔ OS). - Classificação ABC/XYZ e criticidade. - Previsão de demanda e cálculo de parâmetros de reposição (SS, ROP, EOP/EOQ). - Sugestão de compra e interface com o módulo de Compras (recebimento e three-way match do ponto de vista do estoque). - Valorização de estoque e interface com o Financeiro (custo médio/PEPS, provisões, CMV).

Fora do escopo (delegado a outros documentos): - Ciclo completo de aprovação e cotação de compras, gestão de fornecedores e contratos → **módulo Compras / Doc de Compras**. - Contabilização detalhada, plano de contas, DRE, conciliação bancária → **módulo Financeiro / Doc Financeiro**. - Ciclo de vida da OS (abertura, execução, apontamento) → **módulo Ordens de Serviço**; aqui tratamos apenas do **consumo de peças** pela OS. - Cadastro mestre do equipamento e a **autoridade sobre o Serial Number** → **registry-service** (Doc 08); o KSI **referencia** o SN, não o cria. - Sincronização offline e resolução de conflito → **KCI (Doc 13)**; o KSI **consome** essa infraestrutura. - Autenticação/autorização e trilha criptográfica → **KCD (Doc 14)**.

4. Regras de Negócio

#	Regra	Descrição
R1	Tenant sempre presente	Todo item, saldo, movimento, reserva e política carrega <code>tenant_id</code> . Nenhuma consulta ou movimento cruza empresas sem uma transferência interempresa explícita.
R2	Saldo nunca negativo (por padrão)	O disponível não pode ficar negativo. Saída sem saldo disponível é bloqueada, salvo item/depósito com flag <code>permite_negativo</code> (exceção auditada).
R3	Toda movimentação é imutável e auditável	Movimentos são <i>append-only</i> (livro-razão de estoque). Correção se faz por movimento de estorno , nunca por edição/exclusão do original.
R4	Item controlado por lote exige lote	Se <code>controla_lote = true</code> , nenhuma entrada/saída ocorre sem informar o lote. Idem para <code>controla_serial</code> e número de série.
R5	Serial é único e unitário	Um número de série representa uma unidade. Sua quantidade em qualquer localização é 0 ou 1. Não pode existir em dois lugares simultaneamente.
R6	Reserva antecede consumo	Peça planejada em OS gera reserva (reduz o disponível, não o físico). A baixa efetiva ocorre na conclusão; o cancelamento estorna a reserva.
R7	Baixa automática ao concluir OS	Ao evento <code>OSConcluida</code> , o KSI baixa as peças efetivamente aplicadas apontadas na OS, vinculando cada baixa ao SN do equipamento e à OS.
R8	Recebimento passa por inspeção quando exigido	Itens com <code>requer_inspecao = true</code> entram em quarentena ; só migram para disponível após aprovação de qualidade.
R9	Valorização consistente por método	Cada item tem um método de custeio fixo por empresa (Custo Médio Ponderado ou PEPS/FIFO). Trocar de método é evento controlado, versionado e datado.
R10	Reposição respeita política do item	O gatilho de reposição usa os parâmetros vigentes (SS, ROP, EOQ, lead time, cobertura) da política do item/depósito ; alterações são versionadas.
R11	FEFO/FIFO na separação	Itens com validade seguem FEFO (First-Expired-First-Out); os demais, FIFO . A alocação de lote na baixa obedece essa ordem por padrão.
R12	Contagem cega no inventário	O contador não vê o saldo teórico ao contar (contagem cega). O ajuste só ocorre após reconciliação e, acima de tolerância, requer recontagem/aprovação.
R13	Idempotência de eventos	A baixa de uma mesma OS/linha é idempotente: reprocessar <code>OSConcluida</code> não gera baixa dupla (chave de idempotência por <code>os_id + linha</code>).
R14	Custo é imutável no passado	Recebimento retroativo ou correção de custo não reescreve movimentos já valorizados; gera lançamento de ajuste no período corrente.
R15	Offline reconcilia, não sobrescreve	Movimentos capturados offline pelo técnico entram como provisórios ; a sincronização (KCI) os confirma na ordem temporal e resolve conflito por regra determinística.

5. Arquitetura do Módulo

O KSI é o `inventory-service` — dono exclusivo dos dados de estoque (Doc 09, R2/R3). Internamente adota **arquitetura em camadas com DDD**: domínio rico (agregados de item, saldo, movimento), aplicação (casos de uso/serviços), infraestrutura (persistência, mensageria, integrações) e adaptadores de entrada (REST, gRPC, consumidores de evento).

5.1 Camadas internas

1. **Interface / Adaptadores de entrada - REST** (via BFF) para telas de gestão de estoque (web) e para o app do técnico (mobile). - **gRPC** interno para consultas de disponibilidade de baixa latência (ex.: `workorder-service` pergunta "há saldo para reservar?"). - **Consumidores de evento**: `OSAberta`, `OSConcluida`, `OSCancelada`, `PedidoCompraRecebido`, `VendaFaturada`.

2. **Aplicação (casos de uso)** - Serviços de: Movimentação, Reserva, Recebimento, Inventário, Reposição, Classificação ABC, Previsão, Valorização.
3. **Domínio (agregados)** - `Item` (SKU + políticas), `SaldoLocalizacao` (posição por depósito/localização/lote/SN), `Movimento` (livro-razão), `Reserva`, `PoliticaReposicao`, `PlanoInventario`.
4. **Infraestrutura** - PostgreSQL (OLTP, banco próprio do serviço), **outbox** para publicação confiável de eventos, cache de leitura para disponibilidade, motor de previsão (job batch / integração KCI-analytics).

5.2 Integrações (context map do KSI)

Contraparte	Direção	Mecanismo	Conteúdo
Ordens de Serviço (<code>workorder-service</code>)	⇌	gRPC (consulta disponibilidade/reserva) + eventos (<code>OSConcluida</code> → baixa)	Reserva de peças, baixa automática, devolução de peça não usada
Compras (<code>procurement-service</code>)	⇌	Evento <code>PontoDePedidoAtingido</code> (KSI → Compras) + <code>PedidoCompraRecebido</code> (Compras → KSI)	Sugestão/necessidade de compra e recebimento
Financeiro (<code>finance-service</code>)	→	Evento <code>EstoqueValorizado</code> , <code>EstoqueBaixado</code>	Valorização, CMV, provisões, custo médio
Cadastro/SN (<code>registry-service</code>)	←	gRPC/consulta	Validação do SN do equipamento na baixa/rastreio
KCI (Doc 13)	⇌	SDK offline-first + fila de sync	Captura offline no app do técnico, reconciliação
KCD (Doc 14)	⇌	Gateway/mTLS + trilha assinada	AuthZ por papel, assinatura de movimentos sensíveis
BI (<code>bi-service</code>)	→	Eventos (read models)	Giro, cobertura, acuracidade, ruptura

5.3 Estados de quantidade (modelo de saldo)

O saldo de um item numa localização é decomposto em estados que somam o **físico**:

Físico = Disponível + Reservado + Quarentena + Bloqueado
 Em Trânsito (transferência) é contabilizado à parte, "a caminho" do destino.
 Disponível para venda/consumo = Físico - Reservado - Quarentena - Bloqueado

5.4 Identificação automática — camada de captura

Tecnologia	Uso típico no KSI	Portador	Alcance/leitura	Observação
Código de barras (1D)	SKU e etiqueta de prateleira	Etiqueta impressa (Code128/EAN)	Ótica, linha de visão	Universal, barato; base do fluxo
QR Code (2D)	Lote, SN, endereço de localização, OS	Etiqueta impressa	Ótica, câmera do celular	Alta densidade; codifica payload assinado
RFID (UHF/HF)	Inventário em massa, ativos de alto valor	Tag RFID	Rádio, sem linha de visão, múltiplas tags	Contagem em lote; requer leitor
NFC	Ferramenta/kit do técnico, ativo unitário	Tag NFC	Rádio, ~4 cm, toque	Nativo em muitos smartphones; 1:1

A captura é abstraída por uma interface única no app (`ScanResult { tipo, payload, assinatura }`), de modo que a origem (câmera, leitor RFID/NFC pareado por Bluetooth) é transparente para os casos de uso de movimentação. Quando RFID/NFC **não** está disponível no dispositivo, o fluxo faz *fallback* automático para QR/barcode.

6. Diagramas

6.1 Modelo de domínio (agregados de estoque)

```

classDiagram
    class Item {
        +uuid id
        +string sku
        +string descricao
        +bool controla_lote
        +bool controla_serial
        +string metodo_custeio
        +string classe_abc
    }
    class Deposito {
        +uuid id
        +string codigo
        +uuid empresa_id
    }
    class Localizacao {
        +uuid id
        +string endereco
        +uuid deposito_id
    }
    class SaldoLocalizacao {
        +uuid id
        +decimal qtd_fisica
        +decimal qtd_reservada
        +decimal qtd_quarentena
        +decimal custo_medio
    }
    class Lote {
        +uuid id
        +string numero
        +date validade
    }
    class NumeroSerie {
        +uuid id
        +string serial
        +string status
    }
    class Movimento {
        +uuid id
        +string tipo
        +decimal quantidade
        +decimal custo_unit
        +timestamp data
    }
    class Reserva {
        +uuid id
        +decimal quantidade
        +string origem
        +uuid os_id
    }
    class PoliticaReposicao {
        +decimal estoque_seguranca
        +decimal ponto_pedido
        +decimal lote_economico
        +int lead_time_dias
    }

    Item "1" --> "*" SaldoLocalizacao
    Item "1" --> "1" PoliticaReposicao
    Item "1" --> "*" Lote
    Item "1" --> "*" NumeroSerie
    Deposito "1" --> "*" Localizacao
    Localizacao "1" --> "*" SaldoLocalizacao
    SaldoLocalizacao "1" --> "*" Movimento
    SaldoLocalizacao "1" --> "*" Reserva
    Lote "0..1" --> "*" Movimento
    NumeroSerie "0..1" --> "*" Movimento

```

6.2 Rastreabilidade ponta a ponta (peça ↔ SN ↔ OS)

```
flowchart LR
    subgraph Compra
        OC[Ordem de Compra] --> REC[Recebimento]
        REC --> LOTE[Lote / SN da peça]
    end
    subgraph Estoque_KSI
        LOTE --> SALDO[Saldo por localização]
        SALDO --> RES[Reserva p/ OS]
    end
    subgraph Servico
        RES --> APLIC[Peça aplicada]
        APLIC --> SNEQ[SN do equipamento / drone]
        APLIC --> OS[Ordem de Serviço]
        OS --> TEC[Técnico]
        OS --> CLI[Cliente]
    end
    LOTE -. rastro p/ frente .-> APLIC
    SNEQ -. rastro p/ trás .-> LOTE
```

6.3 Ciclo de vida da quantidade

```
stateDiagram-v2
    [*] --> EmTransito: OC emitida
    EmTransito --> Quarentena: Recebido (requer inspeção)
    EmTransito --> Disponivel: Recebido (sem inspeção)
    Quarentena --> Disponivel: Aprovado QA
    Quarentena --> Bloqueado: Reprovado QA
    Disponivel --> Reservado: Reserva p/ OS/Venda
    Reservado --> Disponivel: Cancelamento
    Reservado --> Consumido: Baixa (OS concluída)
    Disponivel --> EmTransito: Transferência
    Consumido --> [*]
    Bloqueado --> [*]: Descarte/Devolução
```

6.4 Arquitetura de integração do módulo

```
flowchart TB
    APP[App do Técnico<br/>Flutter offline-first] -->|scan barcode/QR/Rfid/NFC| BFF[BFF Mobile]
    WEB[Painel Estoque Web] --> BFFW[BFF Web]
    BFF --> KSI[(inventory-service / KSI)]
    BFFW --> KSI
    KSI <-->|gRPC disponibilidade| WO[workorder-service]
    KSI -->|evento PontoDePedidoAtingido| PROC[procurement-service]
    PROC -->|evento PedidoCompraRecebido| KSI
    KSI -->|evento EstoqueValorizado/Baixado| FIN[finance-service]
    KSI -->|consulta SN| REG[registry-service]
    KSI -->|eventos read-model| BI[bi-service]
    KSI -.outbox.-> BUS[{Barramento de Eventos}]
    APP -.sync offline.-> KCI[KCI / fila de sync]
    KCI --> KSI
```

7. Fluxogramas

7.1 Baixa automática de peças ao concluir a OS

```
flowchart TD
    A[OS concluída pelo técnico] --> B{OS tem peças apontadas?}
    B -- Não --> Z[Fim: nada a baixar]
    B -- Sim --> C[Para cada peça aplicada]
    C --> D{Item controla lote/serial?}
    D -- Sim --> E[Resolver lote por FEFO/FIFO<br/>ou SN informado no scan]
    D -- Não --> F[Alocar saldo disponível na localização]
    E --> G{Há reserva prévia da OS?}
    F --> G
    G -- Sim --> H[Converter reserva em consumo]
    G -- Não --> I{Há saldo disponível?}
    I -- Não --> J[Bloqueio: gerar pendência<br/>de acerto / permite_negativo?]
    I -- Sim --> H
    H --> K[Registrar Movimento SAÍDA<br/>tipo=CONSUMO_OS, custo por método]
    K --> L[Vincular baixa: item+lote/SN ↔ SN equipamento ↔ OS]
    L --> M[Recalcular saldo e custo médio]
    M --> N[Publicar evento EstoqueBaixado + EstoqueValorizado]
    N --> O{Saldo cruzou ponto de pedido?}
    O -- Sim --> P[Disparar avaliação de reposição]
    O -- Não --> Q[Fim da linha]
    J --> Q
    P --> Q
```

7.2 Reposição preditiva (ponto de pedido → sugestão/OC)

```
flowchart TD
    A[Gatilho: movimento de saída<br/>ou job diário de reposição] --> B[Recuperar posição do item/  
depósito<br/>Disponível + Em Trânsito]
    B --> C[Obter previsão de demanda<br/>e variabilidade sigma]
    C --> D[Calcular Estoque de Segurança<br/> $SS = z \cdot \sigma_{DLT}$ ]
    D --> E[Calcular Ponto de Pedido<br/> $ROP = \bar{d} \cdot LT + SS$ ]
    E --> F{Posição projetada <= ROP?}
    F -- Não --> Z[Sem ação: estoque suficiente]
    F -- Sim --> G[Calcular quantidade a repor<br/>EOQ ou cobertura-alvo]
    G --> H{Política do item = automática?}
    H -- Sim --> I[Gerar Ordem de Compra<br/>fornecedor preferencial]
    H -- Não --> J[Gerar Sugestão de Compra<br/>para aprovação do comprador]
    I --> K[Publicar PedidoCompraCriado → Compras]
    J --> L[Publicar PontoDePedidoAtingido → Compras]
    K --> M[Item entra Em Trânsito ao confirmar OC]
    L --> M
    M --> N[Recebimento futuro reabastece o saldo]
```

8. Boas Práticas

- **Endereçamento disciplinado.** Toda localização tem endereço legível por máquina (QR na prateleira). Movimento sempre confirma **origem e destino** por scan, não por digitação.
- **Contagem cega e cíclica por classe.** Itens A contados com maior frequência; nunca exibir saldo teórico ao contador (evita viés de confirmação).
- **FEFO como padrão para itens com validade.** Reduz obsolescência e descarte; FIFO para os demais.
- **Reserva antes de prometer.** Só se compromete peça a uma OS/venda com reserva efetiva; evita venda/aplicação de saldo inexistente.
- **Parâmetros de reposição vivos.** SS/ROP/EOQ recalculados periodicamente (não fixos "no chute"); revisados quando lead time ou demanda mudam de patamar.
- **Segregar disponível de reservado nas telas.** O operador vê o que **pode** usar, não só o físico.
- **Movimentos atômicos e idempotentes.** Uma baixa = uma transação; reprocessos não duplicam.

- **Custo por método consistente.** Não misturar custo médio e PEPS no mesmo item; documentar qualquer troca.
- **Offline com confirmação.** Movimento capturado sem rede é **provisório** e visualmente marcado até confirmar na sync.
- **Etiqueta durável no ambiente de campo.** Tags RFID/NFC e QR resistentes a poeira, umidade e vibração (contexto drone/campo).

9. Padrões

Domínio	Padrão adotado
Unidade de medida	UN base por item + fatores de conversão (ex.: caixa → unidade); toda quantidade persistida em UN base.
Código de barras 1D	Code128 (interno) / EAN-13 (comercial).
QR Code	Payload JSON compacto assinado (KCD): <code>{t:"item loc sn os", id, sig}</code> .
RFID	EPC UHF Gen2 (Class 1) para inventário; HF/NFC (ISO 14443/15693) para unitário.
NFC	NDEF com registro de tipo próprio do Kairós (URI + payload assinado).
Custeio	Custo Médio Ponderado Móvel (padrão) ou PEPS/FIFO (opcional por item).
Alocação de lote	FEFO (com validade) / FIFO (default).
Classificação	ABC por valor de consumo (Pareto) + XYZ por variabilidade + criticidade operacional.
Eventos de domínio	<code>EstoqueBaixado</code> , <code>EstoqueValorizado</code> , <code>PontoDePedidoAtingido</code> , <code>ItemRecebido</code> , <code>TransferenciaConcluida</code> , <code>InventarioAjustado</code> (versionados, com <code>tenant_id</code> e <code>traceparent</code>).
Idempotência	Chave <code>tenant_id + origem + referencia</code> em todo consumidor de evento e endpoint de escrita.
Precisão numérica	Quantidade e custo em <code>decimal</code> (nunca <code>float</code>); arredondamento contábil documentado.

10. Casos de Uso

UC-01 — Recebimento de peças de uma OC

Ator: Almoxarife. **Fluxo:** escaneia a etiqueta da OC/nota → confere itens → informa lote/validade ou coleta SN → itens com `requer_inspecao` vão para **quarentena**; os demais para **disponível** → three-way match (OC × recebido × nota) → publica `ItemRecebido` e `EstoqueValorizado` (atualiza custo médio). **Resultado:** saldo e custo atualizados, Financeiro provisiona a entrada.

UC-02 — Reserva de peças na abertura da OS

Ator: `workorder-service` (automático) / Planejador. **Fluxo:** OS planeja peças → KSI verifica disponível via gRPC → cria **reserva** (reduz disponível) → se faltar, sinaliza necessidade (aciona reposição). **Resultado:** peças comprometidas para a OS, sem baixa física ainda.

UC-03 — Baixa automática ao concluir a OS

Ator: Técnico (app). **Fluxo:** técnico aponta peças efetivamente aplicadas (scan por barcode/QR/RFID/NFC) → conclui a OS → evento `OSConcluida` → KSI converte reserva em **consumo**, registra movimento de saída vinculado ao **SN do equipamento**, recalcula custo, publica `EstoqueBaixado`. **Resultado:** estoque reduzido, rastro peça ↔ SN ↔ OS gravado, CMV para o Financeiro. (Ver fluxograma §7.1.)

UC-04 — Devolução de peça não utilizada

Ator: Técnico. **Fluxo:** peça reservada/retirada e não aplicada → devolução ao estoque via scan → estorno da reserva ou movimento de **entrada por devolução** → saldo restabelecido. **Resultado:** sem perda de acuracidade.

UC-05 — Transferência entre depósitos/empresas

Ator: Logística. **Fluxo:** origem gera saída → item em **trânsito** → destino confirma recebimento por scan → entra como disponível. Interempresa gera lançamento de custo entre `tenant_id`. **Resultado:** posição consistente nas duas pontas.

UC-06 — Inventário cíclico de itens classe A

Ator: Conferente. **Fluxo:** sistema gera plano por classe ABC → contagem **cega** por scan → reconciliação → dentro da tolerância ajusta; acima, recontagem/aprovação → `InventarioAjustado`. **Resultado:** acuracidade mantida sem parar a operação.

UC-07 — Reposição preditiva dispara compra

Ator: Sistema (job) / Comprador. **Fluxo:** saída faz posição cruzar o ROP → cálculo de SS/ROP/EOQ → política automática gera OC; manual gera **sugestão** → evento a Compras. **Resultado:** reabastecimento antes da ruptura. (Ver §7.2.)

UC-08 — Reclassificação ABC trimestral

Ator: Especialista de Estoque. **Fluxo:** job agrega valor de consumo do período → ordena (Pareto) → atribui A/B/C e faixa XYZ → ajusta políticas de contagem/reposição por classe. **Resultado:** foco de controle onde há mais valor/risco.

UC-09 — Captura offline em campo (sem rede)

Ator: Técnico. **Fluxo:** sem conexão, movimentos ficam **provisórios** no app (KCI) → ao reconectar, sync confirma na ordem temporal e resolve conflito determinístico → KSI consolida. **Resultado:** operação de campo não bloqueia por falta de rede.

11. Modelagem de Dados (entidades de estoque)

Modelo lógico resumido (banco próprio do `inventory-service`; toda tabela possui `tenant_id`, `criado_em`, `atualizado_em`).

11.1 Diagrama Entidade-Relacionamento

```
erDiagram
    ITEM ||--o{ SALDO_LOCALIZACAO : possui
    ITEM ||--|| POLITICA_REPOSICAO : tem
    ITEM ||--o{ LOTE : "pode ter"
    ITEM ||--o{ NUMERO_SERIE : "pode ter"
    ITEM ||--o{ CLASSIFICACAO_ABC : historico
    DEPOSITO ||--o{ LOCALIZACAO : contem
    LOCALIZACAO ||--o{ SALDO_LOCALIZACAO : abriga
    SALDO_LOCALIZACAO ||--o{ MOVIMENTO : origina
    SALDO_LOCALIZACAO ||--o{ RESERVA : possui
    LOTE ||--o{ MOVIMENTO : referencia
    NUMERO_SERIE ||--o{ MOVIMENTO : referencia
    MOVIMENTO }o--o| OS_REF : "vincula (baixa)"
    MOVIMENTO }o--o| SN_EQUIPAMENTO : "vincula (rastros)"
    ORDEM_COMPRA ||--o{ RECEBIMENTO : gera
    RECEBIMENTO ||--o{ MOVIMENTO : produz
```

11.2 Dicionário de entidades

Entidade	Campos-chave	Descrição
item	id, tenant_id, sku, descricao, un_base, controla_lote, controla_serial, metodo_custeio, requer_inspecao, ativo	Cadastro mestre do SKU e suas políticas de controle e custeio.
deposito	id, tenant_id, empresa_id, codigo, nome, tipo	Depósito/almoxarifado de uma empresa (multiempresa).
localizacao	id, tenant_id, deposito_id, endereco, zona, qr_endereco	Posição endereçável dentro do depósito.
saldo_localizacao	id, tenant_id, item_id, localizacao_id, lote_id?, serial_id?, qtd_fisica, qtd_reservada, qtd_quarentena, qtd_bloqueada, custo_medio	Posição do item por localização/lote/série — a "verdade" do saldo.
lote	id, tenant_id, item_id, numero, validade, fabricacao, fornecedor_id	Lote rastreável (FEFO usa validade).
numeroSerie	id, tenant_id, item_id, serial, status, localizacao_atual_id	Unidade serializada (qtd 0/1); status: disponivel , reservado , aplicado , defeito .
movimento	id, tenant_id, item_id, tipo, quantidade, custo_unit, valor_total, localizacao_origem_id?, localizacao_destino_id?, lote_id?, serial_id?, os_id?, sn_equipamento?, recebimento_id?, data, chave_idempotencia	Livro-razão <i>append-only</i> de todas as movimentações.
reserva	id, tenant_id, item_id, saldo_id, quantidade, origem, os_id?, venda_id?, status	Compromisso de saldo (hard/soft) antes do consumo.
politica_reposicao	id, tenant_id, item_id, deposito_id, estoque_seguranca, ponto_pedido, lote_economico, lead_time_dias, cobertura_dias, fornecedor_preferencial_id, modo (sugestao automatico)	Parâmetros vigentes de reposição (versionados).
classificacao_abc	id, tenant_id, item_id, periodo, classe_abc, faixa_xyz, criticidade, valor_consumo	Snapshot histórico da classificação.
previsao_demanda	id, tenant_id, item_id, deposito_id, periodo, demanda_prevista, sigma, metodo	Saída do motor de previsão por item/depósito/período.
plano_inventario	id, tenant_id, deposito_id, tipo, classe_alvo, status, tolerancia	Cabeçalho de inventário cíclico/geral.
contagem	id, tenant_id, plano_id, saldo_id, qtd_contada, qtd_teorica, divergencia, status	Linha de contagem (cega) e sua reconciliação.

11.3 Tipos de movimento (`movimento.tipo`)

Tipo	Sentido	Origem típica
<code>ENTRADA_RECEBIMENTO</code>	+	OC/recebimento
<code>ENTRADA_DEVOLUCAO</code>	+	Peça não usada retorna
<code>ENTRADA_AJUSTE</code>	+	Inventário (sobra)
<code>SAIDA_CONSUMO_OS</code>	-	Baixa automática por OS
<code>SAIDA_VENDA</code>	-	Venda faturada
<code>SAIDA_AJUSTE</code>	-	Inventário (falta), perda/descarte
<code>TRANSFERENCIA_SAIDA</code> / <code>TRANSFERENCIA_ENTRADA</code>	- / +	Transferência entre localizações/empresas
<code>RESERVA</code> / <code>ESTORNO_RESERVA</code>	(não altera físico)	Reserva de OS/venda

12. Checklist de Prontidão

Cadastro & estrutura - [] Todos os itens têm UN base, método de custeio e flags de lote/serial definidos. - [] Depósitos e localizações endereçados, com QR de prateleira impresso. - [] Fornecedores preferenciais e lead times cadastrados por item.

Identificação automática - [] Impressão de etiquetas (1D/QR) integrada ao recebimento. - [] App do técnico lê barcode, QR, RFID e NFC, com *fallback* automático. - [] Payloads QR/NFC assinados (KCD) e validados na leitura.

Movimentação & rastreabilidade - [] Baixa automática por `OSConcluida` funcionando e idempotente. - [] Reserva na abertura da OS e estorno no cancelamento. - [] Rastro peça ↔ lote/SN ↔ SN equipamento ↔ OS consultável nos dois sentidos. - [] Inventário cíclico por classe com contagem cega e trilha de ajuste.

Reposição & compras - [] SS/ROP/EOQ calculados e revisados por job periódico. - [] Previsão de demanda alimentando os parâmetros. - [] Sugestão/OC disparada ao cruzar ROP; three-way match no recebimento.

Financeiro - [] Custo médio/PEPS conciliado a cada movimento valorizado. - [] CMV publicado ao Financeiro na baixa. - [] Provisões de obsolescência/perda parametrizadas.

Não-funcionais - [] `tenant_id` em toda linha e evento (RLS ativo). - [] Operação offline-first testada (movimentos provisórios + sync). - [] Trilha de auditoria imutável de todos os movimentos.

13. Riscos

#	Risco	Prob.	Impacto	Mitigação
RK1	Divergência de saldo (físico ≠ sistema) por movimento não registrado em campo	Média	Alto	Baixa automática obrigatória, inventário cíclico por classe A, movimentos provisórios offline confirmados na sync.
RK2	Baixa dupla ao reprocessar <code>OSConcluida</code>	Média	Alto	Idempotência por <code>os_id+linha</code> ; movimento <i>append-only</i> com chave única.
RK3	Ruptura de peça crítica em campo	Média	Alto	Reposição preditiva com SS por classe/criticidade; alerta antecipado ao cruzar ROP.
RK4	Excesso/obsolescência por previsão superestimada	Média	Médio	XYZ + revisão de parâmetros; provisão de obsolescência; FEFO.
RK5	Erro de leitura RFID/NFC (tag danificada, colisão)	Média	Médio	<i>Fallback</i> para QR/barcode; leitura com confirmação; verificação de assinatura.
RK6	Custo médio distorcido por recebimento retroativo	Baixa	Médio	Regra R14: ajuste no período corrente, sem reescrever passado; conciliação com Financeiro.
RK7	Conflito de sync offline (dois técnicos, mesma peça)	Média	Médio	Reserva serializada; resolução determinística por ordem temporal (KCI).
RK8	Vazamento entre empresas (multi-tenant)	Baixa	Crítico	RLS por <code>tenant_id</code> ; testes de isolamento; transferência interempresa explícita.
RK9	Lead time real >> cadastrado	Média	Alto	Medição contínua do lead time efetivo; recalcular SS/ROP com o realizado.
RK10	Rastreabilidade incompleta (peça aplicada sem vincular SN)	Baixa	Alto	Bloquear conclusão da OS sem apontar peça+SN quando o item é rastreável.

14. Melhorias Futuras

- **Reposição com ML (KCI-analytics)**: substituir médias móveis por modelos de série temporal (sazonalidade por safra agrícola/uso do drone, promoções, campanhas de recall).
- **Previsão orientada a eventos preditivos**: consumir telemetria/horas de voo do drone para prever falha de componente e **pré-posicionar** peça antes da OS existir.
- **RFID em portal (gate reading)**: contagem automática de inventário na passagem por portais, sem contagem manual.
- **Otimização multi-echelon**: balancear estoque entre almoxarifado central e vans dos técnicos (estoque móvel).
- **Digital twin de estoque**: simulação de políticas (SS/ROP/EOQ) antes de aplicá-las em produção.
- **Sugestão de substituto**: ao faltar a peça, sugerir peça equivalente/compatível por catálogo de intercambiabilidade.
- **Sustentabilidade / economia circular**: rastreio de peças reconcondicionadas e crédito de retorno (core exchange).
- **Contagem por visão computacional**: contagem assistida por câmera no app (estimativa de quantidade em prateleira).

15. Auditoria

O KSI mantém **trilha de auditoria imutável** subordinada ao KCD (Doc 14) e ao princípio de movimentos *append-only* (R3).

- **Livro-razão de estoque**. Toda alteração de saldo é um `movimento` imutável, com autor (`usuario_id`/serviço), origem, `traceparent`, timestamp e `chave_idempotencia`. Correções são movimentos de estorno — nunca edição.

- **Assinatura de movimentos sensíveis.** Ajustes de inventário, mudança de método de custeio e transferências interempresa carregam assinatura/registro reforçado (KCD).
- **Genealogia rastreável.** Consulta bidirecional: dado um SN de equipamento, listar todas as peças aplicadas (com lote/SN, OS, data, técnico); dado um lote de peça, listar todos os equipamentos/OS onde foi aplicado (essencial para recall).
- **Versionamento de políticas.** Alterações em `politica_reposicao` e no método de custeio são versionadas com vigência (quem, quando, valores anterior/novo).
- **Conciliação contábil.** O valor total do estoque ($\sum \text{qtd_fisica} \times \text{custo_medio}$) é conciliado periodicamente com o razão do Financeiro; divergências geram alerta auditável.
- **Trilha de inventário.** Cada `plano_inventario / contagem` guarda saldo teórico, contado, divergência, contador e aprovador do ajuste.
- **Eventos auditáveis.** `EstoqueBaixado`, `InventarioAjustado`, `TransferenciaConcluida`, `PontoDePedidoAtingido` são persistidos com correlação ponta a ponta para reconstrução histórica completa.

Anexo A — Fórmulas de reposição e classificação

Curva ABC (por valor de consumo):

Valor de consumo do item = `demanda_período` × `custo_unitário`
 Ordenar itens desc. por valor de consumo → acumular % → Pareto:
 Classe A ≈ itens que somam ~80% do valor (poucos itens)
 Classe B ≈ próximos ~15% do valor
 Classe C ≈ últimos ~5% do valor (muitos itens)

Estoque de Segurança (demanda e lead time variáveis):

$SS = z \cdot \sqrt{LT \cdot \sigma_d^2 + \bar{d}^2 \cdot \sigma_{LT}^2}$
 z = fator de serviço (ex.: 95% → $z \approx 1,65$; 98% → $z \approx 2,05$)
 LT = lead time médio
 σ_d = desvio-padrão da demanda
 $\bar{d}$ = demanda média por período
 σ_{LT} = desvio-padrão do lead time

Ponto de Pedido (Reorder Point):

$ROP = \bar{d} \cdot LT + SS$

Lote Econômico de Compra (EOQ / Wilson):

$EOQ = \sqrt{2 \cdot D \cdot S / H}$
 D = demanda anual
 S = custo de emitir/pedir por ordem
 H = custo de manter uma unidade em estoque por ano

Custo Médio Ponderado Móvel (na entrada):

$\text{custo_medio_novo} = \frac{\text{qtd_atual} \cdot \text{custo_medio_atual} + \text{qtd_entrada} \cdot \text{custo_entrada}}{\text{qtd_atual} + \text{qtd_entrada}}$

Classificação XYZ (variabilidade da demanda):

Coeficiente de variação $CV = \sigma_d / \bar{d}$
 X : CV baixo → demanda estável (previsível)
 Y : CV médio → demanda variável
 Z : CV alto → demanda errática (difícil prever)

16 Business Intelligence

Documento: 16 – Business Intelligence **Versão:** 1.0 · **Data:** 21 de julho de 2026 · **Status:** Ativo **Dependências:** 00, 01, 02, 03, 04, 05, 06, 07, 08, 09, 10, 11, 12, 13, 14, 15 **Responsável:** Especialista BI

1. Resumo Executivo

Este documento define a **plataforma de Business Intelligence & Analytics** do Drone Kairós ERP: como os dados operacionais gerados pelo ecossistema de drones (ERP, Ordens de Serviço, KSI/estoque, Financeiro, CRM, KCI/diagnósticos e telemetria por Serial Number) são capturados, transformados, modelados e servidos como **indicadores acionáveis por perfil** — fabricante, revenda, técnico, cliente e administrador —, sem jamais violar o princípio constitucional de **multiempresa como cidadã de primeira classe** e **segurança por padrão**.

A tese central é a **separação de planos**: o plano transacional (OLTP, 08/09) permanece otimizado para escrita e consistência; o plano analítico (OLAP) vive em um **lakehouse** próprio, alimentado por **ELT event-driven** a partir do backbone de eventos de domínio (09) e por **CDC** (Change Data Capture) dos bancos de serviço. Entre os dois planos há uma fronteira explícita — nenhuma consulta analítica pesada toca o OLTP em produção. O modelo de consumo combina **near-real-time** (métricas operacionais que não podem esperar o *batch*: OS em atraso, ruptura de estoque, alertas KCI) com **batch curado** (indicadores gerenciais, curva ABC, sazonalidade, garantia por lote).

A arquitetura adota **camadas medalhão** (raw → staged → curated), **modelagem dimensional em star schema** na camada de consumo, um **catálogo de dados** com linhagem automatizada e — o requisito não-negociável — **isolamento multiempresa de ponta a ponta**: particionamento físico por `tenant_id` na ingestão, *Row-Level Security analítico* (RLS) na camada semântica e políticas de coluna para PII. Sobre isso se apoiam **dashboards por perfil**, **self-service BI** governado e integrações verticais com **KCI** (indicadores preditivos de falha) e **KSI** (giro e curva ABC).

O documento entrega o **catálogo de KPIs por perfil**, a arquitetura de dados em diagramas Mermaid, o **fluxograma do pipeline** (ingestão → transformação → publicação), um **exemplo completo de modelo dimensional** (star schema de Ordens de Serviço), padrões de governança, avaliação **BI embutido vs. externo**, casos de uso, checklist de prontidão, riscos e trilha de auditoria.

2. Objetivos

- **O1.** Definir o **catálogo de KPIs por perfil** com fórmula, granularidade, fonte, frequência e dono de cada métrica — eliminando "números órfãos".
- **O2.** Estabelecer a **arquitetura analítica** (lakehouse, camadas raw/staged/curated, ELT, star schema) desacoplada do plano transacional.
- **O3.** Garantir **isolamento multiempresa** no BI com o mesmo rigor do OLTP: nenhum tenant vê dado de outro, em nenhuma camada, por padrão.
- **O4.** Entregar **near-real-time** para métricas operacionais críticas via consumo dos eventos de domínio, sem sacrificar a consistência do *batch* curado.
- **O5.** Prover **self-service BI governado**: analistas de tenant exploram dados sem escrever SQL bruto e sem furar o isolamento.
- **O6.** Integrar **KCI** (preditivo: probabilidade de falha, MTBF, saúde por SN) e **KSI** (giro, cobertura, curva ABC) como domínios analíticos de primeira classe.
- **O7.** Instituir **governança de dados**: catálogo, glossário de negócio, qualidade (testes), linhagem e trilha de auditoria de acesso a dados.
- **O8.** Decidir a **estratégia de ferramentas** (BI embutido no produto vs. BI externo corporativo) com critérios objetivos e caminho de convivência.

3. Escopo

No escopo: modelo de camadas do lakehouse; pipelines ELT/CDC e *streaming*; modelagem dimensional (fatos/dimensões, SCD); camada semântica e métricas; RLS analítico e isolamento multiempresa; catálogo, qualidade e linhagem; dashboards por perfil e self-service; integração analítica com KCI e KSI; avaliação de ferramentas (embutido vs. externo); trilha de auditoria de dados.

Fora do escopo (referência a outros documentos): modelagem OLTP e estratégia de SN (08); contratos de API e catálogo de eventos de domínio (09); IAM/RBAC-ABAC e KCD (07); motores de ML e *feature engineering* do KCI (13); regras de negócio de estoque do KSI (15); infraestrutura/IaC, clusters e FinOps de plataforma (08/operação). Aqui tratamos do **plano analítico** e de seus contratos de consumo — consumimos os eventos e as tabelas descritas nesses documentos, não os redefinimos.

4. Regras

- **R-01. Fronteira OLTP/OLAP.** Nenhuma carga analítica consulta bancos de serviço em produção. A entrada no lakehouse é sempre por evento de domínio ou CDC de réplica.
- **R-02. Tenant em toda linha.** Todo registro em toda camada (raw, staged, curated) carrega `tenant_id`. Ausência de `tenant_id` é rejeição na ingestão, não *default*.
- **R-03. RLS analítico "negar por padrão".** A camada semântica nega acesso na ausência de política explícita de tenant + papel. Isolamento não depende de o dashboard "filtrar certo".
- **R-04. Métrica tem dono e definição única.** Todo KPI existe uma vez no catálogo de métricas (camada semântica). Proibido redefinir a mesma métrica em cada dashboard.
- **R-05. Curado é imutável por reprocesso.** A camada *curated* é reconstruível 100% a partir de raw+staged; nunca sofre `UPDATE` manual. Correção = reprocessar.
- **R-06. PII minimizada e mascarada.** Dado pessoal só sobe ao analítico se um KPI exigir; quando sobe, é mascarado/pseudonimizado por política de coluna (LGPD, 08 §11).
- **R-07. Linhagem obrigatória.** Toda tabela curated e todo KPI têm linhagem rastreável até a fonte. Sem linhagem, não publica.
- **R-08. Contrato de dados versionado.** Mudança de esquema de evento/fonte respeita compatibilidade retroativa; quebra exige nova versão e janela de migração (alinha 09 R-07).
- **R-09. Freshness declarada.** Todo dashboard/KPI expõe sua **latência de dados** (ex.: "near-real-time ≤ 60 s" ou "batch D-1 04:00"). Número sem carimbo de frescor é proibido.
- **R-10. Qualidade como gate.** Nenhuma promoção staged → curated ocorre sem passar nos testes de qualidade (unicidade, não-nulo, integridade referencial, faixa). Falha bloqueia publicação.
- **R-11. Sem exportação que fure isolamento.** Exportações/downloads herdam o RLS do usuário; agregados que permitam reidentificação cruzada entre tenants são bloqueados.
- **R-12. Auditoria de acesso a dado.** Toda consulta a dado sensível e todo acesso administrativo *cross-tenant* (suporte) é registrado em trilha imutável (§15).

5. Arquitetura (analítica)

5.1 Planos e princípios

O sistema opera em **dois planos com contrato explícito entre eles**:

1. **Plano transacional (OLTP)** — bancos por serviço (08 / 09), otimizados para escrita/consistência. **Não** é consultado por BI.
2. **Plano analítico (OLAP)** — lakehouse próprio, otimizado para varredura, agregação e histórico, com governança e isolamento nativos.

A ponte entre os planos tem **dois caminhos complementares**:

- **Streaming (near-real-time):** o BI é apenas mais um consumidor do **backbone de eventos** (09) — `OSConcluida`, `EstoqueBaixado`, `DiagnosticoRegistrado`, `EquipamentoRegistrado`. Latência-alvo de segundos.

- **Batch/CDC (curado):** *Change Data Capture* das réplicas de leitura + cargas incrementais programadas, para consistência e para dimensões de mudança lenta. Latência-alvo D-1 / intradiário.

5.2 Camadas do lakehouse (medalhão)

Camada	Apelido	Conteúdo	Formato/modelo	Consumidor
Raw (Bronze)	<i>landing</i>	Cópia fiel e imutável do evento/registro-fonte, particionada por <code>tenant_id</code> /data	Arquivos colunares append-only + tabela de lakehouse (ACID)	Somente pipelines
Staged (Silver)	<i>conformed</i>	Dados limpos, tipados, deduplicados, conformados (chaves de negócio, SN normalizado), testes de qualidade aplicados	Tabelas normalizadas/wide, versionadas	Pipelines, <i>data scientists</i> (KCI)
Curated (Gold)	<i>serving</i>	Star schemas por domínio, agregados, métricas materializadas, exposição semântica	Modelagem dimensional + camada semântica	Dashboards, self-service, APIs de BI

- **Raw** é a **fonte de verdade reconstruível**: guarda o dado como chegou, sem interpretação. Retenção longa, custo frio.
- **Staged** aplica **qualidade e conformidade** (R-10): normaliza SN, resolve duplicidade de eventos idempotentes, alinha domínios (status de OS, categorias de estoque).
- **Curated** entrega **modelos dimensionais** e a **camada semântica** onde vivem as métricas (R-04) e o **RLS analítico** (R-03).

5.3 Camada semântica e serviço

Sobre a *curated* existe uma **camada semântica** (o "cérebro" do BI): define **métricas versionadas**, **hierarquias** (tempo, geografia, produto, tenant), **relacionamentos** entre fatos e dimensões e — crucialmente — as **políticas de RLS**. Toda ferramenta de consumo (dashboard embutido, BI externo, API de indicadores) atravessa esta camada; **ninguém consulta a curated cru diretamente em produção**. É aqui que "faturamento", "giro de estoque" ou "taxa de retrabalho" ganham uma definição única e auditável.

5.4 Isolamento multiempresa no plano analítico

O isolamento é **defesa em profundidade**, replicando no OLAP a filosofia do OLTP (08 §5):

1. **Ingestão**: particionamento físico por `tenant_id` (R-02); um evento sem tenant não entra.
2. **Armazenamento**: *partition pruning* garante que consultas de um tenant nem leem arquivos de outro.
3. **Semântica**: **RLS analítico** injeta o predicado de tenant + papel em toda query, "negar por padrão" (R-03).
4. **Exposição**: exportações herdam RLS; agregados reidentificáveis são bloqueados (R-11).
5. **Auditoria**: todo acesso *cross-tenant* (suporte/admin) é registrado (R-12).

Tenants *enterprise* podem receber **isolamento físico** (schema/catálogo dedicado no lakehouse) como *tier premium*, análogo ao schema dedicado do OLTP (08 §5.3).

6. Diagramas

6.1 Arquitetura de dados (visão macro)

```
flowchart LR
    subgraph OLTP["Plano Transacional (08/09)"]
        ERP[ERP]
        OS[Ordens de Serviço]
        KSI[KSI · Estoque]
        FIN[Financeiro]
        CRM[CRM]
        KCI[KCI · Diagnósticos]
        TEL[Telemetria por SN]
    end

    subgraph BUS["Backbone de Eventos (09)"]
        EV["Eventos de Domínio<br/>OSConcluida · EstoqueBaixado<br/>DiagnosticoRegistrado ..."]
    end

    subgraph LH["Lakehouse Analítico (OLAP)"]
        RAW["Raw / Bronze<br/>imutável · part. por tenant"]
        STG["Staged / Silver<br/>limpo · conformado · testado"]
        CUR["Curated / Gold<br/>star schemas · agregados"]
        SEM["Camada Semântica<br/>métricas · RLS analítico"]
    end

    subgraph CONS["Consumo"]
        DEMB["BI Embutido<br/>(dashboards por perfil)"]
        DEXT["BI Externo<br/>(analistas)"]
        APIBI["API de Indicadores"]
        DS["Data Science / KCI"]
    end

    ERP & OS & KSI & FIN & CRM & KCI & TEL -->|CDC / réplica| RAW
    ERP & OS & KSI & FIN & CRM & KCI & TEL -->|publica| EV
    EV -->|streaming near-real-time| RAW
    RAW --> STG --> CUR --> SEM
    STG --> DS
    SEM --> DEMB & DEXT & APIBI
    DS -->|scores preditivos| CUR
```

6.2 Isolamento multiempresa (defesa em profundidade)

```
flowchart TB
    Q["Consulta de um usuário<br/>(tenant T, papel P)"] --> G1
    G1["tenant_id presente<br/>e válido?"] -->|não| DENY1["Negado (R-02)"]
    G1 -->|sim| G2["Política RLS<br/>tenant T + papel P<br/>existe?"]
    G2 -->|não| DENY2["Negado por padrão (R-03)"]
    G2 -->|sim| PRUNE["Partition pruning<br/>lê só partições de T"]
    PRUNE --> MASK["Mascaramento de PII<br/>por política de coluna (R-06)"]
    MASK --> AGG["Agregado permite<br/>reidentificação?"]
    AGG -->|sim| DENY3["Bloqueado (R-11)"]
    AGG -->|não| OUT["Resultado + carimbo de freshness (R-09)"]
    OUT --> LOG["Trilha de auditoria (R-12)"]
```

6.3 Convivência de latências (Lambda-like)

```
flowchart LR
    EV[Eventos] --> SPEED["Camada de velocidade<br/>materializações near-real-time"]
    CDC["CDC / Batch"] --> BATCH["Camada batch<br/>curated D-1 / intradiário"]
    SPEED --> SERVE["Camada de serviço<br/>(semântica unifica)"]
    BATCH --> SERVE
    SERVE --> DASH["Dashboards por perfil"]
```

7. Fluxogramas (pipeline de dados)

7.1 Pipeline ELT ponta a ponta (ingestão → publicação)

```
flowchart TD
    A["Fonte: evento de domínio ou CDC"] --> B{"tenant_id presente?"}
    B -->|não| R1["Rejeita → fila de quarentena"]
    B -->|sim| C["Ingestão em RAW<br/>append-only · part. tenant/data"]
    C --> D["Transformação → STAGED<br/>tipagem · dedupe · SN normalizado"]
    D --> E{"Testes de qualidade<br/>(unicidade, nulos, FK, faixa)"}
    E -->|falha| Q["Bloqueia publicação<br/>alerta + quarentena (R-10)"]
    E -->|passa| F["Modelagem → CURATED<br/>fatos + dimensões (SCD)"]
    F --> G["Atualiza camada semântica<br/>métricas + RLS"]
    G --> H["Registra linhagem + freshness"]
    H --> I["Publica em dashboards / API / self-service"]
    Q -->|correção + reprocesso| D
```

7.2 Reprocessamento e correção (imutabilidade da curated · R-05)

```
flowchart LR
    ERR["Erro detectado em curated"] --> ROOT{"Causa na fonte<br/>ou na transformação?"}
    ROOT -->|fonte| FIX1["Corrige raw via nova carga<br/>(raw permanece append-only)"]
    ROOT -->|transformação| FIX2["Corrige código do modelo<br/>(versionado)"]
    FIX1 & FIX2 --> REPLAY["Replay determinístico<br/>raw → staged → curated"]
    REPLAY --> VALID["Revalida qualidade + reconciliação"]
    VALID --> PUB["Republica (sem UPDATE manual)"]
```

7.3 Integração near-real-time (alerta operacional)

```
sequenceDiagram
    participant SVC as Serviço de OS (09)
    participant BUS as Backbone de Eventos
    participant SPD as Camada de Velocidade
    participant SEM as Semântica (RLS)
    participant DSH as Dashboard Técnico
    SVC->>BUS: OSAtasada {tenant, os_id, sla}
    BUS->>SPD: consome (idempotente)
    SPD->>SPD: atualiza métrica "OS em atraso"
    SPD->>SEM: materializa (part. tenant)
    DSH->>SEM: consulta (tenant T, papel técnico)
    SEM-->>DSH: OS em atraso ≤ 60s + freshness
```

8. Boas Práticas

- **ELT sobre ETL.** Carrega bruto primeiro (raw imutável), transforma dentro do lakehouse com SQL versionado e testável. Reprocesso barato e auditável.
- **Idempotência no consumo.** Todo consumidor de evento é idempotente (chave de deduplicação por `event_id`), espelhando [09 R-06](#); *replays* não inflam métricas.
- **Modelagem dimensional na serving.** Star schema (não *snowflake* excessivo) para desempenho de agregação e clareza de negócio; *snowflake* só onde a dimensão exige.
- **SCD explícito.** Dimensões com histórico relevante (cliente, produto, tenant, técnico) usam **Slowly Changing Dimension Tipo 2**; o que só precisa do estado atual usa Tipo 1.
- **Métrica única, muitos gráficos.** Definir a métrica uma vez na semântica (R-04); dashboards são *views*, nunca redefinições.
- **Freshness visível.** Todo painel carimba latência de dados (R-09); "não sei quão fresco" corrói confiança mais que atraso conhecido.
- **Grão declarado.** Cada tabela-fato declara seu **grão** (uma linha = ?) antes de qualquer medida; medidas nunca misturam grãos.

- **PII fora por padrão.** Só sobe PII ao analítico o que um KPI justifica; o resto fica no OLTP (R-06). Pseudonimização preserva `SN` /histórico sem expor a pessoa.
- **Testes de dados como código.** Qualidade versionada junto ao modelo; *pull request* de modelo roda testes de dados (R-10), como teste de software.
- **Contratos de dados.** Fonte e BI acordam esquema versionado (R-08); mudança quebra-contrato não vaza silenciosamente para dashboards.
- **Custo consciente (FinOps analítico).** Particionar e *cluster* por chaves de filtro comuns (tenant, tempo); materializar o que é caro e repetido; expirar raw frio.

9. Padrões

- **P-01. Camadas medalhão** (raw/staged/curated) como topologia obrigatória do lakehouse (§5.2).
- **P-02. Star schema** como padrão da camada de serviço; *conformed dimensions* compartilhadas entre fatos (tempo, tenant, produto, SN).
- **P-03. SCD2** para dimensões historicamente sensíveis; chave substituta (*surrogate key*) + `valido_de` / `valido_ate` / `corrente`.
- **P-04. Nomenclatura:** fatos `fato_<processo>` (grão no nome quando útil), dimensões `dim_<entidade>`, agregados `agg_<metrica>_<grão>`, staging `stg_<fonte>`.
- **P-05. Chave de negócio vs. substituta:** SN e IDs de negócio preservados; *joins* dimensionais por *surrogate key*.
- **P-06. RLS analítico declarativo:** política por (tenant, papel) na semântica; nunca filtro hardcoded no dashboard.
- **P-07. Métricas versionadas:** definição de KPI em código, com dono, descrição de negócio e fórmula (catálogo, §11.4).
- **P-08. Linhagem automatizada:** derivada do grafo de transformações; toda coluna curated rastreável à origem (R-07).
- **P-09. Convenção de freshness:** rótulos padronizados — `NRT` (≤ 60 s), `INTRADAY`, `D-1`, `D-7`.
- **P-10. Camada semântica única:** uma definição por métrica alimenta BI embutido, BI externo e API de indicadores (single source of truth).

10. Casos de Uso

- **UC-01 — Fabricante, garantia por lote.** Detecta lote de produção com taxa de defeito anômala cruzando OS de garantia + KCI + SN. **Fonte:** OS, KCI, produto/lote. **Freshness:** D-1. **Perfil:** fabricante.
- **UC-02 — Revenda, funil de pós-venda.** Vendas por período + taxa de retorno de garantia + NPS de CRM, isolado ao tenant revenda. **Fonte:** ERP/Financeiro, OS, CRM. **Freshness:** intradiário. **Perfil:** revenda.
- **UC-03 — Técnico, produtividade de OS.** OS concluídas, tempo médio de reparo (MTTR), taxa de retrabalho, backlog em atraso. **Fonte:** OS, KCI. **Freshness:** NRT para "em atraso", D-1 para produtividade. **Perfil:** técnico.
- **UC-04 — Cliente, disponibilidade do equipamento.** Uptime por SN, histórico de manutenção, alertas preditivos KCI do próprio parque. **Fonte:** telemetria, OS, KCI. **Freshness:** NRT. **Perfil:** cliente.
- **UC-05 — Admin, saúde da plataforma.** Volume por tenant, latência de pipeline, *freshness* SLA, uso por módulo, sem violar isolamento (agregados). **Fonte:** metadados de plataforma. **Perfil:** administrador.
- **UC-06 — KSI, curva ABC + giro.** Classifica SKUs A/B/C por valor de saída e calcula giro e cobertura para reposição. **Fonte:** KSI, Financeiro. **Freshness:** D-1. **Perfil:** revenda/fabricante.
- **UC-07 — KCI, risco preditivo agregado.** Distribuição de probabilidade de falha por modelo/lote/tenant, alimentando planejamento de peças (liga KSI). **Fonte:** KCI, KSI. **Freshness:** D-1. **Perfil:** fabricante/técnico.
- **UC-08 — Suporte cross-tenant auditado.** Admin de suporte inspeciona dado de um tenant sob *break-glass*, tudo registrado em trilha imutável (R-12). **Perfil:** administrador (privilegiado).
- **UC-09 — Self-service governado.** Analista de tenant monta relatório novo arrastando métricas certificadas, sem SQL e sem furar RLS. **Perfil:** analista de tenant.

11. Modelagem (dimensional + catálogo de KPIs)

11.1 Exemplo de modelo dimensional — Star schema de Ordens de Serviço

O processo de negócio "**execução de Ordem de Serviço**" é o coração operacional. Grão do fato: **uma linha por OS concluída** (medidas aditivas por OS).

```

erDiagram
    FATO_ORDEM_SERVICO }o--|| DIM_TEMPO : "data_conclusao"
    FATO_ORDEM_SERVICO }o--|| DIM_TENANT : "tenant_sk"
    FATO_ORDEM_SERVICO }o--|| DIM_EQUIPAMENTO : "equipamento_sk (SN)"
    FATO_ORDEM_SERVICO }o--|| DIM_TECNICO : "tecnico_sk"
    FATO_ORDEM_SERVICO }o--|| DIM_CLIENTE : "cliente_sk"
    FATO_ORDEM_SERVICO }o--|| DIM_DIAGNOSTICO : "diagnostico_sk (KCI)"
    FATO_ORDEM_SERVICO }o--|| DIM_TIPO_SERVICO : "tipo_servico_sk"

    FATO_ORDEM_SERVICO {
        bigint os_sk PK
        int tenant_sk FK
        int tempo_sk FK
        int equipamento_sk FK
        int tecnico_sk FK
        int cliente_sk FK
        int diagnostico_sk FK
        int tipo_servico_sk FK
        string os_id "chave de negócio"
        int tempo_reparo_min "medida"
        int tempo_espera_min "medida"
        decimal custo_pecas "medida"
        decimal custo_mao_obra "medida"
        boolean em_garantia "flag"
        boolean retrabalho "flag"
        boolean sla_cumprido "flag"
    }

    DIM_TEMPO {
        int tempo_sk PK
        date data
        int ano
        int mes
        int semana
        string dia_semana
        boolean feriado
    }

    DIM_EQUIPAMENTO {
        int equipamento_sk PK
        string serial_number "business key"
        string modelo
        string lote_producao
        date data_fabricacao
        int tenant_sk
    }

    DIM_TECNICO {
        int tecnico_sk PK
        string tecnico_id
        string nome_mascarado "PII pseudonimizada"
        string nivel
        boolean corrente "SCD2"
    }

    DIM_TENANT {
        int tenant_sk PK
        string tenant_id
        string tipo "fabricante|revenda|..."
        string regioao
    }

    DIM_DIAGNOSTICO {
        int diagnostico_sk PK
        string categoria_falha
        string severidade
        decimal probab_falha_prevista "KCI"
    }

```

Dimensões conformadas (`dim_tempo`, `dim_tenant`, `dim_equipamento`) são compartilhadas com os fatos de **estoque** (`fato_movimento_estoque`, KSI), **financeiro** (`fato_lancamento`) e **vendas** (`fato_venda`), garantindo que "por SN", "por lote", "por período" e "por tenant" signifiquem o mesmo em todos os domínios (P-02).

11.2 DDL ilustrativa da tabela-fato (camada curated)

```
-- Grão: 1 linha por Ordem de Serviço concluída
CREATE TABLE curated.fato_ordem_servico (
  os_sk          BIGINT      NOT NULL,          -- surrogate key
  tenant_sk      INT         NOT NULL,          -- isolamento (R-02)
  tempo_sk       INT         NOT NULL,
  equipamento_sk INT         NOT NULL,          -- SN
  tecnico_sk     INT         NOT NULL,
  cliente_sk     INT         NOT NULL,
  diagnostico_sk INT,                -- KCI (pode ser nulo)
  tipo_servico_sk INT         NOT NULL,
  os_id          VARCHAR(32) NOT NULL,          -- business key
  tempo_reparo_min INT       NOT NULL,
  tempo_espera_min INT       NOT NULL,
  custo_pecas    NUMERIC(12,2) NOT NULL DEFAULT 0,
  custo_mao_obra NUMERIC(12,2) NOT NULL DEFAULT 0,
  em_garantia    BOOLEAN     NOT NULL,
  retrabalho     BOOLEAN     NOT NULL,
  sla_cumprido   BOOLEAN     NOT NULL,
  _fonte         VARCHAR(64) NOT NULL,          -- linhagem (R-07)
  _ingestao_ts   TIMESTAMP   NOT NULL,          -- freshness (R-09)
  PRIMARY KEY (os_sk)
)
PARTITION BY (tenant_sk, tempo_sk);           -- pruning por tenant/tempo
```

11.3 Catálogo de KPIs por perfil

Cada KPI declara **fórmula**, **granularidade**, **fonte**, **frescor** e **dono** (O1, R-04). **NRT** = near-real-time (≤ 60 s); **D-1** = *batch* diário.

Fabricante — qualidade, garantia e engenharia

KPI	Fórmula (resumo)	Grão	Fonte	Frescor
Taxa de defeito por lote	OS de garantia do lote ÷ unidades do lote	Lote/modelo	OS, Equipamento	D-1
MTBF por modelo	Σ horas em operação ÷ nº de falhas	Modelo/SN	Telemetria, KCI	D-1
Custo de garantia	Σ (peças + mão de obra) de OS em garantia	Lote/período	OS, Financeiro	D-1
Índice de recall potencial	lotes com defeito > limiar preditivo KCI	Lote	KCI, OS	D-1
Falha prevista vs. realizada	acurácia do preditivo KCI vs. OS efetivas	Modelo	KCI, OS	D-7

Revenda — vendas e pós-venda

KPI	Fórmula (resumo)	Grão	Fonte	Frescor
Receita de vendas	Σ valor de vendas	Loja/período	ERP, Financeiro	Intraday
Ticket médio	receita ÷ nº de vendas	Loja/período	ERP	D-1
Taxa de retorno em garantia	OS de garantia ÷ unidades vendidas	Modelo/período	OS, ERP	D-1
NPS / satisfação	média de pesquisa CRM	Cliente/período	CRM	D-1
Cobertura de estoque	estoque atual ÷ giro médio (KSI)	SKU/loja	KSI	D-1

Técnico — produtividade e OS

KPI	Fórmula (resumo)	Grão	Fonte	Frescor
OS concluídas	contagem de OS fechadas	Técnico/período	OS	NRT
MTTR (tempo médio de reparo)	média de <code>tempo_reparo_min</code>	Técnico/tipo	OS	D-1
Taxa de retrabalho	OS reabertas ÷ OS concluídas	Técnico	OS	D-1
Backlog em atraso	OS abertas com SLA estourado	Técnico	OS	NRT
Aderência ao SLA	OS com <code>sla_cumprido</code> ÷ total	Técnico/tipo	OS	D-1

Cliente — disponibilidade do equipamento

KPI	Fórmula (resumo)	Grão	Fonte	Frescor
Uptime do equipamento	horas disponíveis ÷ horas totais	SN	Telemetria	NRT
Alertas preditivos abertos	contagem de alertas KCI ativos	SN	KCI	NRT
Histórico de manutenção	nº e custo de OS por SN	SN/período	OS	D-1
Tempo até próxima manutenção	previsão KCI (RUL)	SN	KCI	D-1

Administrador — saúde da plataforma

KPI	Fórmula (resumo)	Grão	Fonte	Frescor
Freshness SLA cumprido	pipelines dentro do SLA ÷ total	Pipeline	Metadados	NRT
Volume por tenant	eventos/linhas processadas	Tenant/dia	Metadados	D-1
Qualidade de dados	testes aprovados ÷ testes totais	Modelo	Metadados	D-1
Custo analítico por tenant	consumo de compute/armazenamento	Tenant	FinOps	D-1
Acessos cross-tenant	contagem de eventos break-glass	Plataforma	Trilha (\$15)	NRT

11.4 Catálogo de dados e glossário de negócio

O **catálogo de dados** indexa toda tabela, coluna, métrica e dashboard, com: descrição de negócio, dono, classificação de sensibilidade (público/interno/PII), **linhagem** (R-07) e frescor (R-09). O **glossário** amarra termos de negócio ("giro", "retrabalho", "em garantia") às métricas da camada semântica, eliminando ambiguidade entre áreas. Cada entrada do catálogo é a fonte para descoberta no **self-service** — o analista encontra a métrica certificada em vez de recriá-la.

11.5 Integração KCI (preditivo) e KSI (giro/ABC)

- **KCI → BI.** Os *scores* preditivos (probabilidade de falha, RUL — vida útil remanescente, severidade) produzidos pelos modelos do 13 são **materializados na staged** e projetados em `dim_diagnostico` e em fatos por SN. O BI **consome** o preditivo; não o calcula. Isso alimenta *recall* potencial (fabricante), próxima manutenção (cliente) e priorização de backlog (técnico).
- **KSI → BI.** O `fato_movimento_estoque` (grão: uma linha por movimento) permite calcular **giro** (custo de saída ÷ estoque médio) e **curva ABC** (classificação por valor acumulado de saída). A ligação KCI×KSI é o caso de maior valor: **falha prevista puxa demanda de peça** — o preditivo de KCI, agregado por peça/lote, antecipa a curva de reposição do KSI.

```

flowchart LR
    KCI["Modelos KCI (13)"] -->|scores por SN| STG2["Staged"]
    STG2 --> DIMD["dim_diagnostico"]
    KSI["Eventos KSI (15)"] --> STG3["Staged"]
    STG3 --> FME["fato_movimento_estoque"]
    DIMD --> ABC["Curva ABC preditiva<br/>(peças por falha prevista)"]
    FME --> ABC
    ABC --> REP["Recomendação de reposição"]

```

11.6 Ferramentas — BI embutido vs. BI externo

Decisão orientada por **quem consome** e **quanto isolamento** o caminho exige.

Critério	BI Embutido (no produto)	BI Externo (corporativo)
Público	Todos os perfis, dentro do ERP	Analistas, área de dados
Isolamento multiempresa	Nativo (herda RLS da semântica)	Exige integração cuidadosa de RLS
Experiência	Contextual, "no fluxo"	Exploração livre, ad-hoc
Governança	Total (mesma semântica)	Requer conectar à camada semântica
Curva de adoção	Baixa (já no produto)	Média/alta
Custo/licenças	Incluso no produto	Licenciamento por usuário
Flexibilidade analítica	Média (dashboards curados)	Alta (modelagem própria)

Estratégia adotada — convivência:

1. **BI embutido** é o padrão para os cinco perfis: dashboards por perfil e self-service governado **dentro do ERP**, herdando RLS e semântica automaticamente (isolamento nativo, R-03).
2. **BI externo** é habilitado **para o perfil analista/administrador**, conectado **exclusivamente à camada semântica** (nunca à curated cru), preservando o RLS e o catálogo único de métricas (P-10).
3. **Regra de ouro:** independentemente da ferramenta, a **definição de métrica e a política de isolamento vivem uma única vez na camada semântica**. A ferramenta é intercambiável; a governança não.

12. Checklist

- ☒ Fronteira OLTP/OLAP definida; BI consome eventos/CDC, nunca o transacional (R-01).
- ☒ Camadas medalhão (raw/staged/curated) especificadas com dono e formato.
- ☒ `tenant_id` obrigatório em toda camada; particionamento físico por tenant (R-02).
- ☒ RLS analítico "negar por padrão" na camada semântica (R-03).
- ☒ Catálogo de KPIs por perfil com fórmula, grão, fonte, frescor e dono.
- ☒ Star schema de OS entregue (fato + dimensões conformadas + SCD2 + DDL).
- ☒ Pipeline ELT e fluxo de reprocessamento (curated imutável) diagramados.
- ☒ Near-real-time via eventos + batch curado (convivência de latências).
- ☒ Integração KCI (preditivo) e KSI (giro/ABC) modeladas.
- ☒ Estratégia de ferramentas (embutido + externo) decidida com convivência.
- ☒ Governança: catálogo, glossário, qualidade (gate) e linhagem definidos.
- ☐ ADR-0016 (fronteira OLTP/OLAP e escolha de lakehouse) a formalizar.
- ☐ ADR-0017 (RLS analítico e isolamento multiempresa no OLAP) a formalizar.
- ☐ Benchmark de motor de consulta analítica e política de materialização (Fase de operação).
- ☐ SLAs de freshness por dashboard a ratificar com áreas de negócio.

13. Riscos

#	Risco	Impacto	Mitigação
RK-01	Vazamento entre tenants no analítico (agregado ou export reidentificável).	Crítico (confiança/legal)	RLS "negar por padrão" + partition pruning + bloqueio de agregado reidentificável (R-03, R-11) + auditoria (R-12).
RK-02	Métrica divergente entre dashboards ("qual número está certo?").	Alto (credibilidade)	Métrica única na semântica (R-04, P-10); dashboards são views.
RK-03	BI consultando OLTP e degradando produção.	Alto (disponibilidade)	Fronteira dura OLTP/OLAP (R-01); ingestão só por evento/CDC.
RK-04	Dado stale sem sinalização levando a decisão errada.	Alto	Freshness carimbada em todo KPI (R-09); alerta de SLA de pipeline.
RK-05	Explosão de custo do lakehouse (varredura/materialização).	Médio/Alto	Particionamento por tenant/tempo, materialização seletiva, expiração de raw frio (FinOps §8).
RK-06	PII no analítico violando LGPD.	Alto (legal)	PII fora por padrão; pseudonimização/mascaramento por coluna (R-06).
RK-07	Falha de qualidade publicada (nulos, duplicidade, FK quebrada).	Alto	Qualidade como gate staged → curated (R-10); quarentena e reprocesso.
RK-08	Preditivo KCI tratado como verdade sem monitorar acurácia.	Médio	KPI "previsto vs. realizado" (§11.3); BI consome, não decide sozinho.
RK-09	Reprocessamento inconsistente (curated alterada à mão).	Médio	Curated imutável (R-05); só replay determinístico (§7.2).
RK-10	Self-service furando governança (SQL bruto, joins cross-tenant).	Alto	Self-service só sobre semântica certificada (UC-09); sem acesso à curated crua.

14. Melhorias Futuras

- Formalizar **ADR-0016** (fronteira OLTP/OLAP + escolha de lakehouse) e **ADR-0017** (RLS analítico multiempresa).
- **Data mesh por domínio** conforme a organização escalar: cada domínio (OS, KSI, KCI, Financeiro) como *data product* com dono e SLA próprios.
- **Feature store** compartilhada entre BI e KCI, evitando reengenharia de atributos por SN (integra 13).
- **Deteção de anomalia automatizada** sobre KPIs (garantia por lote, giro, retrabalho) com alerta proativo.
- **Métricas semânticas via API** para embutir indicadores certificados em qualquer superfície (portal do cliente, app do técnico).
- **Contratos de dados versionados** (schema registry) entre fontes e BI, com quebra bloqueante em CI (R-08).
- **Linhagem em nível de coluna** ponta a ponta e catálogo com busca em linguagem natural.
- **Otimização de custo automatizada** (right-sizing de materializações, tiering de armazenamento por acesso).
- **Data clean rooms** para *benchmarking* anônimo entre tenants opt-in, sem reidentificação (ex.: "seu retrabalho vs. mediana do setor").

15. Auditoria

- **Consistência constitucional:** ✓ alinhado aos Artigos de **multiempresa como 1ª classe** (isolamento em toda camada, R-02/R-03), **segurança por padrão** (RLS "negar por padrão", auditoria de acesso), **SN universal** (dimensão conformada de equipamento por Serial Number) e **privacidade/histórico** (PII minimizada, LGPD, R-06).
- **Consistência com o Bible:** ✓ cobre os módulos **KCI** (preditivo) e **KSI** (giro/ABC) como domínios analíticos e as fontes ERP/OS/Financeiro/CRM/Telemetria (02).

- **Rastreabilidade de roadmap:** entrega a etapa de **Business Intelligence & Analytics** da plataforma, consumindo os contratos de dados de 08 (modelo OLTP/SN), 09 (eventos de domínio), 13 (KCI) e 15 (KSI).
 - **Trilha de auditoria de dados (operacional):** todo acesso a dado sensível, toda exportação e todo acesso *cross-tenant* (break-glass de suporte, UC-08) são registrados em log **imutável e encadeado por hash** (padrão de 08 §11.4), com quem/quando/qual tenant/qual dado.
 - **Pendências abertas:** ADR-0016/0017; benchmark de motor analítico e política de materialização; SLAs de freshness por dashboard a ratificar.
 - **Estado:** aprovado como **v1.0**; fornece o catálogo de KPIs, a modelagem dimensional e os contratos de consumo analítico para as camadas de produto (dashboards embutidos), para o BI externo e para a operação (19).
-

Fim do 16 – Business Intelligence · v1.0

17 CRM

Documento: 17 – CRM **Versão:** 1.0 · **Data:** 21 de julho de 2026 · **Status:** Ativo **Dependências:** 00 a 16 **Responsável:** Especialista CRM / Relacionamento

1. Resumo Executivo

Este documento especifica o **módulo de CRM (Customer Relationship Management)** do Drone Kairós ERP: o *bounded context* responsável por **contas, contatos, funil comercial, pós-venda, histórico 360°, automações de relacionamento e conformidade LGPD** dentro de uma plataforma **multiempresa de primeira classe** para o ecossistema de drones.

O diferencial estratégico do CRM Kairós não é a mecânica genérica de pipeline — é o **acoplamento nativo ao ativo físico rastreado por Serial Number (SN)** e ao **histórico vitalício** do equipamento. No ecossistema de drones, o relacionamento não termina na venda: um drone tem ciclo de vida de anos, atravessa OS (Ordens de Serviço), recalls, atualizações de firmware, sinistros e revendas. O CRM é a camada que transforma essa cadeia de eventos técnicos e financeiros em **relacionamento acionável** — quem é o dono atual, qual a saúde do parque instalado, quando o próximo contato deve acontecer e por qual gatilho.

Três decisões estruturam o módulo:

1. **A conta é multiperfil e o vínculo com o equipamento é por SN, não por venda.** Uma mesma organização pode ser simultaneamente *revenda* (compra do fabricante) e *cliente* (opera frota própria); um SN migra de titularidade sem perder histórico. O CRM projeta o **parque instalado por conta** a partir do histórico vitalício, não a partir de notas fiscais.
2. **Pós-venda e comercial compartilham a mesma linha do tempo 360°.** Ticket, oportunidade, fatura, OS e interação de marketing são eventos sobre a **mesma entidade Conta/Contato**, com correlação por `tenant_id + conta_id +` (quando aplicável) `sn`.
3. **Gatilhos de contato são orientados a evento — inclusive preditivos.** O CRM consome sinais do **Financeiro** (inadimplência, renovação) e do **KCI** (Kairós Cognitive Intelligence — recomendações e predição de manutenção) para disparar réguas de relacionamento e criar oportunidades/ tickets automaticamente. Manutenção preditiva vira **motivo de contato comercial e de retenção**, não apenas alerta técnico.

O documento entrega o **modelo de entidades do CRM**, os **diagramas de funil e ciclo de vida**, o **fluxograma de atendimento pós-venda com SLA**, as regras de segmentação e consentimento (LGPD), a matriz de integração com OS/ Financeiro/KCI, casos de uso, checklist de prontidão, riscos e trilha de auditoria.

2. Objetivos

- **O1.** Modelar **contas e contatos** multiperfil (fabricante, revenda, técnico, cliente final) com hierarquia organizacional e relação com equipamentos por **SN**.
- **O2.** Estruturar o **funil comercial** ponta a ponta: lead → qualificação → oportunidade → proposta → fechamento, com estágios, previsibilidade (*forecast*) e motivos de perda.
- **O3.** Definir o **pós-venda**: tickets/chamados, **SLAs** por prioridade e tipo de contrato, base de conhecimento e **integração bidirecional com OS** (serviço técnico).
- **O4.** Consolidar o **Histórico 360° do cliente**: compras, serviços, equipamentos, faturas e interações em uma linha do tempo única e auditável.
- **O5.** Habilitar **automações de relacionamento**: campanhas segmentadas, réguas de comunicação, pesquisas de NPS/CSAT e nutrição de leads.

- **O6.** Integrar com **Financeiro** (faturas, inadimplência, cobrança) e com o **KCI** (recomendações e preditivo de manutenção como gatilho de contato).
- **O7.** Garantir **segmentação e conformidade LGPD** por padrão: base legal, consentimento granular, finalidade, retenção e direitos do titular.
- **O8.** Preservar os princípios do canon: **multiempresa de 1ª classe, segurança/privacidade por padrão e rastreabilidade total** por SN.

3. Escopo

No escopo: gestão de contas/contatos e do relacionamento com equipamentos por SN; funil comercial (leads, oportunidades, propostas, pipeline, forecast); pós-venda (tickets, SLA, base de conhecimento) e sua integração com OS; histórico 360º; automações de marketing/relacionamento (campanhas, réguas, NPS/CSAT); consumo de gatilhos do Financeiro e do KCI; segmentação e conformidade LGPD do módulo; modelo de entidades do CRM.

Fora do escopo (referência a outros documentos): execução técnica da Ordem de Serviço, mão de obra, peças e garantia operacional (**Doc de OS**); contabilidade, emissão fiscal, régua de cobrança e conciliação (**Financeiro**); lógica interna do motor KCI/RAG e modelos preditivos (**Docs 13/KCI**); IAM, RBAC/ABAC, criptografia e Zero Trust (**Doc 07/14**); modelagem física, RLS e histórico vitalício no banco (**Doc 08**); contratos de API e eventos de domínio (**Doc 09**); observabilidade e pipelines (**Docs de operação**). Este documento define **entidades, regras e comportamento do relacionamento**; consome os contratos dos módulos citados.

4. Regras

#	Regra (inviolável no módulo CRM)	Origem / Justificativa
R-01	Toda entidade do CRM possui <code>tenant_id NOT NULL</code> sob RLS; nenhum dado de relacionamento cruza fronteira de tenant sem trilha de auditoria.	Constituição — multiempresa 1ª classe
R-02	O vínculo Conta ↔ Equipamento é por SN e titularidade vigente , projetado do histórico vitalício — nunca inferido apenas da NF de venda.	Rastreabilidade total (Doc 08)
R-03	Toda comunicação outbound (e-mail, SMS, push, WhatsApp) exige base legal LGPD válida e registrada para a finalidade; sem base legal, o envio é bloqueado.	LGPD — Art. 7º/8º
R-04	Consentimento é granular por finalidade e canal , versionado, revogável, com data/hora e origem; opt-out é honrado em ≤ 24h em todos os canais.	LGPD — Art. 8º, §5º / Art. 18
R-05	Lead, Oportunidade, Ticket e Interação referenciam a mesma Conta/Contato ; duplicidade é resolvida por <i>matching e merge</i> auditado, nunca por cópia silenciosa.	Integridade do 360º
R-06	Todo Ticket tem SLA calculado (resposta e resolução) a partir de prioridade + contrato; violação iminente gera escalonamento automático.	Governança de pós-venda
R-07	Transições de estágio no funil e de status no ticket são eventos append-only ; motivo é obrigatório em perda de oportunidade e em reabertura de ticket.	Auditoria (§15)
R-08	Gatilho de contato originado no KCI (preditivo) ou no Financeiro cria artefato rastreável (oportunidade/ticket/tarefa) com <i>origem</i> e <i>evidência</i> , nunca ação anônima.	Explicabilidade / confiança
R-09	Dados sensíveis do titular seguem minimização : coleta-se o necessário à finalidade declarada; campos livres não recebem dado sensível por convenção.	LGPD — minimização
R-10	Segmentos e campanhas respeitam isolamento de tenant ; um segmento nunca combina titulares de tenants distintos.	Multiempresa / privacidade
R-11	Retenção de dados de relacionamento segue política por finalidade; expirado o prazo, aplica-se anonimização ou descarte auditado.	LGPD — Art. 15/16
R-12	Toda automação (régua/campanha) tem critério de saída e <i>frequency capping</i> ; nenhum contato entra em loop de comunicação.	Boas práticas de relacionamento

5. Arquitetura (do módulo CRM)

5.1 Posição no ecossistema

O CRM é um **bounded context** próprio (dono exclusivo de contas, contatos, leads, oportunidades, tickets, campanhas e consentimentos). Ele **não possui** o equipamento, a OS, a fatura nem o modelo preditivo — **consome** esses fatos por API/evento e projeta visões de relacionamento.

```
flowchart LR
    subgraph EDGE["Borda"]
        BFFW["BFF Web"]
        BFFM["BFF Mobile"]
    end

    subgraph CRM["Bounded Context: CRM"]
        ACC["Contas & Contatos"]
        PIPE["Funil Comercial"]
        DESK["Service Desk / Tickets"]
        KB["Base de Conhecimento"]
        MKT["Automação & Campanhas"]
        CONS["Consentimento & LGPD"]
        V360["Projeção 360º"]
    end

    subgraph EXT["Outros contextos (fonte de verdade)"]
        EQP["Equipamentos / SN + Histórico Vitalício"]
        OS["Ordens de Serviço"]
        FIN["Financeiro"]
        KCI["KCI - Preditivo & Recomendação"]
        IAM["IAM / Identidade"]
    end

    BFFW --> CRM
    BFFM --> CRM

    EQP -- "eventos: EquipamentoRegistrado, TitularidadeAlterada" --> V360
    OS -- "eventos: OSAberta, OSConcluida" --> DESK
    OS --> V360
    FIN -- "eventos: FaturaEmitida, FaturaVencida, PagamentoConfirmado" --> V360
    FIN -- "InadimplenciaDetectada" --> MKT
    KCI -- "RecomendacaoManutencao, RiscoFalhaPreditivo" --> PIPE
    KCI --> DESK
    IAM -- "identidade & RBAC/ABAC" --> CRM

    DESK -- "AbrirOS / consultar OS" --> OS
    PIPE -- "PropostaAceita -> pedido" --> FIN
    CONS -- "valida base legal" --> MKT
```

5.2 Camadas lógicas internas

1. **Domínio de Contas:** contas (org e pessoa física), contatos, papéis/perfis, hierarquia, relação com SN.
2. **Domínio Comercial:** leads, oportunidades, propostas, pipeline, forecast, atividades.
3. **Domínio de Atendimento:** tickets, filas, SLA, base de conhecimento, satisfação (CSAT).
4. **Domínio de Relacionamento:** segmentos, campanhas, réguas, NPS, jornadas.
5. **Domínio de Privacidade:** consentimentos, bases legais, preferências de contato, requisições do titular (DSAR).
6. **Projeção 360º:** *read model* que materializa a linha do tempo do cliente a partir de eventos próprios e externos (CQRS leve).

5.3 Padrões de integração

Interface	Direção	Mecanismo	Exemplos
Equipamentos/ SN	Entrada	Evento + consulta	<code>EquipamentoRegistrado</code> , <code>TitularidadeAlterada</code> ; consulta de parque instalado
Ordens de Serviço	Bidirecional	Evento + comando	Entrada: <code>OSAberta</code> , <code>OSConcluida</code> ; Saída: <code>AbrirOS(ticket_id)</code>
Financeiro	Entrada + comando	Evento + comando	Entrada: <code>FaturaVencida</code> , <code>PagamentoConfirmado</code> ; Saída: <code>GerarPedido(proposta_id)</code>
KCI	Entrada	Evento (recomendação)	<code>RecomendacaoManutencao</code> , <code>RiscoFalhaPreditivo</code> , <code>PropensaoUpsell</code>
IAM	Entrada	Token/claims	Identidade, tenant, papéis, atributos ABAC

Regra arquitetural: o CRM **não escreve** em tabelas de outros contextos. Toda ação transversal (abrir OS, gerar pedido) é **comando** publicado ao contexto dono, com correlação (`traceparent`, `tenant_id`, `conta_id`).

6. Diagramas

6.1 Funil comercial (pipeline)

```
flowchart TD
    L["Lead capturado<br/>(site, evento, indicação, KCI)"] --> Q{"Qualificado?<br/>(BANT / fit)"}
    Q -- "Não" --> NUT["Nutrição / Reciclagem<br/>(régua de marketing)"]
    NUT --> Q
    Q -- "Sim" --> OPP["Oportunidade aberta"]
    OPP --> DIAG["Diagnóstico / Descoberta"]
    DIAG --> PROP["Proposta enviada"]
    PROP --> NEG["Negociação"]
    NEG --> DEC{"Decisão"}
    DEC -- "Ganha" --> WON["Fechamento (Won)<br/>→ Pedido no Financeiro"]
    DEC -- "Perdida" --> LOST["Perda (Lost)<br/>motivo obrigatório"]
    WON --> ONB["Onboarding / Ativação<br/>→ vínculo SN à conta"]
    LOST --> NUT
```

6.2 Ciclo de vida do cliente (customer lifecycle)

```
stateDiagram-v2
    [*] --> Lead
    Lead --> Prospect: qualificação
    Prospect --> Cliente: 1ª compra (Won)
    Cliente --> Ativo: onboarding + uso do parque (SN)
    Ativo --> Recorrente: recompra / expansão / renovação
    Ativo --> EmRisco: inadimplência OU risco preditivo (KCI) OU NPS detrator
    EmRisco --> Ativo: recuperação / retenção
    EmRisco --> Inativo: churn
    Recorrente --> Advogado: NPS promotor + indicações
    Inativo --> Reativado: campanha win-back
    Reativado --> Ativo
    Advogado --> [*]
    Inativo --> [*]
```

6.3 Modelo de entidades do CRM (visão relacional)

```
erDiagram
    CONTA ||--o{ CONTATO : possui
    CONTA ||--o{ CONTA : "hierarquia (matriz/filial)"
    CONTA ||--o{ CONTA_PAPEL : "assume perfis"
    CONTA ||--o{ EQUIP_CONTA : "titular de (por SN)"
    CONTA ||--o{ LEAD : origina
    CONTA ||--o{ OPORTUNIDADE : tem
    CONTA ||--o{ TICKET : abre
    CONTA ||--o{ CONSENTIMENTO : concede
    CONTA ||--o{ INTERACAO : registra
    CONTATO ||--o{ CONSENTIMENTO : titulariza
    CONTATO ||--o{ INTERACAO : participa
    LEAD ||--o{ OPORTUNIDADE : "converte em"
    OPORTUNIDADE ||--o{ PROPOSTA : gera
    OPORTUNIDADE ||--o{ ATIVIDADE : contém
    PROPOSTA ||--o{ PROPOSTA_ITEM : detalha
    TICKET ||--o{ TICKET_EVENTO : "linha do tempo"
    TICKET ||--o{ OS_REF : "escala para OS"
    TICKET }o--o{ ARTIGO_KB : "resolvido por"
    SEGMENTO ||--o{ CAMPANHA : direciona
    CAMPANHA ||--o{ CAMPANHA_ALVO : atinge
    CAMPANHA_ALVO }o--|| CONTATO : "envia a"
    EQUIP_CONTA }o--|| EQUIPAMENTO_SN : "referencia SN (externo)"
    INTERACAO }o--o{ CANAL : "por"
```

`EQUIPAMENTO_SN` e `OS_REF` são **referências** a contextos externos (Equipamentos e OS); o CRM guarda o identificador (SN / os_id) e uma projeção mínima, não a fonte de verdade.

7. Fluxogramas (atendimento pós-venda)

7.1 Ciclo de vida do ticket com SLA e integração com OS

```
flowchart TD
    A["Cliente/Contato aciona<br/>(portal, e-mail, telefone, WhatsApp)"] --> B["Abertura de Ticket<br/>>identifica Conta + SN (opcional)"]
    B --> C["Classificação:<br/>tipo, prioridade, contrato"]
    C --> D["Cálculo de SLA<br/>(resposta + resolução)"]
    D --> E["Triagem N1"]
    E -- "Resolvível N1" --> F["Consulta Base de Conhecimento"]
    F --> G["Resolvido?"]
    G -- "Sim" --> H["Primeira resposta<br/>dentro do SLA"]
    G -- "Requer campo/peça" --> I["Escalonamento → abre OS<br/>(comando ao contexto OS)"]
    G -- "Não" --> I
    I --> J["OS em execução<br/>(eventos OS → ticket)"]
    J --> K["OS Concluída → atualiza ticket"]
    K --> H
    H --> L["SLA em risco?"]
    L -- "Sim" --> M["Escalonamento automático<br/>(N2 / gestor)"]
    L -- "Não" --> N["Resolução / Encerramento"]
    M --> N
    N --> O["Pesquisa CSAT<br/>+ registro no 360º"]
    O --> P["Reabertura solicitada?"]
    P -- "Sim (motivo obrigatório)" --> C
    P -- "Não" --> Q["Fechado"]
```

7.2 Gatilho preditivo (KCI) → contato proativo

```
flowchart LR
    K["KCI emite<br/>RiscoFalhaPreditivo(SN, score, evidência)"] --> V["CRM resolve titular vigente do SN<br/>(via histórico vitalício)"]
    V --> D{"Decisão de roteamento"}
    D -- "Risco técnico alto" --> T["Cria Ticket proativo<br/>(prioridade por score)"]
    D -- "Oportunidade de serviço/peça" --> O["Cria Oportunidade<br/>(upsell manutenção)"]
    T --> C["Régua de contato proativo<br/>(canal conforme consentimento)"]
    O --> C
    C --> R["Registra no 360º<br/>origem=KCI, evidência anexada"]
```

8. Boas Práticas

- **Conta única, verdade única.** Deduplicação por *matching* determinístico (CNPJ/CPF, e-mail, domínio) + probabilístico; *merge* sempre auditado e reversível.
- **SN como eixo do relacionamento pós-venda.** Sempre que possível, associe o ticket/oportunidade ao SN — isso conecta o atendimento ao histórico vitalício e ao parque instalado.
- **Estágios de funil com critérios de saída objetivos.** Cada estágio tem definição de "pronto para avançar" (*exit criteria*), não avança por sentimento do vendedor.
- **Forecast por ponderação de probabilidade x estágio,** não por otimismo individual; motivo de perda taxonomizado alimenta melhoria contínua.
- **SLA orientado a contrato.** Prioridade e tipo de contrato (SLA Bronze/Prata/Ouro) determinam metas; *pausas de SLA* (aguardando cliente) são explícitas e auditáveis.
- **Base de conhecimento como ativo vivo.** Todo ticket recorrente vira artigo; artigos alimentam autoatendimento e sugestões de resolução (com apoio do KCI).
- **Automação com respeito ao contato.** *Frequency capping* global por titular, janelas de silêncio, critério de saída e supressão pós-conversão evitam fadiga e reclamação.
- **Privacidade por padrão (privacy by design).** Coleta mínima, finalidade explícita, consentimento verificável antes de qualquer *outbound*; preferências centralizadas e honradas em todos os canais.
- **Proatividade orientada a evidência.** Contato preditivo sempre carrega o "porquê" (evidência do KCI/Financeiro) — aumenta conversão e confiança, reduz percepção de spam.
- **NPS/CSAT fechando o loop.** Detrator gera tarefa de recuperação com dono e prazo; promotor entra em trilha de indicação/advocacy.

9. Padrões

9.1 Estágios e taxonomias padronizadas

Domínio	Enumeração padrão
Estágio de oportunidade	Aberta → Descoberta → Proposta → Negociação → Ganha / Perda
Motivo de perda	Preço, Concorrência, Sem orçamento, Sem fit, Timing, Sem resposta, Interno
Prioridade de ticket	Baixa, Média, Alta, Crítica (drone parado / risco operacional)
Status de ticket	Novo → Em triagem → Em andamento → Aguardando cliente → Escalado (OS) → Resolvido → Fechado (+ Reaberto)
Origem de lead	Site, Evento, Indicação, Parceiro/Revenda, Inbound MKT, KCI (preditivo), Base instalada
Base legal LGPD	Consentimento, Execução de contrato, Legítimo interesse, Obrigação legal, Proteção ao crédito

9.2 SLA de referência (parametrizável por tenant e contrato)

Prioridade	1ª resposta (Ouro)	Resolução (Ouro)	1ª resposta (Prata)	Resolução (Prata)
Crítica	15 min	4 h	1 h	8 h
Alta	1 h	8 h	4 h	24 h
Média	4 h	24 h	8 h	48 h
Baixa	8 h	48 h	24 h	72 h

Valores são **defaults de referência**; cada tenant configura suas metas. O relógio de SLA pausa em **Aguardando cliente** e retoma na resposta.

9.3 Convenções de dados e eventos

- **Identidade:** **UUID** técnico + chaves naturais para *matching* (CNPJ/CPF/e-mail normalizados).
- **Eventos de domínio (saída):** **LeadCriado**, **OportunidadeGanha**, **OportunidadePerdida**, **TicketAberto**, **TicketResolvido**, **ConsentimentoConcedido**, **ConsentimentoRevogado** — versionados e com **tenant_id**.
- **Correlação:** todo artefato transporta **tenant_id**, **conta_id**, **traceparent** e, quando aplicável, **sn** e **os_id**.
- **Nomenclatura:** eventos no passado (fato ocorrido); comandos no imperativo (**AbrirOS**, **GerarPedido**).

10. Casos de Uso

ID	Ator	Cenário	Resultado esperado
UC-01	Vendedor (revenda)	Captura um lead de feira e qualifica por fit e orçamento.	Lead vira oportunidade com estágio, valor e probabilidade; atividades agendadas.
UC-02	Vendedor	Envia proposta de 3 drones + contrato de manutenção.	Proposta versionada com itens; ao aceitar, comando GerarPedido ao Financeiro.
UC-03	Sistema (onboarding)	Pedido faturado e equipamentos registrados.	TitularidadeAlterada / EquipamentoRegistrado vinculam SNs à conta no parque instalado.
UC-04	Cliente final	Abre chamado: drone não liga (crítico).	Ticket crítico com SLA de 15 min; triagem N1; escala para OS se necessário.
UC-05	Técnico	Resolve OS e conclui serviço.	OSConcluida atualiza o ticket e a linha do tempo 360°; dispara CSAT.
UC-06	KCI	Detecta desgaste preditivo em SN sob garantia.	CRM identifica titular vigente e cria oportunidade de manutenção + ticket proativo.
UC-07	Financeiro	Fatura vence sem pagamento.	FaturaVencida marca conta EmRisco ; régua de cobrança relacional (não abusiva) inicia.
UC-08	Marketing	Cria campanha para promotores do NPS com base instalada > 5 SNs.	Segmento respeita consentimento e isolamento de tenant; campanha com critério de saída.
UC-09	Titular de dados	Solicita exclusão/portabilidade (DSAR).	Requisição registrada, prazo LGPD acompanhado; anonimização/descarte auditado.
UC-10	Gestor comercial	Analisa forecast e motivos de perda do trimestre.	Pipeline ponderado, taxa de conversão por estágio e taxonomia de perda.
UC-11	Atendente N2	SLA de ticket em risco.	Escalonamento automático para gestor com alerta e redistribuição.
UC-12	Fabricante	Comunica recall de um lote (SNs específicos).	CRM resolve titulares vigentes dos SNs e dispara campanha/ ticket de recall rastreado.

11. Modelagem (entidades CRM)

11.1 Dicionário de entidades

Entidade	Descrição	Atributos-chave
Conta	Organização ou pessoa que se relaciona com o tenant.	<code>id</code> , <code>tenant_id</code> , <code>tipo</code> (PJ/PF), <code>razao_social/nome</code> , <code>documento</code> , <code>status_ciclo_vida</code> , <code>conta_pai_id</code>
Conta_Papel	Perfis assumidos pela conta (n:n).	<code>conta_id</code> , <code>papel</code> (fabricante/revenda/tecnico/cliente), <code>vigencia</code>
Contato	Pessoa física vinculada a uma conta.	<code>id</code> , <code>conta_id</code> , <code>nome</code> , <code>email</code> , <code>telefone</code> , <code>cargo</code> , <code>is_titular_dados</code>
Equip_Conta	Vínculo de titularidade Conta ↔ SN (projeção).	<code>conta_id</code> , <code>sn</code> , <code>desde</code> , <code>ate</code> , <code>origem</code> (venda/transferência)
Lead	Interesse não qualificado.	<code>id</code> , <code>tenant_id</code> , <code>origem</code> , <code>status</code> , <code>conta_id?</code> , <code>score</code>
Oportunidade	Negócio em andamento.	<code>id</code> , <code>conta_id</code> , <code>estagio</code> , <code>valor</code> , <code>probabilidade</code> , <code>previsao_fechamento</code> , <code>motivo_perda?</code>
Proposta	Oferta formal versionada.	<code>id</code> , <code>oportunidade_id</code> , <code>versao</code> , <code>status</code> , <code>validade</code> , <code>valor_total</code>
Proposta_Item	Linha da proposta (drone/serviço/contrato).	<code>proposta_id</code> , <code>sku</code> , <code>qtd</code> , <code>preco</code> , <code>sn?</code>
Ticket	Chamado de pós-venda.	<code>id</code> , <code>conta_id</code> , <code>contato_id</code> , <code>sn?</code> , <code>tipo</code> , <code>prioridade</code> , <code>status</code> , <code>sla_resposta_em</code> , <code>sla_resolucao_em</code> , <code>os_id?</code>
Ticket_Evento	Linha do tempo append-only do ticket.	<code>ticket_id</code> , <code>tipo</code> , <code>autor</code> , <code>timestamp</code> , <code>payload</code>
Artigo_KB	Artigo da base de conhecimento.	<code>id</code> , <code>titulo</code> , <code>categoria</code> , <code>conteudo</code> , <code>visibilidade</code> , <code>versao</code>
Interacao	Toque de relacionamento (qualquer canal).	<code>id</code> , <code>conta_id</code> , <code>contato_id?</code> , <code>canal</code> , <code>direcao</code> , <code>origem</code> , <code>timestamp</code> , <code>sn?</code>
Segmento	Público-alvo dinâmico ou estático.	<code>id</code> , <code>tenant_id</code> , <code>criterios</code> , <code>tipo</code> , <code>base_legal</code>
Campanha	Ação de relacionamento/marketing.	<code>id</code> , <code>segmento_id</code> , <code>canal</code> , <code>objetivo</code> , <code>criterio_saida</code> , <code>janela</code>
Consentimento	Registro de base legal por finalidade/canal.	<code>id</code> , <code>contato_id</code> , <code>finalidade</code> , <code>canal</code> , <code>base_legal</code> , <code>status</code> , <code>versao</code> , <code>concedido_em</code> , <code>revogado_em?</code>
NPS_Resposta	Resultado de pesquisa NPS/CSAT.	<code>id</code> , <code>conta_id</code> , <code>contato_id</code> , <code>tipo</code> , <code>nota</code> , <code>comentario</code> , <code>origem_evento</code>

11.2 Projeção 360° (read model)

A visão 360° **não é uma tabela transacional**; é um *read model* materializado por CQRS leve, alimentado por eventos próprios (leads, oportunidades, tickets, interações) e externos (equipamentos/SN, OS, faturas, KCI). Chaveamento por `tenant_id` + `conta_id`, com facetas por `sn`.

```

flowchart LR
    subgraph FONTES["Eventos-fonte"]
        E1["CRM: Lead/Oportunidade/Ticket/Interação"]
        E2["Equipamentos: SN + titularidade"]
        E3["OS: aberta/concluída"]
        E4["Financeiro: fatura/pagamento/inadimplência"]
        E5["KCI: recomendação/risco"]
    end
    E1 & E2 & E3 & E4 & E5 --> P["Projeção 360º<br/>(materialized view por conta)"]
    P --> T["Linha do tempo unificada"]
    T --> KPI["Indicadores da conta:<br/>parque instalado, LTV, saúde, risco, NPS"]

```

11.3 Multiperfil e titularidade por SN (nota de modelagem)

Uma conta pode acumular papéis simultâneos (`Conta_Papel n:n`) — por exemplo, uma revenda que também opera frota própria. A titularidade de um SN é **temporal** (`Equip_Conta.desde/ate`): quando o histórico vitalício emite `TitularidadeAlterada`, o CRM fecha o vínculo anterior e abre o novo, **preservando** todo o histórico de relacionamento associado ao SN — recalls, tickets e recomendações permanecem rastreáveis pelo eixo do equipamento.

12. Checklist

Contas & Contatos - [] Conta com `tenant_id`, tipo, documento normalizado e status de ciclo de vida. - [] Deduplicação/*matching* configurado; *merge* auditável e reversível. - [] Papéis (fabricante/revenda/técnico/cliente) e hierarquia matriz/filial. - [] Parque instalado projetado por SN a partir do histórico vitalício.

Funil comercial - [] Estágios com critérios de saída objetivos e forecast ponderado. - [] Motivo de perda obrigatório e taxonomizado. - [] Proposta versionada; `PropostaAceita` → comando ao Financeiro.

Pós-venda - [] SLA por prioridade × contrato, com pausa "aguardando cliente". - [] Escalonamento automático em risco de SLA. - [] Integração bidirecional com OS (abrir OS / receber `OSConcluída`). - [] Base de conhecimento versionada + CSAT no fechamento.

Relacionamento & Automação - [] Segmentos isolados por tenant; campanhas com critério de saída e *frequency capping*. - [] Réguas de NPS/CSAT com tarefa de recuperação para detratores.

Integrações - [] Consumo de `FaturaVencida/InadimplenciaDetectada` (Financeiro). - [] Consumo de `RiscoFalhaPreditivo/RecomendacaoManutencao` (KCI) com evidência anexada.

LGPD & Segurança - [] Base legal por finalidade/canal registrada antes de qualquer *outbound*. - [] Consentimento granular, versionado e revogável; opt-out ≤ 24h. - [] Política de retenção/anonimização por finalidade. - [] RLS por `tenant_id` e trilha de auditoria em toda mutação sensível.

13. Riscos

#	Risco	Impacto	Prob.	Mitigação
RK-01	Duplicidade de contas fragmenta o 360º e o forecast.	Alto	Média	<i>Matching</i> determinístico+probabilístico; <i>merge</i> auditado; bloqueio de criação com alerta de similaridade.
RK-02	Envio <i>outbound</i> sem base legal válida (violação LGPD).	Crítico	Média	Gate de consentimento por finalidade/canal; bloqueio automático; auditoria de todo envio.
RK-03	SLA calculado incorretamente (contrato mal mapeado).	Alto	Média	Motor de SLA parametrizável testado; <i>pausas</i> explícitas; alertas de configuração.
RK-04	Titularidade de SN desatualizada gera contato ao dono errado.	Alto	Baixa	Sincronização por evento <i>TitularidadeAlterada</i> ; resolução sempre do vínculo vigente.
RK-05	Automação em loop / fadiga de comunicação (fadiga → churn/ reclamação).	Médio	Média	<i>Frequency capping</i> global, janelas de silêncio, supressão pós-conversão, critério de saída obrigatório.
RK-06	Vazamento entre tenants em segmentos/campanhas.	Crítico	Baixa	RLS + testes de isolamento; segmento nunca combina tenants; revisão de queries.
RK-07	Gatilho preditivo (KCI) com falso positivo gera contato indevido.	Médio	Média	Limiar de score configurável; evidência exigida; feedback loop para recalibrar o KCI.
RK-08	Dado sensível em campo livre (violação de minimização).	Médio	Média	Convenção anti-dado-sensível; <i>masking</i> e varredura; treinamento e validação de formulário.
RK-09	Cobrança relacional agressiva prejudica relacionamento.	Médio	Média	Régua de inadimplência escalonada e respeitosa; separação clara entre cobrança e relacionamento.
RK-10	DSAR (direitos do titular) não atendido no prazo.	Alto	Baixa	Fluxo DSAR com SLA legal, dono e trilha; painel de acompanhamento.

14. Melhorias Futuras

- **Next Best Action orientado por KCI:** recomendação da próxima melhor ação de relacionamento por conta (contato, oferta, retenção) com explicabilidade.
- **Health Score da conta:** índice composto (uso do parque, tickets, NPS, inadimplência, risco preditivo) para priorizar Customer Success.
- **Copiloto de atendimento:** sugestão de resposta e de artigo de KB ao atendente, com resumo automático do ticket (sem exposição de dado sensível).
- **Modelagem de propensão (upsell/churn):** scores preditivos alimentando segmentação dinâmica.
- **Portal de autoatendimento** com base de conhecimento + abertura/consulta de tickets e status de OS por SN.
- **Orquestração de jornadas** multicanal visual (canvas) com testes A/B e *holdout* para medir incremento.
- **Central de preferências do titular** self-service, com histórico de consentimento e exportação de dados (portabilidade LGPD).
- **Attribution de receita** ligando campanhas → oportunidades → pedidos faturados no Financeiro.

15. Auditoria

- **Trilha imutável (append-only):** toda transição de estágio de oportunidade, mudança de status de ticket, *merge* de conta, concessão/revogação de consentimento e envio *outbound* é registrada com *quem* , *quando* , *de-para* , *motivo* e *transparent* .
- **Rastreabilidade de origem:** artefatos criados por gatilho (KCI/Financeiro) carregam *origem* e *evidência* ; nenhuma ação automática é anônima (R-08).

- **Auditoria de consentimento:** cada envio referencia o `consentimento_id` /base legal vigente no momento; revogações são honradas e auditadas; opt-out rastreado por canal.
 - **Isolamento multiempresa:** logs segregados por `tenant_id`; qualquer acesso cross-tenant (suporte/admin) é registrado e justificado.
 - **Retenção e descarte:** eventos de anonimização/descarte por expiração de finalidade são auditáveis, com prova de execução.
 - **Métricas de governança:** taxa de violação de SLA, tempo de atendimento a DSAR, cobertura de base legal em *outbound*, taxa de duplicidade e reversões de *merge* — reportadas periodicamente ao responsável pelo módulo e ao encarregado (DPO).
 - **Reprodutibilidade:** a projeção 360° pode ser reconstruída a partir dos eventos-fonte, garantindo que a linha do tempo apresentada seja verificável.
-

Fim do documento 17 – CRM. Este documento define entidades, regras e comportamento de relacionamento; a execução técnica (OS), a contabilidade/cobrança (Financeiro), o motor preditivo (KCI), a identidade (IAM) e a persistência (Banco de Dados) são governados por seus respectivos documentos e consumidos por contrato.

18 Financeiro

Documento: 18 – Financeiro **Versão:** 1.0 · **Data:** 21 de julho de 2026 · **Status:** Ativo **Dependências:** 00, 01, 02, 03, 04, 05, 06, 07, 08, 09, 10, 11, 12, 13, 14, 15, 16, 17 **Responsável:** Especialista Financeiro/ERP

1. Resumo Executivo

Este documento define o **módulo Financeiro** do Drone Kairós ERP: o núcleo transacional e contábil que transforma eventos operacionais (ordens de serviço concluídas, recebimentos de compra, movimentações de estoque) em **títulos, lançamentos, obrigações fiscais e caixa**, com rastreabilidade ponta a ponta e conformidade auditável.

O Financeiro do Kairós é concebido sob os princípios do canon: **multiempresa de 1ª classe** (cada `empresa_id` / `tenant_id` possui seu próprio plano de contas, política de câmbio, calendário fiscal e caixa, sem vazamento entre empresas), **segurança e auditoria por padrão** (todo lançamento é imutável e rastreável até seu documento de origem) e **rastreabilidade** (todo valor financeiro reconstrói sua cadeia causal: OS → faturamento → título → recebimento → conciliação → lançamento contábil).

A arquitetura separa três planos com responsabilidades distintas: o **plano operacional** (contas a pagar/receber, tesouraria, faturamento), o **plano fiscal** (emissão de documentos fiscais — NF-e/NFS-e no Brasil, com abstração para outras jurisdições) e o **plano contábil** (partidas dobradas, plano de contas, centros de custo, apuração de resultado). O princípio-guia é o **razão como fonte da verdade contábil**: nenhum saldo é editado diretamente; todo estado deriva do somatório de lançamentos imutáveis.

A integração com o **KSI (Kairós Smart Inventory)** é tratada como cidadã de primeira classe: valorização de estoque (custo médio ponderado ou PEPS), reconhecimento de **CMV** (Custo da Mercadoria/Peça Vendida) no faturamento de peças da OS, e provisões. A **multi-moeda** é entregue como capacidade estrutural (moeda funcional por empresa, moeda de transação, moeda de apresentação) preparada para o roadmap internacional. O detalhamento de **conformidade fiscal por região** é delegado ao Doc 23; aqui definimos a **abstração** que o torna plugável.

Entregas deste documento: modelo de entidades financeiras, diagramas Mermaid (faturamento a partir da OS, conciliação bancária, ciclo de fechamento), fluxogramas operacionais, catálogo de regras, casos de uso, checklist de prontidão, matriz de riscos e trilha de auditoria.

2. Objetivos

- **O1.** Estabelecer os ciclos completos de **Contas a Pagar (AP)** e **Contas a Receber (AR)**, do documento de origem à baixa e à conciliação.
- **O2.** Definir **fluxo de caixa** (realizado e projetado) e **tesouraria** (posição consolidada, aplicações, provisões) por empresa e consolidada.
- **O3.** Modelar o **faturamento** de serviços e peças a partir das **Ordens de Serviço**, com geração de documentos fiscais abstraídos por jurisdição.
- **O4.** Especificar **conciliação bancária** e integração com bancos/meios de pagamento (Open Finance, extratos, PIX, boleto, cartão, gateways).
- **O5.** Definir o **núcleo contábil**: plano de contas, partidas dobradas, centros de custo, lançamentos e apuração de resultado.
- **O6.** Formalizar a **integração com o KSI**: valorização de estoque (custo médio/PEPS), CMV, provisões e movimentos contábeis de estoque.
- **O7.** Prover a **abstração fiscal e tributária** (motor de regras plugável por região), com o detalhe de conformidade delegado ao Doc 23.

- **O8. Entregar a visão de multi-moeda e câmbio** (moeda funcional, transação, apresentação; reavaliação e variação cambial).
- **O9. Garantir controles, auditoria e fechamento** (segregação de funções, imutabilidade, períodos, trilha completa).

3. Escopo

No escopo: contas a pagar e receber; fluxo de caixa e tesouraria; faturamento de OS (serviços e peças); documentos fiscais (NF-e/NFS-e no Brasil, abstração internacional); conciliação bancária e integração com bancos/meios de pagamento; plano de contas, centros de custo e lançamentos contábeis; integração financeira com KSI (valorização, CMV, provisões); abstração de impostos e regras fiscais; visão de multi-moeda e câmbio; fechamento contábil e trilha de auditoria financeira.

Fora do escopo (referência a outros documentos): modelagem de dados global e schemas físicos (Doc 08); arquitetura de microsserviços e eventos de domínio (Doc 09); IAM/segurança detalhada, papéis e KCD (Doc 07); mecânica interna do KSI — códigos de barras, RFID, curva ABC, reposição (Doc 15); **conformidade fiscal por jurisdição** — cálculo detalhado de ICMS/ISS/PIS/COFINS/IRPJ/CSLL, layouts de SPED, regras por município/estado/país (Doc 23); BI/relatórios gerenciais e DRE analítica (Doc de BI). Aqui tratamos das **entidades, ciclos e interfaces financeiras**, não da implementação regulatória por região.

4. Regras

- **R1. Partida dobrada obrigatória.** Todo lançamento contábil tem débito(s) e crédito(s) de igual valor na moeda funcional; um lançamento desbalanceado é rejeitado na persistência.
- **R2. Razão imutável.** Lançamentos contábeis não são editados nem apagados; correções ocorrem por **estorno** (lançamento reverso) e novo lançamento, ambos rastreáveis.
- **R3. Multiempresa isolada.** Todo título, lançamento, conta bancária e documento fiscal carrega `empresa_id / tenant_id`; consultas sem esse filtro são erro de contrato. Plano de contas, calendário fiscal e política de câmbio são por empresa.
- **R4. Origem rastreável.** Todo título financeiro referencia seu **documento de origem** (OS, pedido de compra, contrato, lançamento manual justificado). Nenhum recebível/pagável nasce órfão.
- **R5. Idempotência de faturamento.** A conclusão de uma OS gera **no máximo um** faturamento por evento; reprocessamento usa chave de idempotência (`os_id` + versão) e nunca duplica títulos.
- **R6. Reconhecimento no momento correto.** Receita e CMV são reconhecidos no faturamento (regime de competência); o caixa é reconhecido na baixa (regime de caixa para fluxo). Os dois planos coexistem sem se contaminar.
- **R7. Período aberto para escrita.** Lançamentos só ocorrem em **período contábil aberto**. Período fechado é somente leitura; ajustes exigem período de ajuste explícito e autorizado.
- **R8. Conciliação antes do fechamento.** Nenhum período é fechado com conta bancária não conciliada ou com partidas em suspenso não resolvidas.
- **R9. Segregação de funções (SoD).** Quem cadastra um pagável não o aprova; quem aprova não executa a baixa bancária. Papéis são disjuntos por política (ver Doc 07).
- **R10. Moeda explícita.** Todo valor monetário carrega moeda e, quando de transação estrangeira, a **taxa de câmbio** aplicada e sua data/fonte. Não há valor sem moeda.
- **R11. Impostos por motor de regras.** Nenhum imposto é *hardcoded*; todo cálculo passa pelo **motor fiscal** parametrizado por jurisdição (Doc 23). O Financeiro consome o resultado, não o codifica.
- **R12. Valorização consistente com KSI.** O custo de peça vendida (CMV) usa a **mesma** política de valorização do KSI (custo médio ou PEPS) vigente na empresa; divergência entre estoque e contábil é reconciliada e alertada.
- **R13. Auditoria total.** Toda ação financeira (criar, aprovar, baixar, estornar, conciliar, fechar) gera evento de auditoria imutável com ator, timestamp, valor antes/depois e correlação (`transparent`).
- **R14. Casas decimais e arredondamento.** Valores monetários com precisão definida por moeda (2 casas para BRL/USD/EUR); arredondamento **bankers rounding** ou meio-para-cima conforme política da empresa, aplicado de forma determinística e registrada.

5. Arquitetura (do módulo financeiro)

5.1 Planos lógicos

O Financeiro organiza-se em três planos com contratos claros entre si:

1. **Plano Operacional** — Contas a Pagar, Contas a Receber, Tesouraria/Fluxo de Caixa, Faturamento. Trabalha com **títulos** (obrigações e direitos) e **movimentos de caixa**.
2. **Plano Fiscal** — Emissão e gestão de documentos fiscais (NF-e/NFS-e/abstração), cálculo tributário via motor de regras, obrigações acessórias. Consome o motor fiscal do Doc 23.
3. **Plano Contábil** — Razão (ledger) de partidas dobradas, plano de contas, centros de custo, apuração e fechamento. É a **fonte da verdade contábil**.

O acoplamento entre planos é por **eventos** e **postagem contábil** (posting): eventos operacionais geram títulos; a baixa/emissão gera **regras de contabilização** que produzem lançamentos no razão. O razão nunca é editado diretamente pelos planos superiores.

5.2 Componentes do módulo

Componente	Responsabilidade nuclear	Consome	Produz
Faturamento	Converter OS concluída em fatura (serviços + peças)	Evento OSConcluída (Doc 09), preços, motor fiscal	Fatura, título AR, pedido de documento fiscal
Contas a Receber (AR)	Gerir direitos, cobrança, baixa	Faturas, meios de pagamento	Títulos AR, baixas, eventos de recebimento
Contas a Pagar (AP)	Gerir obrigações, aprovação, pagamento	Pedidos de compra (Compras), notas de entrada	Títulos AP, baixas, remessas de pagamento
Tesouraria/Fluxo de Caixa	Posição de caixa, projeção, aplicações	Títulos AR/AP, extratos	Fluxo realizado/projetado, previsões
Documentos Fiscais	Emitir/receber/gerir NF-e/NFS-e (abstraído)	Faturas, motor fiscal (Doc 23)	Documentos fiscais autorizados, XMLs/DANFE
Conciliação Bancária	Casar extrato ↔ títulos/lançamentos	Extratos (Open Finance/OFX), títulos	Partidas conciliadas, ajustes
Motor de Contabilização	Traduzir eventos financeiros em lançamentos	Eventos AR/AP/estoque/fiscal, regras de posting	Lançamentos no razão
Razão Contábil (Ledger)	Guardar partidas dobradas imutáveis	Lançamentos	Saldos, balancetes, DRE, balanço
Câmbio	Cotações, reavaliação, variação cambial	Fontes de cotação	Taxas, lançamentos de variação
Fechamento	Orquestrar apuração e travamento de período	Todos os planos	Período fechado, apurações

5.3 Contexto e integrações externas

O Financeiro é um **bounded context** que se integra a outros contextos do ERP exclusivamente por API e eventos (nunca por banco compartilhado — R2/R3 do Doc 09):

- **KSI (Estoque/Custos)**: fornece custo de peça e movimentos; recebe reconhecimento de CMV e provisões. Ver §5.4 e §11.
- **Compras**: origem dos pagáveis (pedido → recebimento → nota de entrada → título AP).
- **Ordens de Serviço**: origem dos recebíveis (OS concluída → faturamento).
- **CRM**: cliente/contato, condições comerciais, limite de crédito.
- **Motor Fiscal (Doc 23)**: cálculo tributário e conformidade por região.

- **Bancos/Meios de Pagamento:** extratos, PIX, boleto, cartão, gateways, Open Finance.

5.4 Princípio de contabilização orientada a evento

Cada fato financeiro relevante emite um **evento de domínio** versionado (alinhado ao Doc 09), consumido pelo Motor de Contabilização com **outbox + idempotência**:

Evento	Efeito operacional	Efeito contábil (posting)
FaturaEmitida	Cria título AR	D: Clientes / C: Receita de Serviços + Receita de Peças; C: Impostos a Recolher
PecaFaturada (via OS)	Baixa estoque no KSI	D: CMV / C: Estoque (pela política de valorização)
RecebimentoConfirmado	Baixa título AR	D: Banco / C: Clientes
PagavelAprovado	Cria/confirma título AP	D: Despesa/Estoque / C: Fornecedores
PagamentoExecutado	Baixa título AP	D: Fornecedores / C: Banco
NotaEntradaRegistrada	Entrada de estoque (KSI)	D: Estoque + Impostos a Recuperar / C: Fornecedores
PeriodoReavaliado (câmbio)	Reavaliação de saldos em ME	D/C: Variação Cambial (ativo/passivo)

6. Diagramas

6.1 Arquitetura do módulo (planos e integrações)

```
flowchart TB
    subgraph EXT[Contextos externos]
        OS[Ordens de Serviço]
        CMP[Compras]
        KSI[KSI - Estoque/Custos]
        CRM[CRM]
        FISC[Motor Fiscal - Doc 23]
        BANK[Bancos / Meios de Pagamento]
    end

    subgraph FIN[Módulo Financeiro]
        subgraph OP[Plano Operacional]
            FAT[Faturamento]
            AR[Contas a Receber]
            AP[Contas a Pagar]
            TES[Tesouraria / Fluxo de Caixa]
        end
        subgraph FS[Plano Fiscal]
            DOCF[Documentos Fiscais]
        end
        subgraph CT[Plano Contábil]
            POST[Motor de Contabilização]
            LED[(Razão Contábil)]
            CAMB[Câmbio]
            FECH[Fechamento]
        end
        CONC[Conciliação Bancária]
    end

    OS -->|OSConcluida| FAT
    CRM -->|cliente/crédito| FAT
    FAT -->|título| AR
    FAT -->|pedido doc fiscal| DOCF
    DOCF <-->|cálculo tributário| FISC
    CMP -->|pedido/recebimento| AP
    KSI <-->|custo/CMV/provisão| POST
    AR --> TES
    AP --> TES
    BANK -->|extrato| CONC
    AR <--> CONC
    AP <--> CONC
    FAT --> POST
    AR --> POST
    AP --> POST
    DOCF --> POST
    CAMB --> POST
    POST --> LED
    LED --> FECH
    CONC --> FECH
```

6.2 Modelo de entidades financeiras (ER)

```
erDiagram
    EMPRESA ||--o{ PLANO_CONTAS : possui
    EMPRESA ||--o{ CENTRO_CUSTO : possui
    EMPRESA ||--o{ CONTA_BANCARIA : possui
    EMPRESA ||--o{ PERIODO_CONTABIL : possui
    EMPRESA ||--o{ MOEDA_POLITICA : define

    FATURA ||--|{ FATURA_ITEM : contem
    FATURA ||--o| DOCUMENTO_FISCAL : gera
    FATURA ||--|{ TITULO : origina
    OS ||--o{ FATURA : faturada_em
    CLIENTE ||--o{ FATURA : para

    TITULO ||--o{ PARCELA : parcela
    PARCELA ||--o{ BAIXA : baixada_por
    BAIXA }o--|| MOV_CAIXA : movimenta
    MOV_CAIXA }o--|| CONTA_BANCARIA : em

    LANCAMENTO ||--|{ LANCAMENTO_LINHA : contem
    LANCAMENTO_LINHA }o--|| PLANO_CONTAS : classifica
    LANCAMENTO_LINHA }o--o| CENTRO_CUSTO : rateia
    LANCAMENTO }o--|| PERIODO_CONTABIL : postado_em

    TITULO }o--o| LANCAMENTO : contabiliza
    BAIXA }o--o| LANCAMENTO : contabiliza
    DOCUMENTO_FISCAL }o--o| LANCAMENTO : contabiliza

    EXTRATO_LINHA }o--o| BAIXA : concilia
    EXTRATO_LINHA }o--|| CONTA_BANCARIA : de

    TAXA_CAMBIO }o--|| MOEDA_POLITICA : aplica
```

6.3 Máquina de estados do título (AR/AP)

```
stateDiagram-v2
    [*] --> Provisionado: origem registrada
    Provisionado --> Aprovado: aprovação (SoD)
    Aprovado --> Aberto: documento fiscal/vencimento
    Aberto --> ParcialmenteBaixado: baixa parcial
    ParcialmenteBaixado --> Baixado: baixa final
    Aberto --> Baixado: baixa integral
    Aberto --> Vencido: passou vencimento
    Vencido --> Baixado: baixa com juros/multa
    Aberto --> Cancelado: estorno autorizado
    Baixado --> Estornado: estorno (período aberto)
    Baixado --> Conciliado: casado no extrato
    Conciliado --> [*]
    Cancelado --> [*]
```

7. Fluxogramas (OS → faturamento → recebimento)

7.1 Faturamento a partir da Ordem de Serviço

flowchart TD

```
A[OS concluída pelo técnico] --> B{OS faturável?}
B -->|Não - garantia/cortesia| Z1[Registra sem receita<br/>baixa CMV se houver peça]
B -->|Sim| C[Coleta itens: serviços + peças]
C --> D[Busca preços/contrato no CRM]
D --> E[Motor Fiscal calcula tributos<br/>por jurisdição - Doc 23]
E --> F[Monta Fatura: itens, impostos, total]
F --> G{Valida crédito do cliente}
G -->|Excede limite| Z2[Bloqueia / exige aprovação]
G -->|OK| H[Emite Fatura - idempotente por os_id+versão]
H --> I[Solicita Documento Fiscal<br/>NF-e peças / NFS-e serviço]
I --> J{Autorização fiscal}
J -->|Rejeitada| K[Trata rejeição / reemite]
J -->|Autorizada| L[Gera título AR + parcelas]
L --> M[Baixa peças no KSI<br/>reconhece CMV]
M --> N[Contabiliza: Receita, Impostos, CMV, Estoque]
N --> O[Envia cobrança: boleto/PIX/cartão]
O --> P[Aguarda recebimento]
P --> Q{Recebimento confirmado?}
Q -->|Sim| R[Baixa título AR<br/>D: Banco / C: Clientes]
Q -->|Vencido| S[Régua de cobrança / juros]
R --> T[Concilia no extrato bancário]
S --> T
T --> U[Título Conciliado - fim do ciclo]
```

7.2 Conciliação bancária

flowchart TD

```
A[Importa extrato<br/>Open Finance / OFX / API banco] --> B[Normaliza linhas: data, valor, descrição, identificadores]
B --> C[Motor de matching]
C --> D{Match automático?}
D -->|1:1 exato<br/>valor+data+identificador| E[Concilia automaticamente]
D -->|N:1 ou aproximado| F[Sugestão com score]
D -->|Sem correspondência| G[Fila de exceções]
F --> H{Operador confirma?}
H -->|Sim| E
H -->|Não| G
G --> I{Natureza da linha}
I -->|Tarifa/juros/imposto banco| J[Cria lançamento de ajuste]
I -->|Recebimento não previsto| K[Cria título AR retroativo]
I -->|Pagamento não previsto| L[Cria título AP retroativo]
E --> M[Marca partida como conciliada]
J --> M
K --> M
L --> M
M --> N{Saldo extrato = saldo razão?}
N -->|Sim| O[Conta conciliada no período]
N -->|Não| P[Relatório de divergência / investigação]
O --> Q[Libera conta para fechamento]
P --> Q
```

7.3 Ciclo de fechamento contábil

```
flowchart LR
    A[Corte de período] --> B[Concilia todas as contas bancárias]
    B --> C[Reavaliação cambial de saldos em ME]
    C --> D[Provisões e apropriações<br/>depreciação, folha, provisões KSI]
    D --> E[Apuração de tributos - Doc 23]
    E --> F[Apuração de resultado<br/>Receitas - CMV - Despesas]
    F --> G[Balancete de verificação]
    G --> H{Balancete confere?}
    H -->|Não| I[Ajustes em período de ajuste]
    I --> G
    H -->|Sim| J[Trava período - somente leitura]
    J --> K[Gera DRE, Balanço, DFC]
    K --> L[Arquiva com hash/assinatura para auditoria]
```

8. Boas Práticas

- **BP1. Razão como fonte da verdade.** Saldos são derivados, nunca armazenados como estado mutável autoritativo; qualquer saldo materializado é cache reconstruível a partir dos lançamentos.
- **BP2. Contabilização declarativa.** Regras de *posting* (qual conta debitar/creditar por tipo de evento) são configuração versionada, não código espalhado. Facilita auditoria e mudança de plano de contas.
- **BP3. Idempotência em todo ponto de integração.** Faturamento, baixa e conciliação usam chaves de idempotência; reprocessos são seguros por design (essencial em event-driven — Doc 09).
- **BP4. Reconciliação estoque ↔ contábil contínua.** A conta contábil de estoque é confrontada periodicamente com a valorização do KSI; divergências geram alerta antes do fechamento, não depois.
- **BP5. Multi-moeda desde o núcleo.** Mesmo operando só em BRL hoje, todo valor carrega moeda e taxa; retrofit de multi-moeda em ERP é caro e propenso a erro.
- **BP6. Precisão monetária correta.** Usar tipo decimal de precisão fixa (nunca ponto flutuante binário) para dinheiro; arredondamento determinístico e registrado por moeda.
- **BP7. Segregação de funções aplicada por política.** SoD não é convenção verbal; é regra de autorização (Doc 07) que o sistema recusa violar.
- **BP8. Documento fiscal desacoplado do faturamento.** A fatura existe mesmo se a autorização fiscal falhar temporariamente; a emissão do documento fiscal é assíncrona e resiliente (retry, contingência).
- **BP9. Régua de cobrança automatizada.** Vencimentos disparam ações graduais (lembrete, notificação, protesto/negativação conforme política), reduzindo inadimplência sem intervenção manual.
- **BP10. Fechamento como processo orquestrado.** O fechamento é uma saga com checkpoints e reversibilidade até o travamento, não um botão único e opaco.

9. Padrões

9.1 Padrões técnicos e contábeis adotados

Padrão	Aplicação no Kairós
Partidas dobradas (double-entry)	Todo lançamento no razão; base da integridade contábil
Regime de competência + regime de caixa	Competência para resultado/impostos; caixa para fluxo/tesouraria
Event Sourcing (parcial)	Razão como log imutável de lançamentos; saldos como projeção
Outbox + Idempotência	Publicação de eventos financeiros consistente com a escrita (Doc 09)
Saga	Faturamento e fechamento como fluxos de múltiplos passos com compensação
Money pattern	Valor + moeda encapsulados; nunca número solto
Custo médio ponderado / PEPS	Valorização de estoque e CMV, alinhado ao KSI (política por empresa)
ISO 4217	Códigos de moeda
ISO 20022	Mensageria de pagamentos (visão para integração bancária/tesouraria)
Open Finance / OFX	Padrões de importação de extrato e iniciação de pagamento

9.2 Convenções

- **Plano de contas hierárquico** com codificação por nível (Ativo/Passivo/PL/Receita/Despesa/Custo), configurável por empresa, com contas de uso restrito para *posting* automático.
- **Centros de custo** ortogonais ao plano de contas, permitindo rateio (por OS, por equipe, por unidade de negócio, por drone/frota).
- **Numeração de documentos** sequencial e sem lacunas por empresa e por tipo (exigência fiscal), com controle de série.
- **Nomenclatura de eventos** em português, no passado, versionados: `FaturaEmitida.v1`, `RecebimentoConfirmado.v1`.
- `tenant_id` / `empresa_id` sempre presente em toda entidade, evento e consulta.

9.3 Abstração fiscal (fronteira com o Doc 23)

O Financeiro define uma **interface de motor fiscal** e consome seu resultado; não implementa cálculo tributário. Contrato conceitual:

```
CalcularTributos(contexto) -> ResultadoFiscal
contexto: { jurisdicao, empresa_regime, itens[], cliente, natureza_operacao, data }
ResultadoFiscal: {
  tributos: [ { codigo, base, aliquota, valor, conta_contabil, recuperavel } ],
  total_tributos, total_liquido,
  exigencias_documento: { tipo_doc, campos_obrigatorios }
}
```

Isso permite plugar **Brasil** (ICMS, IPI, ISS, PIS/COFINS, retenções; NF-e modelo 55, NFS-e municipal) e, futuramente, outras jurisdições (VAT/GST, sales tax) sem alterar o núcleo financeiro. O detalhe de cada tributo, alíquota, benefício e obrigação acessória vive no **Doc 23**.

10. Casos de Uso

10.1 UC-01 — Faturar OS com serviço e peça (Brasil)

Ator: Faturista / evento automático `OSConcluida`. **Pré:** OS concluída, cliente com cadastro fiscal válido, peças com custo no KSI. **Fluxo:** Sistema coleta 2h de serviço + 1 hélice; CRM retorna tabela de preço do contrato; motor fiscal calcula ISS

sobre o serviço e ICMS/PIS/COFINS sobre a peça; emite fatura idempotente; solicita **NFS-e** (serviço) ao município e **NF-e 55** (peça) à SEFAZ; ao autorizar, gera título AR em 1 parcela (boleto 30 dias), baixa a hélice no KSI reconhecendo CMV pelo custo médio, e contabiliza receita, impostos, CMV e estoque. **Pós:** Título AR aberto, documentos fiscais autorizados, lançamentos no razão, estoque reduzido. **Exceções:** rejeição fiscal (fila de reprocessamento); cliente sem limite (bloqueio/aprovação); peça sem saldo (alerta, impede baixa).

10.2 UC-02 — Contas a Pagar a partir de compra

Ator: Compras (recebimento) → Financeiro. **Fluxo:** Recebimento de pedido gera nota de entrada; KSI dá entrada no estoque valorizando o custo; Financeiro cria título AP (provisionado → aprovado por alçada, SoD aplicada); no vencimento, gera remessa de pagamento (PIX/boleto/TED); ao executar, baixa o título e contabiliza; concilia no extrato. **Pós:** Estoque valorizado, obrigação registrada e liquidada, caixa atualizado.

10.3 UC-03 — Conciliação bancária diária

Ator: Tesoureiro / rotina automática. **Fluxo:** Importa extrato via Open Finance; motor de matching concilia 1:1 automaticamente; tarifas bancárias viram lançamento de ajuste; um recebimento PIX não previsto é casado a um título AR aberto; exceções vão para fila humana; ao final, saldo do extrato = saldo do razão, conta liberada para fechamento.

10.4 UC-04 — Projeção de fluxo de caixa e tesouraria

Ator: Controller. **Fluxo:** Sistema projeta caixa combinando títulos AR/AP por vencimento, recorrências e sazonalidade; apresenta posição consolidada multiempresa e por empresa, em moeda de apresentação; sinaliza dias de caixa negativo e sugere antecipação de recebíveis ou reprogramação de pagáveis.

10.5 UC-05 — Fechamento mensal multiempresa

Ator: Contador. **Fluxo:** Concilia contas, reavalia saldos em moeda estrangeira (variação cambial), lança provisões (inclusive provisão de estoque do KSI), apura tributos (Doc 23), gera balancete; ajusta em período de ajuste; trava o período; emite DRE, Balanço e DFC por empresa e consolidado; arquiva com hash para auditoria.

10.6 UC-06 — Estorno auditável

Ator: Contador com alçada. **Fluxo:** Fatura emitida com erro; em período aberto, sistema gera **lançamento reverso** vinculado ao original, cancela o documento fiscal (carta de correção ou cancelamento conforme Doc 23), reverte a baixa de estoque no KSI e registra a cadeia completa na auditoria. O lançamento original permanece — nada é apagado (R2).

11. Modelagem (entidades financeiras)

11.1 Entidades nucleares

Entidade	Descrição	Atributos-chave
Empresa	Unidade multiempresa	<code>empresa_id</code> , moeda_funcional, regime_tributário, calendário_fiscal
PlanoContas	Conta contábil hierárquica	<code>conta_id</code> , código, natureza (A/P/PL/R/D/C), pai_id, <code>empresa_id</code>
CentroCusto	Dimensão de rateio	<code>cc_id</code> , código, tipo (OS/equipe/frota/unidade), <code>empresa_id</code>
PeriodoContabil	Janela de competência	<code>periodo_id</code> , ano/mês, status (aberto/ajuste/fechado), <code>empresa_id</code>
Fatura	Documento comercial de venda	<code>fatura_id</code> , <code>os_id</code> , cliente_id, moeda, total_bruto, total_impostos, total_liquido
Faturaltem	Linha da fatura	<code>item_id</code> , tipo (serviço/peça), ref_peca/serviço, qtd, preço_unit, tributos
Titulo	Direito (AR) ou obrigação (AP)	<code>titulo_id</code> , tipo (AR/AP), origem_ref, valor, moeda, status, <code>empresa_id</code>
Parcela	Fração do título com vencimento	<code>parcela_id</code> , <code>titulo_id</code> , vencimento, valor, saldo
Baixa	Liquidação (total/parcial)	<code>baixa_id</code> , <code>parcela_id</code> , valor, data, meio_pagamento, juros/multa/desconto
MovCaixa	Movimento em conta bancária	<code>mov_id</code> , <code>conta_id</code> , valor, sinal, data, <code>baixa_id</code>
ContaBancaria	Conta em banco/gateway	<code>conta_id</code> , banco, agência/conta, moeda, saldo_razão, <code>empresa_id</code>
ExtratoLinha	Linha de extrato importado	<code>ext_id</code> , <code>conta_id</code> , data, valor, identificadores, status_conciliação
DocumentoFiscal	NF-e/NFS-e/abstração	<code>docf_id</code> , tipo, série/número, chave, status, xml_ref, <code>fatura_id</code>
Lancamento	Cabeçalho de partida dobrada	<code>lanc_id</code> , data, <code>periodo_id</code> , origem_ref, estorno_de, <code>empresa_id</code>
LancamentoLinha	Débito/crédito	<code>linha_id</code> , <code>lanc_id</code> , <code>conta_id</code> , <code>cc_id?</code> , sinal (D/C), valor, moeda
MoedaPolitica	Política cambial da empresa	moeda_funcional, moedas_transação[], regra_reavaliação
TaxaCambio	Cotação	<code>taxa_id</code> , par (ex. USD/BRL), data, valor, fonte

11.2 Modelo relacional (visão lógica)

```

erDiagram
    EMPRESA {
        uuid empresa_id PK
        string moeda_funcional
        string regime_tributario
    }
    FATURA {
        uuid fatura_id PK
        uuid empresa_id FK
        uuid os_id FK
        uuid cliente_id FK
        string moeda
        decimal total_bruto
        decimal total_impostos
        decimal total_liquido
        string status
    }
    FATURA_ITEM {
        uuid item_id PK
        uuid fatura_id FK
        string tipo
        string ref
        decimal qtd
        decimal preco_unit
        decimal cmv_unit
    }
    TITULO {
        uuid titulo_id PK
        uuid empresa_id FK
        string tipo
        string origem_ref
        decimal valor
        string moeda
        string status
    }
    PARCELA {
        uuid parcela_id PK
        uuid titulo_id FK
        date vencimento
        decimal valor
        decimal saldo
    }
    BAIXA {
        uuid baixa_id PK
        uuid parcela_id FK
        decimal valor
        date data
        string meio_pagamento
    }
    LANCAMENTO {
        uuid lanc_id PK
        uuid empresa_id FK
        uuid periodo_id FK
        date data
        string origem_ref
        uuid estorno_de FK
    }
    LANCAMENTO_LINHA {
        uuid linha_id PK
        uuid lanc_id FK
        uuid conta_id FK
        uuid cc_id FK
        string sinal
        decimal valor
        string moeda
    }
    DOCUMENTO_FISCAL {
        uuid docf_id PK
        uuid fatura_id FK
        string tipo
        string serie_numero
    }

```

```

    string chave
    string status
}

EMPRESA ||--o{ FATURA : possui
FATURA ||--|{ FATURA_ITEM : contem
FATURA ||--|{ TITULO : origina
FATURA ||--o{ DOCUMENTO_FISCAL : gera
TITULO ||--|{ PARCELA : parcela
PARCELA ||--o{ BAIXA : baixa
EMPRESA ||--o{ LANCAMENTO : registra
LANCAMENTO ||--|{ LANCAMENTO_LINHA : contem

```

11.3 Integração de valorização com o KSI

O KSI é o dono da **quantidade** e do **custo unitário** de cada item de estoque; o Financeiro é o dono do **saldo contábil de estoque** e do **CMV**. A fronteira:

Momento	KSI (quantidade/custo)	Financeiro (contábil)
Entrada (compra)	Incrementa qtd; recalcula custo médio (ou empilha camada PEPS)	D: Estoque / C: Fornecedores (pelo valor da nota)
Saída (peça na OS)	Decrementa qtd; retorna custo pela política vigente	D: CMV / C: Estoque (pelo custo retornado)
Ajuste/inventário	Corrige qtd/valor	D/C: Estoque × Ajuste de Inventário
Provisão de perda/obsolescência	Sinaliza itens	D: Despesa de Provisão / C: Provisão para Perdas

Regra de ouro (R12): o custo usado na contabilização do CMV é **exatamente** o custo retornado pelo KSI na baixa — nunca recalculado no Financeiro. A conta contábil de estoque deve reconciliar com $\Sigma(qtd \times custo)$ do KSI a cada fechamento; divergência é *stopper* de fechamento.

11.4 Multi-moeda (modelo)

Três moedas coexistem por transação estrangeira:

- **Moeda funcional** — moeda da empresa (a contabilidade "pensa" nela). Ex.: BRL.
- **Moeda de transação** — moeda em que o negócio ocorreu. Ex.: USD (peça importada).
- **Moeda de apresentação** — moeda de relatórios/consolidação. Ex.: EUR para matriz internacional.

Cada valor em moeda estrangeira guarda a **taxa aplicada** (data + fonte). No fechamento, saldos monetários em ME são **reavaliados** à taxa de fechamento, gerando lançamento de **variação cambial** (realizada na baixa; não realizada na reavaliação). O detalhe de conformidade cambial e retenções fica no Doc 23; aqui garantimos a **estrutura**.

12. Checklist

12.1 Prontidão operacional (por empresa)

- [] Plano de contas configurado e mapeado às regras de *posting*.
- [] Centros de custo definidos e política de rateio ativa.
- [] Calendário fiscal e períodos contábeis abertos corretamente.
- [] Moeda funcional, política cambial e fontes de cotação configuradas.
- [] Motor fiscal (Doc 23) plugado e validado para a jurisdição.
- [] Contas bancárias e credenciais Open Finance/gateways ativas.
- [] Alçadas de aprovação e SoD (Doc 07) parametrizadas.
- [] Integração KSI validada (entrada, saída, CMV, política de valorização).

12.2 Por transação de faturamento

- ☐ OS marcada como faturável e itens completos (serviços + peças).
- ☐ Preços/contrato obtidos do CRM.
- ☐ Tributos calculados pelo motor fiscal.
- ☐ Limite de crédito do cliente verificado.
- ☐ Fatura emitida de forma idempotente.
- ☐ Documento fiscal autorizado (ou em contingência controlada).
- ☐ Título AR + parcelas gerados.
- ☐ CMV reconhecido e estoque baixado no KSI.
- ☐ Lançamentos contábeis balanceados e postados.

12.3 Por fechamento de período

- ☐ Todas as contas bancárias conciliadas.
- ☐ Reavaliação cambial executada.
- ☐ Provisões e apropriações lançadas.
- ☐ Reconciliação estoque KSI ↔ conta contábil OK.
- ☐ Tributos apurados (Doc 23).
- ☐ Balancete fecha (débitos = créditos).
- ☐ Período travado e relatórios (DRE/Balanço/DFC) arquivados com hash.

13. Riscos

ID	Risco	Impacto	Prob.	Mitigação
RF1	Divergência entre estoque (KSI) e saldo contábil	Alto	Média	Reconciliação contínua §11.3; <i>stopper</i> de fechamento; alerta automático
RF2	Duplicação de faturamento em reprocessamento de evento	Alto	Média	Idempotência por <code>os_id</code> +versão (R5); outbox (Doc 09)
RF3	Erro de arredondamento/precisão monetária	Médio	Média	Decimal de precisão fixa; arredondamento determinístico registrado (R14/BP6)
RF4	Rejeição/indisponibilidade da autoridade fiscal	Alto	Alta	Emissão assíncrona, retry, modo contingência; fatura desacoplada do documento (BP8)
RF5	Vazamento de dados entre empresas (multiempresa)	Crítico	Baixa	<code>empresa_id</code> obrigatório em toda query/evento (R3); isolamento por schema (Doc 08)
RF6	Violação de segregação de funções (fraude interna)	Crítico	Baixa	SoD como política de autorização (R9); trilha de auditoria (R13)
RF7	Conciliação bancária com falso-positivo de match	Médio	Média	Score de confiança; confirmação humana para matches não exatos; divergência de saldo bloqueia fechamento
RF8	Erro de conversão/variação cambial	Alto	Média	Taxa registrada com fonte/data (R10); reavaliação padronizada; testes de fechamento em ME
RF9	Lançamento em período fechado	Alto	Baixa	Período somente-leitura (R7); ajustes só via período de ajuste autorizado
RF10	Inadimplência não gerenciada	Médio	Alta	Régua de cobrança automatizada (BP9); limite de crédito no faturamento
RF11	Regra de imposto <i>hardcoded</i> desatualizando	Alto	Média	Motor fiscal externo parametrizado (R11); versionamento no Doc 23
RF12	Perda de rastreabilidade origem → lançamento	Alto	Baixa	<code>origem_ref</code> obrigatório (R4); razão imutável (R2); auditoria (§15)

14. Melhorias Futuras

- **MF1. Multi-moeda plena e consolidação internacional** — reavaliação automática, tradução de demonstrações (moeda funcional → apresentação), hedge accounting básico.
- **MF2. Conciliação assistida por padrões aprendidos** — motor de matching que aprende padrões recorrentes de descrição/valor para elevar a taxa de conciliação automática, sempre com confirmação humana em exceções.
- **MF3. Previsão de fluxo de caixa avançada** — projeção considerando sazonalidade da operação de drones (safras, campanhas), pipeline de OS e comportamento histórico de pagamento por cliente.
- **MF4. Antecipação de recebíveis integrada** — conexão a mercados de crédito/factoring com cálculo automático de custo efetivo e impacto no caixa.
- **MF5. Faturamento recorrente e por assinatura** — contratos de manutenção de frota (mensalidade + franquia de horas de voo), medição de uso e faturamento automático.
- **MF6. Cobrança omnichannel** — PIX cobrança, boleto híbrido, cartão recorrente, carteira digital, com conciliação nativa do gateway.
- **MF7. Fechamento contínuo (soft close)** — apuração incremental diária reduzindo o esforço de fechamento mensal.
- **MF8. Painel de tesouraria consolidado multiempresa** — posição de caixa em tempo real, cash pooling entre empresas do grupo, política de aplicação.
- **MF9. Trilha fiscal internacional** — expansão da abstração do Doc 23 para VAT/GST e sales tax conforme entrada em novos países do roadmap.

15. Auditoria

15.1 Princípios de auditabilidade

O Financeiro é **auditável por construção**: cada valor reconstrói sua cadeia causal e cada ação registra quem, quando, o quê e a partir de qual estado.

- **Imutabilidade do razão (R2)**: lançamentos nunca são alterados; correção é sempre estorno + novo lançamento, ambos vinculados (`estorno_de`).
- **Rastreabilidade de origem (R4)**: `titulo.origem_ref` → OS/compra/contrato; `lançamento.origem_ref` → título/baixa/documento fiscal. Navegação bidirecional completa.
- **Correlação distribuída**: `traceparent` (W3C, Doc 09) propagado de OS → faturamento → título → lançamento, ligando ações entre contextos.
- **Segregação de funções (R9)**: trilha registra o par ator×ação para provar que cadastro, aprovação e execução foram atores distintos.

15.2 Eventos de auditoria financeira

Evento auditado	Dados registrados
Criação/edição de título	ator, timestamp, valores, origem_ref, <code>empresa_id</code> , <code>traceparent</code>
Aprovação (alçada)	ator aprovador, alçada aplicada, valor, referência SoD
Baixa/pagamento	ator, meio, valor, conta bancária, juros/multa/desconto
Emissão/cancelamento fiscal	tipo, chave, status, protocolo da autoridade, xml_ref
Estorno	lançamento original, motivo, autorização, cadeia revertida
Conciliação	linha de extrato, título casado, score, decisão (auto/humana)
Abertura/fechamento de período	ator, período, balancete-hash, timestamp
Alteração de configuração fiscal/plano de contas	ator, versão anterior/nova, vigência

15.3 Controles de fechamento e conformidade

- **Travamento de período**: após fechado, o período é somente-leitura; qualquer escrita é rejeitada (R7).
- **Balancete assinado**: cada fechamento gera balancete com **hash** (integridade) e assinatura do responsável, arquivado para auditoria externa.
- **Reconciliações obrigatórias**: bancária (R8) e estoque ↔ contábil (R12/§11.3) são pré-condições verificadas antes do travamento.
- **Retenção e arquivamento**: documentos fiscais (XMLs), extratos, balancetes e trilhas são retidos pelos prazos legais aplicáveis (parametrizados por jurisdição no Doc 23) e imutáveis.
- **Relatórios de auditoria**: razão por conta, livro-caixa, aging de AR/AP, mapa de conciliação, log de estornos e relatório de exceções de SoD, todos exportáveis e filtráveis por `empresa_id` e período.

Notas de fronteira entre documentos. A mecânica interna do estoque (barras/Rfid/curva ABC/reposição) é do **Doc 15 (KSI)**; o cálculo tributário e a conformidade regulatória por região são do **Doc 23**; schemas físicos e tenancy são do **Doc 08**; eventos de domínio, sagas e outbox são do **Doc 09**; papéis, IAM e SoD são do **Doc 07**. Este documento define **entidades, ciclos, interfaces e controles financeiros** e como o Financeiro consome essas fronteiras sem duplicá-las.

19 DevOps

Documento: 19 – DevOps **Versão:** 1.0 · **Data:** 21 de julho de 2026 · **Status:** Ativo **Dependências:** 00, 01, 02, 03, 04, 05, 06, 07, 08, 09, 10, 11, 12, 13, 14, 15, 16, 17, 18 **Responsável:** DevOps / SRE

1. Resumo Executivo

Este documento define a **engenharia de entrega e operação** do Drone Kairós ERP: a disciplina que transforma código versionado em software confiável rodando em produção, com o menor atrito e o maior grau de segurança e observabilidade possíveis. Ele materializa três princípios do canon — **contratos primeiro, segurança por padrão e observabilidade desde a arquitetura** — no plano operacional, cobrindo desde o *commit* de um desenvolvedor até o SLO honrado para um tenant em produção.

A plataforma é **multiempresa** e **evolutiva** (monólito modular → microsserviços, conforme Doc 07/09), com três frentes de artefato de natureza distinta: **backend** (serviços containerizados), **web** (SPAs e painéis estáticos servidos por CDN) e **mobile** (apps Flutter distribuídos por lojas). Cada frente tem um pipeline de CI/CD próprio, mas todos compartilham os mesmos **portões de qualidade** (testes, análise estática, varredura de segurança, testes de contrato) e a mesma **cadeia de suprimento de software** endurecida (imagens assinadas, SBOM, proveniência).

A estratégia de infraestrutura é **tudo-como-código (IaC)**: nenhum recurso é criado manualmente; ambientes (**dev, homolog, prod**) são descritos declarativamente, versionados e promovidos por **GitOps**. A orquestração de contêineres avalia e adota **Kubernetes** como plano de execução do backend, com *registry* privado, gestão de configuração externalizada e **segredos** nunca versionados. Os **deploys** usam estratégias progressivas (**blue-green, canary, feature flags**) para reduzir raio de explosão e permitir *rollback* em segundos.

A operação é regida por **SRE**: **SLIs/SLOs/error budget** governam o ritmo entre entregar e estabilizar; **on-call, runbooks, gestão de incidentes e postmortems sem culpa** fecham o ciclo de confiabilidade. **Backup e Disaster Recovery** são projetados em conjunto com o **KCD** (Doc 14), com RPO/RTO por classe de dado. Fecha o documento a **governança de custos (FinOps)**, tornando o gasto de nuvem uma métrica de engenharia de primeira classe.

2. Objetivos

- **01.** Definir pipelines de **CI/CD** para backend, web e mobile, com portões de qualidade e segurança idênticos e automatizados (fail-fast, sem deploy manual em prod).
- **02.** Estabelecer **Infraestrutura como Código** e a gestão dos três ambientes (dev/homolog/prod) com promoção por GitOps e paridade dev ↔ prod.
- **03.** Padronizar **containers e orquestração** (Kubernetes), *registry* privado, gestão de configuração e **segredos** externalizados e rotacionáveis.
- **04.** Adotar **estratégias de deploy progressivo** (blue-green, canary, feature flags) com *rollback* automático guiado por SLO.
- **05.** Tornar a **observabilidade** (logs, métricas, tracing distribuído, dashboards, alertas) um requisito operável, com **SLIs/SLOs/error budget** formalizados.
- **06.** Instituir a prática de **SRE**: on-call, runbooks, gestão de incidentes por severidade e postmortems sem culpa.
- **07.** Garantir **Backup/DR** por classe de dado em conjunto com o KCD, com RPO/RTO medidos e testados.
- **08.** Implantar **governança de custos (FinOps)** com visibilidade por ambiente, serviço e tenant, e mecanismos de contenção.
- **09.** Endurecer a **cadeia de suprimento de software** (assinatura de imagens, SBOM, proveniência, varredura contínua).

3. Escopo

No escopo: esteira de CI/CD (build, teste, análise estática/SAST/SCA/DAST, empacotamento, deploy) para as três frentes; IaC e ciclo de vida de ambientes; contêineres, orquestração Kubernetes, *registry*, configuração e segredos; estratégias de deploy e *feature flags*; stack de observabilidade e prática de SLI/SLO/error budget; processo de SRE (on-call, runbooks, incidentes, postmortems); backup/DR operacional; FinOps; cadeia de suprimento de software.

Fora do escopo (delegado): modelagem de dados e estratégia de SN → Doc 08; contratos de API/eventos e resiliência de aplicação (circuit breaker, saga) → Doc 09; políticas de segurança, Zero Trust, criptografia, SOC e o **desenho** de backup/DR → Doc 14 (KCD); arquitetura de aplicação e bounded contexts → Doc 07; padrões de código → Doc 10. Aqui tratamos do **como entregar e operar** esses artefatos com confiabilidade — não do que eles fazem internamente.

4. Regras

- **R1. Nada manual em produção.** Todo recurso de infra e todo deploy nascem de código versionado e revisado; console/ `kubectl apply` ad-hoc em prod é proibido (quebra-vidro auditado apenas em incidente).
- **R2. Pipeline é o único caminho.** Nenhum artefato chega a homolog/prod sem passar por todos os portões de qualidade e segurança. Não há *bypass*.
- **R3. Fail-fast e fail-closed.** Falha em teste, SAST, SCA ou varredura de imagem **bloqueia** a promoção. Segurança falha fechada.
- **R4. Imutabilidade.** Artefatos são imutáveis e endereçados por *digest* (não por tag móvel). A mesma imagem que passou em homolog é a que vai a prod (*build once, promote many*).
- **R5. Segredo nunca no Git.** Credenciais, chaves e tokens vivem em cofre externo; o repositório contém apenas **referências**. Varredura de segredo roda em cada push.
- **R6. Paridade de ambientes.** Dev, homolog e prod compartilham a mesma IaC e as mesmas imagens; divergem apenas por parâmetros (escala, tamanho, quotas), nunca por arquitetura.
- **R7. Tenant e trace onipresentes.** Toda telemetria (log, métrica, span) carrega `tenant_id` e `trace_id`; ausência é defeito de instrumentação.
- **R8. Deploy progressivo por padrão.** Mudanças de produção entram por canary ou blue-green com verificação automática de SLO; *big bang* é exceção justificada.
- **R9. SLO governa cadência.** Enquanto o *error budget* estiver esgotado, o fluxo prioriza estabilidade sobre novas features (política de congelamento).
- **R10. Cadeia de suprimento verificável.** Toda imagem em prod é **assinada**, tem **SBOM** publicado e proveniência rastreável; nós de execução recusam imagem não assinada.
- **R11. Reversível sempre.** Todo deploy tem *rollback* testado; toda migração de schema é retrocompatível (expand/contract) para permitir reversão sem perda.
- **R12. Custo é métrica de engenharia.** Cada serviço e ambiente tem orçamento; estouro gera alerta e revisão, como qualquer SLO.

5. Arquitetura (de entrega e infraestrutura)

5.1 Fluxo de valor (do commit ao SLO)

```
flowchart LR
    DEV[Desenvolvedor] -->|push / PR| SCM[(Repositório Git)]
    SCM --> CI[CI: build + testes + segurança]
    CI --> REG[(Registry privado<br/>imagens assinadas + SBOM)]
    REG --> CD[CD / GitOps]
    CD --> DEVENV[Dev]
    CD --> HML[Homolog]
    CD --> PRD[Prod]
    PRD --> OBS[Observabilidade<br/>logs · métricas · tracing]
    OBS --> SLO[SLO / Error Budget]
    SLO -->|dentro| DEV
    SLO -->|estourado| FREEZE[Congela features<br/>prioriza confiabilidade]
```

5.2 Topologia de ambientes

Três ambientes permanentes + ambientes efêmeros de PR:

Ambiente	Propósito	Dados	Escala	Acesso	Deploy
Efêmero (PR)	Validar um PR isoladamente	Sintéticos/seed	Mínima	Time do PR	Automático no PR, destruído no merge
Dev	Integração contínua da branch principal	Sintéticos/seed	Mínima	Engenharia	Automático a cada merge
Homolog	Aceite, testes E2E, ensaio de release, carga	Mascarados/anonimizados	~30% da prod	QA, PO, parceiros	Automático (promoção)
Prod	Produção multiempresa	Reais (isolados por tenant)	Plena, autoscaling	SRE (quebra-vidro)	Promoção com aprovação + canary

Regra de fluxo: um artefato só sobe (dev → homolog → prod); nunca lateral nem regressivo. A **mesma imagem** promovida difere apenas por *config* injetada em runtime.

5.3 Plano de execução: Kubernetes (avaliação e decisão)

O backend containerizado adota **Kubernetes** como orquestrador. A decisão é justificada por gatilhos, não por moda:

Critério	Sem orquestração / PaaS gerenciado	Kubernetes	Decisão Kairós
Nº de serviços a operar	Poucos (fase monólito)	Muitos (fase microsserviços, Doc 09)	K8s para acompanhar a evolução para serviços
Autoscaling por serviço	Limitado	Nativo (HPA/VPA)	K8s — escala assimétrica (BI, KCI)
Deploys progressivos	Manual/limitado	Nativo + controladores (canary/blue-green)	K8s — exige R8
Portabilidade multi-cloud	Baixa	Alta	K8s — evita <i>lock-in</i>
Custo cognitivo/operacional	Baixo	Alto	Mitigado por plataforma interna + GitOps
Fase inicial (monólito)	Suficiente	Overhead	Iniciar com cluster gerenciado enxuto; crescer com a malha

Recomendação: cluster **Kubernetes gerenciado** (control plane administrado pelo provedor) para reduzir carga operacional. **Mobile** e **web estático** não usam K8s: web é publicado em **CDN/object storage**; mobile é empacotado e distribuído pelas **lojas** (ver §5.6).

Organização do cluster (namespaces): um namespace por ambiente lógico e por camada; isolamento de tenant é **lógico** (row-level, Doc 07/08), com opção de *node pool/namespaces* dedicado para tenants enterprise que exijam isolamento físico.

```
flowchart TB
    subgraph Cluster["Cluster Kubernetes (por ambiente)"]
        direction TB
        subgraph Edge["Ingress / Borda"]
            IGW["Ingress Gateway + WAF"]
        end
        subgraph Core["Namespace: core"]
            MONO["monolito-modular / serviços de domínio"]
            BFFW["bff-web"]
            BFFM["bff-mobile"]
        end
        subgraph Data["Namespace: data (operado/gerenciado)"]
            PG["PostgreSQL – gerenciado"]
            BROKER["Event Broker"]
            CACHE["Cache"]
        end
        subgraph Plat["Namespace: platform"]
            OTEL["OTel Collector"]
            ESO["External Secrets Operator"]
            ARGO["GitOps Controller"]
        end
        IGW --> BFFW & BFFM & MONO
        MONO --> PG & BROKER & CACHE
        MONO -. telemetria .-> OTEL
    end
```

5.4 Cadeia de suprimento de software (supply chain)

Cada artefato de backend/web percorre uma cadeia endurecida antes de ser promovível:

1. **Build reproduzível** em contêiner efêmero (sem estado de máquina do dev).
2. **SBOM** (inventário de dependências) gerado e armazenado junto ao artefato.
3. **Assinatura** da imagem (chaves gerenciadas em cofre) — proveniência atestada (nível SLSA como meta).
4. **Varredura** de vulnerabilidades da imagem e de dependências (SCA) com política de bloqueio por severidade.
5. **Admissão** no cluster **recusa imagem não assinada** ou de *registry* não confiável (R10).

5.5 Configuração e segredos

Aspecto	Regra	Mecanismo de referência
Configuração não sensível	Externalizada por ambiente, versionada	ConfigMap / repositório de config (GitOps)
Segredos	Nunca no Git; injetados em runtime	Cofre (Vault-like) + External Secrets Operator
Rotação	Periódica e sob incidente	Rotação automatizada; TTL curto para credenciais dinâmicas
Criptografia	Em trânsito e repouso (padrão KCD)	mTLS no mesh; envelope encryption (KMS)
Isolamento por tenant	Segredos de tenant nunca cruzam fronteira	Namespaces/paths por tenant no cofre

5.6 Frentes de artefato

Frente	Artefato	Empacotamento	Destino	Distribuição
Backend	Imagem de contêiner (OCI)	Multi-stage, base <i>distroless</i> , não-root	Registry privado → K8s	GitOps + canary/blue-green
Web	Bundle estático + BFF	Build otimizado, <i>hashing</i> de assets	Object storage + CDN	Publicação atômica + invalidação de cache
Mobile	APK/AAB (Android), IPA (iOS)	Flutter build, <i>code signing</i>	Lojas + trilhas beta	Fastlane → <i>staged rollout</i> nas lojas + <i>feature flags</i>

6. Diagramas

6.1 Pipeline de CI/CD (visão unificada)

```
flowchart TB
    subgraph CI["Integração Contínua"]
        C1[Checkout + cache] --> C2[Lint + format]
        C2 --> C3[Testes unitários]
        C3 --> C4[Testes de contrato<br/>consumer-driven]
        C4 --> C5[SAST + secret scan]
        C5 --> C6[SCA / dependências + licenças]
        C6 --> C7[Build artefato]
        C7 --> C8[Scan de imagem + SBOM]
        C8 --> C9[Assinatura + push no registry]
    end
    subgraph CD["Entrega Contínua"]
        D1[Deploy Dev] --> D2[Testes de integração/E2E]
        D2 --> D3[Promoção Homolog]
        D3 --> D4[E2E + carga + DAST]
        D4 --> D5[Aprovação]
        D5 -->|ok| D6[Canary Prod 5%]
        D6 --> D7[SL0/erros ok?]
        D7 -->|sim| D8[Progressão 25% → 50% → 100%]
        D7 -->|não| D9[Rollback automático]
    end
    C9 --> D1
```

6.2 Topologia dos ambientes e promoção

```
flowchart LR
    subgraph Repos["Git (fonte da verdade)"]
        APP[Repos de aplicação]
        IAC[Repo de IaC / GitOps]
    end
    APP --> BUILD[CI Build] --> REG[(Registry)]
    REG --> ENVDEV
    subgraph Envs["Ambientes (IaC idêntica, params distintos)"]
        ENVDEV[Dev] -->|promove digest| ENVHML[Homolog] -->|promove digest| ENVPRD[Prod]
    end
    IAC -. reconcilia .-> ENVDEV
    IAC -. reconcilia .-> ENVHML
    IAC -. reconcilia .-> ENVPRD
    ENVPRD --> OBS[(Stack de Observabilidade)]
```

6.3 Stack de observabilidade

```
flowchart TB
    subgraph App["Serviços instrumentados (OpenTelemetry)"]
        S1[Serviços de domínio]
        S2[BFF Web/Mobile]
        S3[API Gateway]
    end
    S1 & S2 & S3 -->|OTLP| COL[OTel Collector]
    COL --> LOGS[Logs estruturados]
    COL --> METR[Métricas - TSDB]
    COL --> TRAC[Traces distribuídos]
    LOGS & METR & TRAC --> DASH[Dashboards]
    METR --> ALERT[Motor de Alertas]
    ALERT --> ONCALL[On-call / Paging]
    METR --> SLO[SLO / Error Budget]
```

7. Fluxogramas (CI/CD por frente)

7.1 Backend

```
flowchart TD
    A([Push/PR branch]) --> B[Lint + testes unit + contrato]
    B --> C{Verde?}
    C -->|não| X[Bloqueia PR + reporta]
    C -->|sim| D[SAST + secret scan + SCA]
    D --> E{Sem bloqueio crítico?}
    E -->|não| X
    E -->|sim| F[Build imagem multi-stage]
    F --> G[Scan imagem + SBOM + assinatura]
    G --> H[Push registry por digest]
    H --> I[Ambiente efêmero do PR + E2E]
    I --> J{Merge na main?}
    J -->|sim| K[Deploy Dev automático]
    K --> L[Promoção Homolog + carga/DAST]
    L --> M[Aprovação de release]
    M --> N[Canary Prod + verificação SLO]
    N --> O{SLO ok?}
    O -->|sim| P[100% + monitoramento]
    O -->|não| Q[Rollback automático + alerta]
```

7.2 Web

```
flowchart TD
    A([Push/PR]) --> B[Lint + testes unit + componentes]
    B --> C[SAST + SCA + secret scan]
    C --> D[Build estático + hashing de assets]
    D --> E[Testes E2E em preview efêmero]
    E --> F{Aprovado?}
    F -->|sim| G[Publicação atômica no object storage]
    G --> H[Invalidação de cache CDN]
    H --> I[Canary por % de tráfego na borda]
```

7.3 Mobile (Flutter)

```
flowchart TD
  A([Push/PR]) --> B[Analyze + testes widget/unit]
  B --> C[Build AAB/IPA + code signing]
  C --> D[Testes de integração em dispositivos/emuladores]
  D --> E[Distribuição trilha interna/beta]
  E --> F{Aceite}
  F -->|ok| G[Submissão às lojas]
  G --> H[Staged rollout 5% → 100% + feature flags]
  H --> I[Kill-switch remoto por feature flag se regressão]
```

8. Boas Práticas

- **Build once, promote many.** Compilar/empacotar uma única vez; promover o mesmo *digest* por dev → homolog → prod. Nunca recompilar por ambiente.
- **Trunk-based + PRs curtos.** Branch principal sempre entregável; *feature flags* desacoplam *deploy* de *release*.
- **Portões idênticos entre frentes.** Backend, web e mobile passam pelos mesmos tipos de portão (teste, SAST, SCA, secret scan) para evitar elo fraco.
- **Testes em pirâmide.** Muitos unitários, alguns de contrato/integração, poucos E2E; testes de contrato *consumer-driven* protegem as fronteiras do Doc 09.
- **Migrações expand/contract.** Toda mudança de schema é retrocompatível em duas fases, permitindo rollback sem perda (R11).
- **Infra imutável.** Servidores/pods não são consertados no lugar; são substituídos por nova versão (*cattle, not pets*).
- **GitOps como fonte da verdade.** O estado desejado vive no Git; um controlador reconcilia o cluster continuamente e corrige *drift*.
- **Observabilidade como código.** Dashboards, alertas e SLOs versionados junto ao serviço; instrumentação com `tenant_id / trace_id` (R7).
- **Least privilege operacional.** Acessos a prod são *just-in-time*, temporários e auditados; nada de credencial permanente de humano.
- **Menor raio de explosão.** Canary + *feature flags* + limites de blast por tenant; um deploy ruim afeta 5%, não 100%.
- **Alerta acionável.** Alerta que não pede ação humana é ruído; todo alerta aponta para um runbook.
- **Custo visível no pipeline.** Estimativa de custo de mudança de IaC no PR; *tags* de custo obrigatórias em todo recurso.

9. Padrões

9.1 Padrões de entrega

Padrão	Adoção	Observação
Trunk-based development	Obrigatório	Branches efêmeras, integração diária
Versionamento semântico (SemVer)	Obrigatório	<code>MAJOR.MINOR.PATCH</code> em imagens e libs
Conventional Commits	Obrigatório	Habilita <i>changelog</i> e versionamento automáticos
GitOps (pull-based)	Obrigatório	Controlador reconcilia; sem push direto ao cluster
Multi-stage / distroless	Obrigatório	Imagem mínima, não-root, superfície reduzida
12-Factor	Referência	Config no ambiente, processos stateless, logs como stream

9.2 Estratégias de deploy

Estratégia	Como funciona	Quando usar	Rollback
Rolling update	Substitui pods gradualmente	Mudança de baixo risco, sem quebra de contrato	Rollout undo
Blue-Green	Duas cores idênticas; troca de tráfego atômica	Mudança sensível que exige corte instantâneo	Reapontar para a cor anterior (segundos)
Canary	5% → 25% → 50% → 100% com verificação de SLO	Padrão de prod (R8); mudança de risco médio	Congela/reverte na primeira violação
Feature flags	<i>Deploy sem release</i> ; ativação por flag/tenant/%	Desacoplar entrega de exposição; testes A/B; kill-switch	Desligar a flag (instantâneo, sem redeploy)
Shadow / dark launch	Tráfego espelhado sem afetar resposta	Validar carga/comportamento de novo serviço	N/A (sem impacto ao usuário)

9.3 Padrões de observabilidade

- **Método RED** (Rate, Errors, Duration) para serviços de requisição; **USE** (Utilization, Saturation, Errors) para recursos.
- **Logs estruturados** (JSON) com campos canônicos: `timestamp`, `level`, `service`, `tenant_id`, `trace_id`, `span_id`, `message`.
- **Métricas** com convenção de nomes e *labels* padronizados (evitando alta cardinalidade descontrolada).
- **Tracing** W3C Trace Context / OpenTelemetry propagado ponta-a-ponta (herdado do Doc 09).
- **Correlação** log ↔ trace ↔ métrica por `trace_id` em todo dashboard de incidente.

9.4 Convenção de nomes e tags de recurso

Todo recurso de infra carrega tags obrigatórias: `projeto`, `ambiente`, `servico`, `owner`, `custo-centro`, `tenant-tier`. Sem tags → *policy-as-code* bloqueia a criação.

10. Casos de Uso

UC-01 — Feature nova do commit ao usuário. Dev abre PR → CI roda testes, SAST, SCA, secret scan e sobe ambiente efêmero → após merge, deploy automático em Dev, promoção a Homolog com E2E/carga, aprovação, canary 5% em Prod com verificação de SLO, progressão a 100%. Toda a mudança fica atrás de *feature flag* desligada até o *release* de negócio.

UC-02 — Rollback automático guiado por SLO. Canary a 25% eleva a taxa de erro acima do limiar do SLO de disponibilidade. O controlador de progressão detecta a violação, **interrompe** a progressão e **reverte** para a versão anterior em segundos; um alerta abre incidente SEV3 e aponta ao runbook de deploy.

UC-03 — Hotfix de segurança em produção. Uma CVE crítica é detectada pela varredura contínua de imagens. Pipeline expresso: patch → build → scan → assinatura → blue-green com corte instantâneo. Postmortem registra a origem e endurece a política de SCA.

UC-04 — Provisionar novo ambiente/tenant enterprise. Um tenant enterprise exige isolamento reforçado. IaC parametrizada cria *node pool*/namespace dedicado, paths de segredo isolados e quotas próprias, tudo por PR revisado, sem toque manual (R1/R6).

UC-05 — Rotação de credencial comprometida. KCD sinaliza credencial exposta. Cofre rotaciona automaticamente, External Secrets Operator propaga o novo valor aos pods, e a credencial antiga é revogada — sem downtime e sem versionar segredo (R5).

UC-06 — Release mobile com staged rollout. Nova versão do app Flutter é submetida às lojas com liberação escalonada (5% → 100%). Telemetria de crash sobe em uma faixa; a *feature flag* correspondente é desligada remotamente (kill-switch), contendo o impacto sem nova submissão à loja.

UC-07 — Exercício de DR. *Game day* trimestral: restaura backups em região secundária, promove réplicas e mede RTO/RPO reais contra as metas (§11.5), gerando ações de melhoria (em conjunto com Doc 14).

11. Modelagem (ambientes e pipelines)

11.1 Matriz de ambientes

Dimensão	Dev	Homolog	Prod
Origem do deploy	Merge na main	Promoção de digest	Promoção + aprovação
Estratégia	Rolling	Blue-green (ensaio)	Canary (padrão)
Dados	Seed sintético	Mascarado	Real, isolado por tenant
Observabilidade	Básica	Completa	Completa + SLO/alertas
Retenção de logs	7 dias	15 dias	30–90 dias (por classe)
Backup	Não	Diário	Contínuo (ver §11.5)
Acesso humano	Livre (engenharia)	Restrito	Just-in-time / quebra-vidro

11.2 Exemplo de estágios de pipeline (declarativo)

```
# pipeline.yml – esteira de referência (backend)
# "build once, promote many" – o mesmo digest percorre os ambientes.
name: kairos-backend-delivery

stages:
- name: verify
  jobs:
    - lint: { run: "make lint" }
    - test-unit: { run: "make test-unit", coverage_min: 80 }
    - test-contract: { run: "make test-contract" } # consumer-driven (Doc 09)
    - sast: { run: "make sast" } # análise estática de segurança
    - secret-scan: { run: "make secret-scan", fail_on: any }
    - sca: { run: "make sca", fail_on: "high,critical" }
    - license-check: { run: "make license-check" }

- name: package
  needs: [verify]
  jobs:
    - build-image:
        run: "docker build --target runtime -t $IMAGE:$SHA ."
        base: distroless
        user: nonroot
    - image-scan: { run: "trivy image $IMAGE:$SHA", fail_on: "high,critical" }
    - sbom: { run: "syft $IMAGE:$SHA -o spdx-json > sbom.json" }
    - sign: { run: "cosign sign --key $COSIGN_KEY $IMAGE@$DIGEST" }
    - push: { run: "docker push $IMAGE@$DIGEST" } # imutável por digest

- name: deploy-dev
  needs: [package]
  environment: dev
  jobs:
    - gitops-set-image: { image: "$IMAGE@$DIGEST", target: "envs/dev" }
    - e2e: { run: "make e2e", against: dev }

- name: deploy-homolog
  needs: [deploy-dev]
  environment: homolog
  when: branch == "main"
  jobs:
    - gitops-set-image: { image: "$IMAGE@$DIGEST", target: "envs/homolog" }
    - e2e: { run: "make e2e", against: homolog }
    - load-test: { run: "make load", slo_gate: true }
    - dast: { run: "make dast", against: homolog }

- name: deploy-prod
  needs: [deploy-homolog]
  environment: prod
  approval: { required: true, reviewers: ["sre", "tech-lead"] }
  strategy:
    type: canary
    steps: [5, 25, 50, 100] # % de tráfego
    analysis:
      metrics: [error_rate, latency_p99, saturation]
      slo_gate: true # viola SLO => rollback automático
      rollback: automatic
  jobs:
    - gitops-set-image: { image: "$IMAGE@$DIGEST", target: "envs/prod" }
```

11.3 SLIs, SLOs e Error Budget

SLI é o que se mede; SLO é a meta; *error budget* é o quanto se pode falhar antes de congelar entregas (R9).

Serviço/Jornada	SLI	SLO (janela 30d)	Error budget	Consequência ao esgotar
API núcleo (leitura)	% de respostas < 300 ms e sem erro 5xx	99,9% disponibilidade	43 min/mês	Congela features; foco em confiabilidade
API núcleo (escrita/OS)	% de transações concluídas sem erro	99,9% sucesso	0,1%	Revisão de mudanças recentes
Ingestão de eventos	% de eventos processados < 5 s	99,5%	0,5%	Prioriza fila/DLQ
App mobile (sync offline)	% de sincronizações bem-sucedidas	99,0%	1,0%	Revisão do cliente de sync
Pipeline de deploy	Lead time / change failure rate	CFR < 15%	—	Postmortem obrigatório

Política de error budget: budget > 25% restante → cadência normal; entre 25% e 0% → revisão de risco por PR; esgotado → **congelamento** de features não-críticas até recuperação. Métricas **DORA** (deploy frequency, lead time, change failure rate, MTTR) acompanham a saúde do fluxo.

11.4 Alertas e severidades

Severidade	Critério	Resposta	Notificação
SEV1	Indisponibilidade ampla / perda de dados / brecha	On-call imediato + war room + comunicação	Paging + liderança + status page
SEV2	Degradação séria, SLO em risco iminente	On-call imediato	Paging
SEV3	Falha localizada, budget consumido	Horário comercial estendido	Chat + ticket
SEV4	Ruído/baixo impacto, tendência	Backlog	Ticket

Alertas são baseados em **sintoma** (SLO/experiência do usuário), não apenas em causa (uso de CPU), para reduzir fadiga e falso-positivo.

11.5 Backup e DR (com KCD — Doc 14)

Classe de dado	Estratégia de backup	RPO	RTO	Teste
Transacional (OS, Financeiro, SN)	Contínuo (WAL/PITR) + snapshot diário, replicação cross-region	≤ 5 min	≤ 1 h	Restauração mensal
Eventos / broker	Retenção + replicação de partições	≤ 1 min	≤ 30 min	Reprocesso de DLQ
Objetos/anexos	Versionamento + replicação geográfica	≤ 15 min	≤ 2 h	Amostragem trimestral
Configuração/laC	Git (já é a fonte da verdade)	0	≤ 15 min	Recriação via GitOps
Segredos	Cofre com backup cifrado	≤ 15 min	≤ 30 min	Rotação + restore

O **desenho** de criptografia, retenção legal e classificação é do **KCD (Doc 14)**; aqui está a **operação** (automação, verificação e *game days*). Backups são cifrados, imutáveis (proteção contra ransomware) e testados — backup não testado não é backup.

11.6 SRE — operação

- **On-call:** rotação com escala primária/secundária, *handoff* documentado, SLA de resposta por severidade e horas de descanso protegidas.

- **Runbooks:** um por alerta/cenário, versionado junto ao serviço; contém sintoma, diagnóstico, ação de mitigação, escalonamento e *rollback*.
- **Gestão de incidentes:** papéis definidos (Incident Commander, Comms, Ops), linha do tempo registrada, canal dedicado, *status page* para SEV1/SEV2.
- **Postmortem sem culpa:** obrigatório para SEV1/SEV2 (e SEV3 recorrente); foca em causas sistêmicas e ações corretivas com dono e prazo; publicado internamente.

Template de runbook (resumo): Alerta → Sintoma observável → Impacto/tenant → Diagnóstico rápido (dashboards/queries) → Mitigação → Rollback → Escalonamento → Pós-verificação.

Template de postmortem (resumo): Resumo → Impacto (duração, tenants, SLO/budget) → Linha do tempo → Causa raiz → O que funcionou / o que falhou → Ações corretivas (dono, prazo) → Lições.

11.7 Governança de custos (FinOps)

Prática	Descrição
Orçamento por dimensão	Budget por ambiente, serviço e tenant-tier; alerta em 80%/100%
Alocação de custo	Tags obrigatórias (\$9.4) permitem <i>showback/chargeback</i> por tenant
Rightsizing	Recomendação contínua de recursos (requests/limits) via VPA/telemetria
Escala a zero	Ambientes efêmeros e dev <i>scale-to-zero</i> fora de uso
Compromissos	Uso de capacidade reservada/spot para cargas previsíveis/tolerantes
Estimativa no PR	Diferença de custo de IaC exibida na revisão antes do merge

12. Checklist

Prontidão de pipeline e serviço (Definition of Done operacional):

- [] Pipeline com todos os portões (lint, testes unit/contrato, SAST, secret scan, SCA, licença) verdes e bloqueantes.
- [] Build multi-stage, imagem *distroless* não-root, scan sem crítico/alto, SBOM gerado e imagem **assinada**.
- [] Artefato imutável por *digest*; mesma imagem promovida dev → homolog → prod (build once, promote many).
- [] IaC versionada; ambiente reproduzível por GitOps; *drift* reconciliado automaticamente.
- [] Segredos apenas em cofre; nenhuma credencial no repositório; rotação configurada.
- [] Deploy de prod por canary/blue-green com *SLO gate* e **rollback automático** testado.
- [] Migração de schema em modo expand/contract, reversível.
- [] Instrumentação OTel: logs estruturados + métricas RED + traces com `tenant_id / trace_id`.
- [] Dashboards, alertas e SLOs versionados; cada alerta aponta a um runbook.
- [] `/healthz` e `/readyz` expostos; probes de liveness/readiness configurados.
- [] Backup automatizado com RPO/RTO definidos e restauração testada.
- [] On-call e runbook de falha/rollback publicados; postmortem para SEV1/SEV2.
- [] Tags de custo obrigatórias presentes; orçamento e alerta configurados.
- [] Admissão do cluster recusa imagem não assinada / registry não confiável.

13. Riscos

ID	Risco	Prob.	Impacto	Mitigação
RS-01	Deploy ruim atinge 100% dos tenants	Média	Crítico	Canary + blue-green + rollback automático (R8)
RS-02	Segredo vazado no repositório	Média	Crítico	Secret scan bloqueante + cofre + rotação (R5)
RS-03	<i>Drift</i> entre estado real e IaC	Média	Alto	GitOps pull-based reconciliando continuamente (R1)
RS-04	Imagem comprometida na cadeia de suprimento	Baixa	Crítico	Assinatura + SBOM + admissão restritiva + scan (R10)
RS-05	Migração de schema irreversível quebra prod	Média	Alto	Expand/contract + ensaio em homolog (R11)
RS-06	Fadiga de alertas / cegueira operacional	Alta	Médio	Alerta por sintoma/SLO, não por causa; revisão periódica
RS-07	Complexidade de Kubernetes sobrecarrega o time	Média	Médio	Cluster gerenciado + plataforma interna + GitOps
RS-08	Backup não restaurável no incidente	Baixa	Crítico	<i>Game days</i> de DR; restauração testada e medida (§11.5)
RS-09	Estouro de custo de nuvem sem visibilidade	Média	Médio	FinOps: tags, budgets, rightsizing, estimativa no PR
RS-10	Deriva de paridade dev ↔ prod mascara bugs	Média	Alto	IaC idêntica; diferença só por parâmetro (R6)
RS-11	<i>Lock-in</i> de fornecedor de nuvem	Baixa	Médio	K8s + IaC portátil; abstrações padrão (OTel, OCI)
RS-12	Loja mobile rejeita/atrasa release crítico	Média	Médio	<i>Staged rollout</i> + feature flags + kill-switch remoto

14. Melhorias Futuras

- **Progressive delivery avançado:** análise automática de canary com múltiplos SLIs e *auto-promotion*/*auto-rollback* orientados por sinais compostos.
- **Plataforma interna de desenvolvedor (IDP):** portal *self-service* com *golden paths* (criar serviço, ambiente, pipeline padronizados) para reduzir carga cognitiva.
- **Policy-as-code abrangente:** governança de admissão, custo e segurança expressa como código e testada no pipeline.
- **Chaos engineering contínuo:** injeção controlada de falhas em homolog/prod para validar resiliência (complementa Doc 09).
- **SLSA nível alto:** proveniência verificável ponta-a-ponta e *hermetic builds*.
- **FinOps preditivo:** previsão de custo por tenant e *rightsizing* automatizado.
- **Multi-região ativo-ativo:** evolução de DR (ativo-passivo) para ativo-ativo em jornadas críticas conforme demanda enterprise.
- **AIOps para operação:** correlação e detecção de anomalias assistida sobre a telemetria para reduzir MTTR (sem dependência de fornecedor comercial específico).

15. Auditoria

Item	Descrição
Rastreabilidade de mudança	Todo deploy é rastreável a um commit, PR revisado e digest de imagem assinado; GitOps registra quem promoveu o quê e quando.
Trilha de execução	<code>trace_id</code> e <code>tenant_id</code> em logs/métricas/traces permitem reconstruir qualquer requisição em produção ponta-a-ponta.
Integridade de artefato	Assinatura + SBOM + admissão restritiva garantem que só imagens verificadas rodam em prod (R10).
Segregação de acesso	Acesso a prod é just-in-time, temporário e auditado; quebra-vidro registra ator, motivo e janela.
Conformidade de segredos	Nenhum segredo no Git (varredura contínua); rotação e cofre auditáveis, alinhados ao KCD.
Prontidão de recuperação	RPO/RTO definidos, backups cifrados/imutáveis e restauração testada em <i>game days</i> documentados.
Governança de custo	Tags obrigatórias e budgets tornam o gasto auditável por ambiente, serviço e tenant.
Multiempresa	Isolamento de tenant preservado em telemetria, segredos e (quando exigido) execução dedicada.
Versão do documento	v1.0 — 21 de julho de 2026. Alterações futuras registradas no controle de versão do repositório de documentação.
Aprovação	DevOps/SRE (responsável); dependências 00–18 verificadas e coerentes com o PROJECT CANON.

Nota de encerramento. Este documento é o **contrato operacional** do Drone Kairós ERP: define como o software nasce (CI), como chega ao usuário (CD progressivo), como se mantém de pé (SRE + observabilidade + SLO) e como se recupera (DR), sob custo governado. Ele é deliberadamente **conservador na segurança e progressivo na entrega** — nada chega a produção fora do pipeline, nada roda sem ser observável, e nada muda sem poder ser revertido. A confiabilidade não é um estágio final; é uma propriedade construída em cada etapa.

20 QA

Documento: 20 – QA (Garantia de Qualidade e Testes) **Versão:** 1.0 · **Data:** 21 de julho de 2026 · **Status:** Ativo
Dependências: 00 – Master Prompt, 01 – Constituição, 02 – Project Bible, 03 – Roadmap, 04 – Pesquisa de Mercado, 05 – Benchmark, 06 – Requisitos, 07 – Arquitetura, 08 – Banco de Dados, 09 – Microsserviços, 10 – 19 **Responsável:** Engenharia de QA

1. Resumo Executivo

Este documento define a **estratégia de garantia de qualidade e a disciplina de testes** do Drone Kairós ERP — a plataforma multiempresa do ecossistema de drones que conecta **fabricante** → **revenda** → **técnico** → **cliente** com backend orientado a serviços, apps Flutter, painéis web, API pública e os módulos proprietários **KCI (Kairós Core Intelligence)**, **KCD (Kairós Cyber Defense)** e **KSI (Kairós Smart Inventory)**.

O princípio inegociável herdado do canon é direto: **nada é "pronto" sem auditoria**. Em QA isso se traduz em uma regra operacional — nenhum incremento atravessa um *Quality Gate* sem evidência objetiva (testes verdes, cobertura mínima, varreduras de segurança, verificação de isolamento de tenant). Qualidade aqui não é fase final nem responsabilidade de um time isolado: é uma **propriedade construída em cada camada** e verificada por múltiplos níveis de teste complementares.

A espinha dorsal da estratégia é a **pirâmide de testes**: uma base larga de testes **unitários** rápidos e determinísticos, uma camada intermediária de testes de **integração e contrato**, e um topo estreito de testes **end-to-end (e2e)** que valida jornadas críticas de negócio. A pirâmide é reforçada por **eixos transversais** que atravessam todos os níveis e são específicos do domínio: **isolamento multiempresa** (nenhum vazamento entre tenants, jamais), **segurança** (SAST/DAST/pentest em conjunto com o KCD), **performance e carga**, **offline-first** (mobile e KCI em campo, sem rede) e **avaliação de IA** (qualidade de diagnóstico e regressão de modelos do KCI).

O documento entrega: a arquitetura de testes e a divisão de responsabilidades por nível; a especificação dos testes transversais; a estratégia de **testes de contrato** (consumer-driven) para a API pública e entre serviços; a infraestrutura de **automação, dados de teste, ambientes e CI**; o processo de **gestão de defeitos** com matriz de severidade × prioridade; a **Definition of Done** e os **critérios de aceite**; as **métricas de qualidade** (cobertura, densidade de defeitos, *flakiness*, *escaped defects*); e os **Quality Gates** alinhados ao Master Prompt. Diagramas Mermaid ilustram a pirâmide, o fluxo de qualidade e o ciclo de vida do defeito; tabelas mapeiam tipos de teste por módulo.

2. Objetivos

- **O1.** Estabelecer a **pirâmide de testes** (unitário, integração, contrato, e2e) com metas de proporção, propriedade clara e critérios de "verde" por nível.
- **O2.** Especificar os **testes transversais do domínio**: isolamento multiempresa, segurança (com KCD), performance/carga, offline-first (mobile/KCI) e avaliação de IA (KCI).
- **O3.** Definir **testes de contrato** consumer-driven para a **API pública** e para as integrações **entre serviços** (síncronas e assíncronas por evento).
- **O4.** Padronizar **automação, gestão de dados de teste, topologia de ambientes e integração contínua (CI)**, com portões automáticos e evidências rastreáveis.
- **O5.** Instituir a **gestão de defeitos**: matriz de severidade/prioridade, SLA de correção, fluxo de triagem e critérios de reabertura.
- **O6.** Formalizar a **Definition of Done** e os **critérios de aceite** como contrato de conclusão verificável, não subjetivo.
- **O7.** Definir as **métricas de qualidade** e seus limiares, e como alimentam decisão de release.
- **O8.** Amarrar os **Quality Gates de QA** ao Master Prompt (00) e ao Roadmap (03): nenhuma fase avança sem que a anterior passe.

3. Escopo

No escopo: estratégia e pirâmide de testes; responsabilidades por nível; testes funcionais e não-funcionais; testes de isolamento multiempresa; testes de segurança em conjunto com KCD (SAST, DAST, SCA, pentest, secret scanning); testes de performance e carga; testes offline-first para mobile Flutter e KCI; avaliação de qualidade e regressão de modelos do KCI; testes de contrato (API pública e inter-serviços); automação, dados de teste, ambientes e CI; gestão de defeitos; DoD e critérios de aceite; métricas; Quality Gates.

Fora do escopo (referência a outros documentos): requisitos funcionais detalhados (Doc 06); desenho de arquitetura e tenancy (Doc 07); modelagem física de dados e RLS (Doc 08); contratos e comportamento distribuído dos serviços (Doc 09); controles internos e arquitetura do KCD (documento de Segurança); pipelines de entrega/laC e topologia de clusters (documento de DevOps/Operação). Aqui QA **consome** esses contratos e define **como verificá-los**; não os redefine.

Aplicabilidade: toda a plataforma — ERP, apps, painéis, portais, CRM, financeiro, BI, API pública e os módulos KCI/KCD/KSI — em todas as fases do Roadmap.

4. Regras (Definition of Done)

A **Definition of Done (DoD)** é o contrato de conclusão. Um item (história, tarefa, correção) **só está "pronto"** quando **todas** as regras abaixo têm evidência objetiva. Sem evidência, o item volta para "em andamento" — não existe "pronto na palavra".

- **R1. Testes automatizados verdes.** Unitários, de integração e de contrato pertinentes ao item existem e passam no CI. Nenhum teste pulado (`skip`) sem justificativa aprovada e ticket associado.
- **R2. Cobertura mínima atingida.** Cobertura de linha/branch do código novo ou alterado $\geq 80\%$ ($\geq 90\%$ em módulos críticos: KCD, financeiro, IAM, isolamento de tenant). Cobertura medida sobre o `diff`, não só global.
- **R3. Tenant verificado.** Qualquer código que toque dados carrega e respeita `tenant_id`; existe pelo menos um teste que prova que o item **não** vaza dados entre tenants.
- **R4. Segurança sem bloqueadores.** SAST, SCA (dependências) e *secret scanning* sem achados de severidade **Alta/Crítica** em aberto. Endpoints novos cobertos por DAST no pipeline noturno.
- **R5. Contrato respeitado.** Se o item altera uma interface (REST/OpenAPI, gRPC/proto, evento de domínio), o teste de contrato correspondente foi atualizado e é retrocompatível — ou a quebra foi versionada e comunicada.
- **R6. Critérios de aceite satisfeitos.** Todos os critérios de aceite (Gherkin/BDD) da história passam; jornada crítica coberta por e2e quando aplicável.
- **R7. Observabilidade mínima.** Logs correlacionados por `trace_id`, métricas relevantes expostas e sem erro/latência anômala nos testes.
- **R8. Sem regressão.** Suíte de regressão relevante verde; nenhum defeito novo introduzido em fluxo existente.
- **R9. Documentação e ADR.** Mudança de comportamento documentada; decisão relevante registrada como ADR.
- **R10. Revisão e auditoria.** *Code review* aprovado; trilha de auditoria da mudança (autor, revisor, evidências de teste) preservada. **Nada é "pronto" sem auditoria.**

A DoD é **verificada automaticamente** onde possível (Quality Gate no CI, §15/§7) e **inspecionada** onde não é automatizável. Um item que fura qualquer regra não pode ser marcado como concluído nem entrar em release.

5. Arquitetura de Testes

5.1 A pirâmide de testes

A estratégia adota a pirâmide clássica, calibrada para uma plataforma multiempresa orientada a serviços com forte componente mobile e de IA. Quanto mais baixo o nível, **mais** testes, **mais rápidos**, **mais determinísticos** e **mais baratos**; quanto mais alto, **menos** testes, **mais lentos** e **mais caros**, cobrindo jornadas de valor.

```
graph TD
    E2E["<b>E2E / Jornadas de negócio</b><br/>~5% · Playwright web · integration_test Flutter<br/>fluxos  
ponta-a-ponta em ambiente realista"]
    CONTRATO["<b>Contrato</b><br/>~15% · Pact / OpenAPI · schema de eventos<br/>fronteiras entre serviços e  
API pública"]
    INTEG["<b>Integração</b><br/>~20% · serviço + banco + broker (Testcontainers)<br/>RLS, migrações,  
mensageria, adaptadores"]
    UNIT["<b>Unitário</b><br/>~60% · domínio puro, regras, cálculos<br/>rápido, isolado, determinístico"]

    E2E --> CONTRATO --> INTEG --> UNIT

    style UNIT fill:#1b5e20,color:#fff
    style INTEG fill:#2e7d32,color:#fff
    style CONTRATO fill:#388e3c,color:#fff
    style E2E fill:#43a047,color:#fff
```

Anti-padrões proibidos: o *ice-cream cone* (muitos e2e, poucos unitários) e o *hourglass* (base e topo cheios, meio vazio). Ambos produzem suítes lentas, frágeis e de diagnóstico ruim. Testes que exigem UI para validar regra de negócio pura são reprovados em revisão — a regra pertence ao nível unitário.

5.2 Eixos transversais

Ortogonais à pirâmide, cinco eixos atravessam **todos** os níveis e são específicos do canon do Kairós. Não são um "sexto nível"; são propriedades verificadas em cada nível aplicável.

```
graph LR
    subgraph Pirâmide
        U[Unitário] --- I[Integração] --- C[Contrato] --- E[E2E]
    end
    subgraph "Eixos transversais"
        T[Multiempresa/Tenant]
        S[Segurança · KCD]
        P[Performance/Carga]
        O[Offline-first · KCI/Mobile]
        A[IA · Avaliação KCI]
    end
    T -.verifica.-> U & I & C & E
    S -.verifica.-> U & I & C & E
    P -.verifica.-> I & E
    O -.verifica.-> U & I & E
    A -.verifica.-> U & I
```

5.3 Níveis, propriedade e definição de "verde"

Nível	O que valida	Ferramentas (referência)	Quem escreve (propriedade)	Onde roda	"Verde" significa
Unitário	Regra de domínio, cálculo, invariante, borda	JUnit/Kotlin-test (backend), Dart <code>test</code> (Flutter), Vitest/Jest (web)	Dev do time de feature	A cada commit / pré-push	100% dos casos passam; sem I/O real; < poucos ms cada
Integração	Serviço + banco + broker + adaptadores reais	Testcontainers (Postgres, broker), test doubles de borda	Dev do time de feature	CI por PR	Migrações e RLS aplicadas; mensageria e persistência reais verdes
Contrato	Fronteira entre consumidor e provedor; API pública	Pact (consumer-driven), validação OpenAPI 3.1, schema registry	Consumidor define; provedor verifica	CI por PR + verificação cruzada	Pacto publicado e verificado; sem quebra retroativa
E2E	Jornada de negócio ponta-a-ponta	Playwright (web), <code>integration_test</code> (Flutter), API-driven	QA Engineering (com o time)	Pipeline de release / noturno	Jornada crítica completa sem erro em ambiente realista
Não-funcional	Performance, carga, resiliência, segurança dinâmica	k6/Gatling, OWASP ZAP, chaos tooling	QA + SRE + KCD	Noturno / pré-release	Dentro dos SLOs; sem achado Alto/Crítico

Regra de propriedade: o time que **produz** o código é dono dos testes unitários e de integração daquele código. **QA Engineering** é dono da estratégia, dos e2e de jornada, dos testes não-funcionais, do harness de avaliação de IA e da curadoria da suíte de regressão — e atua como **habilitador** (frameworks, dados, ambientes, revisão de cobertura), nunca como "gargalo que testa tudo no fim".

6. Diagramas

6.1 Fluxo de qualidade (do commit ao release)

```

flowchart TD
    DEV[Commit / Push do dev] --> HOOK{Pré-commit<br/>lint + unit rápidos}
    HOOK -->|falha| DEV
    HOOK -->|ok| PR[Pull Request]
    PR --> CI1[CI: unit + integração + contrato]
    CI1 --> QG1{Quality Gate 1<br/>cobertura ≥ 80% · SAST/SCA · tenant test}
    QG1 -->|reprova| PR
    QG1 -->|aprova| REVIEW[Code Review + auditoria]
    REVIEW -->|aprovado| MERGE[Merge na main]
    MERGE --> CI2[Pipeline principal: build + e2e núcleo]
    CI2 --> NIGHTLY[Noturno: DAST · carga · regressão completa · aval. IA]
    NIGHTLY --> QG2{Quality Gate de Fase<br/>Master Prompt / Roadmap}
    QG2 -->|reprova| BLOCK[Release bloqueado · defeitos abertos]
    QG2 -->|aprova| REL[Release candidato · auditoria assinada]
    BLOCK --> DEV
    REL --> DEV
  
```


6.2 Isolamento multiempresa — o teste que nunca pode falhar

```
sequenceDiagram
    participant TA as Ator Tenant A
    participant API as API / Serviço
    participant RLS as Row-Level Security
    participant DB as Banco (tenant_id)
    TA->>API: Requisição c/ token de Tenant A
    API->>API: Extraí tenant_id do contexto de segurança
    API->>RLS: Query com tenant corrente = A
    RLS->>DB: SELECT ... WHERE tenant_id = A (forçado)
    DB->>TA: Somente dados de A
    Note over TA,DB: Teste negativo: token A tentando acessar recurso de B → 404/403, nunca 200
```

7. Fluxogramas (ciclo de vida do defeito)

```
stateDiagram-v2
    [*] --> Novo: defeito reportado (com evidência + repro)
    Novo --> Triagem: QA classifica severidade × prioridade
    Triagem --> Rejeitado: duplicado / não é defeito / esperado
    Triagem --> Backlog: severidade baixa, planejado
    Triagem --> EmCorrecao: aceito para correção
    EmCorrecao --> EmRevisao: fix + teste de regressão que reproduz o bug
    EmRevisao --> EmCorrecao: review reprova
    EmRevisao --> EmVerificacao: merge + deploy em ambiente de teste
    EmVerificacao --> Reaberto: QA reproduz novamente
    Reaberto --> EmCorrecao
    EmVerificacao --> Fechado: QA confirma correção + regressão verde
    Rejeitado --> [*]
    Fechado --> [*]
    Backlog --> Triagem: repriorização
```

Regra de ouro da correção: todo defeito aceito só fecha acompanhado de um **teste automatizado que reproduz o bug** (*regression test*). O teste deve **falhar** sem a correção e **passar** com ela. Isso impede regressões e transforma cada defeito em cobertura permanente.

8. Boas Práticas

- **Testes determinísticos.** Sem dependência de relógio real, ordem de execução, rede externa ou dados compartilhados mutáveis. Tempo e aleatoriedade são injetados (*clock* e *seed* controlados).
- **Isolamento por teste.** Cada teste cria e destrói seu próprio estado (banco por container efêmero, tenant sintético dedicado). Nada de "banco de dev compartilhado".
- **Nomes que descrevem comportamento.** `deveRecusarAcessoDeTenantDiferente`, não `test1`. O nome do teste é documentação viva do requisito.
- **Arrange–Act–Assert** explícito; um conceito por teste; asserções específicas (não `assertNotNull` genérico onde cabe verificação de valor).
- **Test doubles com parcimônia.** Preferir componentes reais em integração (Testcontainers) a *mocks* que codificam suposições. *Mock* só em bordas caras/externas.
- **Contrato antes do código** (herdado do Doc 09): o pacto/consumer test existe antes de o provedor implementar.
- **Dados de teste como código.** *Builders/factories* versionados; nunca *dumps* de produção sem anonimização (§4/§9 — proibido dado real de cliente em teste).
- **Falha rápida e clara.** Mensagem de asserção diz **o que** era esperado e **o que** veio. Teste que falha sem explicar é defeito de teste.
- **Combate ativo a flakiness.** Teste intermitente é tratado como defeito P2: quarentenado (fora do gate, visível no painel) e corrigido em SLA — nunca ignorado com *retry* cego.
- **Pirâmide, não cone.** Empurrar a verificação para o nível mais baixo possível que ainda dá confiança.

9. Padrões

- **BDD/Gherkin** para critérios de aceite (*Dado/Quando/Então*), servindo de fonte para e2e e como linguagem comum com produto.
- **Consumer-Driven Contracts (Pact)** para toda fronteira de serviço e para a API pública.
- **OpenAPI 3.1** como contrato normativo da API pública; **Protobuf** para gRPC interno; **schema de eventos versionado** (schema registry) para o backbone assíncrono.
- **Testcontainers** como padrão de integração (Postgres com RLS, broker de eventos, cache) — paridade com produção, descartável.
- **Page Object / Screen Object** em e2e (web e Flutter) para reduzir fragilidade.
- **Convenção de nomenclatura de tenants de teste:** `t_test_<caso>` sintético; jamais `tenant_id` de cliente real.
- **Dados sensíveis:** massa de teste **sintética ou anonimizada** (mascaramento irreversível de PII); proibido dado real de produção em ambientes de teste — controle verificado pelo KCD.
- **Convenção de severidade/prioridade** unificada (§ Casos de Uso e Modelagem) usada por todos os times.
- **Versionamento de suítes:** testes vivem no mesmo repositório e PR do código que verificam (co-localização), garantindo que contrato e verificação evoluem juntos.

10. Casos de Uso

10.1 Multiempresa — isolamento de tenant

- **UC-01.** Ator do Tenant A, autenticado, tenta ler/gravar recurso do Tenant B via API, ID direto (IDOR), filtro manipulado ou evento forjado. **Esperado:** negação (403/404), **zero** vazamento; registro de auditoria no KCD. Teste negativo obrigatório em todos os endpoints e consumidores de evento.
- **UC-02.** Migração de banco: verificar que a política **RLS** permanece ativa após cada migração (teste de integração que roda a migração e tenta cross-tenant).

10.2 Segurança (com KCD)

- **UC-03.** SAST detecta injeção, deserialização insegura, cripto fraca no *diff* → bloqueia PR (Gate 1).
- **UC-04.** DAST/ZAP noturno exercita endpoints autenticados e anônimos buscando OWASP Top 10 (injeção, autenticação quebrada, exposição de dados) → achados viram defeitos com severidade herdada do CVSS.
- **UC-05.** SCA + *secret scanning*: dependência com CVE Crítico ou segredo commitado bloqueia merge.
- **UC-06.** Pentest (periódico, por fase e pré-GA) conduzido em conjunto com o KCD; findings entram no fluxo de defeitos com SLA agravado.

10.3 Performance / carga

- **UC-07.** Teste de carga do fluxo de OS e de estoque (KSI) sob volume-alvo por tenant e multi-tenant simultâneo; medir p95/p99 de latência e throughput contra SLO.
- **UC-08.** Teste de *stress* e *soak* (resistência prolongada) para detectar vazamento de memória e degradação; *spike* para validar autoscaling.

10.4 Offline-first (mobile / KCI)

- **UC-09.** Técnico em campo **sem rede:** registra diagnóstico, captura fotos/OCR, executa KCI localmente, enfileira mudanças. **Esperado:** operação plena offline.
- **UC-10. Reconciliação/sincronização** ao voltar a rede: resolução de conflitos determinística, sem perda de dado, sem duplicação (idempotência), preservando `tenant_id`.

10.5 IA — avaliação do KCI

- **UC-11.** *Golden set* de diagnósticos rotulados: medir qualidade (acurácia, precisão/recall por classe de falha) do KCI contra baseline; **regressão** reprova se cair além do limiar.

- **UC-12.** Teste de robustez: entradas ruidosas, imagens de baixa qualidade, casos-limite; verificar *fallback* seguro (KCI nunca "inventa" diagnóstico com alta confiança sem evidência) e comportamento previsível quando a IA externa opcional está indisponível.

10.6 Contrato

- **UC-13.** Parceiro consumidor da **API pública** publica pacto; qualquer mudança do provedor que quebre o pacto reprova no CI antes do deploy.
- **UC-14.** Serviço consumidor de evento **05Concluida** declara o schema que espera; produtor não pode remover/renomear campo sem versionar (tolerant reader + registry).

11. Modelagem — matriz de tipos de teste por módulo

11.1 Tipos de teste (catálogo)

Tipo	Objetivo	Nível na pirâmide	Gatilho no CI
Unitário	Regra/cálculo/invariante	Base	Todo PR
Integração	Serviço+banco+broker reais	Meio-baixo	Todo PR
Contrato (CDC)	Fronteira consumidor ↔ provedor	Meio	Todo PR + cruzado
E2E	Jornada de negócio	Topo	Release / noturno
Isolamento de tenant	Não-vazamento multiempresa	Transversal (todos)	Todo PR (mínimo 1)
Segurança SAST/SCA	Falha no código/dependências	Transversal	Todo PR
Segurança DAST/pentest	Falha em execução	Transversal	Noturno / por fase
Performance/carga	Latência, throughput, resistência	Não-funcional	Noturno / pré-release
Offline-first	Operação sem rede + sync	Transversal	PR (mobile) + noturno
Avaliação de IA	Qualidade + regressão de modelo	Transversal	Noturno + por versão de modelo

11.2 Matriz módulo × tipo de teste

Legenda: ● crítico/obrigatório · ○ aplicável — não aplicável.

Módulo	Unit	Integr.	Contrato	E2E	Tenant	Segurança	Perf.	Offline	IA
API Pública	●	●	●	○	●	●	●	—	—
ERP / Núcleo	●	●	●	●	●	●	○	—	—
KCI (Inteligência)	●	●	○	○	●	●	○	●	●
KCD (Segurança)	●	●	○	○	●	●	○	—	○
KSI (Estoque)	●	●	●	●	●	●	●	○	○
App Mobile (Flutter)	●	●	●	●	●	●	○	●	○
Painéis Web	●	○	●	●	●	●	○	—	—
Financeiro	●	●	●	●	●	●	○	—	—
CRM	●	●	●	○	●	●	○	—	—
BI	●	●	○	○	●	●	●	—	○

Observação: **isolamento de tenant** e **segurança** são • em **todos** os módulos que tocam dados — reflexo direto dos princípios "segurança por padrão" e "multiempresa de primeira classe" do canon.

12. Checklist

Checklist de conclusão (por item — espelha a DoD §4):

- ☐ Unit + integração + contrato pertinentes escritos e **verdes** no CI.
- ☐ Cobertura do *diff* $\geq 80\%$ ($\geq 90\%$ em KCD/financeiro/IAM/tenant).
- ☐ Pelo menos **um** teste negativo de isolamento de tenant.
- ☐ SAST, SCA e *secret scanning* sem achado Alto/Crítico.
- ☐ Contrato (OpenAPI/proto/evento) atualizado e retrocompatível — ou quebra versionada.
- ☐ Critérios de aceite (Gherkin) passando; e2e da jornada quando aplicável.
- ☐ Teste de regressão que **reproduz** o defeito (para correções).
- ☐ Logs correlacionados e métricas expostas; sem latência/erro anômalos.
- ☐ Documentação/ADR atualizada.
- ☐ *Code review* aprovado e **trilha de auditoria** preservada.

Checklist de prontidão de release (por fase — Quality Gate):

- ☐ Suíte completa de regressão verde; **zero** defeito Crítico/Alto em aberto.
- ☐ DAST noturno sem achado Alto/Crítico; pentest de fase concluído e findings tratados.
- ☐ Testes de carga dentro dos SLOs (p95/p99, throughput, erro).
- ☐ Testes offline-first e de sincronização verdes (mobile/KCI).
- ☐ Avaliação de IA do KCI sem regressão além do limiar vs. baseline.
- ☐ Isolamento multiempresa verificado ponta-a-ponta em ambiente realista.
- ☐ *Flakiness* da suíte abaixo do limiar; nenhum teste crítico em quarentena.
- ☐ Métricas de qualidade dentro dos limiares (§ Riscos/Auditoria).
- ☐ Auditoria de QA assinada — **nada avança sem auditoria**.

13. Riscos

ID	Risco	Prob.	Impacto	Mitigação
RS-01	Vazamento de dados entre tenants	Baixa	Crítico	Teste negativo obrigatório por endpoint/evento; RLS verificada pós-migração; auditoria KCD
RS-02	Suíte lenta e frágil (anti-padrão cone/ampulheta)	Média	Alto	Manter proporção da pirâmide; empurrar verificação para base; revisão de altitude de teste
RS-03	<i>Flaky tests</i> corroendo confiança no CI	Alta	Alto	Determinismo obrigatório; quarentena + SLA de correção; painel de flakiness
RS-04	Cobertura alta mas testes fracos (asserção pobre)	Média	Alto	<i>Mutation testing</i> amostral; revisão de qualidade de teste, não só do número
RS-05	Regressão de modelo do KCI sem detecção	Média	Alto	<i>Golden set</i> versionado + gate de regressão por versão de modelo
RS-06	Falha só em campo (offline/sync) não reproduzida em teste	Média	Alto	Simulação de rede/partição no CI; testes de reconciliação e idempotência
RS-07	Dado real de produção usado em teste	Baixa	Crítico	Proibição por política; massa sintética/anonimizada; verificação KCD; <i>secret scanning</i>
RS-08	<i>Drift</i> entre contrato e implementação	Média	Alto	CDC verificado em ambos os lados no CI; schema registry com validação
RS-09	Segurança testada tarde (só no fim)	Média	Alto	SAST/SCA a cada PR; DAST noturno; pentest por fase — <i>shift-left</i>
RS-10	Ambiente de teste divergente de produção	Média	Médio	Testcontainers com paridade; IaC compartilhada; dados representativos
RS-11	Gargalo humano de QA no fim do fluxo	Média	Médio	Propriedade de teste no time de feature; QA como habilitador, não portão manual
RS-12	Defeito fechado sem teste de regressão	Média	Alto	Regra de ouro (\$7); gate que exige teste ligado ao defeito

14. Melhorias Futuras

- **Mutation testing** contínuo (não só amostral) para medir a **eficácia real** dos testes, complementando cobertura.
- **Testes de propriedade** (*property-based*) em regras de domínio críticas (cálculos financeiros, curva ABC do KSI, previsão).
- **Chaos engineering** integrado ao pipeline noturno para validar resiliência da malha (herdando do Doc 09).
- **Test data platform** self-service: geração sob demanda de tenants e massa sintética representativa por perfil de negócio.
- **Avaliação contínua de IA (KCI)**: expansão do *golden set*, detecção de *drift* de dados/entrada em produção realimentando o harness de avaliação, testes de robustez adversarial.
- **Contract testing bidirecional** com parceiros externos da API pública em *sandbox* dedicado, com verificação automatizada de pactos publicados.
- **Ambientes efêmeros por PR** (preview environments) para e2e realista sem contenção de ambiente compartilhado.
- **Painel único de qualidade** (cobertura, densidade de defeitos, flakiness, escaped defects, MTTR de defeito) alimentando diretamente os Quality Gates.
- **Testes de acessibilidade e i18n** automatizados nos painéis web e apps.

15. Auditoria

Os **Quality Gates** amarram QA ao **Master Prompt (00)** e ao **Roadmap (03)**: *nenhuma fase avança sem que a anterior passe pelos Quality Gates*. QA fornece a **evidência objetiva** que sustenta cada portão.

Métricas de qualidade e limiares (fonte de decisão de gate):

Métrica	Definição	Limiar
Cobertura de diff	Linha/branch do código novo/alterado	≥ 80% (≥ 90% crítico)
Densidade de defeitos	Defeitos por KLOC ou por história	Tendência decrescente; sem pico não explicado
Escaped defects	Defeitos encontrados após release / total	≤ limiar acordado por fase
Flakiness	% de execuções intermitentes na suíte	Abaixo do limiar; 0 em testes de gate
MTTR de defeito	Tempo médio de correção por severidade	Dentro do SLA por severidade
Regressão de IA (KCI)	Queda de qualidade vs. baseline no golden set	≤ limiar por métrica (acurácia/precisão/recall)
p95/p99 de latência	Sob carga-alvo	Dentro do SLO

Matriz de severidade × SLA (gestão de defeitos):

Severidade	Definição	SLA de correção	Bloqueia release?
S1 — Crítico	Vazamento entre tenants, perda de dado, falha de segurança explorável, sistema indisponível	Imediato / <i>hotfix</i>	Sim
S2 — Alto	Função central quebrada sem <i>workaround</i> ; regressão de IA relevante	≤ 2 dias úteis	Sim
S3 — Médio	Defeito com <i>workaround</i> ; impacto localizado	≤ 1 sprint	Não (avaliar)
S4 — Baixo	Cosmético, texto, borda rara	Backlog priorizado	Não

Item de auditoria	Descrição
Rastreabilidade	Cada critério de aceite → teste; cada defeito → teste de regressão; cada release → evidência de gate arquivada.
Trilha técnica	Resultados de CI (unit/integração/contrato/e2e), varreduras de segurança e relatórios de carga versionados e imutáveis por release.
Conformidade multiempresa	Teste de isolamento de tenant obrigatório e verificado em borda, serviço e persistência; achados registrados no KCD.
Segurança	SAST/SCA/secret a cada PR; DAST noturno; pentest por fase — evidências assinadas em conjunto com o KCD.
Qualidade de IA	Avaliação do KCI por versão de modelo, com baseline e limiar de regressão registrados.
Governança de dados de teste	Massa sintética/anonimizada; proibição de dado real de produção verificada.
Versão do documento	v1.0 — 21 de julho de 2026. Alterações futuras registradas no controle de versão do repositório de documentação.
Aprovação	Engenharia de QA (responsável); dependências 00–19 verificadas e coerentes com o PROJECT CANON.

Nota de encerramento. Este documento estabelece o *contrato de qualidade* do Drone Kairós ERP. A qualidade não é etapa nem departamento: é construída em cada camada e verificada por múltiplos níveis complementares, com eixos transversais que protegem o que é inegociável — **isolamento entre tenants, segurança por padrão e diagnóstico confiável do KCI**. O portão final é sempre o mesmo princípio do canon: **nada é "pronto" sem auditoria**.

21 Implantação

Campo	Valor
Documento	21 — Implantação
Versão	1.0
Data	21 de julho de 2026
Status	Ativo
Dependências	00 a 20 (com ênfase em 07 Arquitetura, 09 Microsserviços, 19 DevOps, 20 QA)
Responsável	Especialista de Implantação / Go-Live Sênior
Classificação	Interno · Confidencial

1. Resumo Executivo

Este documento define a estratégia, os processos e os controles de **implantação (deployment)** e **go-live** do **Drone Kairós ERP**, plataforma multiempresa (SaaS) para o ecossistema de drones. Ele traduz a esteira de entrega definida no documento **19 — DevOps** e os portões de qualidade do documento **20 — QA** em um plano operacional executável de colocar valor em produção com risco controlado.

O Drone Kairós ERP é composto por: backend de microsserviços, painéis web administrativos e operacionais, aplicativos móveis **Flutter** (Flutter Cliente e Flutter Técnico), API pública versionada e os módulos de negócio **KCI** (Kairós Cadastro/ Inventário), **KCD** (Kairós Comercial/Distribuição) e **KSI** (Kairós Serviços/Inspeção). Por ser multiempresa, a unidade fundamental de implantação não é apenas "uma versão em produção", mas **uma versão promovida por lote de tenants (empresas)**, respeitando isolamento, janelas e maturidade de cada cliente.

A abordagem central combina **blue-green** para troca atômica de infraestrutura, **canary** para exposição progressiva por porcentagem de tráfego e por coorte de tenants, e **feature flags** para desacoplar deploy de release (dark launch e ativação seletiva por empresa). Sobre essa base construímos três fluxos críticos: **onboarding de novos tenants** (provisionamento, dados iniciais, importação), **rollout multiempresa** (ondas por região/segmento com gestão de versão por tenant) e **publicação dos apps móveis** nas lojas (App Store e Google Play) com versionamento coordenado da API pública.

O go-live é governado por um **checklist de prontidão**, **critérios de sucesso mensuráveis**, **janelas de manutenção comunicadas** e um **plano de rollback determinístico**. Após a virada, um período de **hypercare** com monitoramento reforçado, SLAs de resposta encurtados e squad dedicada assegura estabilização. Todo o ciclo é **auditável e rastreável**, aderente aos princípios do canon: segurança por padrão, multiempresa e rastreabilidade ponta a ponta.

Resultados esperados: implantações previsíveis com janela reduzida ou zero-downtime, MTTR baixo via rollback ensaiado, onboarding de novos clientes em horas (não semanas) e conformidade de auditoria para cada promoção de versão.

2. Objetivos

2.1 Objetivo geral

Estabelecer um processo de implantação **repetível, seguro, observável e auditável** que permita entregar novas versões do Drone Kairós ERP a múltiplas empresas com o menor risco e o menor impacto operacional possível, reaproveitando integralmente a esteira de CI/CD do documento 19 e os portões de qualidade do documento 20.

2.2 Objetivos específicos

#	Objetivo	Métrica de sucesso
O1	Adotar estratégias de release progressivo (blue-green, canary, feature flags)	100% dos deploys de produção passam por canary antes de exposição total
O2	Padronizar o provisionamento de novos tenants	Tempo de onboarding técnico < 4 horas; provisionamento automatizado > 90%
O3	Habilitar rollout multiempresa em ondas	Rollout de versão maior em ≤ 5 ondas com gate entre ondas
O4	Garantir gestão de versão por tenant	100% dos tenants com versão declarada e rastreável no catálogo de versões
O5	Coordenar release mobile e versionamento de API	Zero quebra de compatibilidade de API sem período de depreciação anunciado
O6	Reduzir risco de go-live	Rollback ensaiado com RTO ≤ 15 min; taxa de rollback não planejado < 5%
O7	Assegurar estabilização pós-go-live	Hypercare com ≥ 99,9% de disponibilidade na primeira semana
O8	Capacitar técnicos e revendas	≥ 90% de conclusão do trilha de onboarding antes do acesso produtivo
O9	Garantir auditabilidade	100% das promoções com registro imutável (quem, o quê, quando, aprovação)

2.3 Não objetivos

- Este documento **não** redefine a esteira de build/test (ver 19 e 20); apenas a **consome**.
- **Não** especifica a arquitetura de software interna dos módulos (ver 07 e 09).
- **Não** trata de política comercial de preços/planos, apenas do impacto técnico do provisionamento por plano.

3. Escopo

3.1 Dentro do escopo

- Estratégias de deploy e release de todos os componentes: backend/microserviços, painéis web, apps Flutter, API pública e módulos KCI/KCD/KSI.
- Migração de dados de release (schema migrations) e migração de dados de onboarding (carga inicial e importação de clientes novos).
- Provisionamento e onboarding de tenants (empresas) em ambiente multiempresa.
- Rollout por lote de clientes/regiões e gestão de versão por tenant.
- Ciclo de publicação mobile (iOS/Android) e versionamento da API pública.
- Plano de go-live: pré-requisitos, checklist, janelas, comunicação, critérios de sucesso e rollback.
- Treinamento/onboarding de técnicos e revendas.
- Pós-implantação: hypercare, monitoramento inicial e suporte.

3.2 Fora do escopo

Item	Documento de referência
Pipelines de CI/CD, IaC, registries, secrets	19 — DevOps
Estratégia de testes, cobertura, automação de QA	20 — QA
Modelagem de dados e catálogo de tabelas	08 — Banco de Dados
Decomposição de serviços e contratos internos	09 — Microsserviços
Requisitos funcionais dos módulos	06 — Requisitos

3.3 Componentes cobertos

```
flowchart LR
    subgraph Clientes
        FC[Flutter Cliente<br/>Flutter iOS/Android]
        FT[Flutter Técnico<br/>Flutter iOS/Android]
        PW[Painéis Web<br/>Admin/Operação]
        API[API Pública<br/>versionada]
    end
    subgraph Backend
        GW[API Gateway]
        KCI[Módulo KCI]
        KCD[Módulo KCD]
        KSI[Módulo KSI]
        CORE[Serviços de plataforma<br/>Auth · Tenancy · Billing]
    end
    FC --> GW
    FT --> GW
    PW --> GW
    API --> GW
    GW --> KCI & KCD & KSI & CORE
    CORE --> DB[(Bancos por tenant/pool)]
```

4. Regras

Regras normativas de implantação. A força de cada regra segue RFC 2119 (**DEVE**, **NÃO DEVE**, **DEVERIA**, **PODE**).

4.1 Regras gerais de release

- **R1 — Deploy ≠ Release.** Toda funcionalidade nova **DEVE** entrar em produção atrás de **feature flag** desligada por padrão; a ativação por tenant é uma decisão de release separada do deploy.
- **R2 — Portões de qualidade obrigatórios.** Nenhum artefato **DEVE** ser promovido a produção sem aprovação nos portões do documento 20 (testes, cobertura mínima, segurança, performance).
- **R3 — Imutabilidade de artefato.** O mesmo artefato assinado promovido em homologação **DEVE** ser o promovido em produção (promote-by-digest); rebuild entre ambientes **NÃO DEVE** ocorrer.
- **R4 — Canary obrigatório.** Toda mudança de backend em produção **DEVE** passar por fase canary com análise automática antes de 100% do tráfego.
- **R5 — Rollback ensaiado.** Nenhum go-live **DEVE** ocorrer sem que o rollback correspondente tenha sido testado em homologação na mesma release.

4.2 Regras multiempresa

- **R6 — Isolamento.** Uma operação de implantação em um tenant **NÃO DEVE** degradar ou expor dados de outro tenant.
- **R7 — Versão declarada por tenant.** Cada tenant **DEVE** ter uma versão-alvo declarada no **catálogo de versões**; divergências entre versão declarada e versão efetiva **DEVEM** gerar alerta.

- **R8 — Ondas com gate.** O avanço de uma onda de rollout para a próxima **DEVE** exigir aprovação humana (change approver) apoiada por métricas da onda anterior.
- **R9 — Compatibilidade de dados.** Migrações de schema **DEVEM** ser compatíveis com a versão N-1 (expand/contract) durante o período de convivência de versões entre tenants.

4.3 Regras de dados e migração

- **R10 — Migração reversível ou compensável.** Toda migração destrutiva **DEVE** possuir plano de reversão ou de compensação documentado antes da execução.
- **R11 — Backup pré-mudança.** Um backup verificado (restore testado) **DEVE** existir imediatamente antes de qualquer migração de produção.
- **R12 — Idempotência de importação.** Rotinas de importação de onboarding **DEVEM** ser idempotentes e re-executáveis sem duplicar dados.

4.4 Regras de API e mobile

- **R13 — Versionamento explícito.** A API pública **DEVE** ser versionada (major na URL, minor via cabeçalho) com política de depreciação anunciada com antecedência mínima definida em contrato.
- **R14 — Compatibilidade retroativa mobile.** O backend **DEVE** suportar as duas últimas versões maiores publicadas dos apps (janela de compatibilidade), pois a atualização em loja não é instantânea.
- **R15 — Atualização forçada controlada.** Somente por motivo de segurança crítica um app **PODE** exigir atualização obrigatória (force update); caso contrário a atualização **DEVERIA** ser opcional/incentivada.

4.5 Regras de segurança e auditoria

- **R16 — Segurança por padrão.** Segredos, chaves e credenciais **NÃO DEVEM** transitar em artefatos ou logs; toda promoção **DEVE** usar o cofre de segredos definido em 19.
- **R17 — Trilha imutável.** Cada promoção, ativação de flag, provisionamento e rollback **DEVE** gerar registro de auditoria imutável e rastreável ao responsável.

5. Arquitetura de Release / Implantação

5.1 Ambientes e promoção

A esteira reaproveita os ambientes definidos em 19, com promoção linear e unidirecional de artefatos assinados por digest.

Ambiente	Propósito	Dados	Origem do artefato
Dev	Integração contínua	Sintéticos	Build de CI
Homologação (Staging)	Validação QA, ensaio de migração e rollback	Anonimizados (subset de produção)	Promote do Dev
Pré-produção (Canary infra)	Espelho de produção; smoke e testes de fumaça	Sintéticos + tenant de teste	Promote do Staging
Produção	Operação real multiempresa	Reais	Promote do Staging (mesmo digest)

5.2 Estratégias de deploy

Blue-Green. Mantém-se dois ambientes idênticos (Blue = ativo, Green = novo). O tráfego é comutado no gateway/load balancer após validação do Green. O rollback é a recomutação para Blue, quase instantâneo. Aplicável a serviços stateless e ao gateway.

```

flowchart LR
    LB[Gateway / Load Balancer]
    subgraph Blue [Blue - versao atual]
        B1[Serviços vN]
    end
    subgraph Green [Green - versao nova]
        G1[Serviços vN+1]
    end
    LB -->|100% hoje| Blue
    LB -->|switch apos validacao| Green
    Green --> DBS[(Banco compartilhado<br/>schema compativel N/N+1)]
    Blue --> DBS

```

Canary. A nova versão recebe tráfego incremental (ex.: 1% → 5% → 25% → 50% → 100%), com **análise automática** de métricas de ouro (latência, taxa de erro, saturação) comparando canary vs. baseline. Falha na análise dispara rollback automático. No Kairós, o canary é **duplo**: por **porcentagem de tráfego** e por **coorte de tenants** (empresas-piloto voluntárias primeiro).

Feature Flags. Ativação/desativação de funcionalidades em runtime, com segmentação por tenant, por região e por perfil de usuário. Suporta *dark launch* (código em produção, desligado), *kill switch* (desligamento emergencial sem deploy) e *gradual rollout* (percentual de usuários).

Estratégia	Melhor para	Rollback	Custo de infra
Blue-Green	Troca atômica, gateway, serviços críticos	Recomutação (segundos)	Alto (2x)
Canary	Redução progressiva de risco, validação em produção	Reversão de peso de tráfego	Médio
Feature Flags	Desacoplar deploy de release; ativação por tenant	Desligar flag (imediate)	Baixo

5.3 Camada multiempresa (tenancy)

O modelo de tenancy (detalhado em 08) influencia diretamente a implantação:

- **Serviço de Tenancy:** resolve o tenant por subdomínio/cabeçalho e injeta contexto em toda requisição.
- **Catálogo de Versões (Version Registry):** fonte da verdade sobre qual versão cada tenant deve/está executando; consultado pelo gateway e pelo orquestrador de rollout.
- **Provisionador (Tenant Provisioner):** serviço que cria a estrutura de um novo tenant (isolamento de dados, seed inicial, credenciais, configuração de módulos contratados).
- **Orquestrador de Rollout:** coordena as ondas, aplica gates e reconcilia versão declarada × efetiva.

```

flowchart TB
    REG[Catálogo de Versões]
    ORCH[Orquestrador de Rollout]
    GW[API Gateway multiempresa]
    FF[Serviço de Feature Flags]
    PROV[Tenant Provisioner]
    ORCH -->|lê/atualiza versão-alvo| REG
    GW -->|resolve versão por tenant| REG
    GW -->|consulta flags por tenant| FF
    ORCH -->|dispara ondas| GW
    PROV -->|registra novo tenant| REG

```

5.4 Reaproveitamento do DevOps (doc 19)

Recurso do doc 19	Uso na implantação
Pipelines CI/CD	Gatilho de promote, execução de migrações, publicação mobile
IaC (infra como código)	Provisionamento de ambientes blue/green e recursos de tenant
Registry de artefatos assinados	Promote-by-digest e verificação de assinatura
Cofre de segredos	Injeção de credenciais por ambiente/tenant
Observabilidade (métricas/logs/traços)	Análise canary, hypercare e critérios de sucesso
Gestão de mudanças (change mgmt)	Aprovação de janelas e gates entre ondas

6. Diagramas

6.1 Pipeline de promoção de release

```
flowchart LR
    CODE[Merge na main] --> CI[CI: build + testes<br/>doc 20]
    CI --> ART[Artefato assinado<br/>registry]
    ART --> STG[Deploy Homologação]
    STG --> QA{Portões QA<br/>aprovados?}
    QA -->|Não| STOP[Bloqueia promoção]
    QA -->|Sim| PPROD[Pré-produção / Smoke]
    PPROD --> CAN[Canary produção 1-5%]
    CAN --> ANALISE{Análise automática<br/>OK?}
    ANALISE -->|Não| RB[Rollback automático]
    ANALISE -->|Sim| PROG[Progressão 25-50-100%]
    PROG --> ONDAS[Rollout por ondas de tenants]
```

6.2 Provisionamento de novo tenant (onboarding)

```
sequenceDiagram
    participant V as Vendas/Revenda
    participant P as Portal de Provisionamento
    participant PR as Tenant Provisioner
    participant DB as Camada de Dados
    participant REG as Catálogo de Versões
    participant N as Notificação
    V->>P: Solicita nova empresa (plano, módulos KCI/KCD/KSI, região)
    P->>PR: Cria requisição de provisionamento
    PR->>DB: Cria isolamento de dados do tenant
    PR->>DB: Aplica seed inicial (dados-base)
    PR->>REG: Registra tenant na versão estável atual
    PR->>P: Retorna credenciais de admin do tenant
    P->>N: Dispara e-mail de boas-vindas + trilha onboarding
    N->>V: Confirmação e acesso ao ambiente
```

6.3 Topologia de rollout multiempresa por ondas

```
flowchart TB
    NEW[Versão N+1 estável] --> W0
    subgraph W0 [Onda 0 - Interna]
        T0[Tenant de teste + empresa-piloto interna]
    end
    end
    subgraph W1 [Onda 1 - Early adopters]
        T1[Clientes voluntários / baixo risco]
    end
    end
    subgraph W2 [Onda 2 - Região A]
        T2[Lote regional A]
    end
    end
    subgraph W3 [Onda 3 - Região B + segmento crítico]
        T3[Lote regional B / grandes contas]
    end
    end
    subgraph W4 [Onda 4 - Restante]
        T4[Demais tenants]
    end
    end
    W0 -->|gate: métricas OK| W1
    W1 -->|gate: aprovação| W2
    W2 -->|gate: aprovação| W3
    W3 -->|gate: aprovação| W4
```

6.4 Ciclo de release mobile e API

```
flowchart LR
    subgraph Mobile
        DEV[Flutter build] --> BETA[TestFlight / Play Internal]
        BETA --> REV[Revisão nas lojas]
        REV --> STAGED[Rollout escalonado na loja 5-100%]
    end
    end
    subgraph API
        APIN[API vN] --> APINEXT[API vN+1<br/>convivência]
        APINEXT --> DEPREC[Depreciação anunciada de vN]
    end
    end
    STAGED -->|apps usam| APINEXT
    APIN -->|janela de compatibilidade| STAGED
```

7. Fluxogramas (Go-Live e Rollback)

7.1 Fluxo de Go-Live

```
flowchart TD
    A[Início: Go-Live agendado] --> B{Checklist de prontidão<br/>100% aprovado?}
    B -->|Não| B1[Adiar / tratar pendências] --> A
    B -->|Sim| C[Abrir janela + comunicar stakeholders]
    C --> D[Congelamento de mudanças freeze]
    D --> E[Backup verificado + restore testado]
    E --> F[Executar migrações expand compatíveis N/N+1]
    F --> G{Migração OK?}
    G -->|Não| RB[Acionar rollback]
    G -->|Sim| H[Deploy Green + smoke tests]
    H --> I{Smoke OK?}
    I -->|Não| RB
    I -->|Sim| J[Canary 1-5% + análise automática]
    J --> K{Métricas dentro do SL0?}
    K -->|Não| RB
    K -->|Sim| L[Progressão de tráfego 25-50-100%]
    L --> M[Ativação de feature flags por tenant]
    M --> N{Critérios de sucesso atingidos?}
    N -->|Não| RB
    N -->|Sim| O[Encerrar janela + comunicar sucesso]
    O --> P[Entrar em Hypercare]
    P --> Q[Comunicar incidente + registrar auditoria]
```

7.2 Fluxo de Rollback

```
flowchart TD
    R0[Gatilho de rollback<br/>>manual ou automático] --> R1{Tipo de mudança?}
    R1 -->|Somente código/infra| R2[Recomutar tráfego Green -> Blue]
    R1 -->|Código + feature| R3[Desligar feature flag kill switch]
    R1 -->|Inclui migração de dados| R4{Migração reversível?}
    R4 -->|Sim| R5[Aplicar migração de reversão contract]
    R4 -->|Não| R6[Executar plano de compensação<br/>+ restaurar backup se necessário]
    R2 --> V[Validar saúde pós-rollback smoke]
    R3 --> V
    R5 --> V
    R6 --> V
    V --> W{Sistema estável?}
    W -->|Não| ESC[Escalonar para war room / incidente P1]
    W -->|Sim| X[Comunicar clientes afetados]
    X --> Y[Post-mortem sem culpa + registro de auditoria]
```

7.3 Árvore de decisão de estratégia por tipo de mudança

Tipo de mudança	Estratégia recomendada	Rollback primário
Correção de bug backend (sem schema)	Canary	Reversão de tráfego
Nova funcionalidade	Feature flag + dark launch	Desligar flag
Mudança de schema aditiva	Expand/contract + canary	Reversão de tráfego (schema compatível)
Mudança de schema destrutiva	Migração faseada + backup	Compensação/restore
Troca de versão de app mobile	Rollout escalonado em loja	Halt do rollout na loja
Mudança maior de API	Nova versão + depreciação	Manter versão anterior ativa

8. Boas Práticas

- **BP1 — Zero-downtime por padrão.** Preferir blue-green/canary com migrações expand/contract para evitar janelas de indisponibilidade.
- **BP2 — Small batches.** Entregas pequenas e frequentes reduzem o raio de impacto e facilitam o diagnóstico de regressões.
- **BP3 — Trunk-based + flags.** Integrar cedo e proteger com flags, evitando branches longevos e big-bang releases.
- **BP4 — Expand/Contract sempre.** Toda migração de schema segue as fases: **Expand** (adiciona sem quebrar) → **Migrate** (backfill de dados) → **Contract** (remove o antigo após convivência).
- **BP5 — Automatizar o provisionamento.** Onboarding de tenant como código; nada de passos manuais que introduzam variabilidade entre empresas.
- **BP6 — Empresas-piloto voluntárias.** Iniciar rollout por coorte que aceitou receber versões cedo, com canal de feedback direto.
- **BP7 — Rollback é feature, não exceção.** Ensaiar rollback a cada release; medir RTO real.
- **BP8 — Observabilidade antes do go-live.** Painéis, alertas e SLOs configurados e validados **antes** da virada, não durante.
- **BP9 — Comunicação proativa.** Avisar clientes de janelas e de mudanças que afetam fluxo de trabalho, especialmente técnicos em campo.
- **BP10 — Congelamento consciente.** Aplicar freeze de mudanças não relacionadas durante janelas de alto risco.
- **BP11 — Documentar a decisão.** Cada release registra: o que muda, por quê, risco, plano de rollback e responsável (rastreadibilidade).
- **BP12 — Segurança por padrão.** Segredos no cofre, princípio do menor privilégio nas credenciais de provisionamento, e varredura de dependências antes de promover.

9. Padrões

9.1 Versionamento

- **Semântico (SemVer)** para backend e API pública: `MAJOR.MINOR.PATCH`. `MAJOR` sinaliza quebra de contrato.
- **Apps móveis:** `versionName` (marketing, ex.: 3.4.0) + `versionCode/build number` monotônico crescente.
- **API pública:** `MAJOR` na URL (`/api/v2/...`), `MINOR` negociável via cabeçalho (`Kairos-Api-Version`), com política de depreciação e cabeçalhos `Deprecation` / `Sunset`.

9.2 Convenção de tags e releases

Artefato	Padrão de tag	Exemplo
Backend/serviço	<code>svc/<nome>/vMAJOR.MINOR.PATCH</code>	<code>svc/ksi/v2.3.1</code>
Painel web	<code>web/vMAJOR.MINOR.PATCH</code>	<code>web/v4.1.0</code>
App Flutter	<code>app/<flutter>/vX.Y.Z+build</code>	<code>app/tecnico/v3.4.0+412</code>
API pública	<code>api/vMAJOR.MINOR</code>	<code>api/v2.5</code>
Release de plataforma	<code>release/AAAA.MM.iteração</code>	<code>release/2026.07.2</code>

9.3 Nomenclatura de feature flags

`kci|kcd|ksi|core.<contexto>.<funcionalidade>` — ex.: `ksi.inspecao.laudo_automatico`. Toda flag possui: dono, data de criação, tipo (release/ops/experimento), critério de remoção e data-limite (evitar dívida de flags).

9.4 Janelas de manutenção

- **Baixo risco:** fora do horário de pico regional, sem downtime esperado.

- **Alto risco (migração destrutiva)**: janela anunciada com antecedência mínima acordada em SLA, preferencialmente em janela de baixa atividade de campo.
- **Emergencial (hotfix de segurança)**: pode ocorrer fora de janela, com comunicação imediata.

9.5 Classificação de mudança (change management)

Classe	Descrição	Aprovação
Standard	Pré-aprovada, baixo risco, automatizada	Automática
Normal	Requer avaliação e janela	Change approver
Emergencial	Correção urgente	Aprovação expedita + revisão posterior

10. Casos de Uso

10.1 UC-01 — Release de correção de bug no módulo KSI

Ator: Engenharia de plataforma. **Gatilho:** Bug corrigido e mergeado. **Fluxo:** Promote do artefato → homologação → smoke em pré-produção → canary 5% em produção → análise automática 30 min → progressão a 100% → rollout imediato para todos os tenants (mudança sem schema, baixo risco). **Rollback:** reversão de tráfego. **Resultado:** correção disponível a todos os tenants no mesmo dia.

10.2 UC-02 — Lançamento de nova funcionalidade no KCD com ativação seletiva

Ator: Produto + Engenharia. **Gatilho:** Feature pronta atrás de flag `kcd.comercial.proposta_dinamica`. **Fluxo:** Deploy com flag desligada (dark launch) → ativação para 3 empresas-piloto → coleta de feedback e métricas por 1 semana → ativação gradual por região → ativação geral. **Rollback:** desligar flag. **Resultado:** exposição controlada sem novo deploy a cada expansão.

10.3 UC-03 — Onboarding de nova revenda (tenant novo)

Ator: Revenda/Vendas. **Gatilho:** Contrato assinado. **Fluxo:** Portal de provisionamento cria o tenant na versão estável → seed inicial → importação do catálogo de drones/clientes da revenda via planilha padronizada (idempotente) → criação de usuários e perfis → disparo da trilha de onboarding. **Resultado:** revenda operacional em < 4 horas, isolada dos demais tenants.

10.4 UC-04 — Migração de dados de cliente legado

Ator: Equipe de implantação. **Gatilho:** Cliente migrando de sistema antigo. **Fluxo:** Extração → mapeamento para o modelo Kairós → carga em ambiente de staging do tenant → validação e reconciliação (contagens, amostragem) → carga em produção em janela → verificação → go-live do tenant. **Rollback:** manter sistema legado em paralelo (dual-run) até validação final. **Resultado:** migração auditável com reconciliação assinada.

10.5 UC-05 — Release maior de app Flutter Técnico

Ator: Mobile + QA. **Gatilho:** Nova versão maior do app. **Fluxo:** Build Flutter → distribuição beta (TestFlight/Play Internal) → validação → submissão às lojas → rollout escalonado (5% → 20% → 50% → 100%) → monitorar crash-free rate → backend mantém compatibilidade com versão anterior (janela). **Rollback:** halt do rollout na loja + kill switch de features novas via flag. **Resultado:** adoção gradual sem quebrar técnicos em campo com versão antiga.

10.6 UC-06 — Rollback emergencial em produção

Ator: On-call/SRE. **Gatilho:** Alerta de erro acima do SLO durante canary. **Fluxo:** Rollback automático dispara reversão de tráfego → validação de saúde → comunicação a clientes afetados → post-mortem sem culpa. **Resultado:** MTTR reduzido, impacto contido ao percentual canary.

10.7 UC-07 — Depreciação de versão da API pública

Ator: Plataforma + Developer Relations. **Gatilho:** Nova versão maior de API. **Fluxo:** Publicar **v2** → anunciar depreciação de **v1** com prazo → adicionar cabeçalhos **Deprecation / Sunset** → monitorar consumidores de **v1** → suporte à migração → desativar **v1** após o prazo. **Resultado:** transição sem quebra surpresa de integrações de parceiros.

11. Modelagem (Plano de Rollout)

11.1 Estrutura de ondas

Onda	Público	% da base	Critério de entrada	Gate de saída
0	Tenant de teste + empresa interna	~1%	Versão estável em pré-produção	Smoke 100% + zero erro crítico
1	Early adopters voluntários	~5%	Onda 0 estável por 48h	SLO mantido + feedback positivo
2	Região A (baixo risco)	~25%	Aprovação change approver	Erro < baseline + sem P1
3	Região B + grandes contas	~40%	Aprovação + comunicação prévia	SLO mantido + suporte estável
4	Restante da base	~29%	Aprovação final	Reconciliação de versões 100%

11.2 Máquina de estados de versão por tenant

```
stateDiagram-v2
    [*] --> Provisionado
    Provisionado --> EstavelN: versão estável atual
    EstavelN --> AlvoNmais1: agendado para rollout
    AlvoNmais1 --> MigrandoNmais1: janela iniciada
    MigrandoNmais1 --> EstavelNmais1: sucesso
    MigrandoNmais1 --> EstavelN: rollback
    EstavelNmais1 --> [*]
```

11.3 Matriz de decisão de agrupamento de tenants

Critério	Peso	Efeito no agrupamento
Criticidade do cliente (SLA)	Alto	Grandes contas em ondas mais tardias
Região/fuso	Médio	Janela na madrugada local
Volume transacional	Médio	Alto volume separado para observar carga
Módulos contratados (KCI/KCD/KSI)	Médio	Agrupar por superfície de mudança
Disposição a receber cedo	Alto	Voluntários nas ondas iniciais

11.4 Modelo de dados de suporte (conceitual)

Entidade	Campos-chave	Propósito
tenant	id, nome, região, plano, módulos, status	Cadastro de empresa
version_registry	tenant_id, versão_alvo, versão_efetiva, atualizado_em	Gestão de versão por tenant
rollout_wave	id, release, ordem, critério_gate, status	Definição de ondas
rollout_assignment	tenant_id, wave_id, estado	Alocação de tenant à onda
deployment_audit	id, ator, ação, artefato_digest, timestamp, aprovação	Trilha imutável
feature_flag_state	flag, tenant_id, habilitada, alterado_por, timestamp	Estado de ativação

12. Checklist de Prontidão (Go-Live Readiness)

12.1 Pré-requisitos técnicos

- [] Artefatos assinados e promovidos por digest (sem rebuild entre ambientes) — R3
- [] Todos os portões de QA (doc 20) aprovados na release candidata
- [] Migrações validadas em homologação (expand/contract) e ensaiadas
- [] Rollback ensaiado com RTO medido ≤ 15 min — R5
- [] Backup verificado com restore testado — R11
- [] Feature flags criadas, desligadas por padrão, com donos definidos
- [] Compatibilidade de API N/N-1 confirmada; janela mobile respeitada
- [] Observabilidade: painéis, SLOs e alertas configurados e testados
- [] Configuração de canary e análise automática ativa em produção

12.2 Pré-requisitos de negócio e pessoas

- [] Janela de manutenção aprovada e comunicada aos tenants afetados
- [] Plano de comunicação (interno e clientes) pronto e agendado
- [] Squad de hypercare escalada com plantão definido
- [] Trilha de onboarding de técnicos/revendas concluída ($\geq 90\%$)
- [] Critérios de sucesso e de rollback documentados e acordados
- [] Change approver e responsáveis de gate identificados por onda
- [] Base de conhecimento/FAQ e roteiro de suporte atualizados

12.3 Critérios de sucesso do go-live

Critério	Meta	Fonte
Disponibilidade na janela	$\geq 99,9\%$	Observabilidade
Taxa de erro pós-virada	$\leq$ baseline + 0,5 p.p.	Métricas de ouro
Latência P95	Dentro do SLO	APM
Crash-free (mobile)	$\geq 99,5\%$	Telemetria de app
Rollback não planejado	Nenhum	Registro de deploy
Chamados críticos (P1) nas primeiras 24h	0	Suporte/ITSM
Reconciliação de versões por tenant	100% consistente	Catálogo de versões

12.4 Go / No-Go

Decisão formal registrada em ata, com representantes de Engenharia, QA, Produto, Suporte e Implantação. **No-Go** se qualquer pré-requisito técnico bloqueante estiver aberto.

13. Riscos

ID	Risco	Prob.	Impacto	Mitigação	Contingência
RSK-01	Migração de schema incompatível quebra tenants em versões diferentes	Média	Alto	Expand/contract obrigatório (R9); testes em staging	Rollback via schema compatível; compensação
RSK-02	Vazamento entre tenants durante provisionamento	Baixa	Crítico	Isolamento por design (R6); testes de segurança	Isolar tenant afetado; incidente P1; notificação
RSK-03	Rejeição/atraso na revisão das lojas (App Store/Play)	Média	Médio	Submissão antecipada; buffer no cronograma	Manter versão anterior; comunicar usuários
RSK-04	App antigo em campo incompatível com backend novo	Média	Alto	Janela de compatibilidade N/ N-1 (R14)	Reativar compatibilidade; hotfix de backend
RSK-05	Canary não detecta regressão de baixa frequência	Baixa	Médio	Duração mínima de canary; métricas por tenant	Rollback ao detectar; alerta estendido
RSK-06	Importação de dados de onboarding duplica registros	Média	Médio	Idempotência (R12); reconciliação	Re-execução limpa; rollback do tenant
RSK-07	Janela de manutenção mal comunicada gera insatisfação	Média	Médio	Plano de comunicação; antecedência de SLA	Compensação comercial; retrospectiva
RSK-08	Dívida de feature flags obsoletas	Alta	Baixo	Data-limite e dono por flag (padrão 9.3)	Faxina periódica de flags
RSK-09	Rollback de migração destrutiva sem reversão	Baixa	Crítico	Proibir destrutivo sem plano (R10); backup (R11)	Restore de backup; war room
RSK-10	Sobrecarga do suporte no hypercare	Média	Médio	Squad dedicada; FAQ; escalonamento claro	Reforço temporário; priorização P1/P2
RSK-11	Depreciação de API surpreende parceiros	Média	Alto	Anúncio antecipado (R13); cabeçalhos Sunset	Prorrogar prazo; suporte à migração

14. Melhorias Futuras

- **MF1 — Rollout progressivo totalmente orientado a SLO.** Evoluir a análise canary para *progressive delivery* automatizada, avançando ondas sem gate manual quando métricas estiverem verdes por tempo definido.
- **MF2 — Autoatendimento de onboarding.** Portal self-service para revendas provisionarem seus próprios sub-tenants dentro de limites contratados.
- **MF3 — Rollback por tenant granular.** Capacidade de reverter a versão de um único tenant sem afetar a onda, via roteamento por versão no gateway.
- **MF4 — Testes de migração automatizados com dados de produção anonimizados** por release.
- **MF5 — Catálogo de versões preditivo.** Recomendação de agrupamento de ondas com base em histórico de incidentes por perfil de tenant.
- **MF6 — Chaos engineering na esteira** para validar resiliência de rollback e isolamento multiempresa.
- **MF7 — Feature flags com limpeza automatizada** (alerta e PR automático de remoção após data-limite).
- **MF8 — Blue-green de banco** para migrações destrutivas de alto risco com corte reversível.

15. Auditoria

15.1 Princípio

Aderente ao canon (**rastreabilidade**), toda ação de implantação é registrada de forma **imutável, atribuível e verificável**. Nenhuma promoção, ativação de flag, provisionamento ou rollback ocorre sem trilha (R17).

15.2 Eventos auditados

Evento	Dados registrados
Promoção de artefato	Digest, origem → destino, ator, aprovação, timestamp
Execução de migração	Script, versão, tenant(s), resultado, backup de referência
Provisionamento de tenant	Empresa, plano, módulos, executor, seed aplicado
Ativação/desativação de flag	Flag, tenant, estado, ator, motivo
Avanço de onda de rollout	Onda, aprovador, métricas do gate
Rollback	Gatilho (manual/auto), tipo, RTO real, impacto
Publicação mobile	Versão, loja, % de rollout, responsável
Depreciação de API	Versão, data de anúncio, data de sunset

15.3 Rastreabilidade e retenção

- Cada evento correlaciona-se por **ID de release** e **ID de mudança**, permitindo reconstruir a linha do tempo completa de qualquer versão em qualquer tenant.
- Logs de auditoria **NÃO DEVEM** conter segredos ou dados sensíveis de tenant (segurança por padrão — R16).
- Retenção conforme política de compliance da plataforma; registros de mudança de produção com retenção estendida.

15.4 Indicadores de conformidade

Indicador	Meta
Promoções com trilha completa	100%
Divergência versão declarada × efetiva	0 (alerta automático se > 0)
Rollbacks com post-mortem registrado	100%
Flags além da data-limite	Tendência a 0
Provisionamentos automatizados vs. manuais	> 90% automatizado

Pós-Implantação — Hypercare, Monitoramento e Suporte

Detalhamento operacional referenciado nas seções 1, 7 e 12.

Hypercare (janela de estabilização — tipicamente 1 a 4 semanas por onda crítica):

Dimensão	Padrão hypercare	Retorno ao normal
SLA de resposta P1	Encurtado (ex.: 15 min)	SLA contratual padrão
Cobertura de plantão	Squad dedicada 24x7	On-call regular
Cadência de status	Diária (war room)	Semanal
Monitoramento	Painéis dedicados + alertas sensibilizados	Alertas padrão
Critério de saída	SLOs estáveis + zero P1 por N dias	Aprovação formal

Monitoramento inicial: foco nas métricas de ouro (latência, erros, saturação, tráfego) segmentadas **por tenant e por módulo (KCI/KCD/KSI)**, além de crash-free rate dos apps Flutter e taxa de sucesso de onboarding. Alertas calibrados para sensibilidade elevada durante o hypercare.

Suporte e treinamento (técnicos/revendas): - Trilha de onboarding obrigatória antes do acesso produtivo ($\geq 90\%$ de conclusão — R8 de negócio). - Material por perfil: técnico de campo (Flutter Técnico), atendente/revenda (painel + Flutter Cliente), administrador de tenant. - Canal de feedback direto durante empresas-piloto e ondas iniciais. - Base de conhecimento e FAQ atualizados a cada release, com roteiro de escalonamento (N1 → N2 → engenharia).

Fim do Documento 21 — Implantação · Drone Kairós ERP · v1.0

22 Comercial

Campo	Valor
Documento	22 — Comercial
Versão	1.0
Data	21 de julho de 2026
Status	Ativo
Dependências	00 a 21 (com ênfase em 05 — Análise Competitiva, 10 — Arquitetura Multiempresa, 14 — KCI, 15 — KCD, 16 — KSI, 18 — Rastreabilidade por Serial Number)
Responsável	Estratégia de Negócio

Aviso sobre valores: todos os preços, percentuais, metas e projeções financeiras neste documento são **EXEMPLOS ILUSTRATIVOS** para fins de modelagem comercial. Não constituem tabela de preço oficial nem compromisso contratual. Os valores oficiais são publicados no *pricebook* vigente controlado pela área Comercial/Finanças.

1. Resumo Executivo

O **Drone Kairós ERP** é uma plataforma multiempresa (SaaS) desenhada para orquestrar o ciclo completo do ecossistema de drones — do **fabricante à revenda**, da **assistência técnica ao cliente final/operador** — com rastreabilidade vitalícia por **Serial Number (SN)** e módulos diferenciados de inteligência (**KCI**), cibersegurança (**KCD**) e estoque inteligente (**KSI**).

Este documento define o **motor comercial** do produto: como monetizamos, quem é o cliente ideal, como precificamos, por quais canais chegamos ao mercado, como nos posicionamos frente aos concorrentes mapeados no documento 05 e quais métricas governam a saúde do negócio.

Tese comercial central. O valor do Kairós cresce com a **rede**: quanto mais elos da cadeia (fabricante, revenda, técnico) operam sobre a mesma plataforma, mais forte fica a rastreabilidade por SN e mais difícil é o *churn*. Isso sustenta uma estratégia **land-and-expand** com forte efeito de rede e receita expansível (add-ons por módulo e por equipamento rastreado).

Modelo de receita (síntese):

- **Assinatura SaaS recurring** por *tenant*, escalonada em **tiers** (Starter → Growth → Enterprise), com componentes de consumo por **usuário ativo** e por **equipamento sob gestão (SN ativo)**.
- **Add-ons por módulo** — KCI, KCD e KSI ativáveis de forma modular sobre qualquer tier.
- **Marketplace futuro** (peças, serviços, integrações de terceiros) com receita de *take rate*.
- **Serviços profissionais** (onboarding avançado, migração, integrações customizadas) como receita não recorrente que acelera adoção.

Metas-âncora (exemplo ilustrativo, horizonte 24 meses):

Indicador	Meta 12 meses (ex.)	Meta 24 meses (ex.)
ARR	R\$ 6,0 mi	R\$ 18,0 mi
Logos ativos (tenants pagantes)	120	380
NRR (Net Revenue Retention)	≥ 108%	≥ 118%
Gross churn lógico (logo)	≤ 2,0% a.m.	≤ 1,2% a.m.
CAC payback	≤ 14 meses	≤ 10 meses
LTV/CAC	≥ 3,0x	≥ 4,0x

2. Objetivos

Objetivo geral. Estabelecer um modelo comercial replicável, previsível e escalável que transforme a proposta técnica do Kairós (multiempresa + rastreabilidade por SN + módulos KCI/KCD/KSI) em receita recorrente saudável e defensável.

Objetivos específicos:

1. **Monetização clara** — definir modelo de negócio, tiers e add-ons com lógica de valor transparente por *tenant*, usuário e SN.
2. **Foco de mercado** — formalizar segmentação e ICP (Ideal Customer Profile) para direcionar esforço comercial ao público de maior *fit* e menor CAC.
3. **Previsibilidade de receita** — instrumentar MRR/ARR, coortes e *forecast* com governança de *pipeline*.
4. **Eficiência de aquisição** — desenhar GTM multicanal (direto, parcerias com fabricantes, revendas) que reduza CAC e acelere *time-to-value*.
5. **Expansão programada** — operacionalizar land-and-expand via módulos e novos elos da cadeia dentro do mesmo *account*.
6. **Retenção estrutural** — usar rastreabilidade e efeito de rede como *moat* comercial, elevando NRR e reduzindo *churn*.
7. **Excelência de pós-venda** — padronizar onboarding, suporte e SLAs comerciais para sustentar renovação e *upsell*.

CrITÉRIOS de sucesso (exemplos): LTV/CAC ≥ 3x; NRR ≥ 110%; *win rate* em oportunidades qualificadas ≥ 25%; *time-to-first-value* ≤ 21 dias no tier Starter.

3. Escopo

Está no escopo deste documento:

- Modelo de negócio SaaS, add-ons modulares e visão de marketplace.
- Segmentação de mercado e definição de ICP e *personas* de compra.
- Estrutura de precificação (tiers, dimensões de cobrança, freemium vs enterprise).
- Estratégia de go-to-market, canais, aquisição e land-and-expand.
- Posicionamento, proposta de valor e diferenciação competitiva.
- Onboarding comercial, suporte e SLAs sob a ótica comercial.
- Métricas de negócio (MRR, ARR, CAC, LTV, churn, NRR) e sua governança.

Fora do escopo (tratado em outros documentos):

- Especificação técnica dos módulos KCI/KCD/KSI (docs 14–16).
- Arquitetura multiempresa e isolamento de dados por *tenant* (doc 10).
- Detalhamento jurídico de contratos, LGPD e termos de uso (docs de Jurídico/Compliance).
- Precificação oficial vigente (pricebook controlado por Comercial/Finanças).

- Operação financeira de *billing*, impostos e reconhecimento de receita (doc de Financeiro/Faturamento).

Premissas: cobrança em BRL (com possibilidade futura de USD para exportação); ciclo de faturamento mensal e anual; contratos B2B; faturamento por *tenant* raiz com consolidação de subunidades quando aplicável.

4. Regras

4.1 Regras de modelo e cobrança (RC)

- **RC-01** — Toda conta paga é ancorada em um **tenant** e um **tier** base; add-ons de módulo só existem sobre um tier ativo.
- **RC-02** — As **dimensões de cobrança** são três: (a) tier base do tenant, (b) usuários ativos, (c) equipamentos sob gestão (SN ativos). Nenhuma cobrança oculta fora dessas dimensões e dos add-ons publicados.
- **RC-03** — **SN ativo** é a unidade de valor da rastreabilidade: um equipamento passa a ser cobrado quando seu SN é colocado em gestão ativa (histórico vitalício, vínculos de garantia/serviço). SNs arquivados/aposentados não contam para *billing*.
- **RC-04** — **Freemium** é limitado por *caps* explícitos (ver §11) e nunca inclui módulos KCD ou KCI premium; serve para aquisição e prova de valor, não para operação em escala.
- **RC-05** — **Anual à vista** recebe desconto padrão frente ao mensal (exemplo: ~2 meses grátis / ~16%); *downgrade* só na renovação; *upgrade* a qualquer momento com cobrança pro-rata.
- **RC-06** — **Enterprise** admite contrato customizado (volume, SLAs reforçados, *private cloud*/isolamento dedicado), sempre documentado em *order form*.
- **RC-07** — **Descontos** seguem matriz de alçada (ver §4.3); descontos acima da alçada exigem aprovação de Estratégia de Negócio/Finanças.
- **RC-08** — **Marketplace** (fase futura) cobra *take rate* sobre transações; nunca substitui a assinatura base.

4.2 Regras de segmentação (RS)

- **RS-01** — Oportunidades são classificadas por **segmento** (fabricante, rede de revenda, assistência técnica/operador) e por **fit de ICP** (Alto/Médio/Baixo) na entrada do *pipeline*.
- **RS-02** — Contas de **fit Baixo** não recebem esforço de venda direto assistido; são direcionadas a *self-service*/freemium.
- **RS-03** — **Fabricantes** são tratados como contas estratégicas (efeito de rede a jusante) e podem habilitar programa de parceria/co-venda.

4.3 Regras de alçada de desconto (exemplo ilustrativo)

Faixa de desconto sobre lista	Alçada aprovadora
0% – 10%	Executivo(a) de Vendas
11% – 20%	Gerência Comercial
21% – 30%	Diretoria Comercial
> 30% ou termos não padronizados	Estratégia de Negócio + Finanças

4.4 Regras de retenção e expansão (RE)

- **RE-01** — Renovações são iniciadas com **90 dias** de antecedência para Enterprise, **45 dias** para Growth.
- **RE-02** — Conta com *health score* vermelho não pode receber ação de *upsell* antes de plano de recuperação (CS).
- **RE-03** — Expansão por novo elo da cadeia (ex.: revenda convidando suas assistências) tem trilha de *referral* com incentivo (ver §10 UC-05).

5. Arquitetura do Modelo Comercial

O modelo comercial é estruturado em **quatro camadas** que se combinam para formar a receita total de um *account*.

```
flowchart TB
    subgraph L1["Camada 1 – Assinatura base (recorrente)"]
        T1["Tier do Tenant<br/>Starter / Growth / Enterprise"]
    end
    subgraph L2["Camada 2 – Consumo escalonado (recorrente)"]
        U["Usuários ativos"]
        SN["Equipamentos sob gestão (SN ativos)"]
    end
    subgraph L3["Camada 3 – Add-ons por módulo (recorrente)"]
        KCI["KCI – Inteligência"]
        KCD["KCD – Cibersegurança"]
        KSI["KSI – Estoque Inteligente"]
    end
    subgraph L4["Camada 4 – Não recorrente e futura"]
        PS["Serviços Profissionais<br/>(onboarding, migração, integrações)"]
        MKT["Marketplace (take rate)<br/>[fase futura]"]
    end

    T1 --> REC["Receita do Account"]
    U --> REC
    SN --> REC
    KCI --> REC
    KCD --> REC
    KSI --> REC
    PS --> REC
    MKT --> REC

    REC --> MRR["MRR / ARR"]
```

Princípios de arquitetura comercial:

1. **Base + expansão** — a assinatura base garante previsibilidade; consumo (usuários/SN) e add-ons capturam o valor crescente conforme a operação do cliente escala.
2. **Modularidade** — KCI/KCD/KSI são independentes e ativáveis sob demanda, permitindo *land* enxuto e *expand* granular.
3. **Ancoragem em valor** — o SN ativo alinha preço a valor entregue (rastreadabilidade vitalícia), não a custo de infraestrutura.
4. **Efeito de rede monetizável** — cada novo elo da cadeia sobre a plataforma aumenta densidade de dados por SN e cria novos pontos de receita.
5. **Consistência multiempresa** — a arquitetura de *tenant* (doc 10) permite consolidar faturamento de grupos econômicos (fabricante com múltiplas unidades, rede com filiais).

6. Diagramas

6.1 Ecossistema comercial e fluxo de valor

```
flowchart LR
    FAB["Fabricante<br/>(conta estratégica)"] -->|"emite SN,<br/>garante origem"| REV["Rede de Venda"]
    REV -->|"vende + registra<br/>equipamento"| CLI["Cliente / Operador"]
    CLI -->|"aciona"| TEC["Assistência Técnica"]
    TEC -->|"atualiza histórico<br/>vitalício do SN"| SNDB["Rastreabilidade<br/>por Serial Number"]
    FAB -->|"assina Kairós"| K["Drone Kairós ERP"]
    REV -->|"assina Kairós"| K
    TEC -->|"assina Kairós"| K
    K --> SNDB
    SNDB -->|"dado alimenta"| KCI["KCI – Inteligência"]
    KCI -->|"insights de rede"| FAB
```

6.2 Funil comercial (pirâmide de conversão)

```
flowchart TB
    A["Visitantes / Leads<br/>(topo – marketing + parcerias)"]
    B["MQL – Marketing Qualified Lead"]
    C["SQL – Sales Qualified Lead<br/>(fit de ICP validado)"]
    D["Oportunidade / POC"]
    E["Proposta / Order Form"]
    F["Cliente ativo (Closed-Won)"]
    G["Expansão (add-ons + novos elos)"]

    A --> B --> C --> D --> E --> F --> G
    F -->|"land"| G
    G -->|"expand"| G
```

6.3 Estados comerciais de um account

```
stateDiagram-v2
    [*] --> Lead
    Lead --> Qualificado: fit ICP >= Médio
    Lead --> SelfService: fit Baixo / freemium
    Qualificado --> POC: interesse validado
    POC --> Cliente: assinatura fechada
    SelfService --> Cliente: upgrade pago
    Cliente --> Expansao: upsell / novo elo
    Expansao --> Cliente: renovado
    Cliente --> EmRisco: health vermelho
    EmRisco --> Cliente: recuperado (CS)
    EmRisco --> Churn: não renovou
    Churn --> [*]
    Cliente --> Renovado: renovação
    Renovado --> Cliente
```

7. Fluxogramas — Jornada de Venda

7.1 Jornada comercial ponta a ponta

```
flowchart TD
    START([Lead entra no pipeline]) --> SEG{Classificar segmento<br/>e fit de ICP}
    SEG -->|Fit Baixo| SS[Direcionar a self-service / freemium]
    SEG -->|Fit Médio/Alto| QUAL[Qualificação assistida<br/>SDR/BDR]
    QUAL --> DESC{Dor + orçamento<br/>+ autoridade + prazo?}
    DESC -->|Não| NUR[Nutrição / reciclagem]
    DESC -->|Sim| DEMO[Demonstração + descoberta técnica]
    DEMO --> POC{POC necessária?}
    POC -->|Sim| POCX[POC com métricas de sucesso acordadas]
    POC -->|Não| PROP[Proposta + order form]
    POCX --> PROP
    PROP --> NEG{Negociação<br/>dentro da alçada?}
    NEG -->|Não| APRV[Aprovação de alçada §4.3]
    NEG -->|Sim| CLOSE[Assinatura Closed-Won]
    APRV --> CLOSE
    CLOSE --> ONB[Handoff para Onboarding §5/§6 comercial]
    ONB --> LAND[Land: tier base + primeiro módulo]
    LAND --> EXP[Expand: add-ons + novos elos da cadeia]
    EXP --> REN([Renovação / NRR])
    NUR --> SEG
    SS -->|upgrade| DEMO
```

7.2 Land-and-expand dentro do account

```
flowchart LR
    L0["LAND<br/>Tenant + Starter/Growth<br/>1 módulo essencial"] --> E1["EXPAND 1<br/>+ usuários / + SN ativos"]
    E1 --> E2["EXPAND 2<br/>+ add-on (KCI / KCD / KSI)"]
    E2 --> E3["EXPAND 3<br/>+ novo elo da cadeia<br/>(revenda -> assistências)"]
    E3 --> E4["EXPAND 4<br/>upgrade Enterprise<br/>+ marketplace (futuro)"]
    E4 --> NRR["NRR crescente"]
```

8. Boas Práticas

Vendas e qualificação:

- Qualificar por **fit de ICP antes de esforço**; não tratar volume como qualidade de *pipeline*.
- Vender **valor de rede e rastreabilidade**, não *features* isoladas — o SN vitalício é o diferencial narrativo.
- Usar **POC com critérios de sucesso escritos** para encurtar ciclo e reduzir *churn* pós-venda.
- Ancorar preço em **valor por SN/usuário**, evitando corrida ao desconto.

Land-and-expand:

- Fazer **land enxuto** (tier base + um módulo) para acelerar *time-to-value*; expandir com dados de uso.
- Mapear o **grafo da cadeia** de cada conta (quais revendas, quais assistências) e usar como roteiro de expansão.
- Instrumentar **gatilhos de expansão** (ex.: uso de SN acima de X% do cap → conversa de *upsell*).

Precificação e descontos:

- Preferir **desconto anual** (fluxo de caixa + retenção) a desconto sobre lista.
- Manter **pricebook versionado** e *grandfathering* transparente em reajustes.
- Publicar **limites de freemium** com clareza para evitar frustração e *support load*.

Pós-venda:

- Definir **health score** com sinais de produto (adoção de módulos, SN ativos, logins) e de relacionamento.
- Tratar **renovação como processo**, iniciado com antecedência (RE-01), não como evento.

9. Padrões

9.1 Padrões de nomenclatura comercial

- **Tiers:** Starter, Growth, Enterprise (não usar sinônimos ad-hoc em propostas).
- **Add-ons:** Add-on KCI, Add-on KCD, Add-on KSI.
- **Dimensões de cobrança:** tier_base, usuario_ativo, sn_ativo.
- **Estágios de pipeline:** Lead → MQL → SQL → Oportunidade → Proposta → Closed-Won/Closed-Lost.

9.2 Padrões de proposta (order form)

Toda proposta deve conter: identificação do *tenant* raiz; tier; quantidades contratadas por dimensão; add-ons; ciclo (mensal/anual); descontos e alçada aprovadora; SLAs aplicáveis; data de início e renovação; escopo de serviços profissionais (se houver).

9.3 Padrões de dados comerciais (CRM)

- Uma **oportunidade** por ciclo de compra, vinculada a um *account*.
- **MRR** registrado por linha (base, consumo, cada add-on) para permitir análise de *waterfall* (new/expansion/contraction/churn).
- **Motivo de perda** obrigatório em Closed-Lost (taxonomia controlada: preço, *fit*, concorrente, *timing*, sem decisão).

9.4 Padrão de reconhecimento de expansão vs. novo negócio

- **New business:** primeiro contrato de um *account*.
- **Expansion:** aumento de MRR no mesmo *account* (mais SN/usuários, novo add-on, novo elo).
- **Contraction:** redução sem cancelamento total. **Churn:** cancelamento total do *account*.

10. Casos de Uso

UC-01 — Fabricante como âncora de rede. Um fabricante adota o Kairós (Enterprise) para emitir e rastrear SNs de fábrica. Ao exigir/estimular que suas revendas registrem vendas na plataforma, gera *pipeline* qualificado a jusante. *Resultado comercial:* CAC reduzido nas revendas (venda referida) e efeito de rede que eleva o valor do KCI para o fabricante.

UC-02 — Rede de revenda multiunidade. Rede com 30 lojas contrata Growth consolidado no *tenant* raiz, com cobrança por usuários e SN das filiais. *Expand:* add-on KSI para estoque inteligente entre lojas. *Métrica-chave:* NRR via crescimento de SN ativos.

UC-03 — Assistência técnica / operador. Assistência entra via Starter (ou freemium) para gerir ordens de serviço e histórico por SN. *Expand:* add-on KCD quando passa a lidar com dados sensíveis de missões/clientes corporativos.

UC-04 — Freemium → pago (self-service). Operador individual usa freemium (cap de SN e usuários). Ao ultrapassar o cap ou precisar de relatórios KCI, converte para Starter/Growth sem toque humano. *Métrica:* taxa de conversão freemium → pago.

UC-05 — Land-and-expand por referral de cadeia. Cliente Growth (revenda) convida suas assistências parceiras; cada assistência ativada gera crédito/incentivo. *Resultado:* expansão de logos com CAC marginal baixo e maior densidade de dados por SN.

UC-06 — Upsell disparado por uso. Conta atinge 85% do cap de SN do tier. Gatilho automático abre tarefa de CS/Vendas para *upgrade*. *Resultado:* expansão proativa, sem atrito de limite atingido.

UC-07 — Enterprise com requisitos de segurança. Cliente corporativo exige isolamento dedicado e SLAs reforçados; contrata Enterprise + KCD com *order form* customizado e SLA de suporte prioritário.

11. Modelagem — Planos e Métricas

11.1 Tabela de planos (EXEMPLO ILUSTRATIVO)

Valores em BRL, ilustrativos, por mês, ciclo mensal. Anual à vista ~16% de desconto (RC-05).

Dimensão	Freemium	Starter	Growth	Enterprise
Preço base / mês	R\$ 0	R\$ 490	R\$ 1.900	Sob consulta (a partir de ~R\$ 6.500)
Usuários inclusos	2	5	20	Customizado
Usuário adicional / mês	— (cap)	R\$ 60	R\$ 45	Negociado por volume
SN ativos inclusos	25	300	2.000	Customizado
SN adicional (pacote 100) / mês	— (cap)	R\$ 70	R\$ 55	Negociado por volume
Rastreabilidade vitalícia por SN	Básica	Completa	Completa	Completa + retenção estendida
Multiempresa (subunidades)	1	1	Até 10	Ilimitado / grupo econômico
Add-on KCI (Inteligência)	x	Opcional	Opcional	Incluído (tier superior)
Add-on KCD (Cibersegurança)	x	Opcional	Opcional	Incluído / reforçado
Add-on KSI (Estoque Inteligente)	x	Opcional	Opcional	Opcional / incluído
Suporte	Comunidade	E-mail (8x5)	Prioritário (12x5)	Dedicado (24x7) + CSM
SLA de uptime	Best effort	99,5%	99,7%	99,9%
Onboarding	Autoguiado	Guiado padrão	Assistido	Serviços profissionais

11.2 Add-ons por módulo (EXEMPLO ILUSTRATIVO)

Add-on	Preço / mês (sobre tier)	Cobrança principal	Observação
KCI — Inteligência	a partir de R\$ 690	por tenant + faixa de SN	insights de rede, previsões, benchmarks
KCD — Cibersegurança	a partir de R\$ 890	por tenant	trilhas de auditoria, hardening, alertas
KSI — Estoque Inteligente	a partir de R\$ 590	por tenant + nº de locais	reposição preditiva, giro por SN

11.3 Freemium vs Enterprise (posição no espectro)

flowchart LR

```

F["Freemium<br>aquisição / PLG<br>caps rígidos"] --> S["Starter<br>self-service assistido"]
S --> G["Growth<br>venda assistida<br>land-and-expand"]
G --> E["Enterprise<br>venda consultiva<br>contrato customizado"]

```

11.4 Métricas de negócio — definições e fórmulas

Métrica	Definição	Fórmula (referência)
MRR	Receita recorrente mensal normalizada	Σ receita recorrente do mês (base + consumo + add-ons)
ARR	Receita recorrente anualizada	$MRR \times 12$
CAC	Custo de aquisição de cliente	$(\text{custo S\&M do período}) \div (\text{nº novos clientes})$
LTV	Valor do tempo de vida do cliente	$(ARPA \times \text{margem bruta}) \div \text{churn de receita}$
Gross Churn	Perda de receita/logos sem contar expansão	$\text{receita perdida} \div \text{receita início período}$
NRR	Retenção líquida de receita	$(MRR \text{ início} + \text{expansão} - \text{contração} - \text{churn}) \div MRR \text{ início}$
CAC Payback	Meses para recuperar o CAC	$CAC \div (ARPA \times \text{margem bruta mensal})$
ARPA	Receita média por conta	$MRR \text{ total} \div \text{nº de contas}$

11.5 MRR waterfall (composição do movimento mensal)

```
flowchart LR
    MI["MRR início"] --> N["+ New"]
    N --> EXP["+ Expansion<br/>(SN, usuários, add-ons, elos)"]
    EXP --> RE["+ Reactivation"]
    RE --> CO["- Contraction"]
    CO --> CH["- Churn"]
    CH --> MF["MRR fim"]
```

11.6 Metas por métrica (EXEMPLO ILUSTRATIVO)

Métrica	Alvo saudável (ex.)	Alerta (ex.)
NRR	≥ 110%	< 100%
LTV/CAC	≥ 3,0x	< 2,0x
CAC Payback	≤ 12 meses	> 18 meses
Gross churn (logo, mensal)	≤ 1,5%	> 3,0%
Margem bruta SaaS	≥ 75%	< 65%
Conversão freemium → pago	≥ 4%	< 2%

12. Checklist

Checklist de qualificação de oportunidade:

- ☐ Segmento identificado (fabricante / revenda / assistência-operador).
- ☐ Fit de ICP classificado (Alto/Médio/Baixo).
- ☐ Dor mapeada e vinculada a módulo/valor (rastreadibilidade, KCI, KCD, KSI).
- ☐ Autoridade e orçamento confirmados.
- ☐ Critérios de sucesso da POC acordados por escrito (se aplicável).

Checklist de proposta (order form):

- ☐ Tenant raiz e tier definidos.
- ☐ Quantidades por dimensão (usuários, SN) dimensionadas com folga de crescimento.
- ☐ Add-ons selecionados e justificados.
- ☐ Ciclo (mensal/anual) e desconto dentro da alçada (§4.3).
- ☐ SLAs e escopo de serviços profissionais explícitos.

Checklist de fechamento e handoff:

- ☐ Order form assinado e registrado no CRM.
- ☐ MRR decomposto por linha (base/consumo/add-ons).
- ☐ Handoff formal para Onboarding/CS com contexto e metas de valor.
- ☐ Data de renovação e responsável definidos.

Checklist de expansão/renovação:

- ☐ Health score verde e adoção de módulos verificada.
- ☐ Gatilhos de uso (SN/usuários próximos do cap) revisados.
- ☐ Oportunidade de novo elo da cadeia avaliada.
- ☐ Renovação iniciada no prazo (RE-01).

13. Riscos

ID	Risco	Prob.	Impacto	Mitigação
R-01	Concorrência de ERPs generalistas ou verticais (ver doc 05) pressiona preço e diferenciação	Média	Alto	Reforçar <i>moat</i> de rastreabilidade por SN + efeito de rede; posicionamento vertical; provas de ROI
R-02	Complexidade de preço (3 dimensões + add-ons) gera atrito na venda	Média	Médio	Calculadora comercial, pacotes pré-configurados, propostas guiadas
R-03	CAC alto em Enterprise sem <i>pipeline</i> de parceiros	Média	Alto	Priorizar parcerias com fabricantes (co-venda) e <i>referral</i> de cadeia
R-04	Freemium canibaliza planos pagos se caps forem generosos demais	Média	Médio	Caps calibrados; bloqueio de módulos premium no freemium (RC-04)
R-05	Churn concentrado em contas pequenas de baixo <i>fit</i>	Alta	Médio	Filtro de ICP na entrada; <i>self-service</i> para fit Baixo (RS-02)
R-06	Dependência de poucas contas âncora (fabricantes)	Média	Alto	Diversificar base; expandir logos por revendas/assistências
R-07	Descontos fora de alçada corroem margem	Média	Médio	Matriz de alçada (§4.3) e governança de aprovação
R-08	Time-to-value longo eleva <i>churn</i> precoce	Média	Alto	Land enxuto, onboarding padronizado, metas de valor no handoff
R-09	Reajustes mal comunicados geram atrito de renovação	Baixa	Médio	Pricebook versionado, <i>grandfathering</i> e comunicação antecipada
R-10	Marketplace futuro sem massa crítica de oferta/demanda	Média	Médio	Lançar após densidade de rede; começar com peças/serviços de parceiros âncora

14. Melhorias Futuras

1. **Marketplace transacional** — peças, serviços de assistência e integrações de terceiros com *take rate*, aproveitando o grafo de SN.
2. **Precificação por valor/uso avançada** — pacotes baseados em resultados (ex.: preço por SN sob garantia gerida, por missão rastreada).
3. **Programa de parcerias formal** — níveis (revenda autorizada, integrador, fabricante âncora) com *revenue share* e co-marketing.
4. **PLG reforçado** — *self-service* completo do Freemium ao Growth com *product-qualified leads* (PQL) automáticos.
5. **Expansão internacional** — cobrança em USD, *pricebook* multimoeda e *localization* comercial para exportação de fabricantes.
6. **Motor de *pricing* dinâmico** — recomendação de tier/add-on por padrão de uso e coorte.
7. **Bundles verticais** — pacotes prontos por perfil (Fabricante Pro, Rede Multiloja, Assistência Certificada).
8. **Marketplace de dados/benchmark (KCI)** — insights agregados de rede como produto premium (respeitando privacidade/LGPD).

15. Auditoria

Controle de versão do documento:

Versão	Data	Autor	Alteração
1.0	21/07/2026	Estratégia de Negócio	Emissão inicial do documento Comercial

Trilha de auditoria comercial (o que deve ser rastreável):

- **Alterações de pricebook** — versão, data, responsável, itens alterados e política de *grandfathering*.
- **Descontos concedidos** — oportunidade, percentual, alçada aprovadora, justificativa (§4.3).
- **Movimentos de MRR** — *log* de new/expansion/contraction/churn por *account* e linha de receita.
- **Renovações** — data-alvo, antecedência (RE-01), resultado e motivo (se perda).
- **Handoffs Vendas → CS** — metas de valor acordadas e cumprimento.

Cadência de revisão:

- **Mensal** — revisão de métricas (MRR, ARR, churn, NRR, CAC/LTV) em comitê comercial.
- **Trimestral** — revisão de tiers, add-ons e desempenho de canais/parcerias.
- **Anual** — revisão estrutural de posicionamento (vs doc 05) e política de preços.

Dependências e consistência: este documento deve permanecer consistente com os documentos 00–21; qualquer mudança em módulos (14–16), arquitetura multiempresa (10) ou rastreabilidade por SN (18) exige reavaliação do modelo comercial aqui descrito.

Nota final: valores, tiers e metas são ilustrativos e servem à modelagem estratégica. A tabela de preços oficial, os SLAs contratuais e as condições comerciais vigentes são governados pelo pricebook e pelos contratos oficiais mantidos por Comercial, Finanças e Jurídico.

23 Internacionalização

Documento: 23 – Internacionalização **Versão:** 1.0 · **Data:** 21 de julho de 2026 · **Status:** Ativo **Dependências:** 00 – Master Prompt ... 22 (todos os documentos anteriores, 00 – 22) **Responsável:** Especialista em Internacionalização (i18n/l10n)

1. Resumo Executivo

Este documento define a estratégia de **internacionalização (i18n)** e **localização (l10n)** do Drone Kairós ERP: a arquitetura, os padrões, os processos e a governança que tornam a plataforma capaz de operar em múltiplos idiomas, moedas, fusos, regimes fiscais e jurisdições regulatórias — **sem alteração de código por mercado**.

O princípio arquitetural nuclear, herdado do canon (01 – Constituição, Artigo de escala internacional de primeira classe), é a **separação estrita entre lógica e locale**: nenhum texto, formato, símbolo monetário, regra de plural, sentido de leitura, fuso ou unidade é embutido (*hardcoded*) no código. Toda decisão dependente de cultura, país ou regulação é resolvida em **tempo de execução** a partir de três eixos de contexto: **locale** (idioma + região cultural), **tenant/região operacional** (jurisdição fiscal, regulatória e de residência de dados) e **preferência do usuário** (idioma, fuso e formatos individuais). Estes três eixos são ortogonais e resolvidos por uma **cadeia de fallback** determinística.

Adotamos os padrões da indústria como fundação não-negociável: **Unicode/UTF-8** ponta a ponta, **CLDR** (Common Locale Data Repository) como fonte canônica de dados de locale, **ICU MessageFormat** para mensagens com plural/gênero/seleção, **BCP 47** para tags de idioma, **ISO 4217** para moedas, **ISO 3166** para países, **IANA Time Zone Database** para fusos e **UTS #35 / bidi (UAX #9)** para RTL. A internacionalização não é um recurso — é uma **propriedade transversal** de toda a plataforma, verificável em CI.

Os idiomas-alvo do lançamento (*wave 1*) são **Português (Brasil)**, **Inglês (Estados Unidos)** e **Espanhol (América Latina)**, com o arcabouço já preparado para **Espanhol (Europa)**, **Francês**, **Alemão** e, na camada de RTL/bidi, **Árabe** como validação de arquitetura. A conformidade por país é tratada por **abstração de política** (privacidade, fiscal e regulatória de drones), com implementações concretas para **LGPD (Brasil)**, **GDPR (União Europeia)**, e ganchos para **CCPA/CPRA (EUA)** — sempre articuladas com o módulo Financeiro (**multi-moeda/fiscal**) e o **KCD** (conformidade/privacidade).

O documento entrega: a matriz de dimensões i18n/l10n, a arquitetura de localização por tenant/região com *feature flags* regionais e **residência de dados** (*data residency*), os diagramas Mermaid da resolução de locale e da topologia regional, os fluxogramas do processo de tradução e do *rollout* regional, a matriz **país × requisitos**, checklist, riscos, evolução e trilha de auditoria.

2. Objetivos

- **O1.** Garantir **externalização total de textos** e de todo dado sensível a locale — zero *hardcode* de strings, formatos ou símbolos na camada de aplicação, verificável por *lint* e testes de pseudo-localização.
- **O2.** Estabelecer os **padrões canônicos** (Unicode/UTF-8, CLDR, ICU MessageFormat, BCP 47, ISO 4217/3166, IANA TZDB) como base única de toda formatação e mensagem.
- **O3.** Definir a **cadeia de resolução de locale** e a hierarquia de *fallback* (usuário → tenant/região → idioma-base → default global).
- **O4.** Especificar a arquitetura **multi-moeda** integrada ao Financeiro: câmbio, arredondamento fiscal, exibição versus contabilização, e separação entre moeda de transação, funcional e de apresentação.
- **O5.** Modelar a **conformidade por país** como política plugável: privacidade (LGPD/GDPR/abstração), regras fiscais/tributárias por região e requisitos regulatórios de drones (ANAC/FAA/EASA) que impactem dados e documentos.
- **O6.** Desenhar a **arquitetura de localização por tenant/região**, com *feature flags* regionais e **residência de dados** por jurisdição.

- **O7.** Definir o **processo e a governança** de i18n/l10n: ciclo de tradução, papéis, *TMS*, memória de tradução, glossário, *string freeze* e critérios de prontidão regional.
- **O8.** Assegurar **RTL/bidi** e acessibilidade multilíngue como requisitos de arquitetura, não adendos.

3. Escopo

No escopo: externalização de textos e catálogos de tradução; pluralização, gênero e seleção via ICU; formatação de data/hora/número/percentual/lista; fusos horários e conversões; unidades e sistemas de medida; RTL/bidi; multi-moeda (câmbio, arredondamento, exibição, contabilização — em conjunto com Financeiro); conformidade por país (privacidade, fiscal, regulatória de drones no que toca a **dados e documentos**); arquitetura de localização por tenant/região; *feature flags* regionais; residência de dados; processo, *TMS*, governança e auditoria de i18n.

Fora do escopo (referência a outros documentos): o modelo de dados e RLS multi-tenant em detalhe ([08 – Banco de Dados](#)); a decomposição de serviços e contratos ([09 – Microsserviços](#)); a lógica contábil e o plano de contas em si (Documento de Financeiro); o detalhamento operacional dos requisitos regulatórios de voo (é tratado aqui **apenas** onde impacta dados, documentos, formatos e retenção); a infraestrutura de rede/regiões de nuvem (Documento de Infraestrutura). Este documento define **como o produto se adapta ao locale e à jurisdição**, não a implementação interna de cada módulo consumidor.

4. Regras

- **R1. UTF-8 ponta a ponta.** Todo armazenamento, transporte e apresentação usa Unicode/UTF-8. Nenhuma coluna, campo, log ou API usa *charset* legado. Normalização Unicode **NFC** na entrada.
- **R2. Zero *hardcode* de texto.** Nenhuma string voltada ao usuário no código; toda mensagem vem de catálogo, referenciada por **chave estável** (nunca pelo texto-fonte).
- **R3. Locale nunca inferido do idioma sozinho.** Idioma (o *quê* se lê) e região (o *como* se formata) são independentes; ambos derivam da tag **BCP 47** resolvida.
- **R4. Formatação sempre por biblioteca CLDR/ICU.** Datas, números, moedas, listas e plurais jamais são montados por concatenação manual. Proibido `"R$" + valor` ou `dia + "/" + mês`.
- **R5. UTC como verdade temporal.** Todo *timestamp* é persistido em **UTC** com precisão e origem; o fuso é aplicado **apenas na apresentação**, a partir do fuso do usuário/tenant (IANA TZDB), nunca de *offset* fixo.
- **R6. Moeda é sempre um par (valor, código ISO 4217).** Nenhum valor monetário existe sem seu código de moeda e sua escala de casas decimais definida pelo CLDR/ISO. Proibido armazenar dinheiro em ponto flutuante binário.
- **R7. Separação exibição × contabilização.** A moeda de **apresentação** (o que o usuário vê) nunca redefine a moeda **funcional/de registro** (o que é contabilizado). Conversão de exibição não altera o razão.
- **R8. Fallback determinístico.** A resolução de mensagem/formato segue a cadeia definida (§5.3) e **nunca** exhibe a chave crua ao usuário: falha resolve para o idioma-base, com marcação de telemetria.
- **R9. Política de conformidade é plugável por jurisdição.** Privacidade, fiscal e regulatória de drones são resolvidas por *policy* selecionada por região do tenant; a aplicação não contém `if país == ...` de regra de negócio regulatória.
- **R10. Residência de dados respeitada por padrão.** Dado pessoal e documento regulatório residem na região exigida pela jurisdição do tenant; movimentação transfronteiriça só com base legal explícita e registrada.
- **R11. Pseudo-localização obrigatória em CI.** Toda *build* passa por *pseudo-locale* (expansão de comprimento + acentuação + marcação de bordas) para detectar *hardcode*, truncamento e concatenação.
- **R12. String freeze antes do envio à tradução.** Nenhuma chave é enviada ao TMS sem congelamento; mudança pós-freeze cria nova chave/versão, nunca reescreve silenciosamente.
- **R13. Contexto obrigatório na chave.** Toda chave carrega descrição/ `context` para o tradutor (uso, limite de caracteres, placeholders, tom). Chave sem contexto é erro de revisão.
- **R14. RTL de primeira classe.** Layout é *logical-property based* (`start/end`, não `left/right`); espelhamento e bidi validados para o conjunto RTL desde o v1.
- **R15. Sem dado cultural no código-fonte de negócio.** Feriados, formatos de endereço, ordem de nome, honoríficos, unidades e semana-início vêm de dados de locale (CLDR + tabela de tenant), não de literais.

5. Arquitetura de i18n/l10n

5.1 Os três eixos de contexto

A localização do Drone Kairós resolve toda decisão a partir de três eixos **ortogonais**, combinados no *request context*:

Eixo	Origem	Governa	Exemplo
Locale (BCP 47)	Preferência do usuário → <i>header Accept-Language</i> → default do tenant	Idioma dos textos, regras de plural, formato de data/número, sentido de leitura	pt-BR , en-US , es-419 , ar-SA
Região operacional (ISO 3166 + jurisdição)	Configuração do tenant	Regime fiscal, política de privacidade, regulador de drone, residência de dados, moeda funcional	BR /LGPD/ANAC · DE / GDPR/EASA · US /CCPA/ FAA
Preferência individual	Perfil do usuário	Fuso horário, formato de data/hora, sistema de unidades, primeira moeda de exibição	fuso America/ Sao_Paulo , 24h, métrico

Um usuário *en-US* pode operar dentro de um tenant cuja região é *BR*: ele lê em inglês, mas o documento fiscal, a moeda funcional e a política de privacidade seguem a **jurisdição do tenant (BR/LGPD/ANAC)**. Essa ortogonalidade é o coração da arquitetura.

5.2 Camadas lógicas

1. **Camada de dados de locale (CLDR)**: fonte canônica de calendários, símbolos, padrões de número/data, plural rules, nomes de idioma/país/moeda, ordem de listas.
2. **Camada de catálogos (mensagens)**: *bundles* por locale, indexados por chave estável, formato ICU MessageFormat, versionados no repositório e servidos por *locale service*.
3. **Camada de formatação (runtime i18n)**: API única de formatação (datas, números, moeda, listas, unidades, tempo relativo) construída sobre ICU/CLDR — o **único** ponto autorizado a produzir texto localizado.
4. **Camada de política por jurisdição**: *policies* plugáveis de privacidade, fiscal e regulatória de drone, selecionadas pela região do tenant.
5. **Camada de configuração regional (tenant/região)**: perfil regional do tenant, *feature flags* regionais e diretiva de residência de dados.
6. **Camada de apresentação (UI/BFF)**: consome a API de formatação e os catálogos; aplica RTL/bidi por *logical properties*; nunca formata manualmente.

5.3 Cadeia de resolução e fallback

A resolução de mensagem e formato é **determinística**:

```
preferência do usuário
└─> locale completo do tenant (ex.: es-419)
    └─> idioma-base (ex.: es)
        └─> idioma-âncora do produto (en, source of truth)
```

Regras: (a) uma chave ausente no locale-alvo cai para o próximo elo, **nunca** para a chave crua; (b) toda queda de *fallback* emite telemetria (*i18n.fallback_hit*) para priorizar tradução; (c) dados de formato (CLDR) seguem *fallback* de locale independente do *fallback* de mensagens; (d) o idioma-âncora (*en*) é sempre 100% completo por definição — é a fonte.

5.4 Modelo de dados de suporte (resumo)

Sem substituir o 08 – Banco de Dados , a i18n exige:

- **locale** e **timezone** no perfil do usuário e **default no tenant**;

- **conteúdo de negócio traduzível** (ex.: nome de produto, descrição de serviço, categoria) modelado como **tabela de tradução** (`entidade_id`, `locale`, campos), não colunas por idioma;
- **valores monetários** como (`amount_minor` `BIGINT`, `currency` `CHAR(3)`, `scale` `SMALLINT`) — inteiro em unidade menor, nunca `float`;
- **região/jurisdição do tenant** e **diretiva de residência** como atributos de configuração regional.

5.5 Multi-moeda (integração com Financeiro)

A arquitetura monetária distingue **três papéis de moeda** (alinhada a boas práticas contábeis multi-moeda):

Papel	Definição	Quem define	Exemplo
Moeda de transação	Moeda em que o fato econômico ocorreu	O evento (nota, pagamento)	Compra em <code>USD</code>
Moeda funcional / de registro	Moeda em que o tenant/entidade contabiliza	Região/configuração do tenant	Razão em <code>BRL</code>
Moeda de apresentação	Moeda em que o usuário visualiza	Preferência do usuário/ relatório	Dashboard em <code>EUR</code>

Regras monetárias (em conjunto com o Financeiro):

- **Câmbio**: taxas obtidas de provedor com *timestamp* e fonte; cada conversão registra (`taxa`, `data`, `fonte`) para auditoria e reprodutibilidade. Taxa de **registro** (contábil) é distinta da taxa de **exibição** (informativa).
- **Arredondamento**: aplicado conforme a escala CLDR da moeda de destino (ex.: `JPY` = 0 casas, `BRL/USD` = 2, `BHD` = 3), com **modo de arredondamento fiscal explícito** (padrão *half-even/bankers* no razão) e preservação de resíduos em rateios (sem "centavo perdido").
- **Exibição**: símbolo, posição, separadores e código seguem CLDR do locale de apresentação (`R$ 1.234,56` em `pt-BR`; `$1,234.56` em `en-US`).
- **Contabilização**: sempre na moeda funcional; a conversão de exibição é **derivada e não persistida no razão**. Diferenças de câmbio realizadas/não-realizadas são responsabilidade do módulo Financeiro (referência), acionadas por eventos.

6. Diagramas

6.1 Arquitetura de localização (visão de componentes)

```
flowchart TB
    subgraph Cliente["Cliente (Web / Mobile)"]
        UI["UI · logical properties · RTL-aware"]
    end

    subgraph Borda["Borda (BFF / Gateway)"]
        RC["Resolvedor de Contexto<br/>(locale · região · usuário)"]
    end

    subgraph Nucleo["Núcleo de i18n"]
        LS["Locale Service<br/>(catálogos ICU por locale)"]
        FMT["API de Formatação<br/>(CLDR/ICU · data · número · moeda · unidade)"]
        CLDR["Dados CLDR"]
        TMDB["Catálogos / TM<br/>versionados"]
    end

    subgraph Politicas["Políticas por Jurisdição"]
        PRIV["Privacidade<br/>LGPD · GDPR · abstração"]
        FISC["Fiscal/Tributária<br/>por região"]
        REG["Regulatória de Drone<br/>ANAC · FAA · EASA"]
    end

    subgraph Config["Configuração Regional"]
        TEN["Perfil Regional do Tenant"]
        FLAG["Feature Flags Regionais"]
        RES["Diretiva de Residência de Dados"]
    end

    subgraph Fin["Financeiro (ref.)"]
        FX["Câmbio · Arredondamento · Razão multi-moeda"]
    end

    UI --> RC
    RC --> LS
    RC --> TEN
    LS --> TMDB
    LS --> FMT
    FMT --> CLDR
    RC --> PRIV & FISC & REG
    TEN --> FLAG & RES
    FISC --> FX
    FMT --> FX
```

6.2 Resolução de locale e fallback (sequência)

```
sequenceDiagram
    participant U as Usuário
    participant BFF as BFF / Resolvedor de Contexto
    participant LS as Locale Service
    participant CLDR as CLDR
    U->>BFF: Requisição (Accept-Language, perfil, tenant)
    BFF->>BFF: Resolve locale (usuário > tenant > default)
    BFF->>BFF: Resolve região/jurisdição (tenant)
    BFF->>LS: getMessages(chave, locale)
    alt chave existe no locale
        LS->>BFF: mensagem localizada
    else ausente
        LS->>LS: fallback (locale > base > âncora en)
        LS->>BFF: mensagem (fallback) + telemetria fallback_hit
    end
    BFF->>CLDR: padrões de data/número/moeda (locale apresentação)
    CLDR->>BFF: formatos
    BFF->>U: Resposta localizada (texto + formatos + fuso do usuário)
```

6.3 Topologia regional e residência de dados

```
flowchart LR
    subgraph GLOBAL["Plano de Controle Global"]
        CATL["Catálogos i18n<br/>(sem dado pessoal)"]
        FLAGS["Serviço de Feature Flags"]
        CFG["Registro de Config Regional"]
    end

    subgraph BR["Região BR (LGPD · ANAC)"]
        BRDATA[("Dados pessoais + docs fiscais BR")]
        BRAPP["Instância de dados BR"]
    end

    subgraph EU["Região EU (GDPR · EASA)"]
        EUDATA[("Dados pessoais + docs EU")]
        EUAPP["Instância de dados EU"]
    end

    subgraph US["Região US (CCPA · FAA)"]
        USDATA[("Dados pessoais + docs US")]
        USAPP["Instância de dados US"]
    end

    CATL --> BRAPP & EUAPP & USAPP
    FLAGS --> BRAPP & EUAPP & USAPP
    CFG --> BRAPP & EUAPP & USAPP
    BRAPP --> BRDATA
    EUAPP --> EUDATA
    USAPP --> USDATA
```

7. Fluxogramas (processo de tradução e rollout regional)

7.1 Processo de tradução (source → produção)

```
flowchart TD
    A["Dev cria/edita chave<br/>(source en + context + limite)"] --> B["Lint i18n<br/>+ pseudo-loc"]
    B -- reprova --> A
    B -- aprova --> C["String Freeze<br/>(marco de release)"]
    C --> D["Sincroniza com TMS<br/>(extração de novas chaves)"]
    D --> E["Memória de Tradução<br/>+ Glossário aplicados"]
    E --> F["Tradução (humana / assistida)"]
    F --> G["Revisão linguística + QA in-context"]
    G --> H{"Aprovado?"}
    H -- não --> F
    H -- sim --> I["Merge do catálogo por locale"]
    I --> J["CI: cobertura de tradução + testes de formato"]
    J --> K{"Cobertura ≥ meta?"}
    K -- não --> F
    K -- sim --> L["Publicação do catálogo (versão)"]
    L --> M["Telemetria de fallback em produção<br/>realimenta backlog"]
    M --> D
```

7.2 Rollout de um novo país/região

```
flowchart TD
  S["Decisão de entrar em nova região"] --> P1["Perfil de país:<br/>locale, moeda, fuso, unidades"]
  P1 --> P2["Seleção de políticas:<br/>privacidade · fiscal · regulatória de drone"]
  P2 --> P3["Definir residência de dados<br/>e base legal de transferência"]
  P3 --> P4["Traduzir catálogo alvo<br/>(processo 7.1)"]
  P4 --> P5["Feature flags regionais em OFF<br/>(dark launch)"]
  P5 --> P6["QA de conformidade + fiscal + i18n<br/>(matriz §11)"]
  P6 --> C["Gate de prontidão<br/>(checklist §12)"]
  C -- falha --> P4
  C -- ok --> P7["Piloto com tenant-farol"]
  P7 --> P8["Métricas OK?"]
  P8 -- não --> P4
  P8 -- sim --> P9["Ativar flags · GA regional"]
  P9 --> P10["Monitorar fallback, fiscal, DSR/privacidade"]
```

8. Boas Práticas

- **Chave estável, nunca o texto-fonte.** Use `invoice.status.overdue`, não a frase em inglês como chave; texto muda, chave permanece.
- **Uma mensagem = uma sentença completa.** Nunca concatene fragmentos traduzidos; a ordem das palavras e a gramática variam por idioma. Use ICU `select / plural` para variação, não *string splicing*.
- **Placeholders nomeados e tipados.** `{count, plural, ...}`, `{date, date, long}`, `{price, number, currency}` — nunca posicionais anônimos que o tradutor não entende.
- **Projete para expansão de 30–40%.** Alemão e francês expandem; a UI e os limites de campo toleram crescimento. Pseudo-loc torna isso visível cedo.
- **Formate no runtime, não no dado.** Persista valores neutros (UTC, `amount_minor`, código de moeda, número puro); formate só na borda de apresentação.
- **Propriedades lógicas de layout.** `margin-inline-start`, `text-align: start`, `dir="auto"` para bidi; nada de `left/right` fixos.
- **Não traduza dados de negócio como se fossem UI.** Nome de produto do cliente é dado (tabela de tradução, editável pelo tenant); rótulo de botão é UI (catálogo do produto).
- **Contexto e capturas de tela para o tradutor.** Ambiguidade ("Post" verbo x substantivo) resolve-se com `context` e QA *in-context*.
- **Números e IDs também têm cultura.** Telefone, CEP/postal, documento fiscal, endereço e ordem de nome variam por país — nunca assuma o formato brasileiro/americano como universal.
- **Fuso do usuário, não do servidor.** "Ontem", agendamentos e cortes de relatório calculam-se no fuso do usuário/tenant, com atenção a horário de verão.
- **Moeda com código sempre visível em ambiguidade.** `$` é ambíguo (USD/CAD/AUD/MXN); exiba código ISO quando o contexto não desambigua.

9. Padrões (normativos)

Domínio	Padrão adotado	Uso no Kairós
Codificação	Unicode / UTF-8, normalização NFC	Armazenamento, transporte, apresentação — universal
Dados de locale	CLDR (UTS #35 / LDML)	Fonte única de calendários, símbolos, plural rules, formatos
Mensagens	ICU MessageFormat	Plural, gênero, seleção, placeholders tipados
Tags de idioma	BCP 47 (<code>pt-BR</code> , <code>es-419</code> , <code>zh-Hant</code>)	Identificação de locale em toda a stack
País/território	ISO 3166-1 alpha-2	Região do tenant, jurisdição, residência
Moeda	ISO 4217 (código + escala)	Todo valor monetário; escala via CLDR
Fuso horário	IANA TZDB (nomes, não <i>offsets</i>)	<code>America/Sao_Paulo</code> , DST correto
Bidi / RTL	UAX #9 + CSS logical properties	Espelhamento e leitura RTL
Colaço/ordenação	UCA (ICU Collator) por locale	Ordenar listas conforme cultura
Unidades	CLDR Units + UN/CEFACT quando fiscal	Métrico/imperial, formatação de unidade
Números	CLDR number patterns	Separadores, agrupamento, sinal, percentual
Transferência de dados	BCP 47 + ISO 8601 (datas em API)	API sempre ISO 8601/UTC; formatação só na UI

Regra de ouro dos padrões: a aplicação nunca reimplementa o que CLDR/ICU já resolvem. Formatação, plural e ordenação são **delegados**, não codificados.

10. Casos de Uso

- **CU-01 — Usuário multilíngue em tenant estrangeiro.** Analista `en-US` opera tenant brasileiro: lê UI em inglês; vê valores no razão em `BRL` ; documentos fiscais emitidos em `pt-BR` conforme ANAC/fisco BR; dados pessoais residem na região BR (LGPD).
- **CU-02 — Fatura multi-moeda.** OS cobrada em `USD` (transação), contabilizada em `BRL` (funcional, taxa de registro auditável), exibida ao gestor europeu em `EUR` (apresentação, taxa informativa). Três moedas, um único fato contábil.
- **CU-03 — Pluralização correta.** "3 drones cadastrados" / "1 drone cadastrado" / árabe com 6 categorias de plural — tudo por ICU `plural` , sem `if count == 1` .
- **CU-04 — Relatório com corte por fuso.** Fechamento "diário" de um tenant em `America/Sao_Paulo` versus operação em `Europe/Lisbon` : o corte respeita o fuso do tenant, com DST tratado pela TZDB.
- **CU-05 — Direito do titular (DSR).** Titular europeu exerce direito de acesso/eliminação (GDPR); titular brasileiro exerce direitos LGPD — mesma abstração de *policy* de privacidade, execução conforme jurisdição e residência.
- **CU-06 — Novo mercado sem código.** Entrada no México: cria-se perfil `es-MX / MXN / America/Mexico_City` , seleciona-se política fiscal e regulatória local, traduz-se o catálogo, ativa-se por *feature flag* — **nenhuma alteração de código de negócio**.
- **CU-07 — Documento regulatório localizado.** Certificado/registro de aeronave gera PDF com rótulos no idioma do documento, unidades locais (altura em metros/pés conforme regulador) e formato de data local, arquivado na região de residência.
- **CU-08 — RTL.** Tenant em árabe: UI espelhada, números e datas corretos, campos bidi (nome árabe + matrícula latina) sem quebra visual.

11. Modelagem — Matriz país × requisitos

11.1 Dimensões de i18n/l10n (referência transversal)

Dimensão	Fonte/Padrão	Exemplo de variação
Idioma / plural / gênero	ICU + CLDR plural rules	<code>en</code> (2 formas) · <code>pt</code> (2) · <code>ar</code> (6)
Formato de data/hora	CLDR + fuso do usuário	<code>21/07/2026</code> · <code>07/21/2026</code> · <code>2026-07-21</code>
Formato de número	CLDR number patterns	<code>1.234,56</code> · <code>1,234.56</code> · <code>1 234,56</code>
Moeda (símbolo, posição, escala)	ISO 4217 + CLDR	<code>R\$ 1.234,56</code> · <code>\$1,234.56</code> · <code>¥1235</code>
Fuso horário	IANA TZDB	<code>America/Sao_Paulo</code> · <code>Europe/Berlin</code>
Unidades/medidas	CLDR Units	métrico × imperial; metros × pés
Sentido de leitura	UAX #9	LTR × RTL
Ordenação/colação	UCA (ICU)	ordem alfabética por locale
Endereço / nome / telefone	Perfil de país	ordem de nome, CEP × ZIP × postcode
Semana / feriados	CLDR + tabela do tenant	início dom/seg; feriados nacionais

11.2 Matriz país × requisitos regulatórios e de localização

País/Região	Locale(s)	Moeda funcional	Privacidade	Fiscal/Tributário (natureza)	Regulador de Drone	Residência de dados	Wave
Brasil	<code>pt-BR</code>	<code>BRL</code>	LGPD (ANPD)	Impostos indiretos multi-nível (federal/estadual/municipal), documentos fiscais eletrônicos	ANAC (+ DECEA/SISANT)	Preferencial BR	1
Estados Unidos	<code>en-US</code>	<code>USD</code>	CCPA/CPRA + setorial	<i>Sales tax</i> por estado/condado	FAA (Part 107/registro)	US	1
México / LatAm hisp.	<code>es-419</code> , <code>es-MX</code>	<code>MXN</code> e outras	Leis locais (abstração)	Imposto sobre valor local, doc. eletrônico	Autoridade local	LatAm	1
União Europeia (DE/FR/ES/PT-EU)	<code>de-DE</code> , <code>fr-FR</code> , <code>es-ES</code> , <code>pt-PT</code>	<code>EUR</code>	GDPR	IVA (VAT) intracomunitário	EASA	UE/EEE	2
Reino Unido	<code>en-GB</code>	<code>GBP</code>	UK GDPR / DPA	VAT	CAA	UK	3
Golfo (validação RTL)	<code>ar-SA</code> , <code>ar-AE</code>	<code>SAR</code> , <code>AED</code>	Leis locais (abstração)	Regime local	GACA / autoridade local	Local	3 (arquitetural)

As linhas de wave 2–3 já são **suportadas pela arquitetura** (política plugável, catálogos, residência); o esforço restante é tradução, homologação fiscal e parametrização — não engenharia estrutural.

11.3 Abstração de políticas por jurisdição

```
classDiagram
class PoliticaPrivacidade {
  <<interface>>
  +baseLegal(finalidade)
  +direitosDoTitular()
  +retencao()
  +transferenciaTransfronteirica()
}
class LGPD_BR
class GDPR_EU
class Privacidade_Generica

class PoliticaFiscal {
  <<interface>>
  +calcularTributos(operacao)
  +documentoFiscal()
  +arredondamentoFiscal()
}
class PoliticaDrone {
  <<interface>>
  +documentosObrigatorios()
  +camposDeRegistro()
  +retencaoDeRegistros()
}

PoliticaPrivacidade <|.. LGPD_BR
PoliticaPrivacidade <|.. GDPR_EU
PoliticaPrivacidade <|.. Privacidade_Generica
PoliticaFiscal <|.. Fiscal_BR
PoliticaFiscal <|.. Fiscal_VAT_EU
PoliticaDrone <|.. Drone_ANAC
PoliticaDrone <|.. Drone_FAA
PoliticaDrone <|.. Drone_EASA
```

Sobre reguladores de drone (ANAC/FAA/EASA), o escopo deste documento é **o que impacta dados e documentos**: campos obrigatórios de registro de aeronave/operador, documentos exigidos (certificados, autorizações), idioma/formato desses documentos, unidades legais (ex.: limites de altura) e **prazos de retenção** de registros — não a lógica operacional de voo.

12. Checklist

Prontidão de i18n (produto) - [] UTF-8/NFC em banco, APIs, logs e arquivos; nenhum *charset* legado. - [] Zero string *hardcoded*; *lint* de i18n verde no CI. - [] Todo texto com chave estável + `context` + limite de caracteres. - [] Pseudo-localização executada sem truncamento/vazamento/concatenação. - [] Datas/números/moedas/listas via API de formatação CLDR/ICU (nenhuma concatenação manual). - [] Plurais/gênero via ICU (`plural/select`), zero `if count == 1`. - [] *Timestamps* em UTC; apresentação no fuso do usuário; DST validado. - [] Layout RTL por propriedades lógicas; bidi testado. - [] Fallback determinístico; chave crua nunca aparece; telemetria de fallback ativa.

Prontidão de localização (por locale) - [] Cobertura de tradução $\geq$ meta definida (ex.: 100% para GA, $\geq 95\%$ para piloto). - [] Glossário e memória de tradução aplicados; termos-chave consistentes. - [] QA linguístico *in-context* aprovado.

Prontidão regional (por país) - [] Perfil de país (locale, moeda, fuso, unidades, formatos de endereço/nome/telefone) configurado. - [] Política de privacidade selecionada (LGPD/GDPR/abstração) e DSR funcional. - [] Política fiscal/tributária parametrizada e homologada. - [] Requisitos regulatórios de drone (dados/documentos) mapeados na matriz. - [] Residência de dados definida; base legal de transferência transfronteiriça registrada. - [] *Feature flags* regionais e *gate* de prontidão satisfeitos.

13. Riscos

ID	Risco	Prob.	Impacto	Mitigação
RS-01	Texto <i>hardcoded</i> escapa para produção	Média	Alto	Lint de i18n + pseudo-loc obrigatórios no CI (R2, R11)
RS-02	Concatenação de fragmentos quebra gramática de outro idioma	Média	Médio	ICU MessageFormat; proibição de <i>splicing</i> (R4, §8)
RS-03	Dinheiro em ponto flutuante gera erro de arredondamento	Baixa	Crítico	<code>amount_minor</code> inteiro + escala CLDR + arredondamento fiscal (R6)
RS-04	Confusão exibição × contabilização adultera o razão	Média	Crítico	Separação de papéis de moeda; conversão de exibição não persiste (R7)
RS-05	<i>Offset</i> fixo em vez de TZDB erra no horário de verão	Média	Alto	IANA TZDB por nome; nunca <i>offset</i> fixo (R5)
RS-06	Violação de residência de dados / transferência sem base legal	Baixa	Crítico	Diretiva de residência por região + registro de base legal (R10)
RS-07	Regra fiscal/regulatória embutida em <code>if país</code>	Média	Alto	Políticas plugáveis por jurisdição (R9, §11.3)
RS-08	RTL/bidi quebrado por layout com left/right fixos	Média	Médio	Propriedades lógicas + validação árabe desde v1 (R14)
RS-09	Cobertura de tradução baixa gera fallback visível excessivo	Média	Médio	Meta de cobertura no <i>gate</i> + telemetria de fallback (R8, §12)
RS-10	Chave reescrita silenciosamente corrompe traduções existentes	Baixa	Médio	<i>String freeze</i> + nova chave/versão (R12)
RS-11	Expansão de texto (DE/FR) trunca UI	Média	Médio	Projeto para +40% + pseudo-loc (§8)
RS-12	Ambiguidade de símbolo monetário (\$) confunde usuário	Baixa	Médio	Código ISO visível em contexto ambíguo (§8)
RS-13	Dessincronia entre catálogos e código após <i>release</i>	Média	Médio	Extração automatizada + CI de cobertura + versão de catálogo
RS-14	Divergência de dado de negócio traduzível entre idiomas	Média	Médio	Tabela de tradução com <i>fallback</i> e propriedade do tenant (§5.4)

14. Melhorias Futuras

- **Idiomas adicionais (waves 2–3):** completar `es-ES`, `fr-FR`, `de-DE`, `pt-PT`, `en-GB`; consolidar RTL com `ar-*` em produção.
- **Tradução assistida com *human-in-the-loop*:** pré-tradução automatizada de novas chaves com revisão humana obrigatória, alimentada por memória de tradução e glossário — reduzindo tempo de *rollout* sem abrir mão de QA linguístico.
- **CLDR/ICU auto-atualizáveis:** pipeline que acompanha novas versões de CLDR e TZDB (fusos e regras de DST mudam por decreto) com testes de regressão de formato.
- **Catálogo servido dinamicamente:** entrega de *bundles* por locale sob demanda (*lazy*), com versionamento e cache por *edge*, evitando *bundle* monolítico.
- **Localização de conteúdo gerado pelo tenant:** fluxo self-service para o tenant traduzir seus próprios dados de negócio (produtos, serviços) com controle de qualidade.
- **Motor de regras fiscais como serviço:** externalizar cálculo tributário multi-jurisdição para um serviço dedicado com tabelas versionadas por vigência.

- **Detecção proativa de gaps:** *dashboard* de telemetria de `fallback_hit` e de formatos não cobertos, priorizando automaticamente o *backlog* de tradução.
- **Acessibilidade multilíngue:** validação de leitores de tela por idioma e de conformidade WCAG por locale, incluindo bidi.

15. Auditoria

Item	Descrição
Rastreabilidade de conformidade	Cada requisito de país (privacidade, fiscal, drone, residência) mapeia a uma linha da matriz §11.2 e a uma política concreta §11.3, com base legal registrada.
Trilha de câmbio	Toda conversão monetária registra (<code>taxa</code> , <code>data</code> , <code>fonte</code> , <code>papel</code>) ; a taxa de registro é reproduzível e distinta da de exibição (R6, R7, §5.5).
Trilha de tradução	Cada chave possui histórico de versão, <code>context</code> , <i>string freeze</i> e cobertura por locale; mudanças pós-freeze geram nova versão (R12, §7.1).
Telemetria de i18n	<code>i18n.fallback_hit</code> , formatos não cobertos e locales requisitados são coletados para auditoria de qualidade e priorização (R8).
Residência de dados	Localização física de dado pessoal e documento regulatório é auditável por tenant/região; transferências transfronteiriças exigem registro de base legal (R10, KCD).
Conformidade de padrões	CI verifica UTF-8/NFC, ausência de <i>hardcode</i> , uso da API de formatação e pseudo-localização (R1, R2, R4, R11).
Integração KCD/ Financeiro	Direitos do titular (DSR) e eventos fiscais são articulados com o KCD (privacidade) e o Financeiro (razão multi-moeda), conforme dependências 00–22.
Versão do documento	v1.0 — 21 de julho de 2026. Alterações futuras registradas no controle de versão do repositório de documentação.
Aprovação	Especialista em Internacionalização (responsável); dependências 00 – 22 verificadas e coerentes com o PROJECT CANON (escala internacional de 1ª classe; segurança/privacidade por padrão).

Nota de encerramento. A internacionalização do Drone Kairós ERP é **estrutural, não cosmética**: o produto foi desenhado para que idioma, formato, moeda, fuso, jurisdição e residência de dados sejam **parâmetros de execução**, resolvidos por padrões abertos (Unicode, CLDR, ICU, BCP 47, ISO 4217/3166, IANA) e por políticas plugáveis por jurisdição. Entrar em um novo mercado é um ato de **configuração e tradução governadas**, não de reescrita de código — cumprindo o princípio constitucional de escala internacional de primeira classe, com segurança e privacidade por padrão.

24 Evolução da Plataforma

Documento: 24 – Evolução da Plataforma **Versão:** 1.0 · **Data:** 21 de julho de 2026 · **Status:** Ativo **Dependências:** 00 – 23 (kit completo) **Responsável:** Líder de Produto/Plataforma

1. Resumo Executivo

Este é o **documento de fechamento do kit** e, ao mesmo tempo, o documento que o mantém vivo. Enquanto os documentos 00 – 23 descrevem *o que a plataforma é* — sua constituição, arquitetura, dados, microsserviços, módulos KCI/KCD/KSI, API pública e operação —, o documento 24 descreve **como a plataforma muda sem se trair**. Ele define o contrato de evolução: quem decide, com que critérios, em que cadência, com que garantias de compatibilidade e sob qual auditoria.

O Drone Kairós ERP é uma **plataforma multiempresa (SaaS)** cujo maior ativo é a **confiança acumulada**: rastreabilidade vitalícia por Serial Number, isolamento entre tenants, uma API pública sobre a qual terceiros constroem negócios, e três módulos proprietários (Kairós Core Intelligence, Kairós Cyber Defense, Kairós Smart Inventory) que amadurecem ao longo do tempo. Evoluir uma plataforma assim não é adicionar recursos — é **preservar promessas** (o SN de um drone registrado em 2026 precisa ser resolvível em 2046) enquanto se expande capacidade, alcance geográfico e inteligência.

A tese central deste documento é que **evolução é uma disciplina de governança, não um resultado de iniciativa individual**. Três instrumentos sustentam essa disciplina: (1) um **modelo de decisão** explícito (RFC → ADR → release) com pesos e fóruns definidos; (2) **versionamento semântico** aplicado tanto à plataforma quanto — separadamente — à API pública, com uma **política de depreciação e compatibilidade** contratual e previsível; e (3) um **ciclo de melhoria contínua** que trata dívida técnica, saúde da plataforma e feedback como fluxo permanente, não como campanha esporádica.

O documento entrega ainda a **estratégia de extensibilidade** (SDKs, plugins, marketplace de integrações e programa de parceiros sobre a API pública), a **estratégia de escala** (multi-região, *data residency*, performance em larga escala e sharding por tenant), o **roadmap de horizontes Now/Next/Later** para os três módulos e novos verticais, o conjunto de **métricas de saúde da plataforma**, a política de **sustentabilidade e ciclo de vida** de recursos e clientes, e a **ligação formal com o Project Bible** — porque toda evolução registrada aqui só é real quando propagada para a fonte única de verdade.

2. Objetivos

- **O1.** Estabelecer o **modelo de decisão de produto/plataforma**: fóruns, papéis, pesos, quórum e o fluxo RFC → ADR → Release.
- **O2.** Definir o **versionamento semântico** da Plataforma e da API Pública como esquemas **independentes**, com regras claras de *major/minor/patch*.
- **O3.** Estabelecer a **política de depreciação e compatibilidade** — janelas mínimas, comunicação, *sunset* e caminhos de migração — como compromisso contratual.
- **O4.** Instituir a **gestão de dívida técnica** como orçamento recorrente e a **melhoria contínua** como ciclo mensurável.
- **O5.** Definir a **extensibilidade**: modelo de plugins, marketplace de integrações, SDKs oficiais e o **programa de parceiros** sobre a API pública.
- **O6.** Especificar a **estratégia de escala**: multi-região, *data residency*, performance em larga escala e **sharding por tenant**.
- **O7.** Publicar o **roadmap de horizontes (Now/Next/Later)** para KCI, KCD, KSI e novos verticais, sem datas rígidas mas com direção firme.
- **O8.** Definir as **métricas de saúde da plataforma** e o **ciclo de feedback** que as retroalimenta.

- **O9. Estabelecer sustentabilidade e ciclo de vida:** como recursos nascem, amadurecem, depreciam e como clientes migram com segurança.
- **O10. Formalizar a ligação com o Project Bible:** toda decisão de evolução é registrada, propagada e auditável.

3. Escopo

No escopo: governança de produto e de plataforma; versionamento e depreciação da plataforma e da API pública; gestão de dívida técnica e melhoria contínua; extensibilidade (SDKs, plugins, marketplace, parceiros); estratégia de escala (multi-região, *data residency*, performance, sharding por tenant); roadmap de horizontes para módulos e verticais; métricas de saúde e ciclo de feedback; sustentabilidade e ciclo de vida de recursos/clientes; integração com o Project Bible.

Fora do escopo (referência a outros documentos): especificação funcional dos módulos e da API (Docs de módulos e Doc de API Pública); arquitetura de microsserviços e malha (Doc 09); modelagem de dados e tenancy físico (Docs 07, 08); pipelines de CI/CD e operação (Docs de operação); segurança/IAM e KCD detalhado (Doc 07 e Doc do KCD); estratégia comercial e precificação (Docs de negócio). Aqui tratamos **do processo e das regras de mudança** que incidem sobre todos eles.

Princípios herdados do canon (não contradizer): governança acima de velocidade; escala internacional desde o design; qualidade e auditoria contínuas; rastreabilidade vitalícia por Serial Number; isolamento entre tenants; API-first.

4. Regras (Governança de Evolução)

- **R1. Nada evolui sem RFC.** Toda mudança de comportamento observável (plataforma ou API) começa por uma RFC pública internamente, antes de código.
- **R2. Toda decisão vira ADR.** Uma RFC aprovada gera um **Architecture Decision Record** imutável; ADRs não se editam, se *supersedem*.
- **R3. O Project Bible prevalece.** Em conflito, o Bible decide (respeitadas as cláusulas pétreas da Constituição, Doc 01). Toda decisão de evolução é propagada ao Bible (Doc 02).
- **R4. Compatibilidade é contrato, não cortesia.** Quebras só ocorrem em *major* anunciado, com janela de depreciação cumprida integralmente. **Nunca** se quebra silenciosamente.
- **R5. Dois SemVer independentes.** A **Plataforma** e a **API Pública** têm versionamento próprio; um *minor* de plataforma **não** implica mudança de versão da API, e vice-versa.
- **R6. Depreciar é um processo, não um evento.** Todo recurso a ser removido passa por **Anunciado → Depreciado → Sunset → Removido**, com janela mínima publicada (§9).
- **R7. Orçamento de dívida é inegociável.** Um percentual fixo de capacidade de cada ciclo é reservado à saúde técnica; não é a primeira coisa a ser cortada — é a última.
- **R8. Extensão não é privilégio de acesso.** Plugins e parceiros operam **exclusivamente** sobre a API pública versionada e escopos OAuth; nenhuma integração acessa dados internos ou o banco.
- **R9. Tenant e residência sempre respeitados.** Nenhuma evolução pode violar isolamento de tenant nem *data residency* declarada; é critério de veto em qualquer release.
- **R10. Rastreabilidade vitalícia é irrevogável.** Nenhuma mudança pode tornar um Serial Number histórico irresolúvel. Migrações preservam a resolução do SN de ponta a ponta.
- **R11. Métrica antes de opinião.** Priorização e depreciação são justificadas por dados de saúde e uso (§8, §11), não por preferência.
- **R12. Reversibilidade por padrão.** Todo release traz plano de *rollback* e *feature flag*; o que não pode ser revertido exige aprovação de fórum superior (§5).

5. Arquitetura de Evolução e Extensibilidade

5.1 Camadas da governança de produto/plataforma

A evolução é decidida em três fóruns encadeados, cada um com escopo e cadência próprios:

Fórum	Composição	Escopo de decisão	Cadência	Instrumento
Guilda Técnica (Tech Guild)	Arquitetos + tech leads por bounded context	RFCs técnicas, ADRs, dívida técnica, padrões	Semanal	RFC/ADR
Conselho de Plataforma (Platform Council)	Líder de Produto/Plataforma, CTO, líderes de KCI/KCD/KSI, segurança, DPO	Roadmap, versionamento <i>major</i> , depreciações, <i>data residency</i>	Quinzenal	Decision Record + Roadmap
Comitê Executivo (Steering)	C-level	Novos verticais, expansão de região, mudanças que afetam contratos/SLAs	Mensal/sob demanda	Decisão estratégica

Modelo de decisão — pesos e quórum. As decisões seguem *consent-based* (avança-se sem objeção fundamentada), com escalonamento por reversibilidade:

Tipo de mudança	Reversibilidade	Fórum decisor	Quórum	Registro
<i>Patch</i> / correção	Alta	Tech Guild	2 revisores	ADR leve
<i>Minor</i> / novo recurso compatível	Alta	Tech Guild + PO	Maioria simples	ADR
<i>Major</i> plataforma ou API	Baixa	Platform Council	Consenso, sem veto de segurança/DPO	ADR + Bible + comunicado
Depreciação de recurso	Baixa	Platform Council	Consenso	ADR + política §9
Novo vertical / nova região	Muito baixa	Steering	Aprovação executiva	Decisão + roadmap

Vetos absolutos: Segurança (KCD) e DPO/*Data Protection* têm **veto** sobre qualquer mudança que afete isolamento de tenant, *data residency* ou postura de risco. Um veto não é derrubável por maioria — é resolvido tecnicamente ou a mudança não avança.

5.2 Fluxo canônico RFC → ADR → Release

```

flowchart LR
    I[Ideia / Feedback / Métrica] --> RFC[RFC aberta]
    RFC --> REV{Revisão<br/>Tech Guild}
    REV -- objeção fundamentada --> RFC
    REV -- consent --> ESC{Reversível?}
    ESC -- sim --> ADR[ADR registrado]
    ESC -- não --> PC[Platform Council]
    PC -- aprovado --> ADR
    PC -- vetado --> RFC
    ADR --> IMPL[Implementação c/ feature flag]
    IMPL --> QG[Quality Gate]
    QG -- passa --> REL[Release versionado]
    QG -- falha --> IMPL
    REL --> BIBLE[Propagação ao Project Bible]
    BIBLE --> AUD[Trilha de auditoria]
  
```

5.3 Arquitetura de extensibilidade

A extensibilidade é construída em **anéis concêntricos**, do núcleo imutável para a borda aberta:


```

flowchart TB
    subgraph Nucleo[Núcleo – Domínio proprietário]
        CORE[Serviços de domínio + KCI/KCD/KSI]
        DATA[(Dados por tenant · SN vitalício)]
    end
    subgraph Contrato[Camada de contrato]
        PUB[API Pública versionada]
        EVT[Webhooks / Eventos assinados]
        OAUTH[OAuth2 · escopos · quotas]
    end
    subgraph Extensao[Camada de extensão]
        SDK[SDKs oficiais]
        PLUG[Plugins / Conectores]
        MKT[Marketplace de integrações]
    end
    subgraph Ecossistema[Ecossistema]
        PART[Parceiros / Integradores]
        CLI[Clientes / Tenants]
    end

    CORE --- DATA
    CORE --> PUB
    CORE --> EVT
    PUB --- OAUTH
    EVT --- OAUTH
    PUB --> SDK
    EVT --> SDK
    SDK --> PLUG
    PLUG --> MKT
    MKT --> PART
    PART --> CLI
    OAUTH -. isola .-> Nucleo

```

Regras estruturais da extensibilidade:

1. **A API pública é a única superfície de extensão.** Não existe "API privada emprestada". Plugins e parceiros veem exatamente o que qualquer parceiro vê.
2. **Escopos mínimos.** Cada integração recebe apenas os escopos OAuth necessários; leitura de SN não implica escrita em estoque.
3. **Quotas e rate limiting por chave/tenant.** Um parceiro mal comportado degrada a si mesmo, nunca a plataforma nem outro tenant.
4. **Eventos assinados.** Webhooks carregam assinatura verificável; o parceiro valida origem e integridade.
5. **Certificação.** Nenhuma integração entra no marketplace sem passar por *review* técnico, de segurança (KCD) e de privacidade.

5.4 Modelo de plugins e marketplace

Elemento	Definição	Governança
SDK oficial	Bibliotecas mantidas pela plataforma (ex.: linguagens de servidor, mobile) que encapsulam auth, paginação, <i>retry</i> e tipos gerados do OpenAPI	Versionadas junto à API pública (§9)
Conector/Plugin	Integração empacotada (ex.: contabilidade, e-commerce, telemetria de fabricante) construída sobre a API/SDK	Certificação obrigatória; <i>sandbox</i> antes de produção
Marketplace	Catálogo curado onde tenants instalam integrações com consentimento explícito de escopos	Revisão + monitoramento contínuo + <i>kill switch</i>
Programa de parceiros	Camadas (Registrado → Verificado → Estratégico) com acesso a <i>sandbox</i> , suporte e co-marketing	Contrato de parceria + SLA de API

6. Diagramas

6.1 Ciclo de evolução da plataforma (visão macro)

```
flowchart LR
    subgraph Entrada
        F1[Feedback de clientes]
        F2[Métricas de saúde]
        F3[Dívida técnica]
        F4[Radar de mercado / verticais]
    end
    subgraph Decisao[Governança]
        RFC[RFC] --> ADR[ADR] --> ROAD[Roadmap Now/Next/Later]
    end
    subgraph Entrega
        BUILD[Build] --> RELEASE[Release versionado] --> DEPREC[Gestão de depreciação]
    end
    subgraph Aprendizado
        MEAS[Medição] --> BIBLE[Project Bible] --> LEARN[Aprendizado]
    end

    F1 & F2 & F3 & F4 --> RFC
    ROAD --> BUILD
    DEPREC --> MEAS
    LEARN --> F2
```

6.2 Governança e vetos

```
flowchart TB
    RFC[RFC] --> TG[Tech Guild]
    TG --> C1{Afeta isolamento/\nresidência/risco?}
    C1 -- sim --> VET[Veto Segurança/DP0]
    C1 -- não --> C2{Reversível?}
    VET -- resolvido --> C2
    VET -- não resolvido --> KILL[Arquivada]
    C2 -- sim --> ADR[ADR + Release]
    C2 -- não --> PC[Platform Council] --> ADR
    ADR --> BIBLE[Project Bible]
```

6.3 Escala multi-região e sharding por tenant

```
flowchart TB
    GDNS[Global DNS / Anycast] --> GW[Gateway global · roteamento por residência]
    GW --> RA[Região A · UE]
    GW --> RB[Região B · Américas]
    GW --> RC[Região C · APAC]

    subgraph RA[Região A]
        RAg[Gateway regional]
        RAg --> SA1[(Shard A1 · tenants)]
        RAg --> SA2[(Shard A2 · tenants)]
    end
    subgraph RB[Região B]
        RBg[Gateway regional]
        RBg --> SB1[(Shard B1 · tenants)]
        RBg --> SB2[(Shard B2 · tenants)]
    end
    subgraph RC[Região C]
        RCg[Gateway regional]
        RCg --> SC1[(Shard C1 · tenants)]
    end

    MAP[Diretório de tenants\ntenant_id → região → shard] -. resolve .-> GW
```

O **Diretório de tenants** é a peça central da escala: mapeia `tenant_id` para sua **região de residência** (imutável sem migração aprovada) e para o **shard** físico. A resolução de Serial Number consulta esse diretório para rotear sem jamais cruzar fronteira de residência indevidamente.

7. Fluxogramas (Release e Depreciação)

7.1 Ciclo de release

```
flowchart TD
    A[Merge em trunk] --> B[Pipeline CI: testes + segurança + contrato]
    B -- falha --> A
    B -- passa --> C[Artefato versionado SemVer]
    C --> D[Deploy canário – 1% tráfego]
    D --> E{Métricas de saúde OK?}
    E -- não --> R[Rollback automático + incidente]
    E -- sim --> F[Rollout progressivo por região]
    F --> G{Feature flag: audiência-alvo}
    G --> H[100% + release notes públicas]
    H --> I[Atualiza changelog API/Plataforma]
    I --> J[Propaga ao Project Bible]
```

7.2 Ciclo de depreciação de recurso ou versão de API

```
flowchart TD
    S0[Identificação: recurso obsoleto / v-antiga] --> S1[Decisão Platform Council + ADR]
    S1 --> S2[Anúncio: changelog + e-mail + header Deprecation]
    S2 --> S3[Estado DEPRECIADO<br/>funciona, porém sinalizado]
    S3 --> S4[Guia de migração + SDK atualizado + suporte]
    S4 --> S5{Uso residual < limiar?}
    S5 -- não --> S6[Engajamento ativo dos tenants remanescentes]
    S6 --> S5
    S5 -- sim --> S7[SUNSET anunciado com data firme]
    S7 --> S8[Header Sunset + degradação anunciada]
    S8 --> S9[REMOÇÃO]
    S9 --> S10[Registro em Bible + auditoria]
```

8. Boas Práticas

- **BP1. Trunk-based com feature flags.** Integração contínua em tronco único; recursos incompletos ficam atrás de *flags*, nunca em *branches* longevas.
- **BP2. Expand/contract para toda mudança de esquema ou contrato.** Primeiro adiciona (expand), migra leitores/escritores, só então remove (contract). Nunca renomeia in-place.
- **BP3. Contrato antes de código.** OpenAPI/Protobuf revisado e versionado precede a implementação (herdado do Doc 09).
- **BP4. Canary + rollback automático.** Todo release passa por audiência reduzida com *guardrails* de métrica que revertem sozinhos.
- **BP5. Dívida técnica visível.** Toda dívida tem registro, impacto estimado e dono; dívida oculta é incidente latente.
- **BP6. Documentar a depreciação no dia do anúncio,** não no dia da remoção. O parceiro precisa de tempo real.
- **BP7. Dogfooding da API pública.** Os próprios apps consomem, sempre que possível, a mesma API pública dos parceiros — o que dói para nós dói para eles antes.
- **BP8. Métricas de saúde como guardrails de deploy,** não apenas *dashboards* de observação.
- **BP9. Migração assistida, nunca imposta a frio.** Ferramentas, *dry-run*, janelas e *rollback* de migração de cliente.
- **BP10. Cada decisão é rastreável ao Bible.** Se não está no Bible, não aconteceu oficialmente.

9. Padrões (SemVer e Políticas)

9.1 Versionamento semântico — dois esquemas independentes

Plataforma e API Pública seguem `MAJOR.MINOR.PATCH`, porém com significados e ciclos próprios.

Incremento	Plataforma	API Pública
MAJOR	Mudança arquitetural com impacto operacional para tenants (ex.: novo modelo de tenancy físico, migração de região obrigatória)	Quebra de contrato observável: remoção de campo/endpoint, mudança de semântica, novo obrigatório
MINOR	Novo módulo/recurso compatível, sem ação do cliente	Adição compatível: novo endpoint, campo opcional, novo escopo
PATCH	Correção/hardening sem mudança de comportamento	Correção de bug sem alterar contrato

Regras de versionamento da API pública:

1. **Versão maior na URL** (`/v1`, `/v2`) — versões maiores coexistem durante a janela de depreciação.
2. **Adições são sempre *minor***. Consumidores toleram campos novos (regra do consumidor robusto / *tolerant reader*).
3. **Nunca reciclar significado**. Um campo depreciado não é reaproveitado com outra semântica.
4. **Deprecation e Sunset headers** (padrão HTTP) sinalizam o estado de cada endpoint/versão.
5. **SDKs acompanham a API**: um SDK `2.x` fala `/v2`; SDKs mantidos por, no mínimo, uma versão maior anterior.

9.2 Política de compatibilidade e janelas de depreciação

Item	Janela mínima de suporte após anúncio	Observações
Versão MAJOR da API pública	24 meses	Duas majors coexistem no mínimo
Endpoint/campo individual (minor)	12 meses	Alternativa disponível antes do anúncio
SDK oficial (major)	12 meses após sucessor	Correções de segurança durante a janela
Recurso de produto (feature)	6 meses	Migração assistida quando houver estado
Formato de webhook/evento	12 meses	Evento novo coexiste com o antigo

Contrato de compatibilidade (o que a plataforma promete):

- **Nunca** quebrar sem *major* + anúncio + janela cumprida.
- **Nunca** tornar um Serial Number histórico irresolúvel (R10).
- **Sempre** oferecer caminho de migração documentado e ferramenta antes do *sunset*.
- **Sempre** comunicar por, no mínimo, três canais: changelog público, *header* HTTP e notificação direta ao tenant/parceiro afetado.

9.3 Padrões de dívida técnica e melhoria contínua

- **Orçamento fixo**: 20% da capacidade de cada ciclo reservada a saúde técnica (R7), com *tracking* separado do roadmap de recursos.
- **Registro estruturado**: cada item de dívida tem `tipo` (código, dados, arquitetura, dependência, operacional), `impacto` (probabilidade × custo), `juros` (custo de mantê-la) e `dono`.
- **Regra do escoteiro**: áreas tocadas saem melhores do que entraram; refatoração oportunista é bem-vinda dentro do escopo.
- **Índice de dívida** publicado no painel de saúde (§8/§11) e revisado pela Tech Guild.

10. Casos de Uso

CU1 — Lançar uma capacidade compatível na API pública. Um novo endpoint de consulta de histórico consolidado do SN é proposto. RFC → ADR → implementação atrás de *flag* → *canary* → *minor* [v1.7](#) → [v1.8](#) da API. Nenhum parceiro precisa agir; SDKs ganham o método na próxima *minor*. Registro no Bible.

CU2 — Quebrar contrato inevitável (major). A semântica de status de OS precisa mudar para suportar novos verticais. Decisão no Platform Council, com veto potencial de segurança avaliado. Publica-se [/v2](#) mantendo [/v1](#) por 24 meses, guia de migração, SDK [2.x](#), headers [Deprecation / Sunset](#), e engajamento ativo dos parceiros de maior uso.

CU3 — Onboarding de parceiro no marketplace. Integrador de contabilidade solicita entrada. Recebe credenciais de *sandbox*, escopos mínimos, passa por certificação técnica + KCD + privacidade, publica conector; tenants instalam com consentimento explícito de escopos. Monitoramento contínuo e *kill switch* disponíveis.

CU4 — Expandir para uma nova região com *data residency*. Demanda de clientes em nova jurisdição. Steering aprova; provisiona-se região; *data residency* declarada torna-se atributo imutável dos novos tenants; diretório de tenants roteia; resolução de SN respeita a fronteira. Auditoria confirma que nenhum dado cruza indevidamente.

CU5 — Migrar um tenant de shard/região. Tenant cresce e precisa de shard dedicado, ou muda de residência por exigência legal. Migração assistida: *dry-run*, janela combinada, cópia consistente, verificação de resolução de SN ponta a ponta, corte com *rollback* pronto, atualização do diretório de tenants. Zero perda de rastreabilidade.

CU6 — Depreciar um recurso subutilizado. Métricas mostram uso residual de um relatório legado. Platform Council decide depreciar; anúncio + guia; após uso cair abaixo do limiar e cumprir 6 meses, *sunset* e remoção. Tudo registrado no Bible.

CU7 — Evoluir o KCI para modo preditivo mais autônomo. Novo motor preditivo entra atrás de *flag*, em *shadow mode* (calcula sem agir), comparado ao comportamento atual; ao provar ganho, promove-se progressivamente por tenant. Mantém-se *offline-first* (canon) e explicabilidade para auditoria.

11. Modelagem — Horizontes Now / Next / Later

Horizontes indicam **direção e confiança**, não datas contratuais. "Now" é comprometido; "Next" é provável e em desenho; "Later" é direção estratégica sujeita a validação.

11.1 Roadmap por horizonte

```
flowchart LR
    subgraph NOW[NOW · comprometido]
        N1[SemVer + política de depreciação da API]
        N2[Painel de saúde da plataforma]
        N3[SDKs oficiais v1 + sandbox de parceiros]
        N4[Sharding por tenant · 1ª região adicional]
        N5[Orçamento de dívida técnica formalizado]
    end
    subgraph NEXT[NEXT · em desenho]
        X1[Marketplace de integrações curado]
        X2[Multi-região com data residency declarável]
        X3[KCI preditivo em shadow mode]
        X4[KCD · maturação de SOC 24x7 + threat intel]
        X5[KSI · otimização de reposição assistida por IA]
    end
    subgraph LATER[LATER · direção]
        L1[KCI autônomo/prescritivo com explicabilidade]
        L2[KCD · resposta automatizada / SOAR maduro]
        L3[KSI · otimização multi-tenant federada]
        L4[Novos verticais: seguros, logística, energia, defesa civil]
        L5[Federação de residência e edge por região]
    end
    NOW --> NEXT --> LATER
```

11.2 Detalhamento dos horizontes

Eixo	Now	Next	Later
Governança/ API	SemVer duplo + política de depreciação ativos	Automação de changelog e <i>contract testing</i> de parceiros	<i>Self-service</i> de <i>deprecation impact</i> por parceiro
Extensibilidade	SDKs oficiais, <i>sandbox</i> , certificação	Marketplace curado, camadas de parceria	Ecossistema com <i>revenue share</i> e templates verticais
Escala	Sharding por tenant, 2ª região	Multi-região + <i>data residency</i> declarável	Edge por região, federação, otimização de custo por tenant
KCI	Preditivo assistido, <i>offline-first</i>	<i>Shadow mode</i> preditivo comparado	Autônomo/prescritivo com explicabilidade auditável
KCD	Zero Trust, auditoria, <i>threat detection</i>	SOC 24x7, <i>threat intel</i> , <i>playbooks</i>	SOAR maduro, resposta automatizada, <i>purple team</i> contínuo
KSI	Curva ABC, reposição automática, previsão	Otimização de reposição assistida por IA	Otimização federada multi-tenant (preservando isolamento)
Verticais	ERP de drones (núcleo)	Adjacências ao ecossistema de drones	Seguros, logística, energia, defesa civil, agro

Nota sobre IA (canon): toda evolução de inteligência (KCI/KSI) mantém-se *offline-first* quando aplicável, com IA externa **opcional**, explicabilidade para auditoria e sem dependência de qualquer modelo comercial específico. A plataforma trata modelos de IA como **componentes substituíveis** atrás de uma interface — nenhum documento ou contrato amarra a plataforma a um fornecedor de IA.

11.3 Critérios de promoção entre horizontes

Transição	Gatilhos objetivos
Later → Next	Sinal de demanda validado (feedback + mercado) + viabilidade técnica confirmada em RFC
Next → Now	ADR aprovado + capacidade alocada + sem veto de segurança/DPO + métrica-alvo definida
Now → Entregue	Passou Quality Gate + <i>canary</i> saudável + release notes + propagado ao Bible

12. Checklist

Checklist de governança de release - ☐ RFC aberta, revisada e com *consent* registrado. - ☐ ADR criado (ou *supersede* explícito de ADR anterior). - ☐ Impacto de compatibilidade classificado (patch/minor/major) para plataforma e API. - ☐ Veto de segurança/DPO consultado quando aplicável. - ☐ *Feature flag* e plano de *rollback* prontos. - ☐ *Canary* + *guardrails* de saúde configurados. - ☐ Release notes + changelog (plataforma e/ou API) atualizados. - ☐ Propagado ao Project Bible.

Checklist de depreciação - ☐ Decisão do Platform Council + ADR. - ☐ Alternativa disponível **antes** do anúncio. - ☐ Janela mínima aplicável definida (§9.2). - ☐ Anúncio em três canais (changelog, *header*, notificação direta). - ☐ Guia de migração + ferramenta + SDK atualizado. - ☐ Monitoramento de uso residual até o limiar. - ☐ *Sunset* com data firme; remoção; registro no Bible.

Checklist de escala/nova região - ☐ Aprovação Steering + requisitos de *data residency* mapeados. - ☐ Diretório de tenants suporta a nova região/shard. - ☐ Resolução de SN validada dentro da fronteira de residência. - ☐ Migração de tenant testada em *dry-run* com *rollback*. - ☐ Auditoria confirma não-vazamento entre regiões/tenants.

Checklist de extensibilidade/parceiro - ☐ Escopos OAuth mínimos definidos. - ☐ Quotas e *rate limits* por chave. - ☐ Certificação técnica + KCD + privacidade aprovada. - ☐ *Sandbox* validado antes de produção. - ☐ *Kill switch* e monitoramento ativos.

13. Riscos

#	Risco	Impacto	Probabilidade	Mitigação
RS1	Quebra silenciosa de contrato de API	Perda de parceiros e confiança	Média	<i>Contract testing</i> de parceiros, <i>tolerant reader</i> , R4/R5, <i>canary</i>
RS2	Dívida técnica sufocando o roadmap	Velocidade colapsa	Alta	Orçamento fixo de 20% (R7), índice de dívida visível
RS3	Depreciação apressada sem migração	Churn e ruptura contratual	Média	Política §9.2, migração assistida, janelas mínimas
RS4	Vazamento entre tenants/regiões na escala	Violação legal e de confiança	Baixa	R9, veto do DPO, auditoria de residência, isolamento por shard
RS5	SN histórico irresolvível após migração	Falha do valor constitucional	Baixa	R10, verificação ponta a ponta em toda migração
RS6	Marketplace com integração maliciosa	Risco de segurança ao tenant	Média	Certificação, escopos mínimos, <i>kill switch</i> , monitoramento
RS7	Dependência de fornecedor de IA	Amarra estratégica	Média	Modelos como componentes substituíveis (§11.2)
RS8	Roadmap capturado por cliente único	Distorce a plataforma	Média	Priorização por métrica (R11), governança de fórum
RS9	Bible desatualizado	Perda da fonte de verdade	Média	Propagação obrigatória (R3), Quality Gate
RS10	Explosão de versões de API coexistentes	Custo operacional e confusão	Média	Limite de majors vivas, <i>sunset</i> disciplinado

14. Melhorias Futuras

- **MF1.** *Contract testing* automatizado contra os principais parceiros, bloqueando *merge* que quebre consumidores reais.
- **MF2.** Geração automática de *changelog* e *deprecation notices* a partir de *diffs* de OpenAPI.
- **MF3.** Painel de "*deprecation impact*" *self-service* para cada parceiro visualizar o que o afeta.
- **MF4.** Automação do índice de estado do Bible a partir dos cabeçalhos dos documentos (herdado do Doc 02).
- **MF5.** *Chaos engineering* regionalizado para validar isolamento de falha por região/shard.
- **MF6.** *Feature flags* com segmentação por tenant/vertical e *rollout* orientado por métrica de saúde.
- **MF7.** Programa de parceiros com *revenue share* e *templates* verticais no marketplace.
- **MF8.** *Explainability service* transversal para KCI/KSI, alimentando a auditoria contínua.

15. Auditoria

Trilha de auditoria da evolução. Toda mudança é rastreável em cadeia contínua: [Feedback/Métrica](#) → [RFC](#) → [ADR](#) → [Release versionado](#) → [Changelog](#) → [Project Bible](#) → [Trilha de auditoria](#). Nenhum elo pode faltar; a ausência de qualquer um é achado de auditoria.

Objeto auditável	Evidência esperada	Frequência
Decisões de evolução	ADR imutável + referência no Bible	A cada decisão
Compatibilidade da API	<i>Diff</i> de OpenAPI + classificação SemVer + <i>contract tests</i>	A cada release
Depreciações	Anúncio datado, <i>headers</i> , guia, evidência de janela cumprida	A cada depreciação
Isolamento tenant/residência	Relatório de auditoria de não-vazamento	Contínua + a cada nova região
Resolução de SN	Verificação ponta a ponta pós-migração	A cada migração
Dívida técnica	Índice de dívida + orçamento consumido	A cada ciclo
Saúde da plataforma	<i>Snapshot</i> de métricas (§8/§11)	Contínua
Marketplace/parceiros	Registro de certificação + monitoramento	Onboarding + contínuo

Métricas de saúde da plataforma (referência do painel): disponibilidade por região (SLA), latência p95/p99 da API pública, taxa de erro por versão, *lead time* de mudança, frequência de deploy, MTTR, taxa de falha de mudança (DORA); adoção de versões de API e SDKs; uso residual de recursos depreciados; índice de dívida técnica; NPS/CSAT de tenants e parceiros; consumo de quota por parceiro. Essas métricas alimentam o **ciclo de feedback** (§6.1) e servem de *guardrail* de deploy (BP8).

Consistência com o canon. Este documento respeita e referencia: Constituição (01), Project Bible (02), Roadmap (03), Arquitetura (07), Banco de Dados (08), Microsserviços (09) e os documentos de módulos, API pública e operação (10 – 23). Não introduz contradição; onde toca outro documento, remete a ele.

Estado. Documento **Ativo v1.0**. Como fecha o kit e governa a mudança, é **revisado ao fim de cada fase** junto ao Project Bible, e atualizado sempre que uma decisão de evolução altere as regras aqui descritas.

Fim do 24 – Evolução da Plataforma · v1.0 — este documento fecha o kit 00 – 24 e permanece vivo enquanto a plataforma evoluir.